BR280

实验指导书

文档版本：V2.0

发布日期：2022-05-06

深圳市史河机器人科技有限公司

目 录

第一章 实验设备安装 1
实验 1 基于 VirtualBox 虚拟机安装 Ubuntu 系统 1
一、实验目的 1
二、实验设备 1
2.1 安装 VirtualBox 虚拟机软件 1
2.2 ubuntu-18.04.5-desktop-amd64.iso 映像文件 1
三、实验步骤 1
3.1 VirtualBox 的安装与配置 1
3.2 创建 Ubuntu 3
3.3 安装 Ubuntu 7
实验 2 ROS 安装与配置 17
一、实验目的 17
二、实验设备 17
三、实验步骤 17
3.1 Ubuntu 更换软件安装源 17
3.2 配置 ROS 的安装源 19
3.3 安装 ROS 桌面完整版 24
3.4 配置并激活环境变量 25
3.5 构建包依赖关系 26
3.6 初始化 rosdep 27
3.7 检查 ROS 是否安装成功 34
第二章 基础类实验 36
实验 3 ROS 文件系统导览 36
一、实验目的 36
二、实验设备 36
三、实验原理 36
3.1 ROS 文件系统概念介绍 36
3.2 文件系统工具 36
四、实验步骤 37
4.1 使用 rospack 37
4.2 使用 roscd 37
4.3 使用 rosls 38

版权所有 © 深圳史河机器人科技有限公司

4.4 Tab 补全 38
五、课堂作业 39
实验 4　创建与编译 ROS 软件包 40
一、实验目的 40
二、实验原理 40
2.1 一个 catkin 软件包的组成成分 40
2.2 工作空间（workspace）的含义 40
2.3 环境变量 40
三、实验步骤 41
3.1 创建 catkin 工作空间 41
3.2 检查环境变量 43
3.3 创建 catkin 软件包 43
3.4 构建一个 catkin 工作区间并生效配置文件 44
3.5 软件包依赖关系 44
3.6 自定义软件包 46
3.7 构建 ROS 软件包 49
四、课堂作业 49
实验 5　理解 ROS 节点 50
一、实验目的 50
二、实验设备 50
三、实验原理 50
3.1 计算图 50
3.2 节点 50
3.3 客户端库 51
四、实验步骤 51
4.1 roscore 51
4.2 使用 rosnode 51
4.3 使用 rosrun 52
五、课堂作业 54
实验 6　理解 ROS 话题 55
一、实验目的 55
二、实验设备 55
2.1 roscore 55
2.2 turtlesim 55

版权所有 © 深圳史河机器人科技有限公司

三、实验原理 56
3.1 ROS 话题 56
四、实验步骤 56
4.1 使用 rqt_graph 56
4.2 rostopic 简介 57
4.3 使用 rostopic echo 57
4.4 使用 rostopic list 58
4.5 ROS 消息 59
4.6 使用 rostopic type 59
4.7 深入学习 rostopic 60
4.8 使用 rqt_plot 63
五、课堂作业 64
实验 7 理解 ROS 服务和参数 65
一、实验目的 65
二、实验设备 65
三、实验原理 65
3.1 ROS 服务 65
3.2 rosparam 65
四、实验步骤 65
4.1 使用 rosservice 65
4.2 使用 rosparam 68
五、课堂作业 70
实验 8 使用 rqt_console 和 roslaunch 71
一、实验目的 71
二、实验设备 71
三、实验原理 71
四、实验步骤 71
4.1 使用 rqt_console 和 rqt_logger_level 71
五、课堂作业 76
实验 9 自定义消息和服务 77
一、实验目的 77
二、实验原理 77
2.1 msg 和 srv 介绍 77
三、实验步骤 78

版权所有 © 深圳史河机器人科技有限公司

3.2 使用 msg 78
3.3 使用 srv 80
3.4 获取帮助 81
四、课堂作业 81
实验 10 编写简单的发布者和订阅者（C++） 83
一、实验目的 83
二、实验设备 83
三、实验原理 83
四、实验操作 83
4.1 编写发布者结点 83
4.2 编写订阅者节点 86
4.3 构建节点 87
4.4 检验发布者和订阅者 89
五、课堂作业 90
实验 11 编写简单的发布者和订阅者（Python） 91
一、实验目的 91
二、实验设备 91
三、实验原理 91
四、实验操作 91
4.1 编写发布者结点 91
4.2 编写订阅者节点 94
4.3 构建节点 95
4.4 检验发布者和订阅者 95
五、课堂作业 96
实验 12 编写简单的服务和客户端（C++） 97
一、实验目的 97
二、实验设备 97
三、实验原理 97
四、实验步骤 97
4.1 编写服务节点 97
4.2 编写客户端节点 98
4.3 构建节点 100
4.4 检验服务和客户端 101
五、课堂作业 101

版权所有 © 深圳史河机器人科技有限公司

实验 13 编写简单的服务和客户端（Python） 102
一、实验目的 102
二、实验设备 102
三、实验原理 102
四、实验步骤 102
4.1 编写服务节点 102
4.2 编写客户端节点 103
4.3 构建节点 104
4.4 检验简单的服务和客户端 105
五、课堂作业 105
实验 14 录制和回放数据 106
一、实验目的 106
二、实验设备 106
三、实验步骤 106
3.1 录制数据（创建 bag 文件） 106
3.2 检查并回放 bag 文件 107
3.3 录制数据子集 108
四、课堂作业 109
实验 15 从 bag 文件中读取消息 110
一、实验目的 110
二、实验设备 110
三、实验步骤 110
3.1 立即回放消息并在多个终端中查看输出 110
四、课堂作业 112
实验 16 Roswtf 的学习 113
一、实验目的 113
二、实验设备 113
三、实验原理 113
四、实验步骤 113
4.1 安装检查 113
4.2 在线检查 114
实验 17 rviz 可视化工具 116
一、实验目的 116
二、实验设备 116

版权所有 © 深圳史河机器人科技有限公司

三、实验原理 116
3.1 RViz 安装与启动 116
3.2 坐标框架（Coordinate Frames） 119
第三章 应用类实验 120
实验 18 BR280 安装与配置 120
一、实验目的 120
二、实验设备 120
三、实验步骤 120
3.1 摄像头和夹持器的安装 120
3.2 网络配置和 IP 获取 124
3.3 PC 连接远程桌面 127
实验 19 BR280 基本运动控制 131
一、实验目的 131
二、实验设备 131
三、实验原理 131
四、实验步骤 131
五、课堂作业 134
实验 20 BR280 传感器数据获取 135
一、实验目的 135
二、实验设备 135
三、实验原理 135
四、实验步骤 137
五、课堂作业 142
实验 21 BR280 定位与建图 143
一、实验目的 143
二、实验设备 143
三、实验原理 143
四、实验步骤 144
五、课堂作业 145
实验 22 BR280 自主导航避障 146
一、实验目的 146
二、实验设备 146
三、实验原理 146
四、实验步骤 149

版权所有 © 深圳史河机器人科技有限公司

五、课堂作业 150
实验 23 BR280 图形化编程控制机械臂 151
一、实验目的 151
二、实验设备 151
三、实验原理 151
四、实验步骤 152
五、课堂作业 156
实验 24 BR280 Python 编程控制机械臂 157
一、实验目的 157
二、实验设备 157
三、实验原理 157
四、实验步骤 158
五、课堂作业 159
实验 25 BR280 ROS+moveit 编程 160
一、实验目的 160
二、实验设备 160
三、实验原理 160
四、实验步骤 161
五、课堂作业 163
实验 26 BR280 手眼标定 164
一、实验目的 164
二、实验设备 164
三、实验原理 164
四、实验步骤 165
五、课堂作业 170
实验 27 BR280 二维码识别抓取积木 171
一、实验目的 171
二、实验设备 171
三、实验原理 171
3.1 视觉抓取中的关键技术 171
3.2 二维码识别 171
四、实验步骤 174
五、课堂作业 178
实验 28 BR280 颜色识别抓取积木 179

版权所有 © 深圳史河机器人科技有限公司

一、实验目的.....179
二、实验设备.....179
三、实验原理.....179
四、实验步骤.....180
五、课堂作业.....184
实验 29 BR280 物体识别抓取积木.....185
一、实验目的.....185
二、实验设备.....185
三、实验原理.....185
四、实验步骤.....186
五、课堂作业.....189
实验 30 BR280 深度学习实验 1—数据集采集与标注.....190
一、实验目的.....190
二、实验设备.....190
三、实验原理.....190
四、实验步骤.....192
4.1 数据集采集.....192
4.2 数据标注.....194
五、课堂作业.....197
实验 31 BR280 深度学习实验 2—模型训练与目标检测.....198
一、实验目的.....198
二、实验设备.....198
三、实验原理.....198
3.1 什么是目标检测.....198
3.2 基于深度学习的目标检测算法.....198
四、实验步骤.....199
4.1 模型训练.....199
4.2 目标检测.....203
五、课堂作业.....204
实验 32 BR280 自主移动抓取实验.....205
一、实验目的.....205
二、实验设备.....205
三、实验原理.....205
3.1 什么是移动抓取.....205

版权所有 © 深圳史河机器人科技有限公司

3.2 自主移动和视觉精准对位功能 205
3.3 二维码积木抓取 206
四、实验步骤 207
4.1 改造小桌子 207
4.2 场地布置 207
4.3 软件操作 208
五、 课堂作业 209

版权所有 © 深圳史河机器人科技有限公司

第一章 实验设备安装

实验 1　基于 VirtualBox 虚拟机安装 Ubuntu 系统

一、实验目的

安装 Ubuntu 系统，所有的开发工作都会在 Ubuntu 系统上完成。

二、实验设备

2.1 安装 VirtualBox 虚拟机软件

VirtualBox 下载链接：

https://pan.baidu.com/s/1-u5pIMDxQPDP2gvGU6DDfw， 提取码：tgn4。

2.2 ubuntu-18.04.5-desktop-amd64.iso 映像文件

Ubuntu 下载链接：

https://pan.baidu.com/s/1yoLnvcHSKscx5s95xz-Y1Q ，提取码：koie。

三、实验步骤

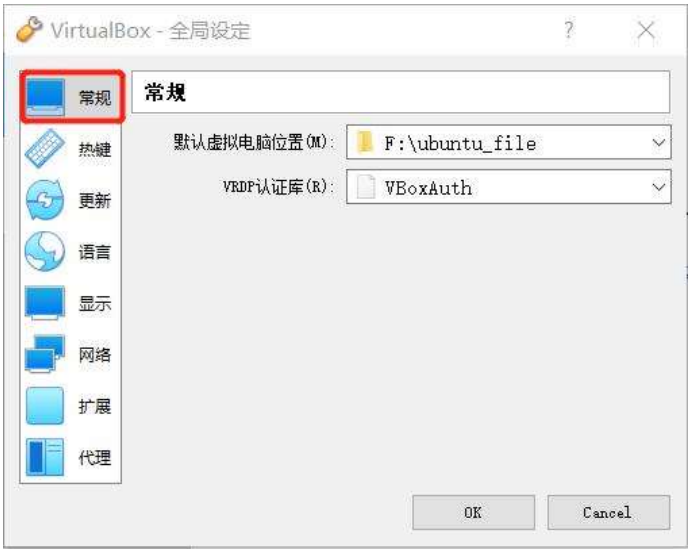
3.1 VirtualBox 的安装与配置

1）下载好 VirtualBox 之后，双击运行开始安装，安装这个软件没有复杂操作，按照提示操作即可。

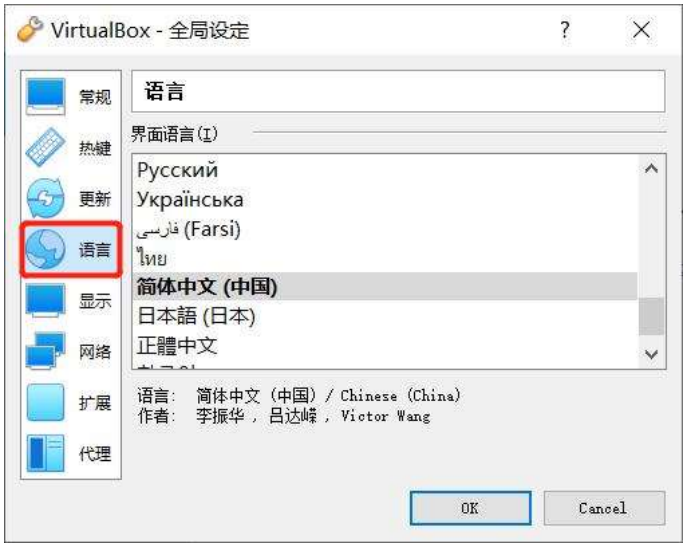
2）运行安装好的 VirtualBox 程序，如下图：

版权所有 © 深圳史河机器人科技有限公司

3）配置 VirtualBox。按 CTRL+G 打开全局设定，根据需要可以设定虚拟电脑位置和界面语言：修改默认虚拟电脑位置比如 F:\ubuntu_file 可以让自己更方便查找、备份安装好的虚拟机。



如果界面是英文，可以改成中文：



版权所有 © 深圳史河机器人科技有限公司

3.2 创建 Ubuntu

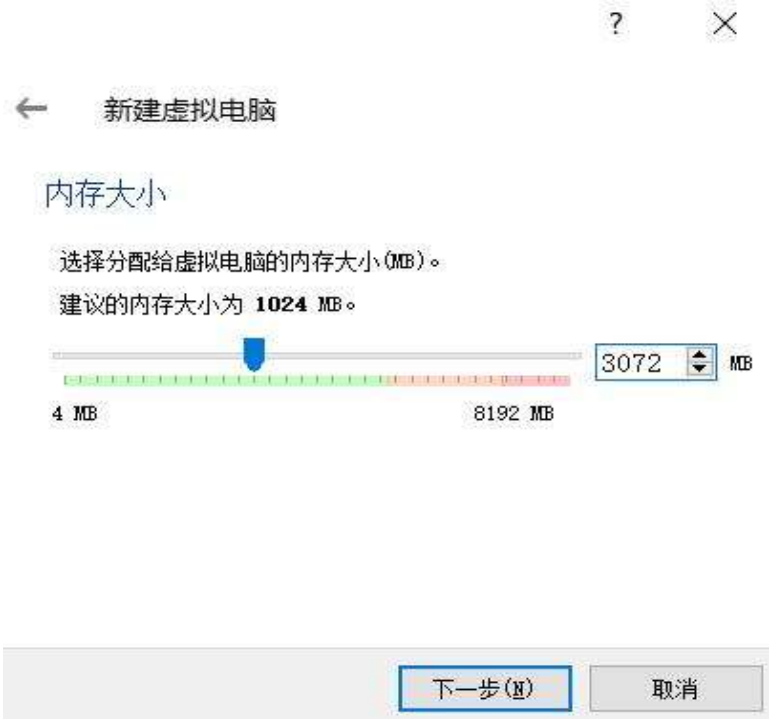
1）运行 VirtualBox 程序，点击“新建”按钮，创建一个虚拟机。

2）虚拟机名称可以随意输入，示例：ros。操作系统选择 Linux，版本选择 Ubuntu，点击“下一步”。

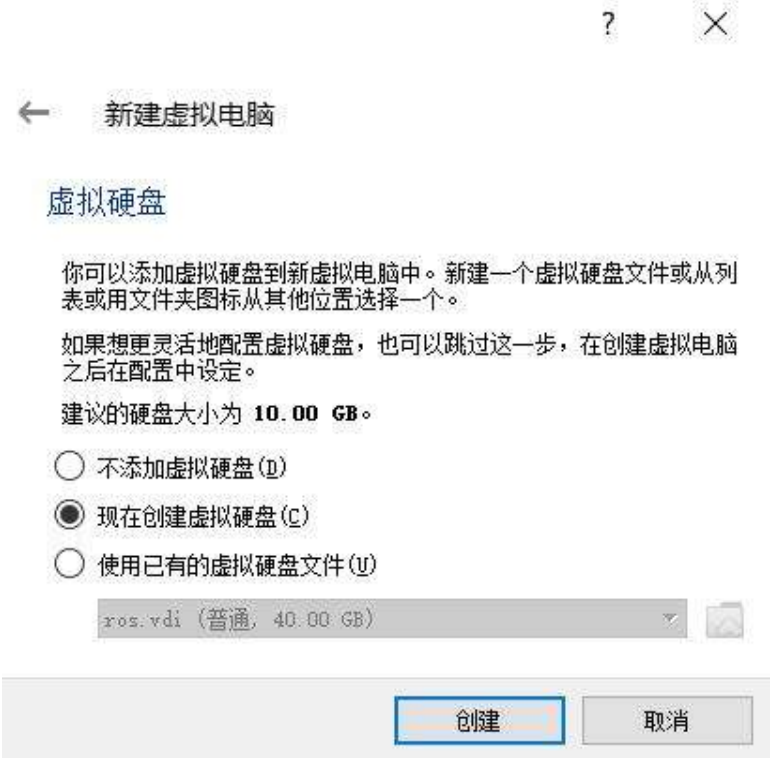
3）设定虚拟机的内存，此内存即为虚拟机所占用的系统内存，可随意修改，建议不要超过系统内存的 1/2。

首先了解自己的笔记本电脑系统内存大小，本电脑的系统内存为 8G，为了方便以后编程学习使用，这里将虚拟内存设为 3G，点击“下一步”。

版权所有 © 深圳史河机器人科技有限公司

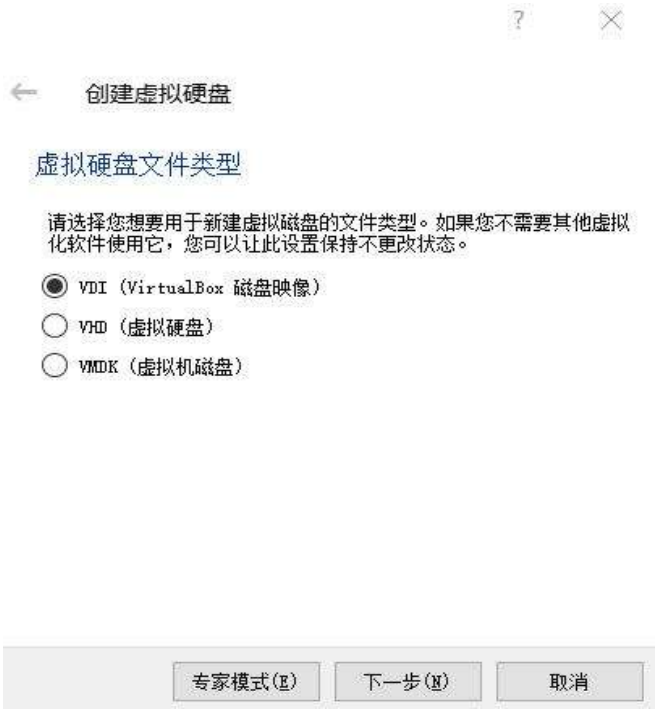


4）为虚拟机创建一块虚拟硬盘。选择“现在创建虚拟硬盘”，并单击“创建”。



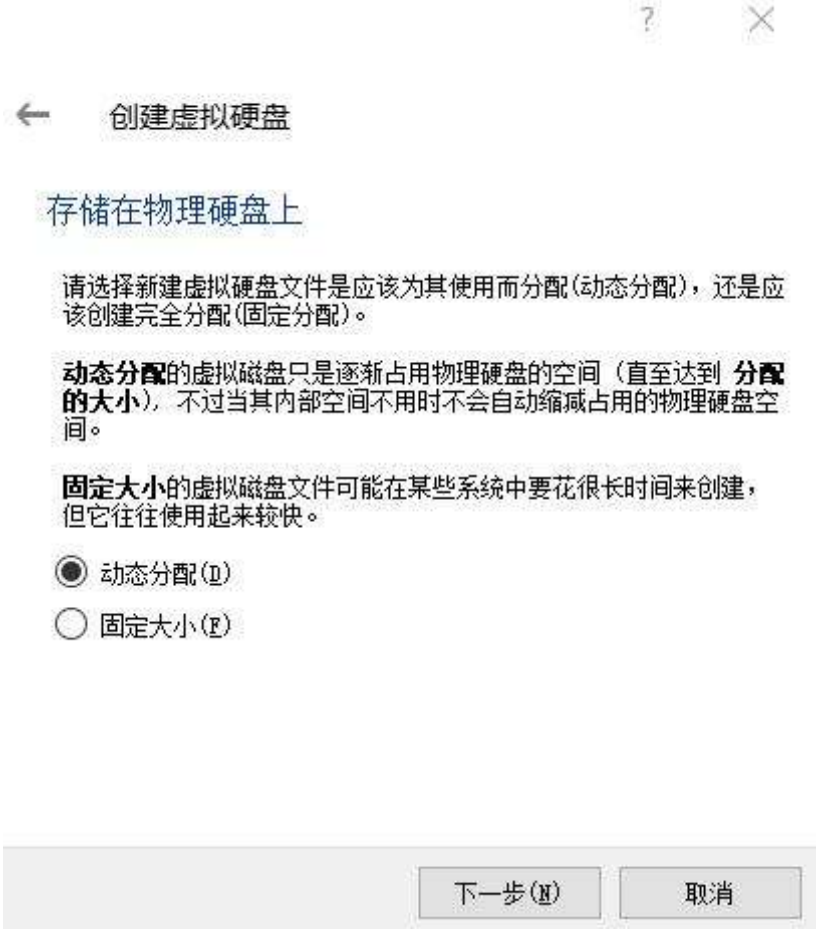
5）选择虚拟硬盘文件类型。选择默认的 VDI (VirtualBox 磁盘映像)，并单击“下一步”。

版权所有 © 深圳史河机器人科技有限公司



6）选择“动态扩展”，并单击“下一步”。

因为分配给虚拟机的内存空间较大，使用时逐渐占用磁盘空间，闲置时自动缩减比较合理，所以选择动态扩展类型。



版权所有 © 深圳史河机器人科技有限公司

7）输入虚拟硬盘文件的名称，并选择保存位置。

选择虚拟硬盘大小时一定先确定保存所在位置磁盘的可用大小。可用空间为270G，虚拟硬盘大小设置为 50G。单击“创建”。

8）虚拟机创建完成。也就是说 Ubuntu 所需的硬件资源准备好了，相当于买了一个没有安装操作系统的电脑主机。

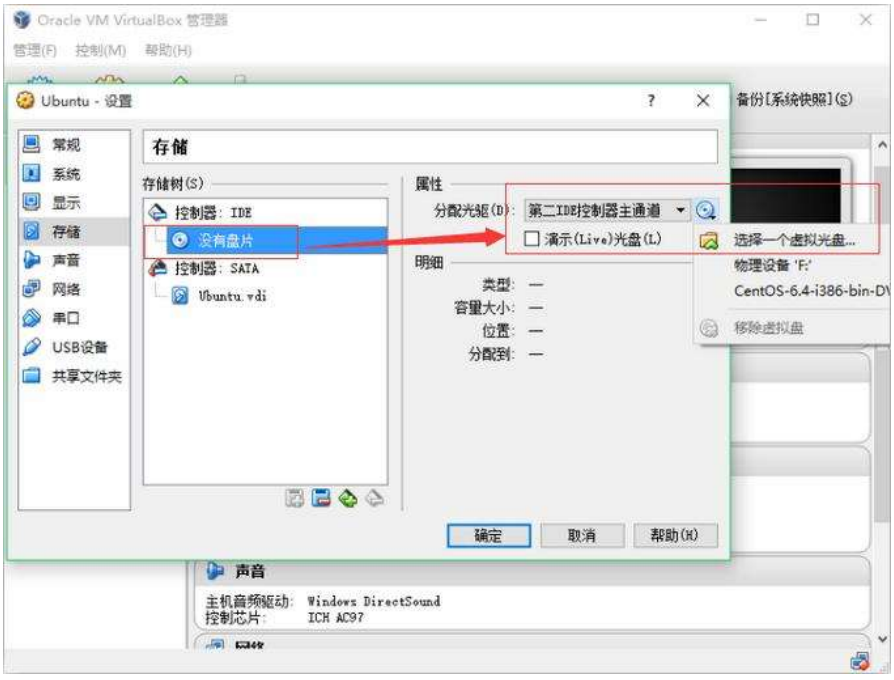
版权所有 © 深圳史河机器人科技有限公司

3.3 安装 Ubuntu

1）选择下载的 Ubuntu LTS 14.04 的 ISO 映像文件

到达此界面，请勿直接点击启动按钮，否则有可能会导致进程中断。应该先设置"存储"，请点击下图中红框中的按钮，打开存储设置界面

2）进入存储设置界面后，点击没有盘片，再点击光盘按钮，选择一个虚拟光驱，添加上面您下载的 ubuntu-18.04.5-desktop-amd64.iso 映像文件。



版权所有 © 深圳史河机器人科技有限公司

3）旋转刚刚创建的虚拟机 Ubuntu，点击“启动”按钮。

4）启动如果看到下面的界面，那么下拉选项选择你刚才选择的 ISO 文件。（如果没有出现此界面,直接跳往下一步即可）

版权所有 © 深圳史河机器人科技有限公司

5）进入安装界面，选择“中文简体”，选择“安装 Ubuntu”。

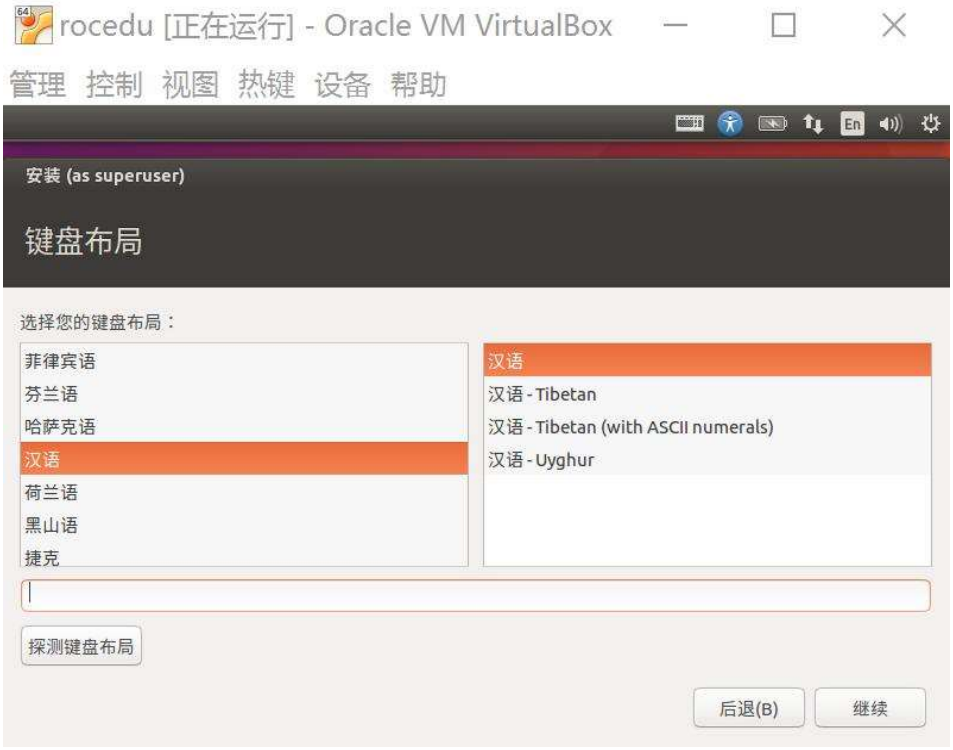
6）选择“继续”。网络状况不好可以不选择"安装 Ubuntu 时下载更新”

7）选择“清除整个磁盘并安装 Ubuntu”单击“现在安装”，弹出警告框，选择“继续”。

版权所有 © 深圳史河机器人科技有限公司

8）安装过程中时区，用鼠标单击地图中的“中国”，就会选择“上海”，单击“继续”。

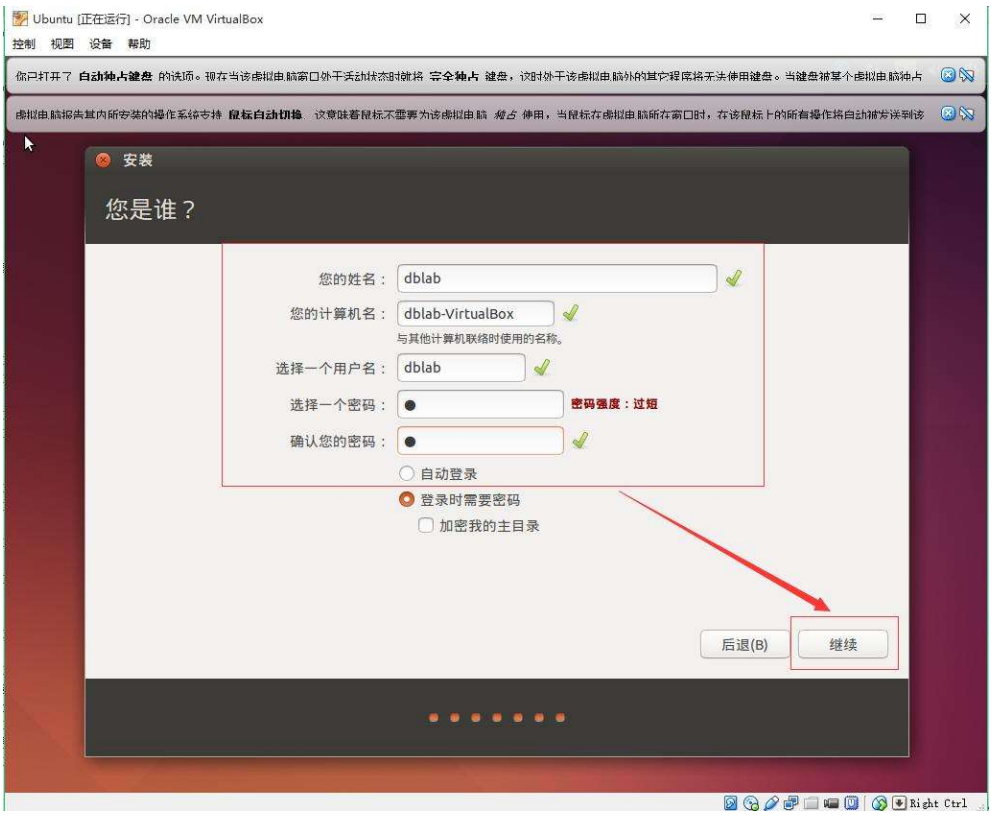
语言选择“汉语”



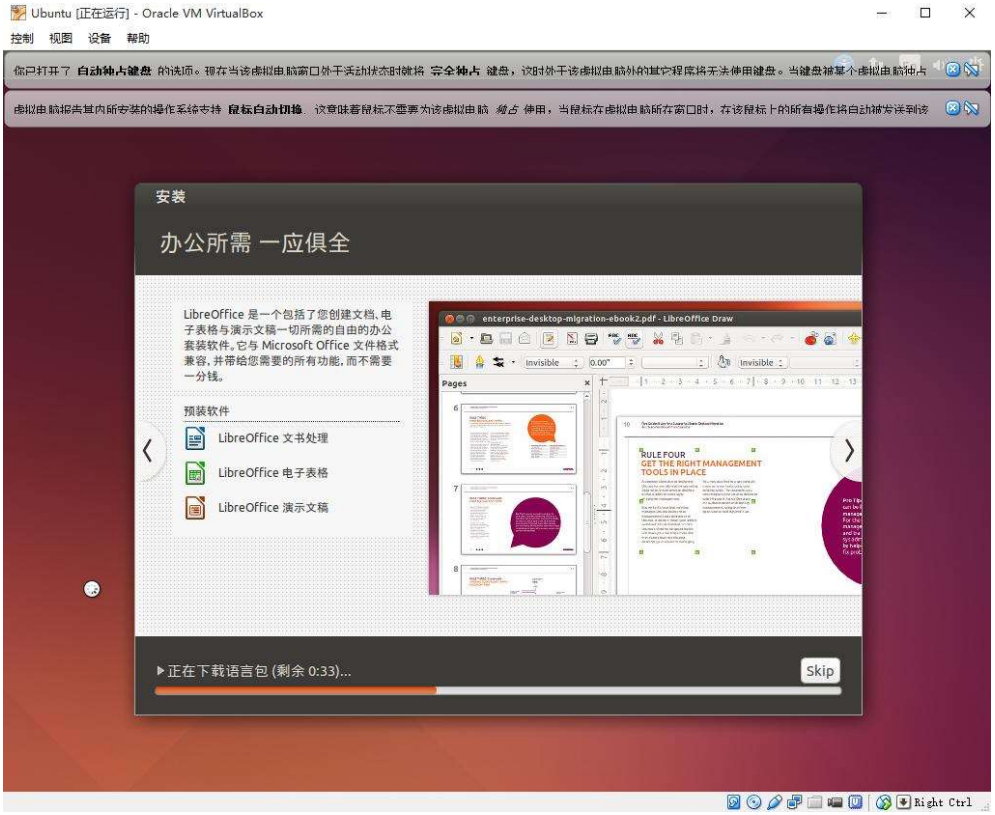
9）设置用户名和密码

建议选择“登录时需要密码”选项。由于我们现在处于学习阶段，不需要考虑安全问题，密码建议使用一位的密码，这样在安装软件需要输入密码的时候也比较方便。

版权所有 © 深圳史河机器人科技有限公司



在安装过程中，系统会自动安装，不要点击 skip，等待系统自动安装完成。

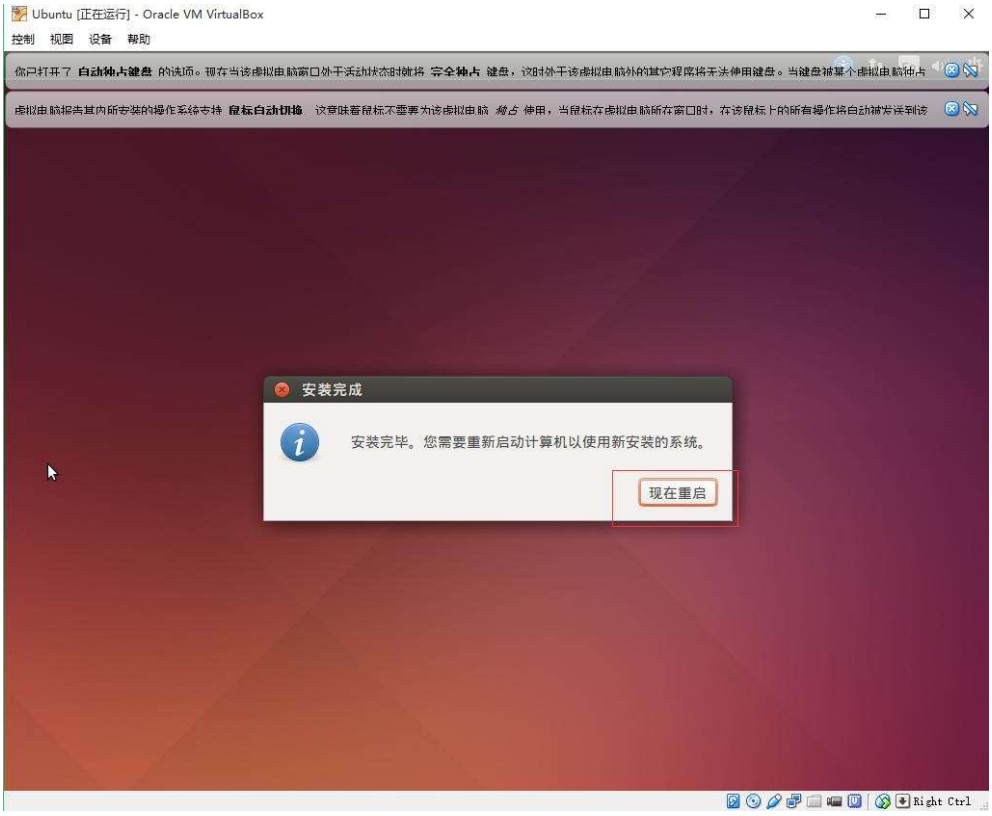


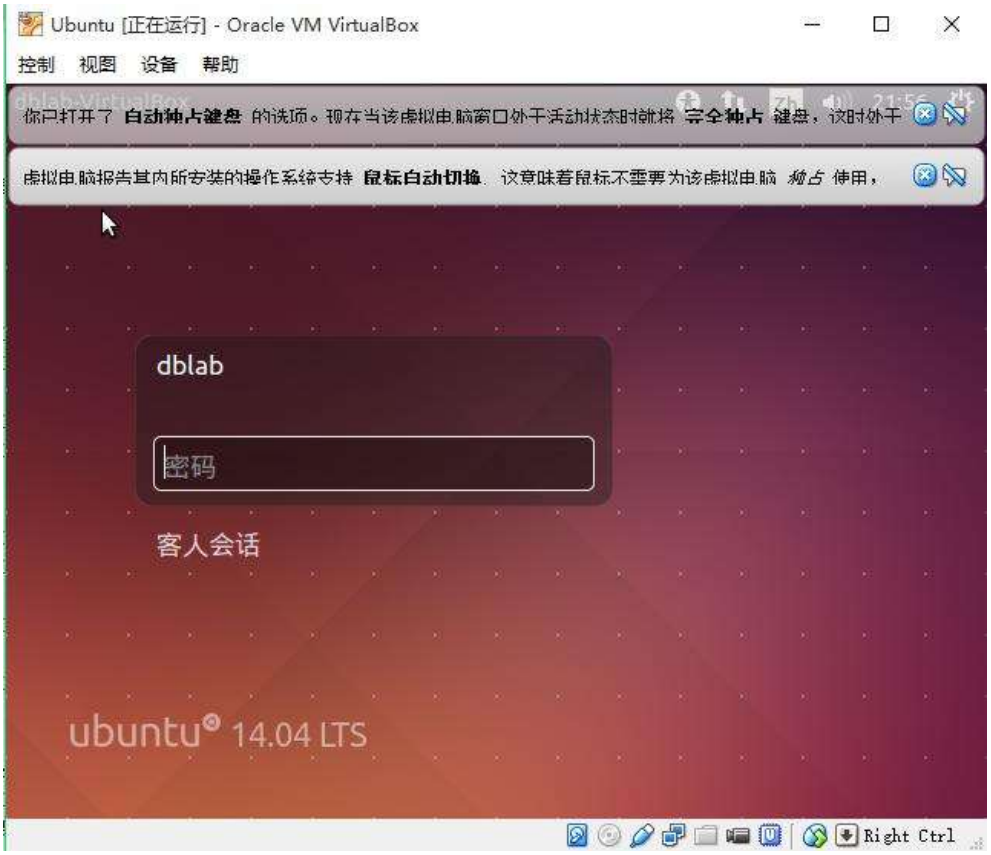
10）安装完成之后重启

安装终于完成，点击“现在重启”即可。“现在重启”只是重启虚拟机系统的

版权所有 © 深圳史河机器人科技有限公司

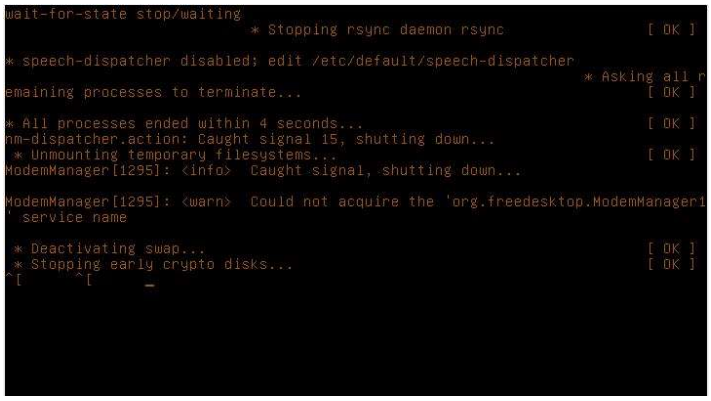
运行，并不是重启 Windows 系统。





重启时，可能会出现如下提醒页面

版权所有 © 深圳史河机器人科技有限公司

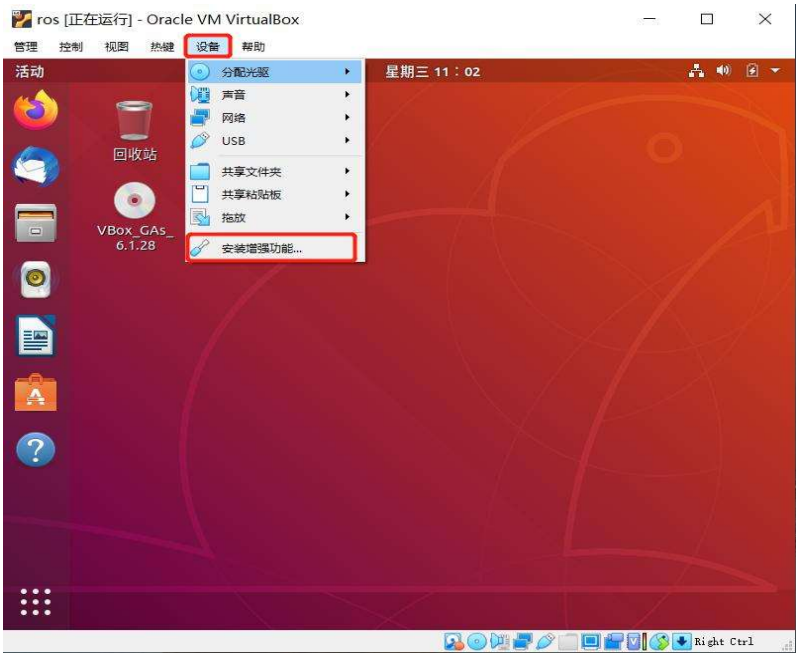


我们可以自己关闭虚拟机系统，选择“强制退出”选项

11）重新启动 Ubuntu 系统，并安装增强功能

启动后，虚拟机 Ubuntu 系统默认是以窗口模式打开,而且分辨率很低。这是 VirtualBox 初始安装完 Ubuntu 的原因，我们需要修复这个问题，让虚拟机支持大的分辨率。

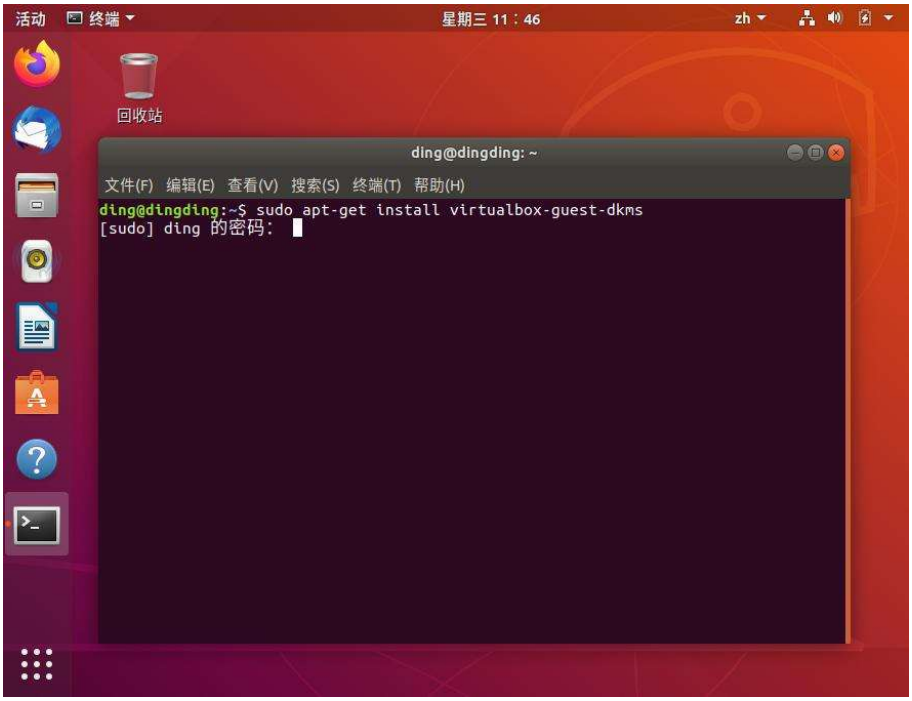
点击 VirtualBox 的菜单“设备”选项，选择“安装增强功能”，系统便会自动安装好增强的功能，如果提示需要确认，按 Enter 键即可。



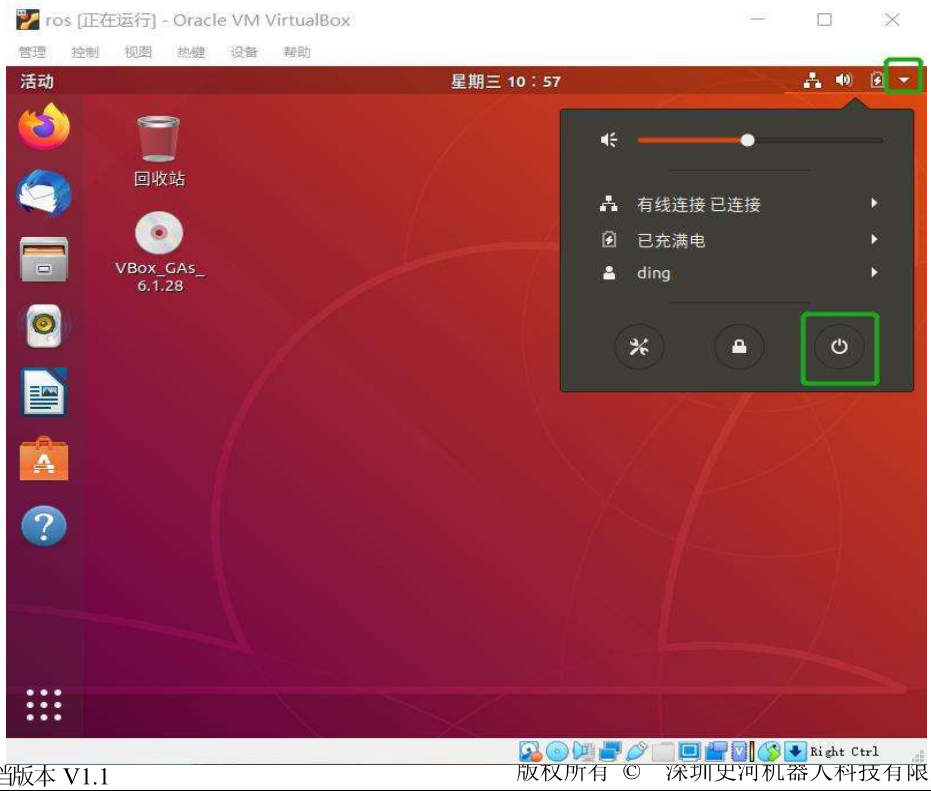
版权所有 © 深圳史河机器人科技有限公司

此时我们可以看到桌面上会多出一个光盘图标，如果没有，请尝试本步骤。按 CTRL+ALT+T 打开虚拟机终端，在弹出的页面中输入如下命令：

sudo apt-get install virtualbox-guest-dkms，然后按 Enter 键，输入自己一开始设置的密码（注：输入密码时屏幕不会有显示，直接输入密码，然后按 Enter 键即可）



再次重启安装好的 Ubuntu 系统，点击右上角倒三角图标，然后点击“电源”按钮，选择“重新启动”。

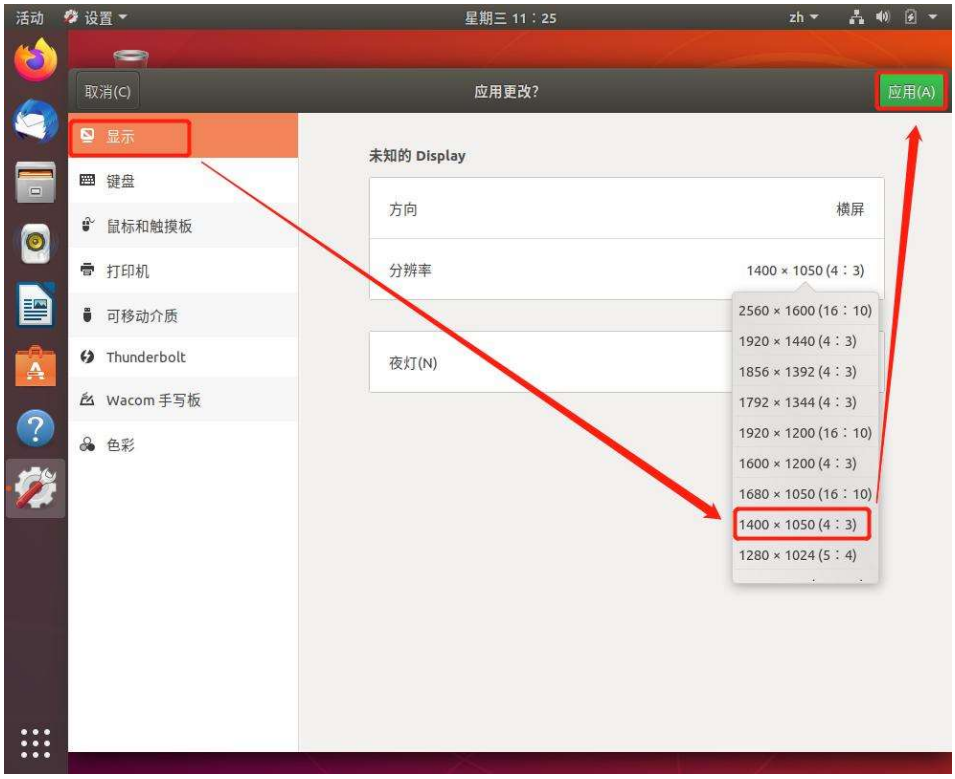


版权所有 © 深圳史河机器人科技有限公司

12）设置分辨率

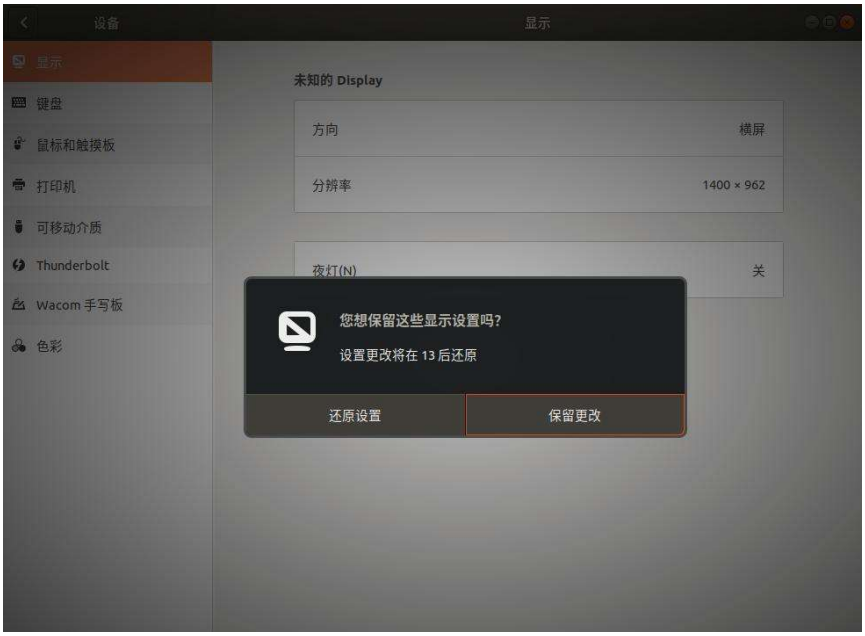
重启后，我们进入系统，选择左侧“显示应用程序”，在弹出的界面中选择“设置”

进入系统设置，选择“显示”栏目，设置合适的分辨率，然后点击“应用”。



在弹出的界面中，选择“保留更改”。

版权所有 © 深圳史河机器人科技有限公司



到此为止，使用 VirtualBox 虚拟机安装 Ubuntu 系统就已经成功了，接下来就是安装 ROS。

版权所有 © 深圳史河机器人科技有限公司

实验 2　ROS 安装与配置

一、实验目的

安装 ROS 来编写机器人软件程序

二、实验设备

Ubuntu 操作系统（务必确保已经连上网络）

三、实验步骤

3.1 Ubuntu 更换软件安装源

打开 Ubuntu 软件，点击软件和更新

然后将下载后面的源，选择其他站点，然后选择 aliyuan，最后点击“选择服务器”，然后输入密码。

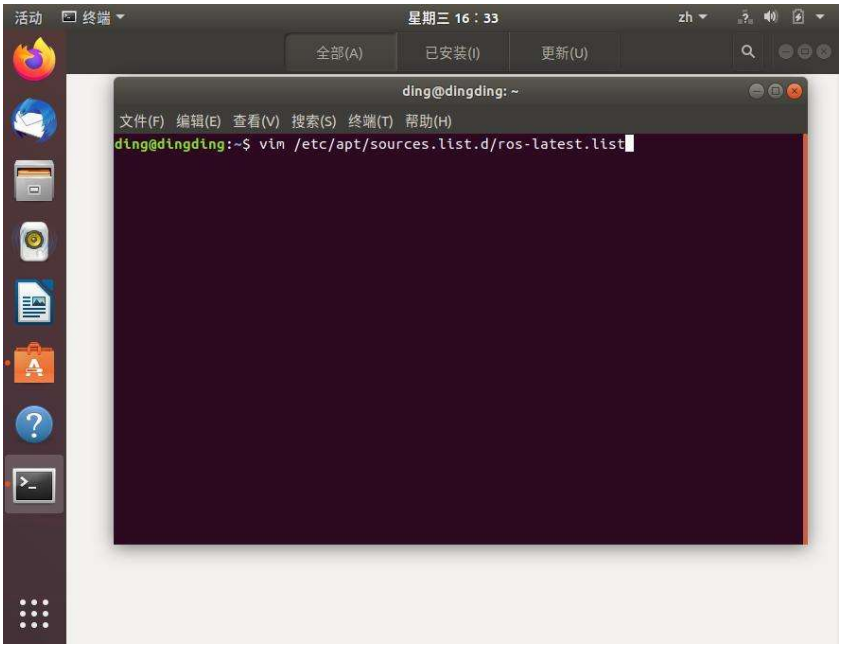
版权所有 ©　深圳史河机器人科技有限公司

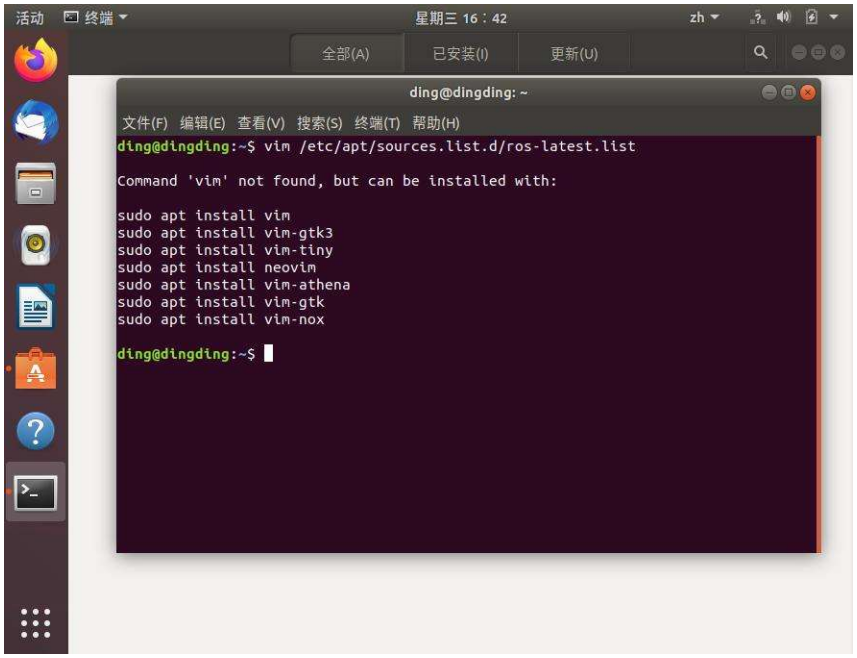
输入密码之后，点击关闭，在系统会提示可用软件的列表信息已过时，选择“重新载入”，更新缓存完毕后，软件安装源即更换成功。

版权所有 © 深圳史河机器人科技有限公司

3.2 配置 ROS 的安装源

按 CTRL+ALT+T 打开终端，新建一个/etc/apt/sources.list.d/ros-latest.list 文件（Ubuntu 的源存放在 /etc/apt/ 目录下的 sources.list 文件中）将清华源的服务地址（deb https://mirrors.tuna.tsinghua.edu.cn/ros/ubuntu/ bionic main）写在里面。如上操作我们使用 vim 编辑器进行编辑。

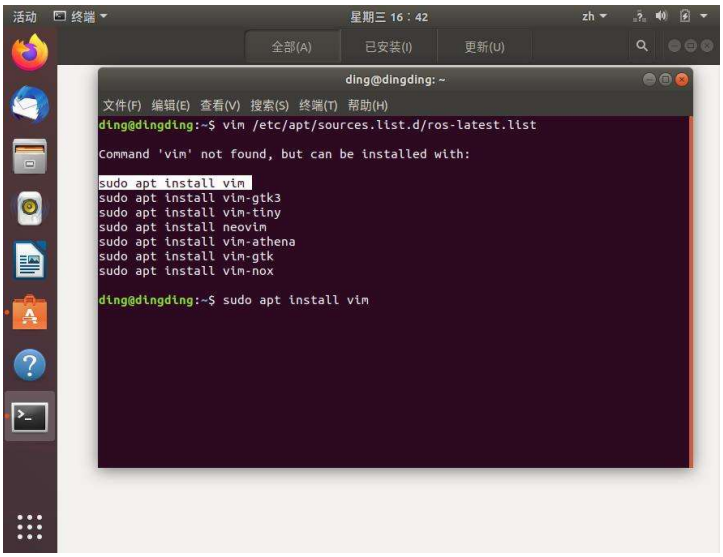




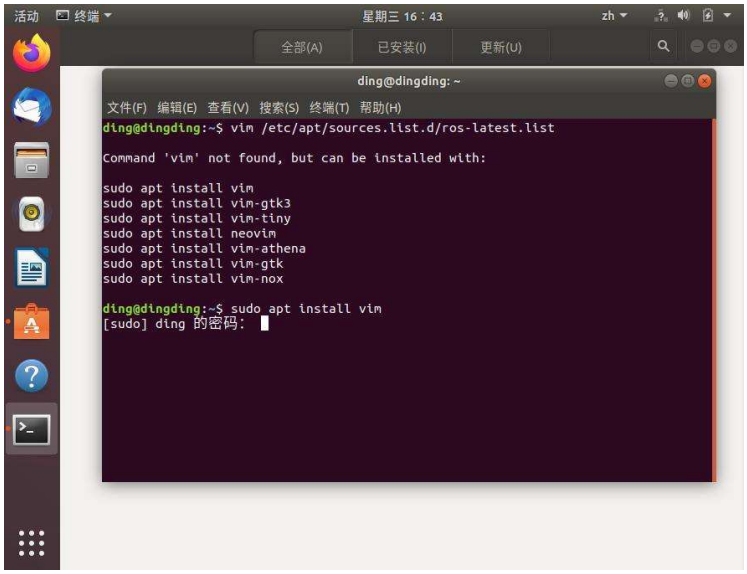
由于系统提示未安装 vim 编辑器，执行以下指令安装 vim 编辑器：

```
sudo apt install vim
```

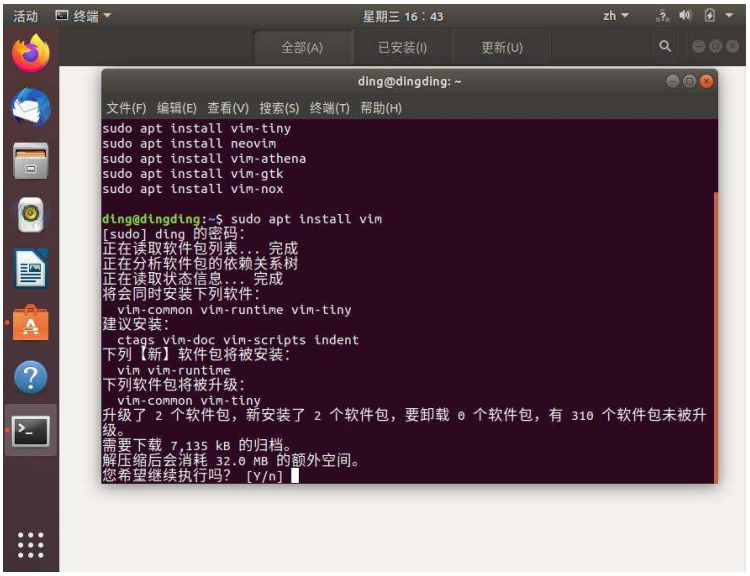

版权所有 © 深圳史河机器人科技有限公司



输入密码（输入密码时，屏幕上不会显示，输入完成之后，按 Enter 键即可）

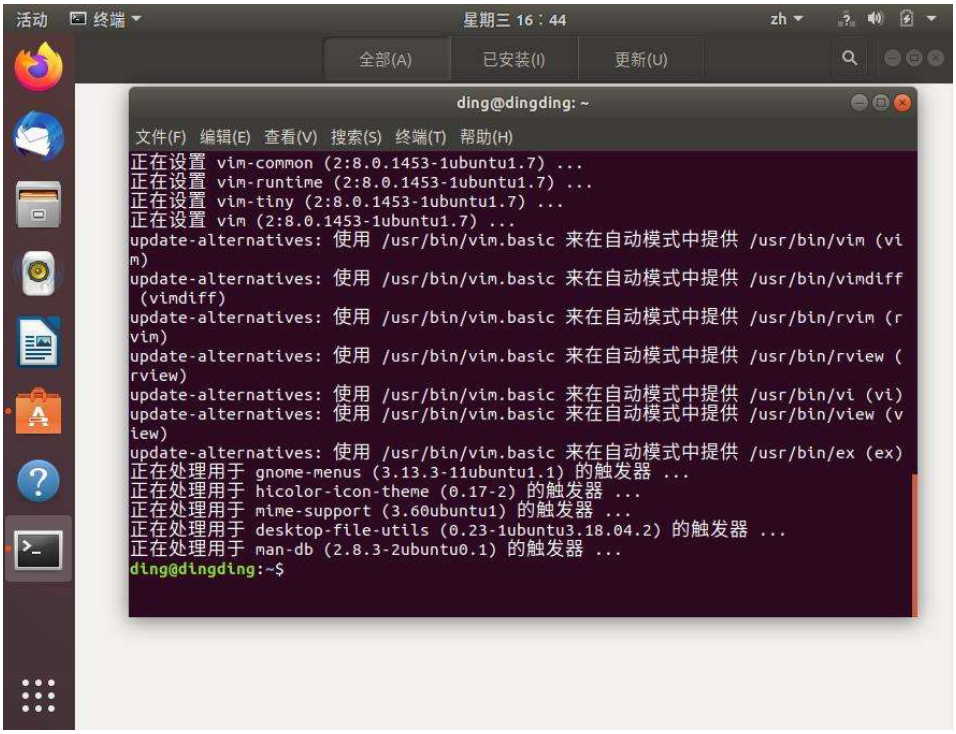


输入 y 之后，按 Enter 键执行命令



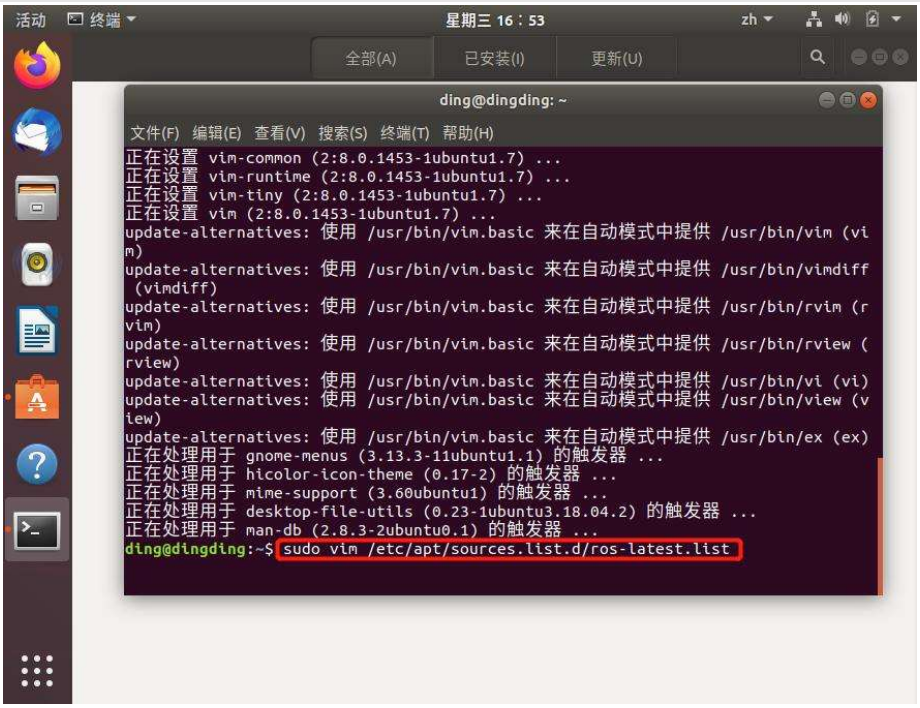
版权所有 © 深圳史河机器人科技有限公司

等待安装完成



安装完成之后，输入以下指令创建/etc/apt/sources.list.d/ros-latest.list 文件：

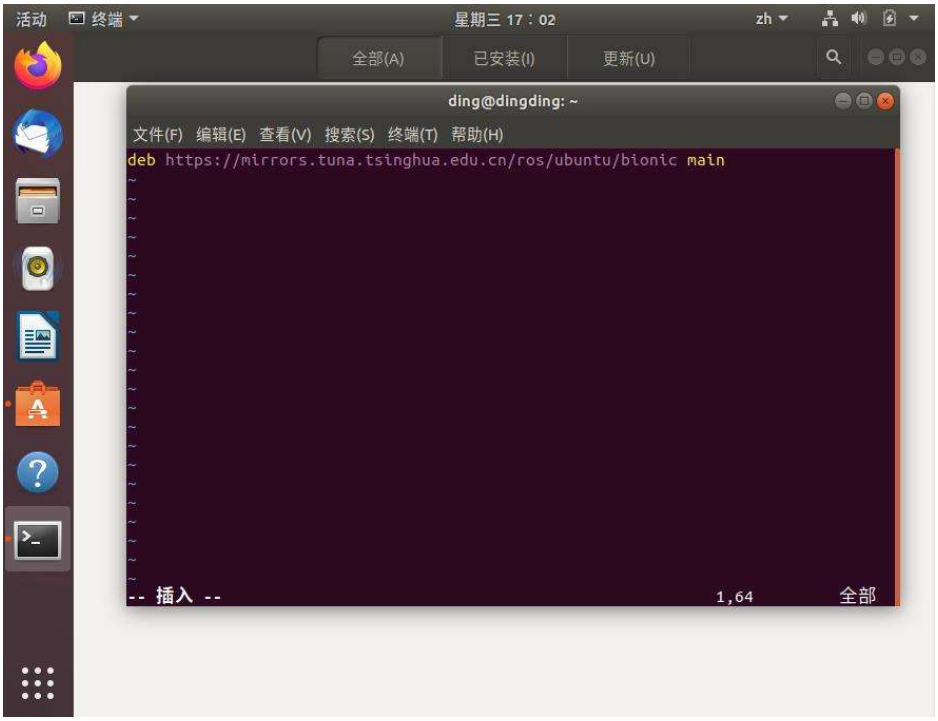
```
sudo vim /etc/apt/sources.list.d/ros latest.list
```



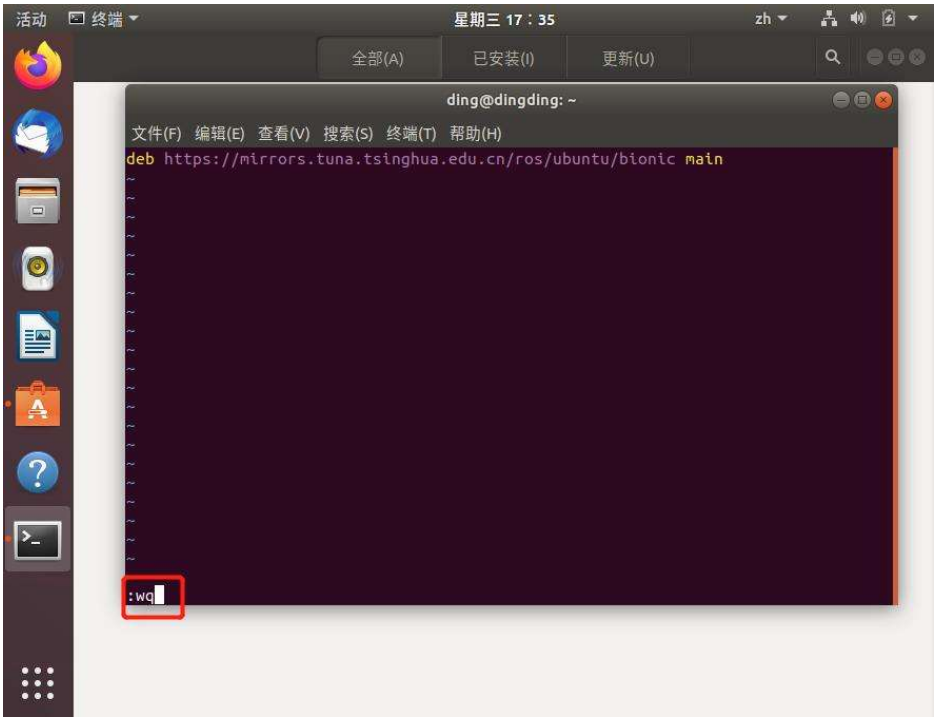
在弹出的界面，先按 i，然后将

deb https://mirrors.tuna.tsinghua.edu.cn/ros/ubuntu/ bionic main 保存进去。

版权所有 © 深圳史河机器人科技有限公司



然后按 ESC，进入命令模式，然后按下：（英文冒号）进入底线模式，输入 wq，按 Enter 键。



然后在命令栏输入以下指令，信任 ROS 的 GPR Key：

```
sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ sudo vim /etc/apt/sources.list.d/ros-latest.list
ding@dingding:~$ sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --
recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
Executing: /tmp/apt-key-gpghome.sLZp5eVa4D/gpg.1.sh --keyserver hkp://keyserver.
ubuntu.com:80 --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
gpg: 密钥 F42ED6FBAB17C654：公钥“Open Robotics <info@osrfoundation.org>”已导入
gpg: 合计被处理的数量：1
gpg:               已导入：1
ding@dingding:~$
```

然后再输入以下指令更新索引：

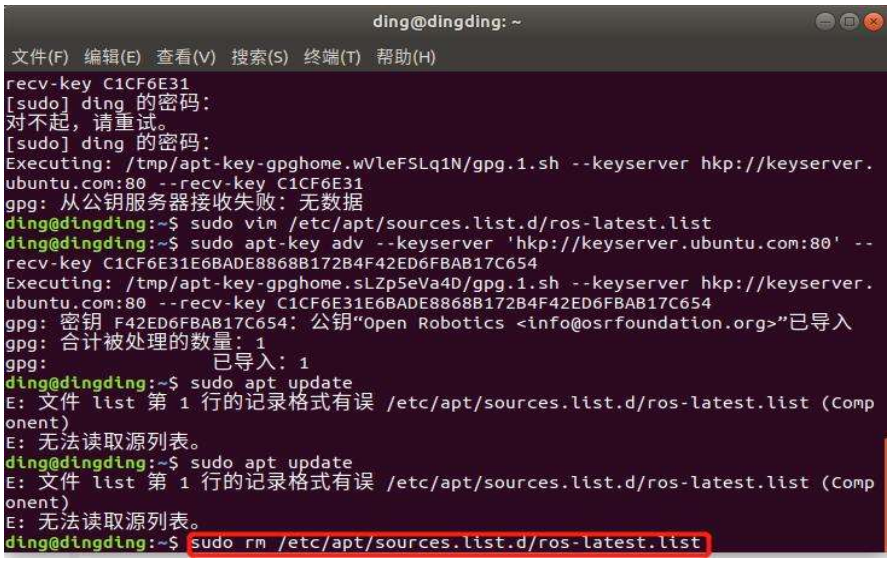
```
sudo apt update
```

```
ding@dingding: ~
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
update-alternatives: 使用 /usr/bin/vim.basic 来在自动模式中提供 /usr/bin/ex (ex)
正在处理用于 gnome-menus (3.13.3-11ubuntu1.1) 的触发器 ...
正在处理用于 hicolor-icon-theme (0.17-2) 的触发器 ...
正在处理用于 mime-support (3.60ubuntu1) 的触发器 ...
正在处理用于 desktop-file-utils (0.23-1ubuntu3.18.04.2) 的触发器 ...
正在处理用于 man-db (2.8.3-2ubuntu0.1) 的触发器 ...
ding@dingding:~$ sudo vim /etc/apt/sources.list.d/ros-latest.list
ding@dingding:~$ sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --
recv-key C1CF6E31
[sudo] ding 的密码：
对不起，请重试。
[sudo] ding 的密码：
Executing: /tmp/apt-key-gpghome.wVleFSLq1N/gpg.1.sh --keyserver hkp://keyserver.
ubuntu.com:80 --recv-key C1CF6E31
gpg: 从公钥服务器接收失败：无数据
ding@dingding:~$ sudo vim /etc/apt/sources.list.d/ros-latest.list
ding@dingding:~$ sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --
recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
Executing: /tmp/apt-key-gpghome.sLZp5eVa4D/gpg.1.sh --keyserver hkp://keyserver.
ubuntu.com:80 --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
gpg: 密钥 F42ED6FBAB17C654：公钥“Open Robotics <info@osrfoundation.org>”已导入
gpg: 合计被处理的数量：1
gpg:               已导入：1
ding@dingding:~$ sudo apt update
```

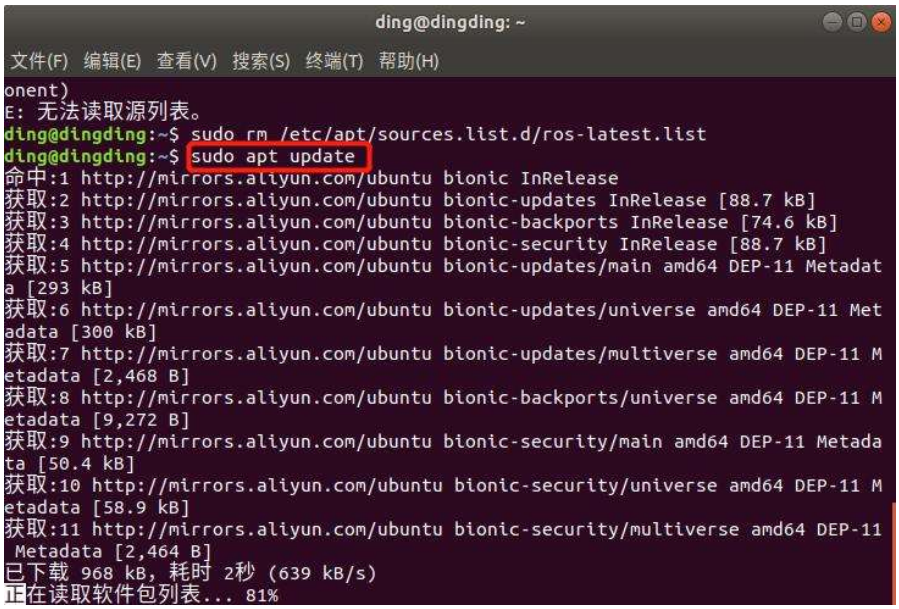
如出现以下错误：记录格式有误，则按下图操作即可解决问题。

```
ding@dingding: ~
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
recv-key C1CF6E31
[sudo] ding 的密码：
对不起，请重试。
[sudo] ding 的密码：
Executing: /tmp/apt-key-gpghome.wVleFSLq1N/gpg.1.sh --keyserver hkp://keyserver.
ubuntu.com:80 --recv-key C1CF6E31
gpg: 从公钥服务器接收失败：无数据
ding@dingding:~$ sudo vim /etc/apt/sources.list.d/ros-latest.list
ding@dingding:~$ sudo apt-key adv --keyserver 'hkp://keyserver.ubuntu.com:80' --
recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
Executing: /tmp/apt-key-gpghome.sLZp5eVa4D/gpg.1.sh --keyserver hkp://keyserver.
ubuntu.com:80 --recv-key C1CF6E31E6BADE8868B172B4F42ED6FBAB17C654
gpg: 密钥 F42ED6FBAB17C654：公钥“Open Robotics <info@osrfoundation.org>”已导入
gpg: 合计被处理的数量：1
gpg:               已导入：1
ding@dingding:~$ sudo apt update
E: 文件 list 第 1 行的记录格式有误 /etc/apt/sources.list.d/ros-latest.list (Comp
onent)
E: 无法读取源列表。
ding@dingding:~$ sudo apt update
E: 文件 list 第 1 行的记录格式有误 /etc/apt/sources.list.d/ros-latest.list (Comp
onent)
E: 无法读取源列表。
ding@dingding:~$
```


版权所有 © 深圳史河机器人科技有限公司



再次更新索引，问题解决。



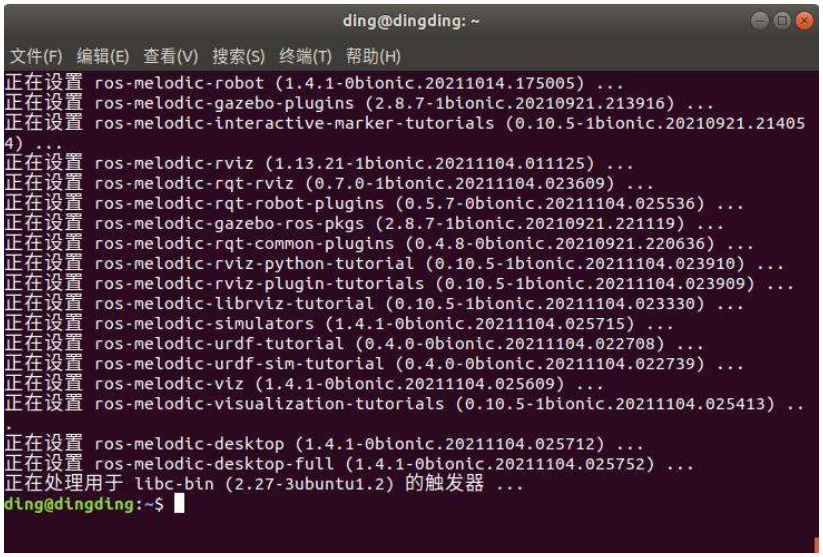
3.3 安装 ROS 桌面完整版

执行以下指令，安装桌面完整版：

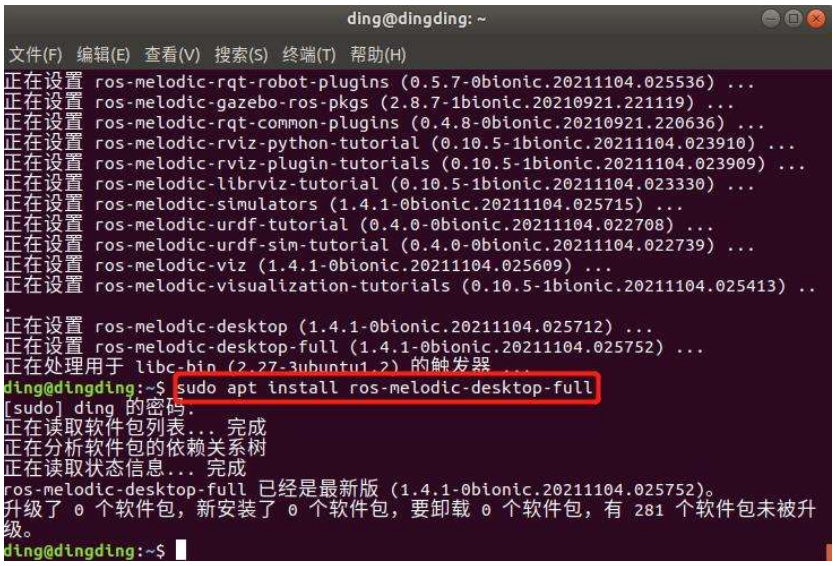
```
sudo_apt install ros-melodic-desktop-full
```

版权所有 © 深圳史河机器人科技有限公司

安装完成界面如下：



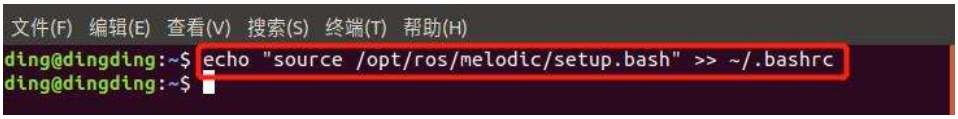
多执行此命令几次，检查是否真的完全安装完成，如下图所示，系统提示已经是最新版本，说明安装成功。



3.4 配置并激活环境变量

输入以下指令，添加环境变量：

```
echo "source /opt/ros/melodic/setup.bash" >> ~/.bashrc
```

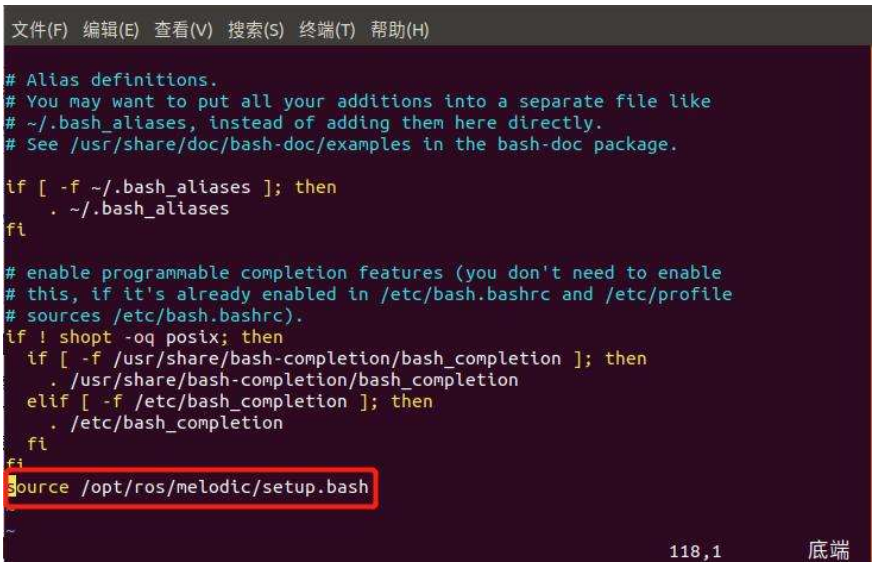


添加完成后我们利用 vim 查看一下这个 bashrc 文件，看看是否添加成功。

```
vim .bashrc
```

版权所有 © 深圳史河机器人科技有限公司

新添加的环境变量在文件的最后一行。可以看到，添加成功。



然后按：（英文冒号），输入 wq，退出 vim，回到终端指令输入界面，输入教程的第二条指令。

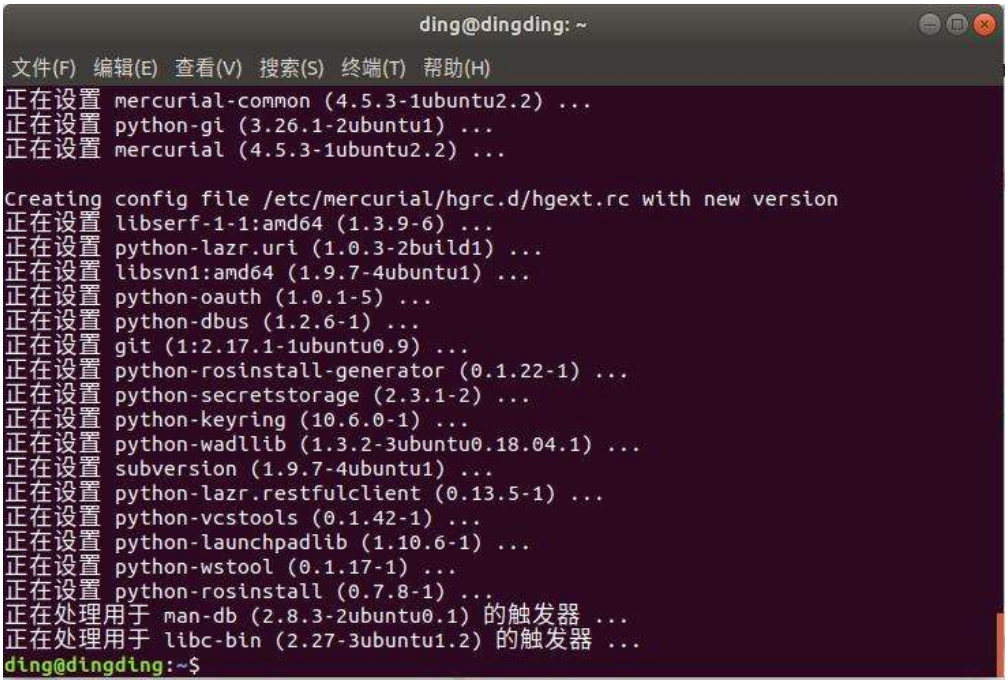
3.5 构建包依赖关系

到目前为止，你已经安装了运行核心 ROS 包所需的内容。为了创建和管理自己的 ROS 工作区，有各种各样的工具和需求分别分布。例如：rosinstall 是一个经常使用的命令行工具，它使你能够轻松地从一个命令下载许多 ROS 包的源树。要安装这个工具和其他构建 ROS 包的依赖项，请运行以下指令：

```
sudo apt-get install python-rosinstall python-rosinstall-generator python-wstool build-essential
```

版权所有 © 深圳史河机器人科技有限公司

安装成功。

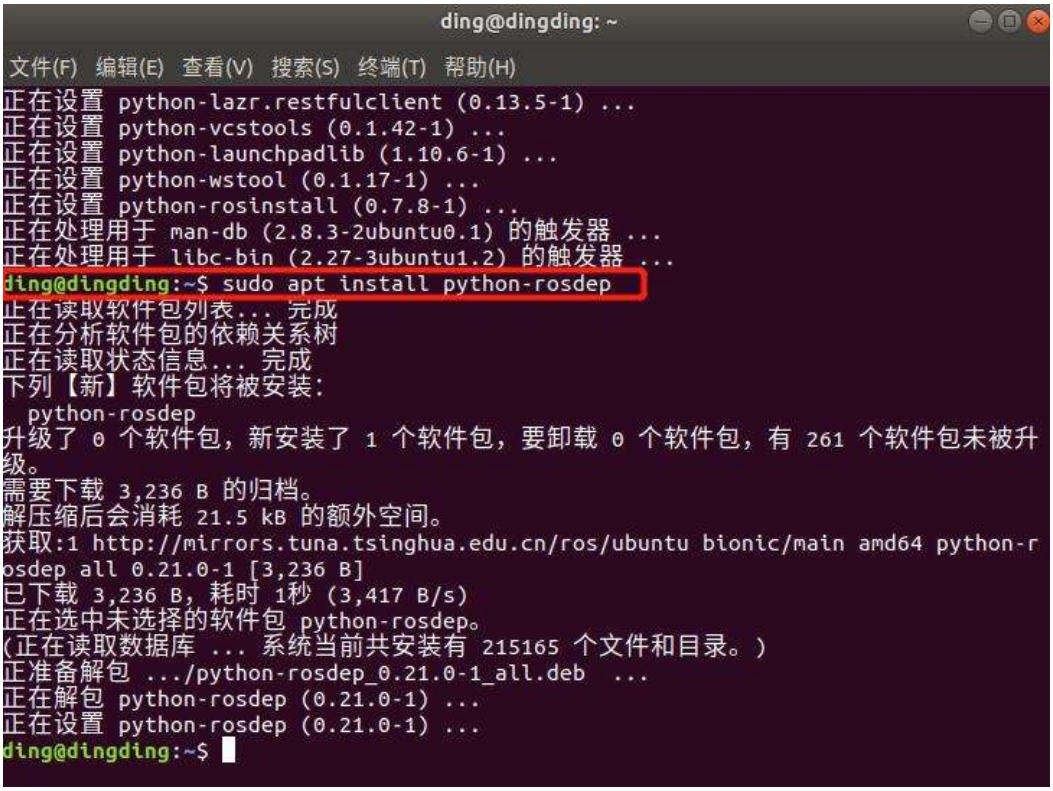


3.6 初始化 rosdep

在你使用 ROS 之前，需要初始化 rosdep。rosdep 让你能够轻松地安装被想要编译的源代码，或被某些 ROS 核心组件需要的系统依赖。

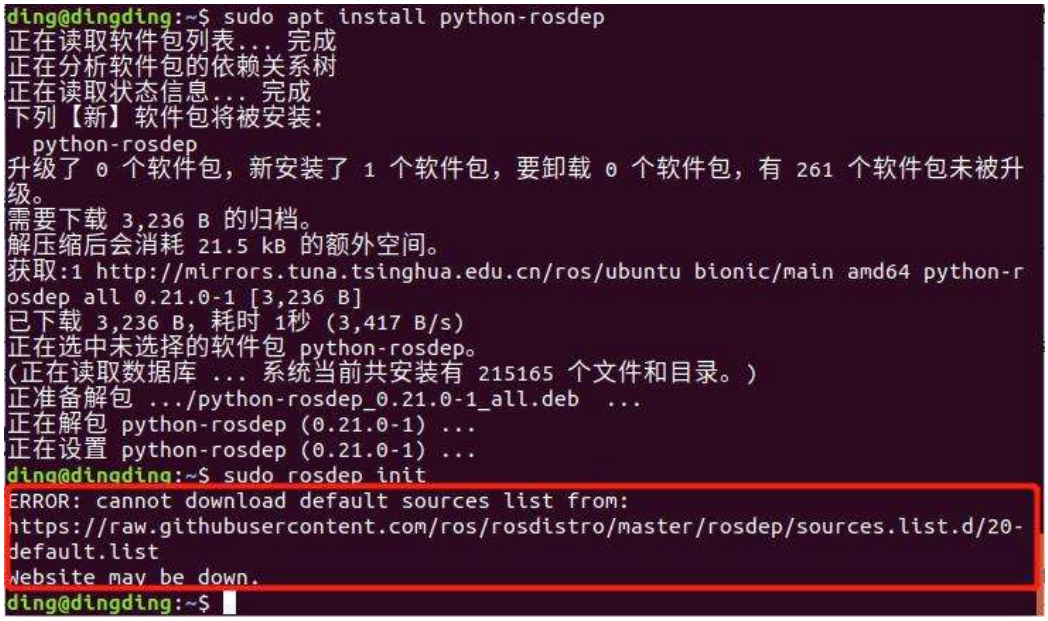
按 CTRL+ALT+T 打开终端，输入以下指令，完成初始化：

```
sudo apt install python-rosdep
```



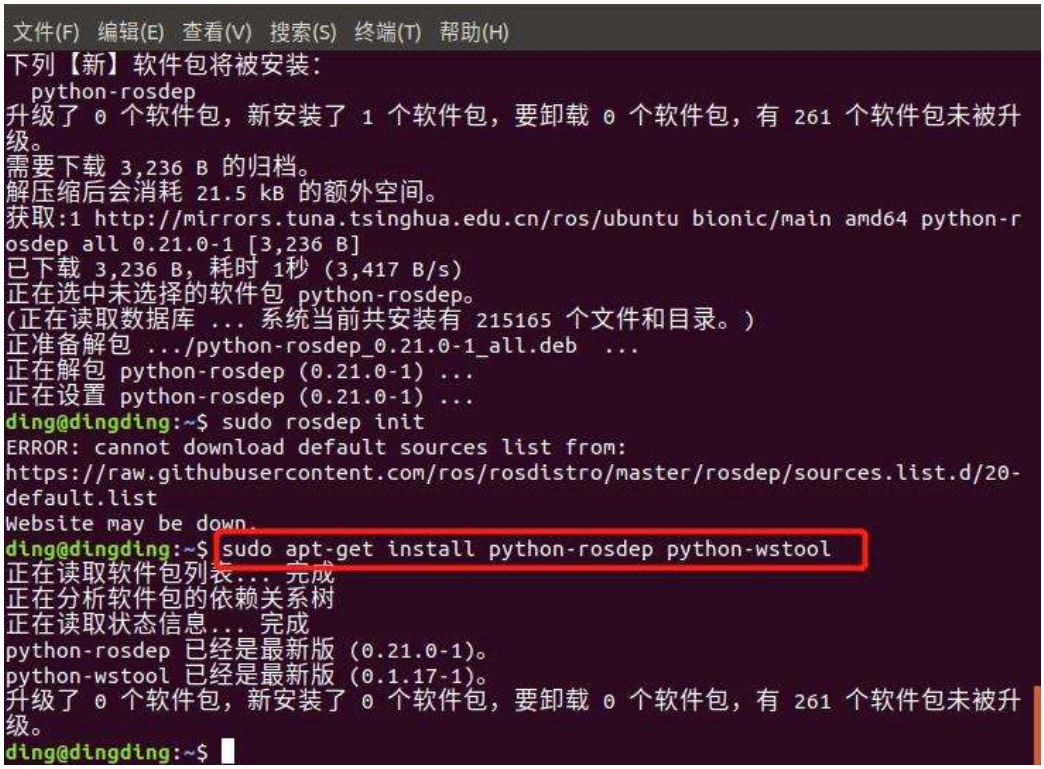
版权所有 © 深圳史河机器人科技有限公司

我们发现 rosdep 初始化失败，如下图所示



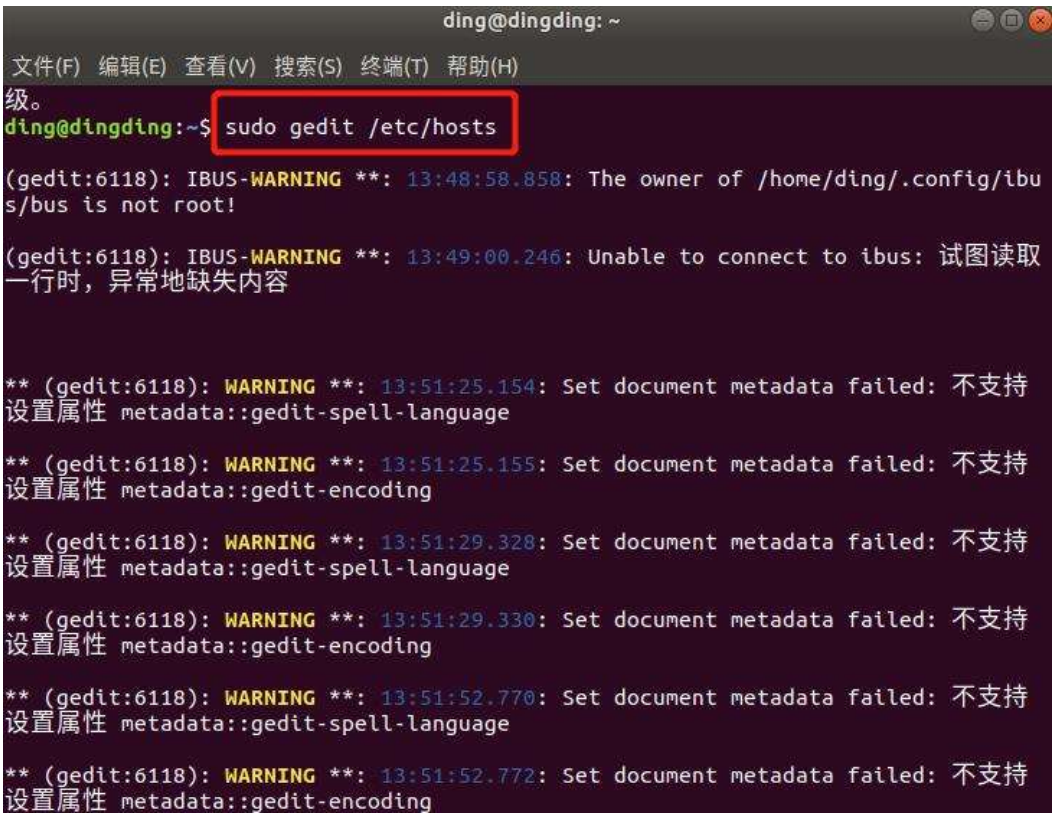
解决方法为。执行如下指令：

```
sudo apt-get install python-rosdep python-wstool
```



如果问题仍然无法解决，首先输入命令 `sudo gedit /etc/hosts` 打开 hosts 文件。不需要在意运行命令后出现的警告，不影响操作。

版权所有 © 深圳史河机器人科技有限公司



然后在文件末尾添加无法访问的网址(raw.githubusercontent.com)的IP地址及其域名即可。例如 185.199.108.133 raw.githubusercontent.com。

然后保存文件并退出，接着再次尝试再次执行 rosdep 初始化操作。

```
sudo rosdep init
```

成功之后再执行 rosdep update 更新操作，我们发现更新失败。

版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.y
aml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
ERROR: unable to process source [https://raw.githubusercontent.com/ros/rosdistro
/master/releases/fuerte.yaml]:
        Failed to download target platform data for gbpdistro:
        <urlopen error timed out>
Query rosdistro index https://raw.githubusercontent.com/ros/rosdistro/master/ind
ex-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Add distro "foxy"
ERROR: error loading sources list:
        <urlopen error <urlopen error timed out> (https://raw.githubusercontent.
com/ros/rosdistro/master/foxy/distribution.yaml)>
```

解决办法如下，然后保存退出，重新执行 rosdep update 指令。

```
# This file is managed by man:systemd-resolved(8). Do not edit.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "systemd-resolve --status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs must not access this file directly, but only through the
# symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a different way,
# replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0
search lan


nameserver 8.8.8.8
nameserver 8.8.4.4
```

```
ding@dingding: ~
文件(F)  编辑(E)  查看(V)  搜索(S)  终端(T)  帮助(H)
ding@dingding:~$ sudo gedit /etc/resolv.conf
[sudo] ding 的密码：

(gedit:6399): IBUS-WARNING **: 14:11:00.952: The owner of /home/ding/.config/ibu
s/bus is not root!

(gedit:6399): IBUS-WARNING **: 14:11:01.382: Unable to connect to ibus: 试图读取
一行时，异常地缺失内容

** (gedit:6399): WARNING **: 14:12:56.043: Set document metadata failed: 不支持
设置属性 metadata::gedit-spell-language

** (gedit:6399): WARNING **: 14:12:56.044: Set document metadata failed: 不支持
设置属性 metadata::gedit-encoding
```

然后我们发现又出现了新的问题

```
ding@dingding:~$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.y
aml
ERROR: error loading sources list:
        ('The read operation timed out',)
ding@dingding:~$
```

输入以下指令：

```
sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/sources_list.py
```

将 url="https://ghproxy.com/"+url 添加到下图所示位置

版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/sources_list.py
[sudo] ding 的密码:
```

```
def download_rosdep_data(url):
    """
    :raises: :exc:`DownloadFailure` If data cannot be
        retrieved (e.g. 404, bad YAML format, server down).
    """
    try:
        # http/https URLs need custom requests to specify the user-agent, since some repositories reject
        # requests from the default user-agent.
        url="https://ghproxy.com/"+url
        if url.startswith("http://") or url.startswith("https://"):
            url_request = request.Request(url, headers={'User-Agent': 'rosdep/{version}'.format(version=__version__)})
        else:
            url_request = url
        f = urlopen(url_request, timeout=DOWNLOAD_TIMEOUT)
        text = f.read()
        f.close()
        data = yaml.safe_load(text)
        if type(data) != dict:
            raise DownloadFailure('rosdep data from [%s] is not a YAML dictionary' % (url))
```

输入以下指令：

```
sudo gedit /usr/lib/python2.7/dist-packages/rosdistro/__init__.py
```

将原来的 DEFAULT_INDEX_URL 进行以下替换

```
ding@dingding:~$ sudo gedit  /usr/lib/python2.7/dist-packages/rosdistro/__init__.py
```

```
# same version as in:
# - setup.py
# - stdeb.cfg
__version__ = '0.8.3'

# index information

DEFAULT_INDEX_URL = 'https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml'


def get_index_url():
    # environment variable has precedence over configuration files
    if 'ROSDISTRO_INDEX_URL' in os.environ:
        return os.environ['ROSDISTRO_INDEX_URL']

    def read_cfg_index_url(fname):
        try:
            with open(fname) as f:
                return yaml.safe_load(f.read())['index_url']
        except (IOError, KeyError, yaml.YAMLError):
            return None
```

```
# same version as in:
# - setup.py
# - stdeb.cfg
__version__ = '0.8.3'

# index information

DEFAULT_INDEX_URL = 'https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro/master/index-v4.yaml'


def get_index_url():
    # environment variable has precedence over configuration files
    if 'ROSDISTRO_INDEX_URL' in os.environ:
        return os.environ['ROSDISTRO_INDEX_URL']

    def read_cfg_index_url(fname):
        try:
            with open(fname) as f:
                return yaml.safe_load(f.read())['index_url']
        except (IOError, KeyError, yaml.YAMLError):
            return None

    cfg_file = 'config.yaml'
```

输入以下指令：

版权所有 © 深圳史河机器人科技有限公司

sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/gbpdistro_support.py

将 https://ghproxy.com/添加到下图所示位置（36 行）

```
ding@dingding:~$ sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/gbpdistro_support.py
```

```
# py3k
try:
    unicode
except NameError:
    basestring = unicode = str

# location of an example gbpdistro file for reference and testing
FUERTE_GBPDISTRO_URL = 'https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro/' \
    'master/releases/fuerte.yaml'

# seconds to wait before aborting download of gbpdistro data
DOWNLOAD_TIMEOUT = 15.0
```

输入以下指令：

sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/sources_list.py

将 https://ghproxy.com/添加到下图所示位置（72 行）

```
ding@dingding:~$ sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/sources_list.py
```

```
# default file to download with 'init' command in order to bootstrap
# rosdep
DEFAULT_SOURCES_LIST_URL = 'https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/sources.list.d/20-default.list'

# seconds to wait before aborting download of rosdep data
DOWNLOAD_TIMEOUT = 15.0

SOURCES_LIST_DIR = 'sources.list.d'
SOURCES_CACHE_DIR = 'sources.cache'
```

输入以下指令：

sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/rep3.py

将 https://ghproxy.com/添加到下图所示位置（39 行）

```
ding@dingding:~$ sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/rep3.py
```

```
try:
    from urllib.request import urlopen
except ImportError:
    from urllib2 import urlopen
import yaml
import warnings

from .core import DownloadFailure
from .rosdistrohelper import PreRep137Warning

# location of targets file for processing gbpdistro files
REP3_TARGETS_URL = 'https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro/master/releases/targets.yaml'

# seconds to wait before aborting download of gbpdistro data
DOWNLOAD_TIMEOUT = 15.0
```

输入以下指令：

sudo gedit

版权所有 © 深圳史河机器人科技有限公司

/usr/lib/python2.7/dist-packages/rosdistro/manifest_provider/github.py

将 https://ghproxy.com/添加到下图所示位置（68、119 行）

```
ding@dingding:~$ sudo gedit /usr/lib/python2.7/dist-packages/rosdistro/manifest_provider/
github.py
```

```
    release_tag = repo.get_release_tag(pkg_name)

    if not repo.has_remote_tag(release_tag):
        raise RuntimeError('specified tag "%s" is not a git tag' % release_tag)

    url = 'https://ghproxy.com/https://raw.githubusercontent.com/%s/%s/package.xml' % (path, release_tag)
    try:
        logger.debug('Load package.xml file from url "%s"' % url)
        return _get_url_contents(url)
    except URLError as e:
        logger.debug('- failed (%s), trying "%s"' % (e, url))
        raise RuntimeError()


    package_xml_paths = list(filter(package_xml_in_parent, package_xml_paths))

    cache = SourceRepositoryCache.from_ref(tree_json['sha'])
    for package_xml_path in package_xml_paths:
        url = 'https://ghproxy.com/https://raw.githubusercontent.com/%s/%s/%s' % \
            (path, cache.ref(), package_xml_path + '/package.xml' if package_xml_path else 'package.xml')
        logger.debug('- load package.xml from %s' % url)
        package_xml = _get_url_contents(url)
        name = parse_package_string(package_xml).name
        cache.add(name, package_xml_path, package_xml)
```

输入以下指令：

sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/gbpdistro_support.py

将 gbpdistro_url = "https://ghproxy.com/" + gbpdistro_url 添加到下图所示位置（204 行）

```
ding@dingding:~$ sudo gedit /usr/lib/python2.7/dist-packages/rosdep2/gbpdistro_support.py
```

```
    # we can convert a gbpdistro file into rosdep data by following a
    # couple rules
    # will output a warning
    targets_data = download_targets_data(targets_url=targets_url)
    try:
        f = urlopen(gbpdistro_url, timeout=DOWNLOAD_TIMEOUT)
        text = f.read()
        f.close()
        gbpdistro_data = yaml.safe_load(text)
        # will output a warning
        gbpdistro_url = 'https://ghproxy.com/' + gbpdistro_url
        return gbprepo_to_rosdep_data(gbpdistro_data,
                                      targets_data,
                                      gbpdistro_url)
    except Exception as e:
        raise DownloadFailure('Failed to download target platform data '
                              'for gbpdistro:\n\t' + str(e))
```

然后再执行 rosdep update，即可更新成功

版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ rosdep update
reading in sources list data from /etc/ros/rosdep/sources.list.d
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/osx-homebrew.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/base.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/python.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/rosdep/ruby.yaml
Hit https://raw.githubusercontent.com/ros/rosdistro/master/releases/fuerte.yaml
Query rosdistro index https://ghproxy.com/https://raw.githubusercontent.com/ros/rosdistro
/master/index-v4.yaml
Skip end-of-life distro "ardent"
Skip end-of-life distro "bouncy"
Skip end-of-life distro "crystal"
Skip end-of-life distro "dashing"
Skip end-of-life distro "eloquent"
Add distro "foxy"
Add distro "galactic"
Skip end-of-life distro "groovy"
Skip end-of-life distro "hydro"
Skip end-of-life distro "indigo"
Skip end-of-life distro "jade"
Skip end-of-life distro "kinetic"
Skip end-of-life distro "lunar"
Add distro "melodic"
Add distro "noetic"
Add distro "rolling"
updated cache in /home/ding/.ros/rosdep/sources.cache
```

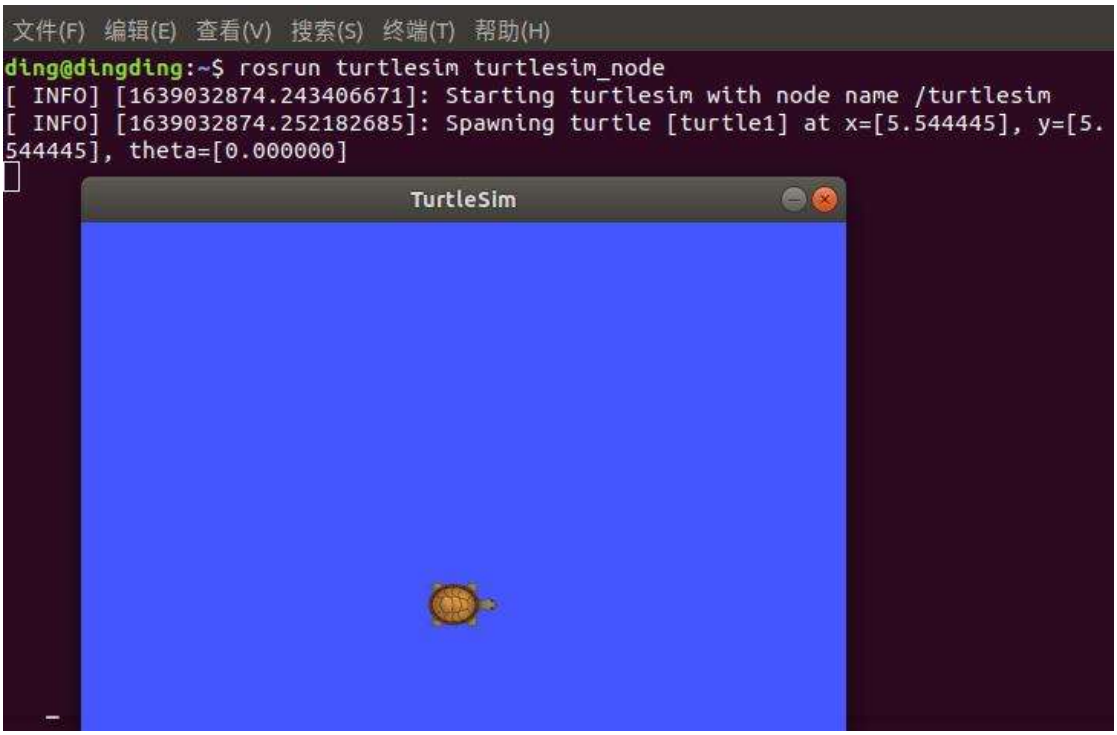
3.7 检查 ROS 是否安装成功

1）按住 Ctrl+Alt+T 打开一个新的终端，然后输入 roscore。

```
ding@dingding:~$ roscore
... logging to /home/ding/.ros/log/72518c2a-58bc-11ec-b7dc-080027226d4a/roslaunc
h-dingding-2310.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://dingding:41605/
ros_comm version 1.14.12


SUMMARY
========

PARAMETERS
 * /rosdistro: melodic
 * /rosversion: 1.14.12

NODES

auto-starting new master
process[master]: started with pid [2321]
ROS_MASTER_URI=http://dingding:11311/

setting /run_id to 72518c2a-58bc-11ec-b7dc-080027226d4a
process[rosout-1]: started with pid [2332]
started core service [/rosout]
```

2）按住 Ctrl+Alt+T 再打开一个新的终端，然后输入 rosrun turtlesim turtlesim_node，会弹出一个小海龟。

版权所有 © 深圳史河机器人科技有限公司



3）按住 Ctrl+Alt+T 再打开一个新的终端，然后输入 rosrun turtlesim turtle_teleop_key，按键盘上的上下左右箭头，控制小海龟移动。

如果上述命令都可以运行，证明 ROS 已经安装成功啦。

版权所有 © 深圳史河机器人科技有限公司

第二章 基础类实验

实验 3 ROS 文件系统导览

一、实验目的

1、对 ROS 文件系统概念有个大概的认识，

2、学会运用一些简单的文件系统工具，例如 rospack、roscd、rosls 等命令行工具。

二、实验设备

本实验中我们将会用到 ros-tutorials 程序包。如果你之前安装的 ROS 不是完整桌面版（Desktop-Full），请先安装 ros-tutorials 程序包。我们使用 apt 命令去安装：

```
sudo apt-get install ros-<distro>-ros-tutorials
```

将<distro>替换成你安装的 ROS 发行版简称（比如 kinetic 或 noetic 等）。从前面可以，我们安装的版本是 melodic，则对应的命令修改为如下：

```
sudo apt-get install ros-melodic-ros-tutorials
```

三、实验原理

3.1 ROS 文件系统概念介绍

1）软件包（Packages）：包是 ROS 代码的软件组织单元，每个软件包都可以包含程序库、可执行文件、脚本或其他构件。

2）Manifests（package.xml）：清单（Manifest）是对软件包的描述。它用于定义软件包之间的依赖关系，并记录有关软件包的元信息，如版本、维护者、许可证等。

3.2 文件系统工具

程序代码散落在许多 ROS 包中。使用 Linux 内置命令行工具（如 ls 和 cd）来进行查找和导航可能非常繁琐，因此 ROS 提供了专门的命令工具来简化这些操作。

版权所有 © 深圳史河机器人科技有限公司

四、实验步骤

4.1 使用 rospack

rospack 允许你获取软件包的有关信息。在本实验中，我们只涉及到 find 参数选项，该选项可以返回软件包的所在路径，具体用法如下：

```
rospack find [package_name]
```

比如我们要找一个名叫 roscpp 的功能包所在位置，可以通过以下指令来实现。

```
rospack find roscpp
```

如果你是在 Ubuntu 操作系统上通过 apt 安装的 ROS，你看到的应该是：

4.2 使用 roscd

roscd 是 rosbash 命令集的一部分，它允许你直接切换目录（cd）到某个软件包或者软件包集当中，具体用法如下：

```
roscd [功能包名称[/子目录] ]
```

比如切换到 roscpp 目录下：

```
roscd roscpp
```

比如切换到 roscpp 的子目录 msg 下：

```
roscd roscpp/msg
```

你可以看到使用 pwd 命令输出的工作目录 YOUR_INSTALL_PATH/share/roscpp 和之前使用 rospack find 输出的路径是一样的。

注：就像 ROS 中的其它工具一样，roscd 只能切换到那些路径已经包含在 ROS_PACKAGE_PATH 环境变量中的软件包。要查看 ROS_PACKAGE_PATH 中包含的路径，可以输入 echo ROS_PACKAGE_PATH

4.2.1 roscd log

roscd log 将带您进入存储 ROS 日志文件的目录。需要注意的是，如果你没有执行过任何 ROS 程序，系统会报错说该目录不存在。

版权所有 © 深圳史河机器人科技有限公司

如果你已经运行过 ROS 程序，那么可以尝试：roscd log

因为本机还未运行过 ROS 程序，所以没有结果。

```
ding@dingding:/opt/ros/melodic/share/roscpp/msg$ roscd log
No active roscore
ding@dingding:~/.ros/log$
```

4.3 使用 rosls

rosls 是 rosbash 命令集的一部分，它允许你直接按软件包的名称执行 ls 命令（而不必输入绝对路径）。rosls 的用法是：

rosls [功能包名称[/子目录]]

比如列出 roscpp 目录下的内容：

rosls roscpp

比如列出 roscpp 的子目录 msg 下的内容：

rosls roscpp/msg

```
ding@dingding:~/.ros/log$ rosls roscpp
cmake  msg  package.xml  rosbuild  srv
ding@dingding:~/.ros/log$ rosls roscpp/msg
Logger.msg
ding@dingding:~/.ros/log$ rosls roscpp/msg/Logger.msg
/opt/ros/melodic/share/roscpp/msg/Logger.msg
ding@dingding:~/.ros/log$
```

4.4 Tab 补全

每次都输入完整的软件包名称是比较繁琐。在之前的例子中，roscpp_tutorials 是个相当长的名称。而一些 ROS 工具支持 TAB 补全的功能。

当我们输入 roscd roscpp_tut<<<按 TAB 键>>>，在按下 TAB 键之后，命令栏会自动补完剩余部分。

```
ding@dingding:~/.ros/log$ roscd roscpp_tutorials/
```

当我们输入 roscd tur<<<按 TAB 键>>>，在按 TAB 键后，命令应该会尽可能地自动补充完整 roscd turtle。当再次按 TAB 键后会列出所有以 turtle 开头的 ROS 软件包：

```
ding@dingding:/opt/ros/melodic/share/roscpp_tutorials$ roscd turtle
turtle_actionlib/  turtlesim/         turtle_tf/         turtle_tf2/
ding@dingding:/opt/ros/melodic/share/roscpp_tutorials$ roscd turtle
```

现在输入 turtles 然后按 TAB 键：

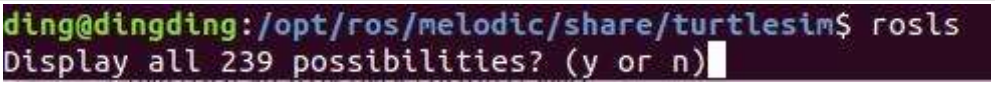
roscd turtles<<<按 TAB 键>>>，因为只有一个软件包的名称以 turtles 开头，所以你应该会看到：

```
ding@dingding:/opt/ros/melodic/share/roscpp_tutorials$ roscd turtlesim/
```


版权所有 © 深圳史河机器人科技有限公司

如果要查看当前安装的所有软件包的列表，你也可以利用 TAB 补全：

```
rosls <<<双击 TAB 键>>>
```



五、课堂作业

理解以下 ROS 命令工具的命名方式以及用法：

- rospack = ros + pack(age)
- oscd = ros + cd
- rosls = ros + ls

版权所有 © 深圳史河机器人科技有限公司

实验 4　创建与编译 ROS 软件包

一、实验目的

1、理解 catkin 软件包的组成成分；
2、学会如何使用 catkin 创建新的 ROS 软件包；
3、学会如何自定义你的软件包；
4、理解 rospack 的意义及用法。

二、实验原理

2.1 一个 catkin 软件包的组成成分

一个包要想称为 catkin 软件包，必须符合以下要求：
1）这个包必须有一个符合 catkin 规范的 package.xml 文件。
• 这个 package.xml 文件提供有关该软件包的元信息。
2）这个包必须有一个 catkin 版本的 CMakeLists.txt 文件。
• 如果它是 Catkin 元包，则需要有一个 CMakeList.txt 文件的相关样板。
3）每个包必须有自己的目录。
• 这意味着在同一个目录下不能有嵌套的或者多个软件包存在。
最简单的软件包看起来就像这样：

```
my_package/
  CMakeLists.txt
  package.xml
```

2.2 工作空间（workspace）的含义

1）src：代码空间（Source Space）
2）build：编译空间（Build Space）
3）devel：开发空间（Development Space）
4）install：安装空间（Install Space）

Catkin 工作空间是创建、修改、编译 catkin 软件包的目录。catkin 的工作空间，直观的形容就是一个仓库，里面装载着 ROS 的各种项目工程，便于系统组织管理调用。在可视化图形界面里是一个文件夹。我们自己写的 ROS 代码通常就放在工作空间中。

2.3 环境变量

环境变量的设置就是为了让计算机在执行命令的时候方便找到命令。换句话

版权所有 © 深圳史河机器人科技有限公司

说，计算机操作系统里面设置环境变量其实就是设置一定的文件路径，让计算机在执行命令的时候能方便地找到这条命令在哪里。

三、实验步骤

3.1 创建 catkin 工作空间

首先创建一个 catkin 工作空间

```
mkdir -p ～/catkin_ws/src     //创建了第二层级的文件夹 src，这是放 ROS 软件包的地方
cd ～/catkin_ws/              //回到工作空间,catkin_make 必须在工作空间下执行；
catkin_make                   //开始编译，调用系统自动完成编译和链接过程，构建生成目标文件
```

```
ding@dingding:~$ mkdir -p ~/catkin_ws/src
ding@dingding:~$ cd ~/catkin_ws/
ding@dingding:~/catkin_ws$ catkin_make
Base path: /home/ding/catkin_ws
Source space: /home/ding/catkin_ws/src
Build space: /home/ding/catkin_ws/build
Devel space: /home/ding/catkin_ws/devel
Install space: /home/ding/catkin_ws/install
Creating symlink "/home/ding/catkin_ws/src/CMakeLists.txt" pointing to "/opt/ros
/melodic/share/catkin/cmake/toplevel.cmake"
####
#### Running command: "cmake /home/ding/catkin_ws/src -DCATKIN_DEVEL_PREFIX=/hom
e/ding/catkin_ws/devel -DCMAKE_INSTALL_PREFIX=/home/ding/catkin_ws/install -G Un
ix Makefiles" in "/home/ding/catkin_ws/build"
####
-- The C compiler identification is GNU 7.5.0
-- The CXX compiler identification is GNU 7.5.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Using CATKIN_DEVEL_PREFIX: /home/ding/catkin_ws/devel
-- Using CMAKE_PREFIX_PATH: /opt/ros/melodic
```

创建成功之后会有三个文件夹：

build：主要是放置程序编译的一些中间文件

devel：这个文件夹存放一些可执行文件以及环境变量配置的脚本文件

src：主要存放程序代码、功能包。

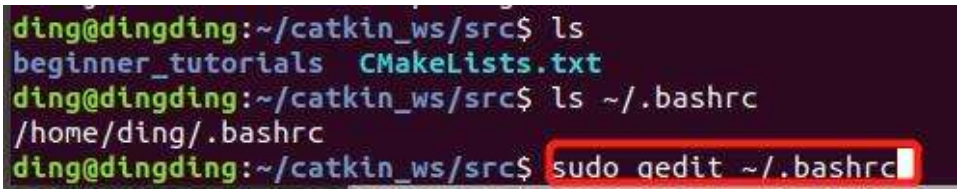
版权所有 © 深圳史河机器人科技有限公司

接着需要配置全局环境变量和 Catkin 工作空间的安装路径，在终端命令行运行：

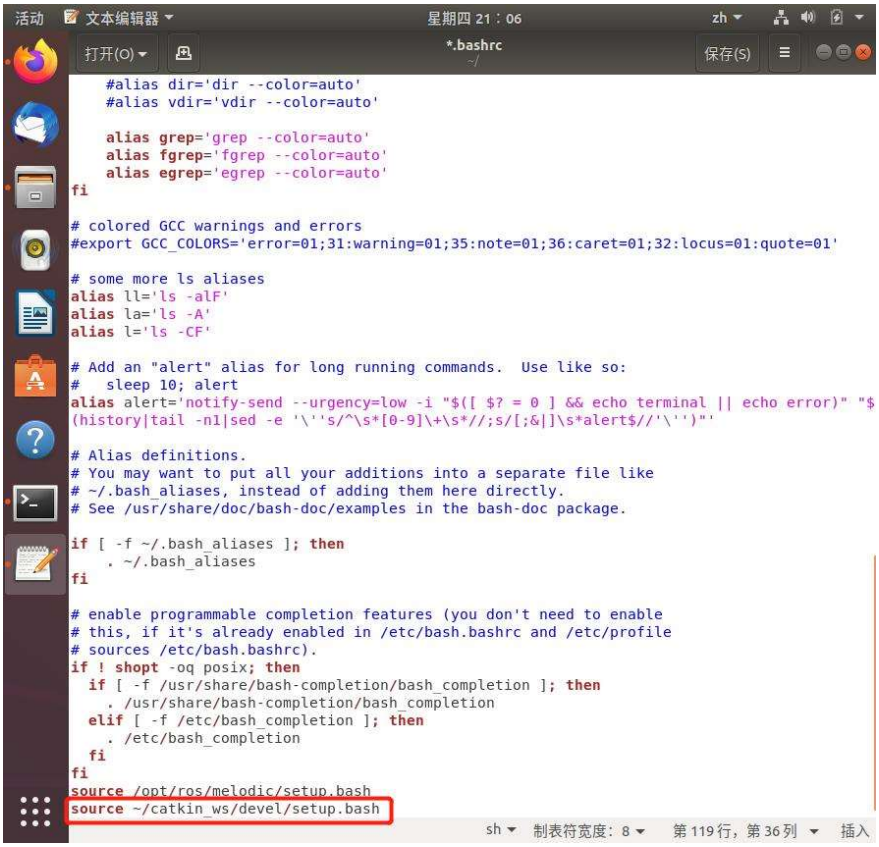
```
source ~/catkin_ws/devel/setup.bash
```

然而每次重新打开一个新的终端 shell，都需要执行上诉命令，比较麻烦。为此，我们可以将该命令加入脚本~/.bashrc 文件中，如此一来，每次打开新的终端，.bashrc 就会自动运行，执行命令如下：

```
sudo gedit ~/.bashrc
```



在弹出的文本编辑器中，将 source ~/catkin_ws/devel/setup.bash 加进去，然后保存退出。



版权所有 © 深圳史河机器人科技有限公司

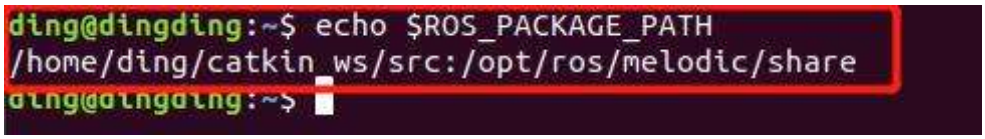
3.2 检查环境变量

要想保证工作空间已配置正确，需确保 ROS_PACKAGE_PATH 环境变量包含你的工作空间目录，采用以下命令查看：

```
echo ROS_PACKAGE_PATH
```

出现 /home/自己计算机的名字/catkin_ws/src:/opt/ros/melodic/share

这个语句中间有一个冒号，冒号前面是自己创建的工作空间的路径，那个/src 文件夹是专门存放源代码和功能包的。冒号后面是 ROS 本身工作空间下源代码和软件包所存放的文件夹的路径。

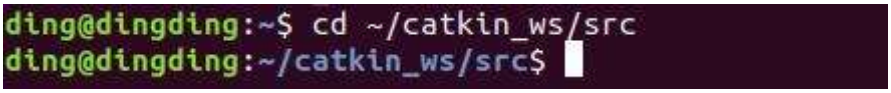


到此为止，你的工作环境已经搭建完成了，接下来就是在 catkin_ws 工作空间下的 src 目录下新建功能包，并进行功能包程序。

3.3 创建 catkin 软件包

首先切换到刚才创建的空白 catkin 工作空间中的源文件空间目录

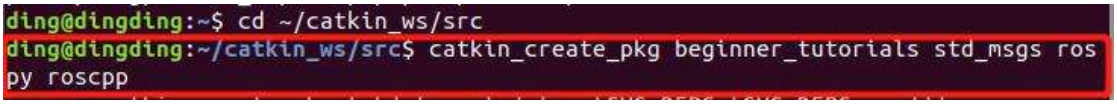
```
cd ~/catkin_ws/src
```



现在使用 catkin_create_pkg 命令创建一个名为 beginner_tutorials 的新软件包，这个软件包依赖于 std_msgs、roscpp 和 rospy：

注：catkin_create_pkg 的用法如下：

```
catkin_create_pkg <package_name> [depend1] [depend2] [depend3]
```



这将会创建一个名为 beginner_tutorials 的文件夹，这个文件夹里面包含一个 package.xml 文件和一个 CMakeLists.txt 文件，这两个文件都已经部分填写了你在执行 catkin_create_pkg 命令时提供的信息。

版权所有 © 深圳史河机器人科技有限公司

3.4 构建一个 catkin 工作区间并生效配置文件

首先在 catkin 工作区间中构建软件包：

```
ding@dingding:~/catkin_ws/src$  cd ~/catkin_ws
ding@dingding:~/catkin_ws$ catkin_make
Base path: /home/ding/catkin_ws
Source space: /home/ding/catkin_ws/src
Build space: /home/ding/catkin_ws/build
Devel space: /home/ding/catkin_ws/devel
Install space: /home/ding/catkin_ws/install
####
#### Running command: "cmake /home/ding/catkin_ws/src -DCATKIN_DEVEL_PREFIX=/hom
e/ding/catkin_ws/devel -DCMAKE_INSTALL_PREFIX=/home/ding/catkin_ws/install -G Un
ix Makefiles" in "/home/ding/catkin_ws/build"
####
```

然后将这个工作空间添加到 ROS 环境中，你需要 source 一下生成的配置文件：

```
. ~/catkin_ws/devel/setup.bash
```

```
ding@dingding:~/catkin_ws$ . ~/catkin_ws/devel/setup.bash
ding@dingding:~/catkin_ws$
```

3.5 软件包依赖关系

3.5.1 一级依赖

之前在使用 catkin_create_pkg 命令时提供了几个软件包作为依赖关系，现在我们可以使用 rospack 命令工具来查看这些一级依赖包。

```
rospack depends1 beginner_tutorials（depends1 中的 1 表示的就是 1 级依赖）
```

```
ding@dingding:~$ rospack depends1 beginner_tutorials
roscpp
rospy
std_msgs
ding@dingding:~$
```

如你所见，rospack 列出了在运行 catkin_create_pkg 命令时作为参数的依赖包，这些依赖关系存储在 package.xml 文件中。

输入以下指令，可以查看储存在 package.xml 文件中的依赖包

```
roscd beginner_tutorials
cat package.xml
```

```
ding@dingding:~$ roscd beginner_tutorials
ding@dingding:~/catkin_ws/src/beginner_tutorials$ cat package.xml
```


版权所有 © 深圳史河机器人科技有限公司

```
<buildtool_depend>catkin</buildtool_depend>
<build_depend>roscpp</build_depend>
<build_depend>rospy</build_depend>
<build_depend>std_msgs</build_depend>
<build_export_depend>roscpp</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>
<exec_depend>roscpp</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

如果在查看一级依赖时出现以下错误，解决方法如下：

```
ding@dingding:~/catkin_ws$ rospack depends1 beginner_tutorials
[rospack] Error: the rosdep view is empty: call 'sudo rosdep init' and 'rosdep update'
```

先找 rosdep 包的路径，执行：rospack find rosdep，如果显示：

[rospack]Error: package 'rosdep' not find,则没有安装 rosdep，执行安装命令：

```
sudo apt install rospack-tools
```

```
ding@dingding:~/catkin_ws$ rospack depends1 beginner_tutorials
[rospack] Error: the rosdep view is empty: call 'sudo rosdep init' and 'rosdep update'
ding@dingding:~/catkin_ws$ rospack find rosdep
[rospack] Error: package 'rosdep' not found
ding@dingding:~/catkin_ws$ sudo apt install rospack-tools
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
将会同时安装下列软件:
  librospack0d
下列【新】软件包将被安装:
  librospack0d rospack-tools
升级了 0 个软件包，新安装了 2 个软件包，要卸载 0 个软件包，有 97 个软件包未被升级。
需要下载 92.0 kB 的归档。
解压缩后会消耗 324 kB 的额外空间。
您希望继续执行吗? [Y/n] y
```

安装后执行：sudo rosdep init 如果出现以下错误

```
ding@dingding:~/catkin_ws$ sudo rosdep init
ERROR: default sources list file already exists:
        /etc/ros/rosdep/sources.list.d/20-default.list
Please delete if you wish to re-initialize
```

执行如下命令：

```
sudo rm /etc/ros/rosdep/sources.list.d/20-default.list
```

然后再重新执行命令：sudo rosdep init

```
ding@dingding:~/catkin_ws$ sudo rm /etc/ros/rosdep/sources.list.d/20-default.list
ding@dingding:~/catkin_ws$ sudo rosdep init
Wrote /etc/ros/rosdep/sources.list.d/20-default.list
Recommended: please run

        rosdep update
```

3.5.2 间接依赖

在很多情况下，一个依赖包还会有它自己的依赖关系，这种就叫间接依赖。

版权所有 © 深圳史河机器人科技有限公司

比如，rospy 就有其它依赖包。输入以下指令，可以查看 rospy 的依赖包。

```
rospack depends1 rospy
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$  rospack depends1 rospy
genpy
roscpp
rosgraph
rosgraph_msgs
roslib
std_msgs
```

一个软件包可以有相当多间接的依赖关系，使用 rospack 可以递归检测出所有嵌套的依赖包。

```
rospack depends beginner_tutorials
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$ rospack depends beginner_tutor
ials
cpp_common
rostime
roscpp_traits
roscpp_serialization
catkin
genmsg
genpy
message_runtime
gencpp
geneus
gennodejs
genlisp
message_generation
rosbuild
rosconsole
std_msgs
rosgraph_msgs
xmlrpcpp
roscpp
rosgraph
ros_environment
rospack
roslib
rospy
```

3.6 自定义软件包

本部分实验将解释 catkin_create_pkg 命令生成的每个文件，并详细描述这些文件的组成部分，以及如何自定义这些文件。

3.6.1 自定义 package.xml

自动生成的 package.xml 文件在你的新软件包中。现在一起来看看新生成的 package.xml 文件以及每一个需要关注的元素。

1）描述 description 标签

```
<package format="2">
  <name>beginner_tutorials</name>
  <version>0.0.0</version>
  <description>The beginner_tutorials package</description>
```

将描述信息修改为任何你喜欢的内容，但是按照惯例，第一句话应该简短一些，因为它包括了软件包的范围。

版权所有 © 深圳史河机器人科技有限公司

2）维护者 maintainer 标签

```
<!-- One maintainer tag required, multiple allowed, one person per tag -->
<!-- Example:  -->
<!-- <maintainer email="jane.doe@example.com">Jane Doe</maintainer> -->
<maintainer email="ding@todo.todo">ding</maintainer>
```

这是 package.xml 中要求填写的一个重要标签，因为它能够让其他人联系到软件包的相关人员。至少需要填写一个维护者，但如果你想的话可以添加多个。除了在标签里面填写维护者的名字外，还应该在标签的 email 属性中填写电子邮件地址。

3）许可证 license 标签

```
<!-- One license tag required, multiple allowed, one license per tag -->
<!-- Commonly used license strings: -->
<!--   BSD, MIT, Boost Software License, GPLv2, GPLv3, LGPLv2.1, LGPLv3 -->
<license>TODO</license>
```

选择种许可证并将它填写到这里。一些常见的开源许可协议有 BSD、MIT、Boost Software License、GPLv2、GPLv3、LGPLv2.1 和 LGPLv3。你可以在 Open Source Initiative 中阅读其中的若干个许可协议的相关信息。

4）依赖项 dependencies 标签

依赖被分为 build_depend、buildtool_depend、run_depend、test_depend。

• Build Tool Dependencies 指定该软件包需要的构建系统工具，以进行自我构建。一般来说，唯一需要的构建工具是 catkin。在交叉编译的情况下，编译工具的依赖性是针对编译所处的架构。

• Build Dependencies 指定哪些软件包需要用来构建这个包。当构建时需要这些包中的任何文件时，就会出现这种情况。这可以是在编译时包括这些包的头文件，针对这些包的库进行链接，或者在构建时需要任何其他资源（特别是当这些包在 CMake 中被 find_package()-ed 时）。在交叉编译的情况下，构建依赖是针对目标架构的。

• Run Dependencies 指定需要哪些包来运行此包中的代码，或针对此包构建库。当你依赖共享库或在此包中的公共头文件中过境包含它们的头文件时，就会出现这种情况（特别是当这些包在 CMake 的 catkin_package() 中被声明为 (CATKIN_)DEPENDS 时）。

• Test Dependencies 仅指定单元测试的额外依赖。他们不应该重复任何已经作为构建或运行依赖项提到的依赖项。

在之前的操作中，因为我们将 roscpp、rospy 和 std_msgs 作为 catkin_create_pkg 命令的参数，因此依赖关系如下所示：

版权所有 © 深圳史河机器人科技有限公司

```
  <!-- The *depend tags are used to specify dependencies -->
  <!-- Dependencies can be catkin packages or system dependencies -->
  <!-- Examples: -->
  <!-- Use depend as a shortcut for packages that are both build and exec dependencies -->
  <!--   <depend>roscpp</depend> -->
  <!--   Note that this is equivalent to the following: -->
  <!--   <build_depend>roscpp</build_depend> -->
  <!--   <exec_depend>roscpp</exec_depend> -->
  <!-- Use build_depend for packages you need at compile time: -->
  <!--   <build_depend>message_generation</build_depend> -->
  <!-- Use build_export_depend for packages you need in order to build against this
package: -->
  <!--   <build_export_depend>message_generation</build_export_depend> -->
  <!-- Use buildtool_depend for build tool packages: -->
  <!--   <buildtool_depend>catkin</buildtool_depend> -->
  <!-- Use exec_depend for packages you need at runtime: -->
  <!--   <exec_depend>message_runtime</exec_depend> -->
  <!-- Use test_depend for packages you need only for testing: -->
  <!--   <test_depend>gtest</test_depend> -->
  <!-- Use doc_depend for packages you need only for building documentation: -->
  <!--   <doc_depend>doxygen</doc_depend> -->
  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>
```

因为在编译和运行时都需要用到所有指定的依赖包，因此还要将每一个依赖包分别添加到 run_depend 标签中

```
<buildtool_depend>catkin</buildtool_depend>
<build_depend>roscpp</build_depend>
<build_depend>rospy</build_depend>
<build_depend>std_msgs</build_depend>
<build_export_depend>roscpp</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>
<exec_depend>roscpp</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>std_msgs</exec_depend>
```

5） 最终的 package.xml

现在看下面最后去掉了注释和未使用标签后的 package.xml 文件就显得更加简洁了：

```
<?xml version="1.0"?>
<package format="2">
  <name>beginner_tutorials</name>
  <version>0.0.0</version>
  <description>The beginner_tutorials package</description>

  <maintainer email="ding@todo.todo">ding</maintainer>

  <license>TODO</license>

  <buildtool_depend>catkin</buildtool_depend>
  <build_depend>roscpp</build_depend>
  <build_depend>rospy</build_depend>
  <build_depend>std_msgs</build_depend>
  <build_export_depend>roscpp</build_export_depend>
  <build_export_depend>rospy</build_export_depend>
  <build_export_depend>std_msgs</build_export_depend>
  <exec_depend>roscpp</exec_depend>
  <exec_depend>rospy</exec_depend>
  <exec_depend>std_msgs</exec_depend>

  <!-- The export tag contains other, unspecified, tags -->
  <export>
    <!-- Other tools can request additional information be placed here -->

  </export>
</package>
```


版权所有 © 深圳史河机器人科技有限公司

3.7 构建 ROS 软件包

按照之前的创建 ROS 软件包教程，你应该已经创建好了一个 catkin 工作空间和一个名为 beginner_tutorials 的 catkin 软件包。现在切换到 catkin 工作空间并查看 src 目录：

```
cd ~/catkin_ws/
ls src
```

```
ding@dingding:~$ cd ~/catkin_ws/
ding@dingding:~/catkin_ws$ ls src
beginner_tutorials  CMakeLists.txt
```

我们可以看到一个名为 beginner_tutorials 的目录，这就是之前用 catkin_create_pkg 命令创建的。现在我们可以使用 catkin_make 来构建它：

```
catkin_make
```

我们可以看到很多 cmake 和 make 的输出信息：

```
ding@dingding:~/catkin_ws$ catkin_make
Base path: /home/ding/catkin_ws
Source space: /home/ding/catkin_ws/src
Build space: /home/ding/catkin_ws/build
Devel space: /home/ding/catkin_ws/devel
Install space: /home/ding/catkin_ws/install
####
#### Running command: "make cmake_check_build_system" in "/home/ding/catkin_ws/build"
####
####
#### Running command: "make -j1 -l1" in "/home/ding/catkin_ws/build"
####
```

ls 可以查看 catkin 工作空间中生成的目录

```
ding@dingding:~/catkin_ws$ ls
build  devel  src
```

build 目录是构建空间的默认位置，同时 cmake 和 make 也是在这里被调用来配置和构建你的软件包。

devel 目录是开发空间的默认位置，在安装软件包之前，这里可以存放可执行文件和库。

到此为止，我们已经成功构建了一个 ROS 软件包。

四、课堂作业

1、熟悉创建 catkin 软件包的流程以及各种指令的含义和用法。

2、不参考实验指导书，尝试自己创建一个完整的 catkin 软件包。

版权所有 © 深圳史河机器人科技有限公司

实验 5　理解 ROS 节点

一、实验目的

1、理解 ROS 图的概念；

2、学会使用 roscore、rosnode、rosrun 等命令行工具

二、实验设备

轻量级模拟器。如果你之前安装的 ROS 不是完整桌面版（Desktop-Full），请先执行以下指令进行安装：

```
sudo apt-get install ros-melodic-ros-tutorials
```

三、实验原理

3.1 计算图

计算图（Computation Graph）是一个由 ROS 进程组成的点对点网络，它们能够共同处理数据。ROS 的基本计算图的概念有节点（Nodes）、主节点（Master）、参数服务器（Parameter Server）、消息（Messages）、服务（Services）、话题（Topics）和袋（Bags），它们都以不同的方式向图（Graph）提供数据。

节点（Nodes）：节点是一个可执行文件，它可以通过 ROS 来与其他节点进行通信。

消息（Messages）：订阅或发布话题时所使用的 ROS 数据类型。

话题（Topics）：节点可以将消息*发布*到话题，或通过*订阅*话题来接收消息。

主节点（Master）：ROS 的命名服务，例如帮助节点发现彼此。

rosout：在 ROS 中相当于 stdout/stderr（标准输出/标准错误）。

roscore：主节点 + rosout + 参数服务器。

3.2 节点

节点实际上只不过是 ROS 软件包中的一个可执行文件。ROS 节点使用 ROS 客户端库与其他节点通信。节点可以发布或订阅话题，也可以提供或使用服务。

举个例子来说，有一个机器人，和一个遥控器，那么这个机器人和遥控器开始工作后，就是两个节点。遥控器起到了下达指令的作用；机器人负责监听遥控器下达的指令，完成相应动作。从这里我们可以看出，节点是一个能执行特定工作任务的工作单元，并且能够相互通信，从而实现一个机器人系统整体的功能。在这里我们把遥控器和机器人简单定义为两个节点，实际上在机器人中根据控制

版权所有 © 深圳史河机器人科技有限公司

器、传感器、执行机构等不同组成模块，还可以将其进一步细分为更多的节点，这个是根据用户编写的程序来定义的。）

3.3 客户端库

ROS 客户端库可以让用不同编程语言编写的节点进行相互通信：

rospy = Python 客户端库

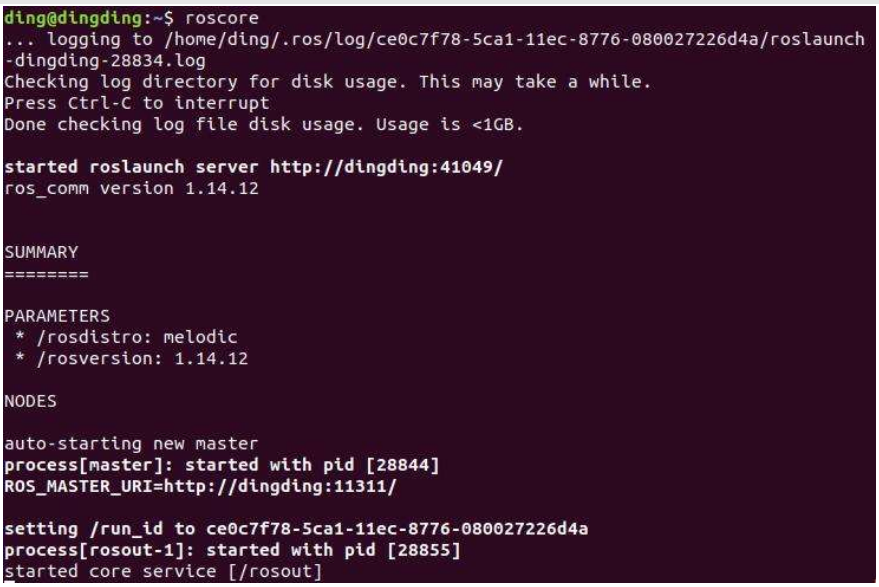
roscpp = C++客户端库

四、实验步骤

4.1 roscore

roscore 是你在运行所有 ROS 程序前首先要运行的命令。执行：

```
roscore
```



如果 roscore 运行后没有初始化，很有可能是网络配置的问题。

如果 roscore 不能初始化并提示缺少权限，可能是因为~/.ros 目录属于 root 用户（只有 root 用户才能访问），可以用以下命令递归地更改该目录的所有权：

```
sudo chown -R <your_username> ~/.ros
```

4.2 使用 rosnode

打开一个新终端，可以使用 rosnode 看看 roscore 运行时干了些什么。要确保以前的终端开着，比如打开一个新的标签页，或者最小化之前的窗口。

注：当打开一个新的终端时，环境将会重置，~/.bashrc 文件将会生效。如果你在运行 rosnode 等命令时出现一些问题，那么可能需要将一些环境设置文件添加到~/.bashrc 或手动 source 一下。具体方法请参考前文。

rosnode 显示当前正在运行的 ROS 节点信息。rosnode list 命令会列出这些活

版权所有 © 深圳史河机器人科技有限公司

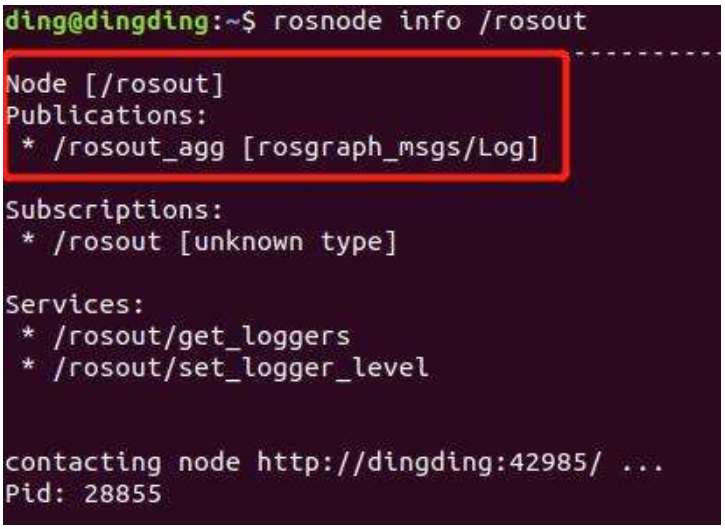
动的节点：

```
rosnode list
```

这表示当前只有一个节点在运行：rosout。因为这个节点用于收集和记录节点的调试输出，所以它总是在运行的。

而 rosnode info 命令返回的是某个指定节点的信息。

```
rosnode info /rosout
```



这给了更多关于 rosout 的信息，实际上它是发布了一个/rosout_agg 话题。

4.3 使用 rosrun

rosrun 可以让你用包名直接运行软件包内的节点（而不需要知道包的路径）。用法：

```
rosrun [package_name] [node_name]
```

打开一个新终端，执行以下指令：

```
rosrun turtlesim turtlesim_node
```

版权所有 © 深圳史河机器人科技有限公司

在一个新终端中，输入以下指令：

```
rosnode list
```

```
ding@dingding:~$ rosnode list
/rosout
/turtlesim
```

ROS 可以通过命令行重新分配名称。

关闭 turtlesim 窗口以停止节点（或回到 rosrun turtlesim 的终端并按 Ctrl+C）。现在让我们重新运行它，但是这一次使用重映射参数来改变节点名称：

```
rosrun turtlesim turtlesim_node __name: =my_turtle
```

```
ding@dingding:~$ rosrun turtlesim turtlesim_node __name:=my_turtle
[ INFO] [1639465654.393869203]: Starting turtlesim with node name /my_turtle
[ INFO] [1639465654.400576818]: Spawning turtle [turtle1] at x=[5.544445], y=[5.5
44445], theta=[0.000000]
```

```
ding@dingding:~$ rosnode list
/my_turtle
/rosout
/turtlesim
```

如果你仍看到/turtlesim 在列表中，这可能因为你是在终端中使用 Ctrl+C 停止的节点而不是关闭窗口，可以尝试清除下 rosnode 列表：

```
rosnode cleanup
```

```
ding@dingding:~$ rosnode cleanup
ERROR: connection refused to [http://dingding:33481/]
ERROR: connection refused to [http://dingding:41193/]
Unable to contact the following nodes:
 * /turtlesim
 * /my_turtle
Warning: these might include alive and functioning nodes, e.g. in unstable networ
ks.
Cleanup will purge all information about these nodes from the master.
Please type y or n to continue:
y
Unregistering /turtlesim
Unregistering /my_turtle
done
```

然后再执行上述指令，就会看到输出结果里面/rosout 和/my_turtle，没有了/turtlesim。

```
ding@dingding:~$ rosrun turtlesim turtlesim_node __name:=my_turtle
[ INFO] [1639466152.188397543]: Starting turtlesim with node name /my_turtle
[ INFO] [1639466152.197140874]: Spawning turtle [turtle1] at x=[5.544445], y=[5.5
44445], theta=[0.000000]
^C
ding@dingding:~$ rosnode list
/my_turtle
/rosout
```

看到新的/my_turtle 节点。使用另外一个 rosnode 指令：ping 来测试它是否正常：

```
rosnode ping my_turtle
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ rosnode list
/my_turtle
/rosout
ding@dingding:~$ rosnode ping my_turtle
rosnode: node is [/my_turtle]
pinging /my_turtle with a timeout of 3.0s
xmlrpc reply from http://dingding:39213/        time=0.743151ms
xmlrpc reply from http://dingding:39213/        time=0.725031ms
xmlrpc reply from http://dingding:39213/        time=0.607967ms
xmlrpc reply from http://dingding:39213/        time=0.571012ms
xmlrpc reply from http://dingding:39213/        time=0.663042ms
xmlrpc reply from http://dingding:39213/        time=0.585079ms
xmlrpc reply from http://dingding:39213/        time=0.640869ms
```

五、课堂作业

理解以下命令行工具的含义并且学会其使用方法：

roscore = ros + core 主节点（为 ROS 提供命名服务) + rosout (stdout/stderr) + 参数服务器

rosnode = ros + node：获取节点信息的 ROS 工具

rosrun = ros + run：运行给定的软件包中的节点

rosnode list：列出活动的节点

rosnode info /节点名称：返回某个指定节点的信息

版权所有 © 深圳史河机器人科技有限公司

实验 6　理解 ROS 话题

一、实验目的

1、理解 ROS 话题的作用；

2、掌握 rostopic 和 rqt_plot 的用途和具体用法。

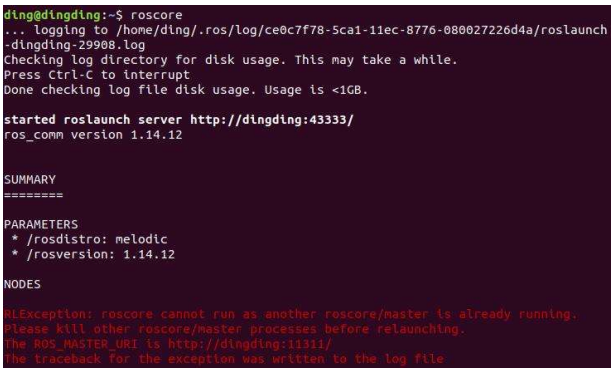
二、实验设备

2.1 roscore

首先确保 roscore 正在运行，打开一个新终端：

```
roscore
```

如果你没有退出在上一个实验中运行的 roscore，则可能会显示错误消息：



2.2 turtlesim

打开一个新的终端：

```
rosrun turtlesim turtlesim_node
```

我们还需要通过键盘来控制 turtle，打开一个新终端：

```
rosrun turtlesim turtle_teleop_key
```

现在就可以通过键盘的方向键来控制小海龟的运动了。

版权所有 ©　深圳史河机器人科技有限公司

三、实验原理

3.1 ROS 话题

turtlesim_node 节点和 turtle_teleop_key 节点之间是通过一个 ROS 话题来相互通信的。turtle_teleop_key 在话题上发布键盘按下的消息，turtlesim 则订阅该话题以接收消息。让我们使用 rqt_graph 来显示当前运行的节点和话题。

四、实验步骤

4.1 使用 rqt_graph

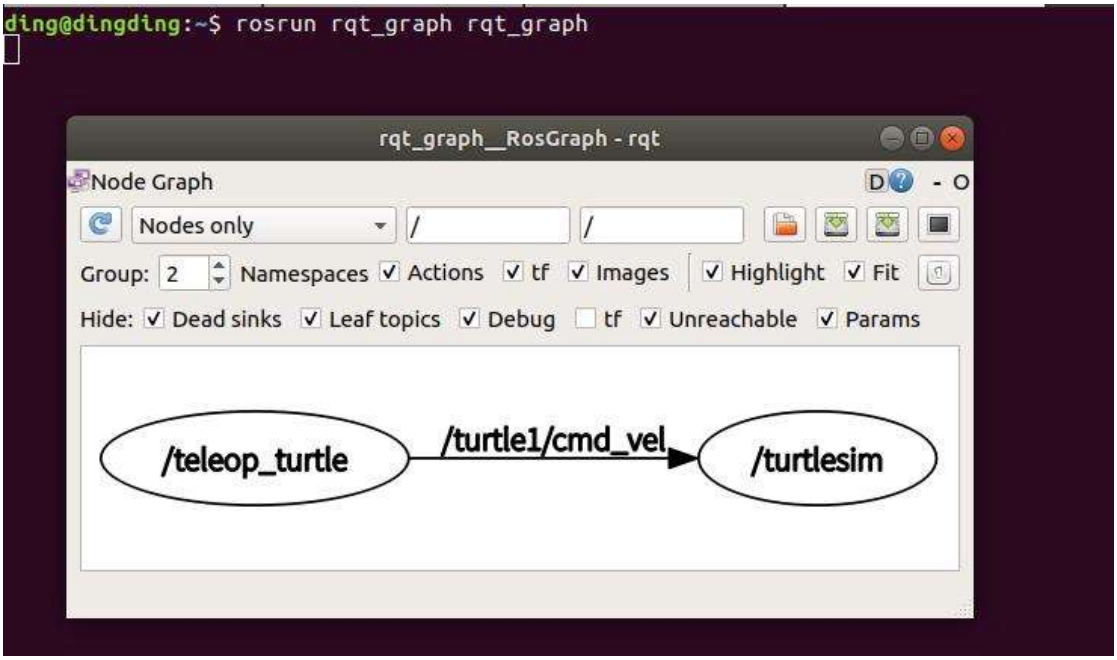
rqt_graph 用动态的图显示了系统中正在发生的事情。rqt_graph 是 rqt 程序包中的一部分。如果你发现没有安装，输入以下指令进行安装：

```
sudo apt-get install ros-<distro>-rqt
sudo apt-get install ros-<distro>-rqt-common-plugins
```

将<distro>替换成你安装的 ROS 发行版简称（kinetic 或 noetic 或 melodic 等）

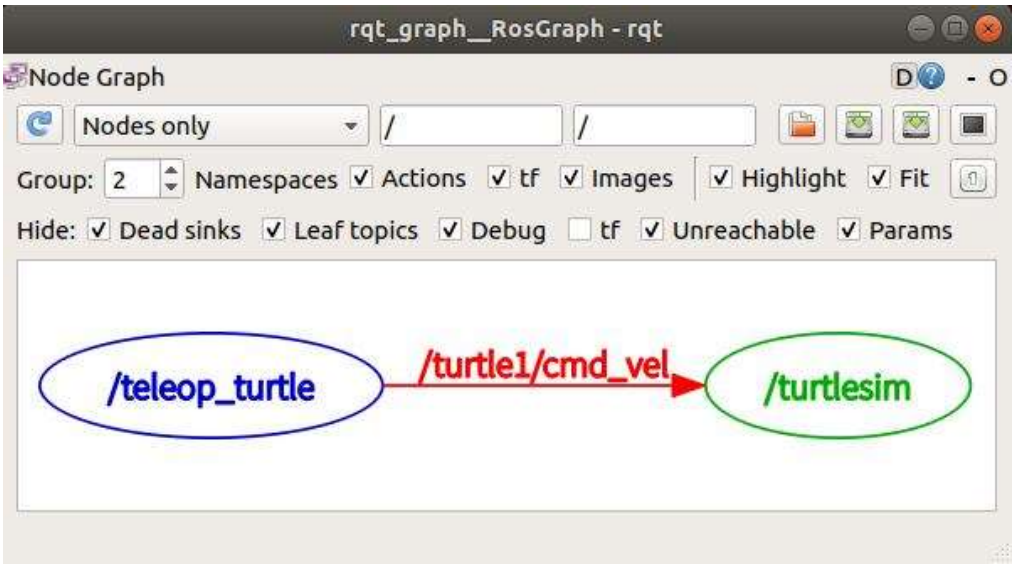
打开一个新终端：

```
rosrun rqt_graph rqt_graph
```



如果把鼠标放在/turtle1/command_velocity 上方，相应的 ROS 节点（这里是蓝色和绿色）和话题（这里是红色）就会高亮显示。可以看到，turtlesim_node 和 turtle_teleop_key 节点正通过一个名为/turtle1/command_velocity 的话题来相互通信。

版权所有 © 深圳史河机器人科技有限公司



4.2 rostopic 简介

rostopic 命令工具能让你获取 ROS 话题的信息。

你可以使用帮助选项查看可用的 rostopic 的子命令，例如：

```
rostopic -h
```

```
ding@dingding:~$ rostopic -h
rostopic is a command-line tool for printing information about ROS Topics.

Commands:
        rostopic bw     display bandwidth used by topic
        rostopic delay  display delay of topic from timestamp in header
        rostopic echo   print messages to screen
        rostopic find   find topics by type
        rostopic hz     display publishing rate of topic
        rostopic info   print information about active topic
        rostopic list   list active topics
        rostopic pub    publish data to topic
        rostopic type   print topic or field type

Type rostopic <command> -h for more detailed usage, e.g. 'rostopic echo -h'
```

或者在输入 rostopic 之后双击 Tab 键输出可能的子命令：

```
ding@dingding:~$ rostopic
bw    echo  find  hz    info  list  pub   type
```

4.3 使用 rostopic echo

rostopic echo 可以显示在某个话题上发布的数据，用法：

```
rostopic echo [topic]
```

打开一个新终端，输入以下指令，查看由 turtle_teleop_key 节点发布的“指令、速度”数据。

```
rostopic echo /turtle1/cmd_vel
```

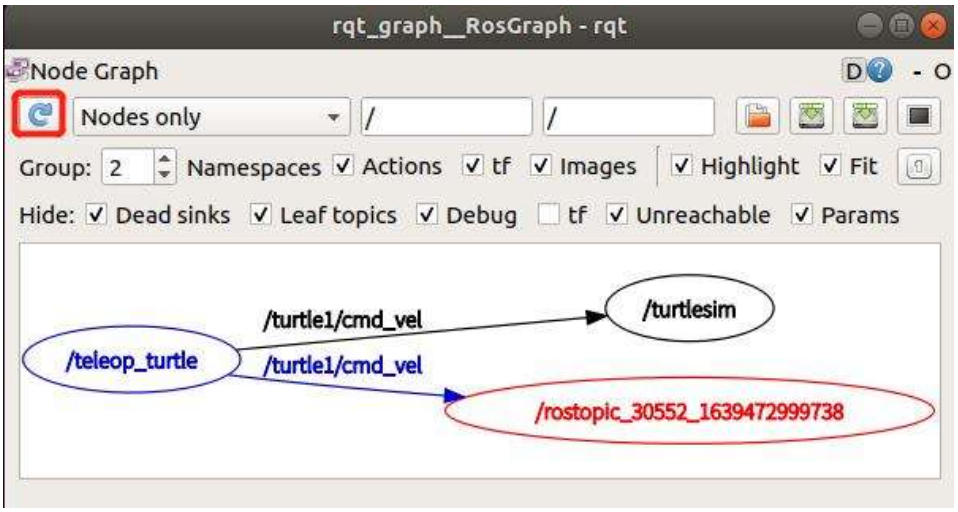

版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ rostopic echo /turtle1/cmd_vel
```

此时你会发现都没发生，因为现在还没有数据被发布到该话题上。可以通过按下键盘方向键让 turtle_teleop_key 节点发布数据。

```
ding@dingding:~$ rostopic echo /turtle1/cmd_vel
linear:
  x: -2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 2.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
```

此时我们再看 rqt_graph，点击左上角刷新按钮显示最新节点，我们发现 rostopic echo（这里红色显示）现在也订阅了 turtle1/command_velocity 话题。



4.4 使用 rostopic list

rostopic list 能够列出当前已被订阅和发布的所有话题。我们查可以查看一下 list 子命令需要的参数。打开一个新终端：

```
rostopic list -h
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ rostopic list -h
Usage: rostopic list [/namespace]

Options:
  -h, --help            show this help message and exit
  -b BAGFILE, --bag=BAGFILE
                        list topics in .bag file
  -v, --verbose         list full details about each topic
  -p                    list only publishers
  -s                    list only subscribers
  --host                group by host name
```

在 rostopic list 中使用 verbose 选项

```
rostopic list -v
```

会列出所有发布和订阅的主题及其类型的详细信息。

```
ding@dingding:~$ rostopic list -v

Published topics:
 * /turtle1/color_sensor [turtlesim/Color] 2 publishers
 * /turtle1/cmd_vel [geometry_msgs/Twist] 1 publisher
 * /rosout [rosgraph_msgs/Log] 5 publishers
 * /rosout_agg [rosgraph_msgs/Log] 1 publisher
 * /turtle1/pose [turtlesim/Pose] 2 publishers

Subscribed topics:
 * /turtle1/cmd_vel [geometry_msgs/Twist] 3 subscribers
 * /rosout [rosgraph_msgs/Log] 1 subscriber
 * /statistics [rosgraph_msgs/TopicStatistics] 1 subscriber
```

4.5 ROS 消息

话题的通信是通过节点间发送 ROS 消息实现的。为了让发布者（turtle_teleop_key）和订阅者（turtulesim_node）进行通信，发布者和订阅者必须发送和接收相同类型的消息。这意味着话题的类型是由发布在它上面消息的类型决定的。使用 rostopic type 命令可以查看发布在话题上的消息的类型。

4.6 使用 rostopic type

rostopic type 命令用来查看所发布话题的消息类型。用法：

```
rostopic type [topic]
```

运行以下指令：

```
rostopic type /turtle1/cmd_vel          //查看发布话题的消息类型
rosmsg show geometry_msgs/Twist   //使用 rosmsg 查看消息的详细信息
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ rostopic type /turtle1/cmd_vel
geometry_msgs/Twist
ding@dingding:~$ rosmsg show geometry_msgs/Twist
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

4.7 深入学习 rostopic

4.7.1 使用 rostopic pub

rostopic pub 可以把数据发布到当前某个正在广播的话题上，用法：

rostopic pub [topic] [msg_type] [args]

以下指令会发送一条信息给 turtlesim，让它以 2.0 大小的线速度和 1.8 大小的角速度移动。

rostopic pub -1 /turtle1/cmd_vel geometry_msgs/Twist -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'

代码解析：

rostopic pub　　//将信息发布到指定的话题

-1　　//让 rostopic 只发布一条消息，然后退出

/turtle1/cmd_vel　　//要发布到的话题的名称

geometry_msgs/Twist　　//发布到话题时要使用的消息的类型

——　　//两个破折号用来告诉选项解析器，表明之后的参数都不是选项。如果参数前有破折号（-）比如负数，那么这是必需的

版权所有 © 深圳史河机器人科技有限公司

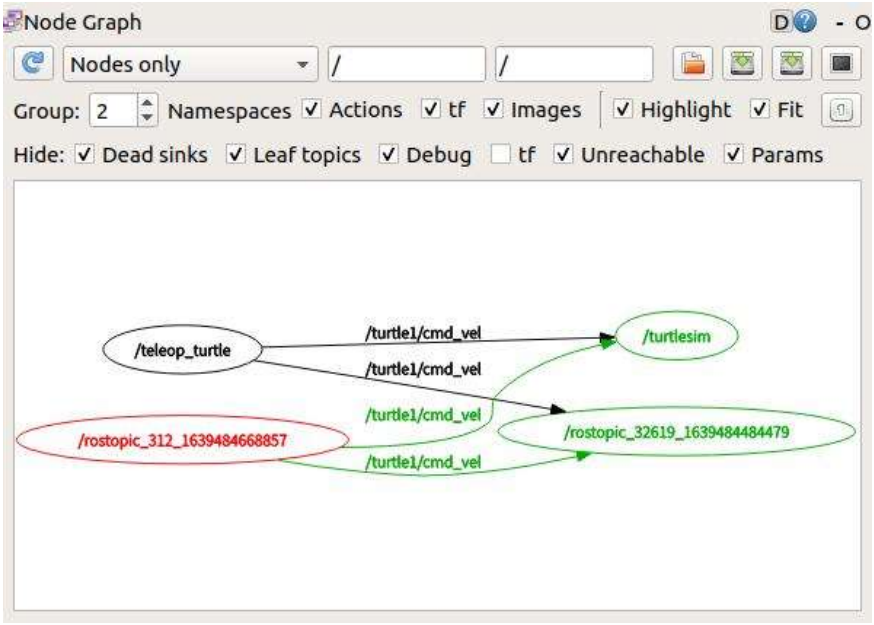
'[2.0, 0.0, 0.0]' '[0.0, 0.0, 1.8]'　　//一个 turtlesim/Velocity 消息有两个浮点型元素：linear 和 angular。在本例中，'[2.0，0.0，0.0]'表示 linear 的值为 x=2.0, y=0.0，z=0.0，而'[0.0，0.0，1.8]'是说 angular 的值为 x=0.0，y=0.0，z=1.8

我们发现，turtle 在移动一段距离之后就停止移动。这是因为 turtle 需要一个稳定的频率为 1Hz 的指令流才能保持移动状态。我们可以使用 rostopic pub -r 命令来发布源源不断的命令：

rostopic pub /turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, -1.8]'

它将以 1 Hz 的速度发布 velocity 指令到 velocity 话题上。

通过 rqt_graph 可以看到 rostopic pub 节点（此处为红色）正在与 rostopic echo 节点（此处为绿色）进行通信



可以看到，turtle 正沿着一个圆形轨迹持续运动。我们可以在新终端中通过 rostopic echo 命令来查看 turtlesim 所发布的数据：

版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~$ rostopic echo /turtle1/pose
x: 8.76436710358
y: 7.45492362976
theta: -1.31699335575
linear_velocity: 2.0
angular_velocity: -1.79999995232
---
x: 8.77150630951
y: 7.42373037338
theta: -1.34579336643
linear_velocity: 2.0
angular_velocity: -1.79999995232
---
x: 8.77774429321
y: 7.39234447479
theta: -1.37459337711
linear_velocity: 2.0
angular_velocity: -1.79999995232
---
x: 8.78307628632
y: 7.3607916832
theta: -1.40339338779
linear_velocity: 2.0
angular_velocity: -1.79999995232
```

4.7.2 使用 rostopic hz

rostopic hz 报告数据发布的速率，用法：

```
rostopic hz [topic]
```

以下指令可以查看 turtlesim_node 发布/turtle/pose 的速度。

```
rostopic hz /turtle1/pose
```

```
ding@dingding:~$ rostopic hz /turtle1/pose
subscribed to [/turtle1/pose]
average rate: 62.596
        min: 0.014s max: 0.017s std dev: 0.00063s window: 62
average rate: 62.558
        min: 0.014s max: 0.018s std dev: 0.00063s window: 125
average rate: 62.541
        min: 0.014s max: 0.018s std dev: 0.00064s window: 187
average rate: 62.523
        min: 0.011s max: 0.021s std dev: 0.00079s window: 250
average rate: 62.519
        min: 0.011s max: 0.021s std dev: 0.00078s window: 313
```

现在我们可以知道了 turtlesim 正以大约 62Hz 的频率发布有关乌龟的数据。我们也可以结合 rostopic type 和 rosmsg show 命令来获取关于某个话题的更深层次信息：

```
rostopic type /turtle1/cmd_vel | rosmsg show
```


版权所有 © 深圳史河机器人科技有限公司

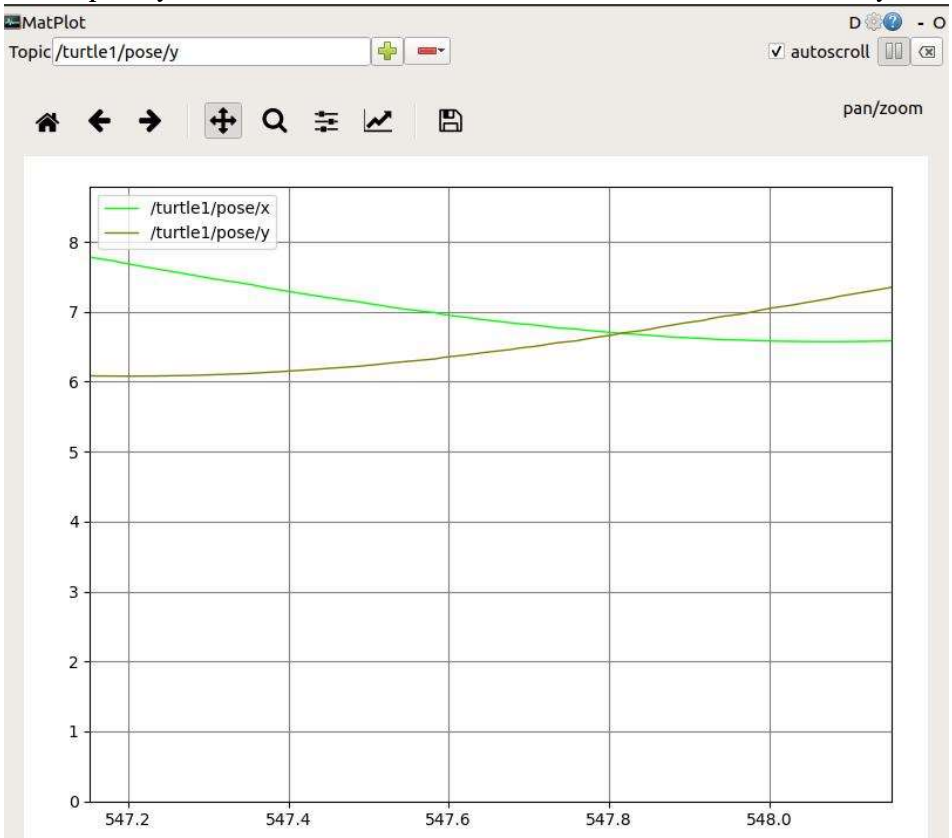
```
ding@dingding:~$ rostopic type /turtle1/cmd_vel | rosmsg show
geometry_msgs/Vector3 linear
  float64 x
  float64 y
  float64 z
geometry_msgs/Vector3 angular
  float64 x
  float64 y
  float64 z
```

4.8 使用 rqt_plot

rqt_plot 命令可以在滚动时间图上显示发布到某个话题上的数据。这里我们将使用 rqt_plot 命令来绘制正被发布到/turtle1/pose 话题上的数据。首先，在一个新终端中输入：

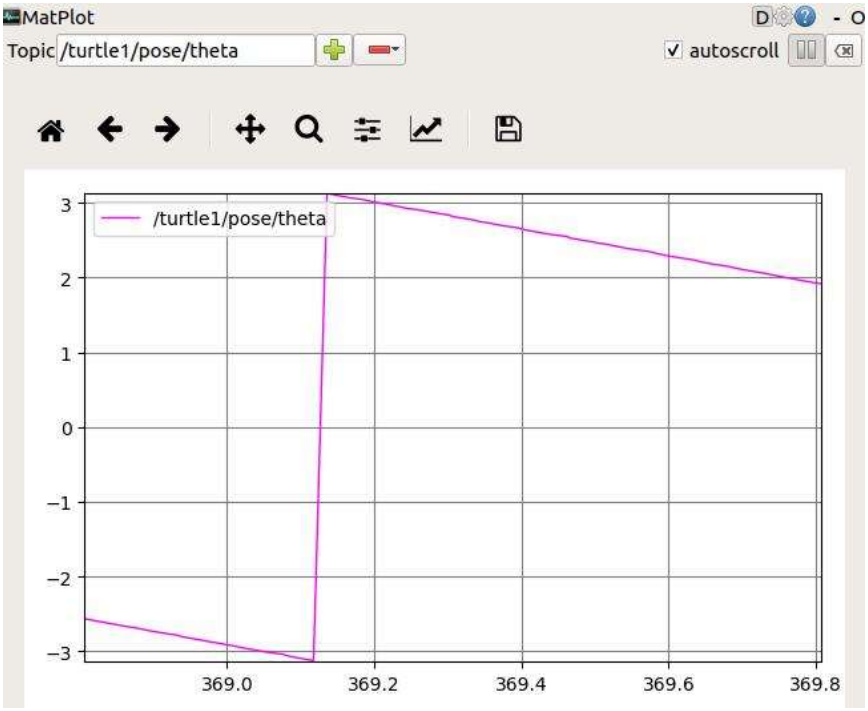
```
rosrun rqt_plot rqt_plot
```

这会弹出一个新窗口，可以在左上角的文本框里面添加任何想要绘制的话题。在里面输入/turtle1/pose/x 后，之前不能按下的加号按钮将会变亮。按一下该按钮，并对/turtle1/pose/y 重复相同的过程。现在你会在图中看到 turtle 的 x-y 位置。



按下减号按钮会显示一组菜单可以让你在图中隐藏指定的话题。现在隐藏掉你刚才添加的话题并添加/turtle1/pose/theta，你会看到如下所示的图：

版权所有 © 深圳史河机器人科技有限公司



五、课堂作业

1、理解 ROS 节点与话题的关系。

2、理解 ROS 话题在信息传输中的作用。

3、熟练掌握 rostopic 的各种用法。

版权所有 © 深圳史河机器人科技有限公司

实验 7　理解 ROS 服务和参数

一、实验目的

1、掌握 ROS 服务和参数的相关知识；

2、学会命令行工具 rosservice 和 rosparam 的使用方法。

二、实验设备

此实验在上一实验的基础上进行，确保 turtlesim_node 仍在运行，如果没有运行 turtlesim_node，请参考上一个实验运行 turtlesim_node。

三、实验原理

3.1 ROS 服务

服务（Services）是节点之间通讯的另一种方式。服务允许节点发送一个请求（request）并获得一个响应（response）。

3.2 rosparam

rosparam 能让我们在 ROS 参数服务器（Parameter Server）上存储和操作数据。参数服务器能够存储整型（integer）、浮点（float）、布尔（boolean）、字典（dictionaries）和列表（list）等数据类型。rosparam 使用 YAML 标记语言的语法。一般而言，YAML 的表述很自然：1 是整型，1.0 是浮点型，one 是字符串，true 是布尔型，[1，2，3]是整型组成的列表，{a：b，c：d}是字典。rosparam 有很多命令可以用来操作参数，如下所示：

- rosparam set　　设置参数
- rosparam get　　获取参数
- rosparam load　　从文件中加载参数
- rosparam dump　　转储参数到文件里
- rosparam delete　　删除参数
- rosparam list　　列出参数名

四、实验步骤

4.1 使用 rosservice

rosservice 可以很容易地通过服务附加到 ROS 客户端/服务器框架上。rosservice 有许多可用于服务的命令，如下所示：

版权所有 © 深圳史河机器人科技有限公司

- rosservice list 输出活跃服务的信息
- rosservice call 用给定的参数调用服务
- rosservice type 输出服务的类型
- rosservice find 按服务的类型查找服务
- rosservice uri 输出服务的 ROSRPC uri

4.1.1 rosservice list

list 命令显示 turtlesim 节点提供了 9 个服务：reset，clear，spawn，kill，turtle1/set_pen，/turtle1/teleport_absolute，/turtle1/teleport_relative，turtlesim/get_loggers，turtlesim/set_logger_level。同时还有两个与 rosout 节点有关的服务：/rosout/get_loggers 和/rosout/set_logger_level。

```
ding@dingding:~$ rosservice list
/clear
/kill
/reset
/rosout/get_loggers
/rosout/set_logger_level
/spawn
/teleop_turtle/get_loggers
/teleop_turtle/set_logger_level
/turtle1/set_pen
/turtle1/teleport_absolute
/turtle1/teleport_relative
/turtlesim/get_loggers
/turtlesim/set_logger_level
ding@dingding:~$
```

4.1.2 rosservice type

用法：rosservice type [service]

以下指令可以 clear 服务的类型：

```
rosservice type /clear
```

```
ding@dingding:~$ rosservice type /clear
std_srvs/Empty
```

服务的类型为 empty（空），这表明调用这个服务时不需要参数（它在发出请求时不发送数据，在接收响应时也不接收数据）。下面使用 rosservice call 命令调用此服务：

4.1.3 rosservice call

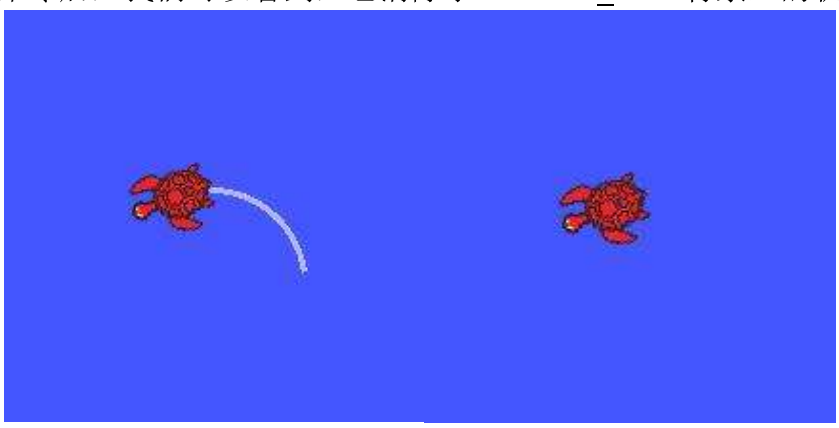
用法：rosservice call [service（服务名称）] [args（参数）]

因为 clear 服务的类型为 empty，所以进行无参数调用：

```
rosservice call /clear
```


版权所有 © 深圳史河机器人科技有限公司

执行命令后，我们可以看到，它清除了 turtlesim_node 背景上的轨迹。



以下指令可以查看具有参数的 spawn（产卵）服务的信息：

```
rosservice type /spawn | rossrv show
```

```
ding@dingding:~$ rosservice type /spawn | rossrv show
float32 x
float32 y
float32 theta
string name
---
string name
```

这个服务能让我们可以在给定的位置和角度生成一只新的乌龟。name 字段是可选的，这里我们不设具体的名字，让 turtlesim 自动创建一个。

```
rosservice call /spawn 2 2 0.2 ""
```

该调用返回了新产生的乌龟的名字：name: turtle2

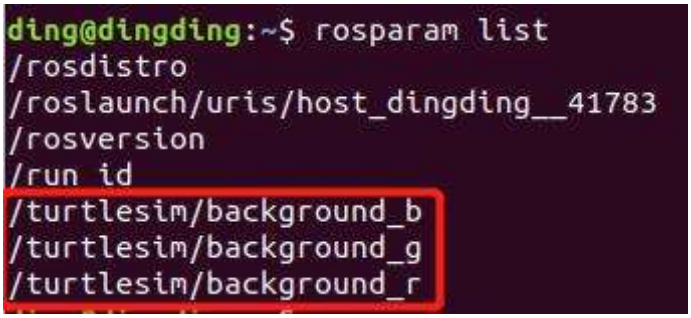
版权所有 © 深圳史河机器人科技有限公司

4.2 使用 rosparam

4.2.1 rosparam list

```
rosparam list
```

可以看到 turtlesim 节点在参数服务器上有 3 个参数用于设定背景颜色：



4.2.2 rosparam set 和 rosparam get

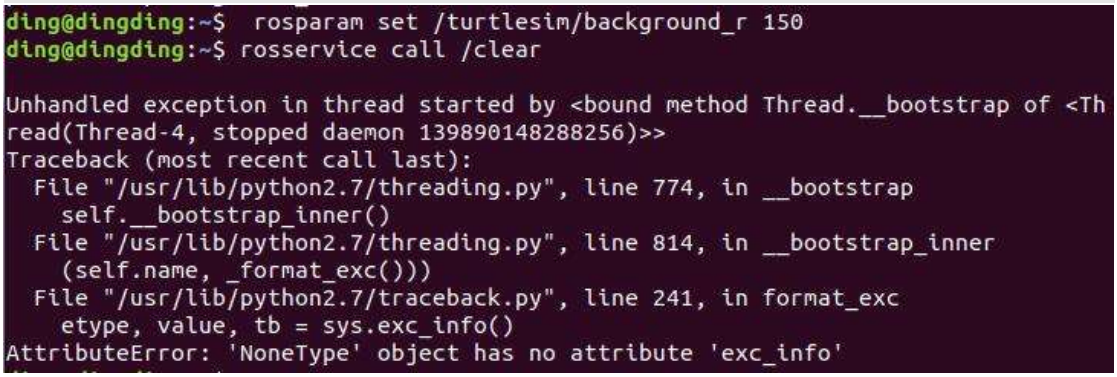
```
用法：rosparam set [param_name]  （设置参数）
      rosparam get [param_name]  （获取参数）
```

使用以下指令修改背景颜色的红色通道值：

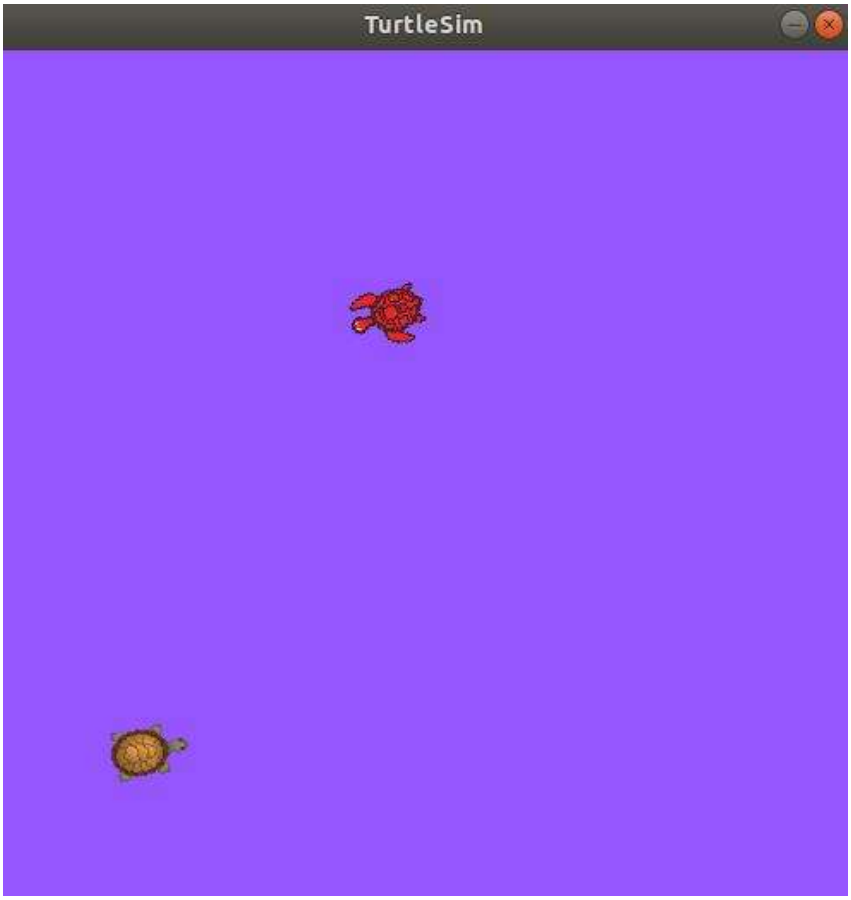
```
rosparam set /turtlesim/background_r 150
```

上述指令修改了参数的值，现在我们需要调用 clear 服务使得参数的修改能生效：

```
rosservice call /clear
```



版权所有 © 深圳史河机器人科技有限公司



我们可以通过以下指令获取背景的绿色通道的参数值：

```
rosparam get /turtlesim/background_g
```

也可以用 rosparam get /来显示参数服务器上的所有内容：

```
rosparam get /
```

```
ding@dingding:~$  rosparam get /turtlesim/background_g
86
ding@dingding:~$ rosparam get /
rosdistro: 'melodic

  '
roslaunch:
  uris: {host_dingding__41783: 'http://dingding:41783/'}
rosversion: '1.14.12

  '
run_id: 5bf41c34-5d46-11ec-b49c-080027226d4a
turtlesim: {background_b: 255, background_g: 86, background_r: 150}
```

4.2.3 rosparam dump 和 rosparam load

用法：

```
rosparam dump [file_name] [namespace]
rosparam load [file_name] [namespace]
```

我们将所有的参数写入 params.yaml 文件：

```
rosparam dump params.yaml
```


版权所有 © 深圳史河机器人科技有限公司

也可以将 yaml 文件重下载到新的命名空间，例如 copy_turtle：

```
rosparam load params.yaml copy_turtle
rosparam get /copy_turtle/turtlesim/background_g
```

```
ding@dingding:~$ rosparam load params.yaml copy_turtle
ding@dingding:~$ rosparam list
/copy_turtle/rosdistro
/copy_turtle/roslaunch/uris/host_dingding__41783
/copy_turtle/rosversion
/copy_turtle/run_id
/copy_turtle/turtlesim/background_b
/copy_turtle/turtlesim/background_g
/copy_turtle/turtlesim/background_r
/rosdistro
/roslaunch/uris/host_dingding__41783
/rosversion
/run_id
/turtlesim/background_b
/turtlesim/background_g
/turtlesim/background_r
ding@dingding:~$ rosparam get /copy_turtle/turtlesim/background_g
86
```

五、课堂作业

1、深入理解 ROS 服务和参数的工作原理。

1、熟悉并掌握 rosservice 各种命令的用途和具体使用方法。

2、熟悉并掌握 rosparam 各种命令的用途和具体使用方法。

版权所有 © 深圳史河机器人科技有限公司

实验 8 使用 rqt_console 和 roslaunch

一、实验目的

1、学会使用 rqt_console 和 rqt_logger_level；

2、学会使用 roslaunch 同时启动多个节点。

二、实验设备

本实验会用到 rqt 和 turtlesim 这两个软件包。若未安装，请先安装：

```
sudo apt-get install ros-melodic-rqt ros-melodic-rqt-common-plugins ros-melodic-turtlesim
```

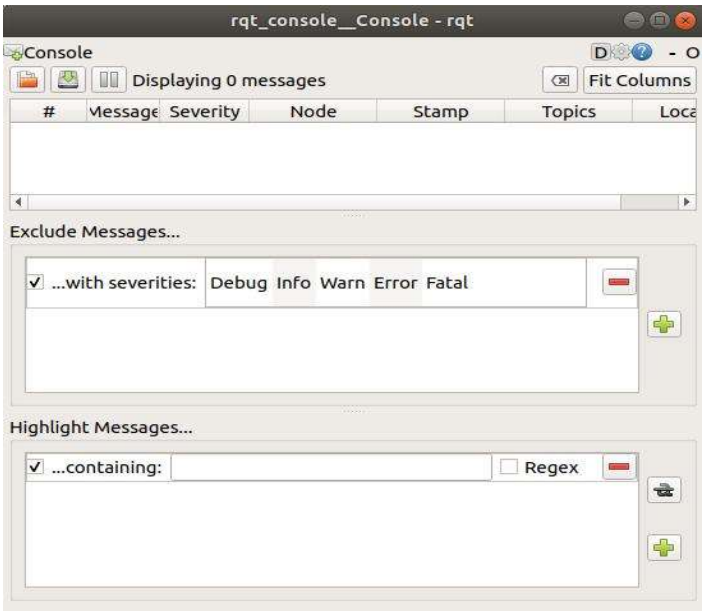
三、实验原理

rqt_console 连接到了 ROS 的日志框架，以显示节点的输出信息。rqt_logger_level 允许我们在节点运行时改变输出信息的详细级别，包括 Debug、Info、Warn 和 Error。

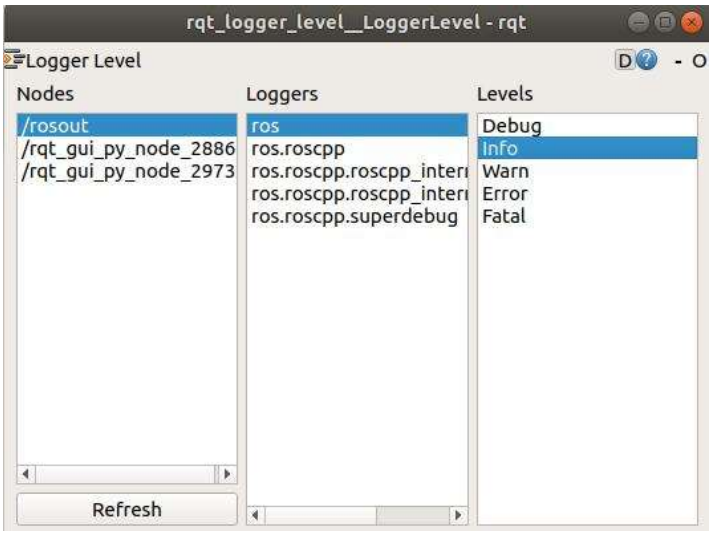
四、实验步骤

4.1 使用 rqt_console 和 rqt_logger_level

现在让我们来看一下 turtlesim 在 rqt_console 中输出的信息，同时在使用 turtlesim 时切换 rqt_logger_level 中的日志级别。在启动 turtlesim 之前先在两个新终端中运行 rqt_console 和 rqt_logger_level：



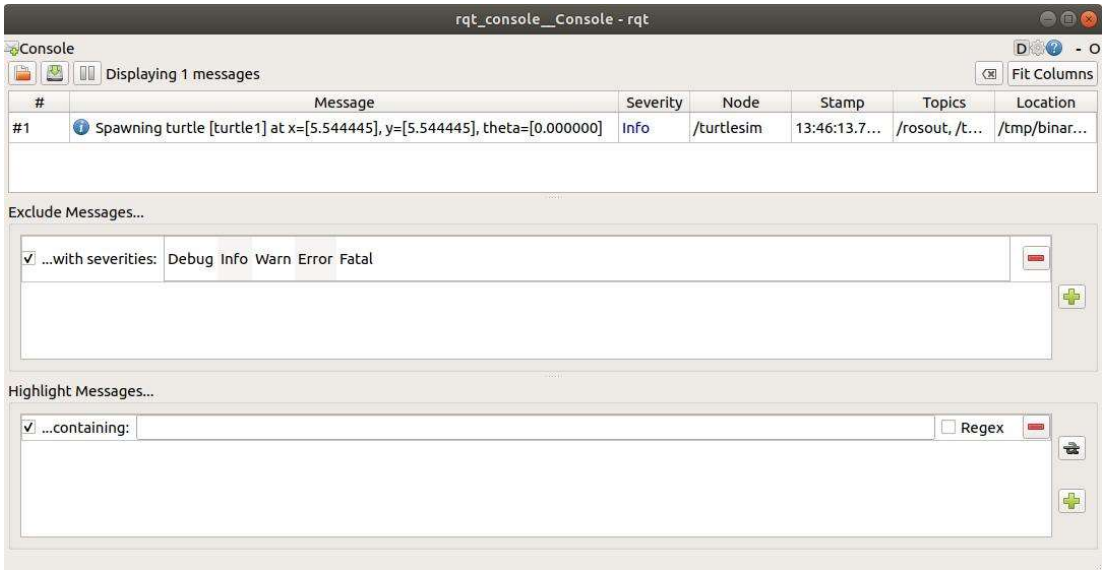
版权所有 © 深圳史河机器人科技有限公司



然后在一个新终端中启动 turtlesim：

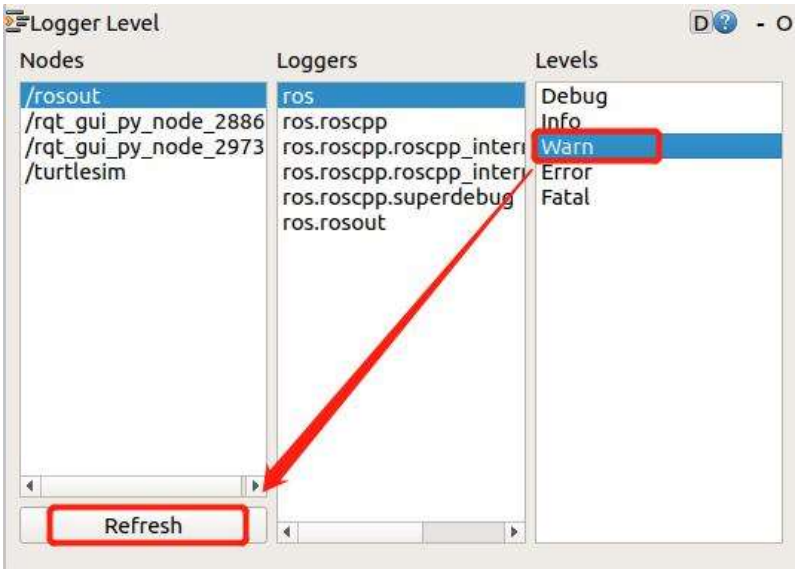
```
rosrun turtlesim turtlesim_node
```

因为默认的日志级别是 Info，所以你会看到 turtlesim 启动后发布的所有信息，如下图所示：

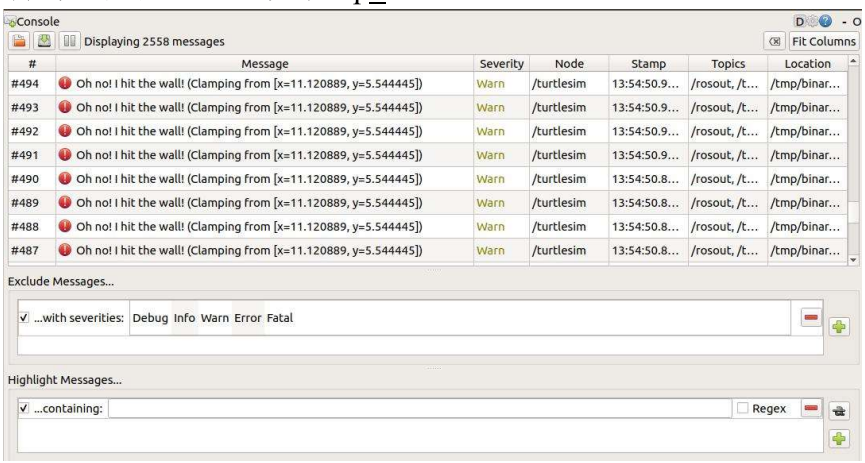


然后我们在 rqt_logger_level 窗口中刷新一下节点并选择 Warn 以修改日志级别，如下图所示：

版权所有 © 深圳史河机器人科技有限公司



然后将乌龟撞在墙上，观察 rqt_console：



4.1.1 日志记录器级别

日志级别的优先级按以下顺序排列：

- Fatal （致命）
- Error （错误）
- Warn （警告）
- Info （信息）
- Debug （调试）

Fatal 是最高优先级，Debug 是最低优先级。通过设置日志级别，你可以获得所有优先级级别，或只是更高级别的消息。比如，将日志级别设为 Warn 时，你会得到 Warn、Error 和 Fatal 这三个等级的日志消息。

现在按 Ctrl+C 退出 turtlesim 节点。接下来我们将使用 roslaunch 来启动多个 turtlesim 节点和一个模仿者节点，来让一个乌龟模仿另一个乌龟。

版权所有 © 深圳史河机器人科技有限公司

4.1.2 使用 roslaunch

roslaunch 可以用来启动定义在 launch（启动）文件中的节点。用法：

```
roslaunch [package] [filename.launch]
```

先切换到我们之前创建和构建的 beginner_tutorials 软件包目录下：

```
roscd beginner_tutorials
```

然后创建一个 launch 目录：

```
mkdir launch
cd launch
```

```
^Cding@dingding:~roscd beginner_tutorials
ding@dingding:~/catkin_ws/src/beginner_tutorials$ mkdir launch
ding@dingding:~/catkin_ws/src/beginner_tutorials$ cd launch
ding@dingding:~/catkin_ws/src/beginner_tutorials/launch$
```

4.1.3 launch 文件

现在创建一个名为 turtlemimic.launch 的 launch 文件，并将以下内容加进去：

```
<launch>

  <group ns="turtlesim1">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <group ns="turtlesim2">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <node pkg="turtlesim" name="mimic" type="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>

</launch>
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials/launch$  gedit turtlemimic.launch
```

```
<launch>

  <group ns="turtlesim1">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <group ns="turtlesim2">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <node pkg="turtlesim" name="mimic" type="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>

</launch>
```


版权所有 © 深圳史河机器人科技有限公司

4.1.4 launch 解析

下面开始拆解 launch XML 文件：

```
<launch>
```

首先是 launch 标签开头，它表明这是一个 launch 文件。

```
<group ns="turtlesim1">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <group ns="turtlesim2">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>
```

此处我们创建了两个分组，并以命名空间（namespace）标签来区分，其中一个名为 turtulesim1，另一个名为 turtlesim2，两个分组中都有相同的名为 sim 的 turtlesim 节点。这样可以让我们同时启动两个 turtlesim 模拟器，而不会产生命名冲突。

```
<node pkg="turtlesim" name="mimic" type="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>
```

在这里我们启动模仿节点，话题的输入和输出分别重命名为 turtlesim1 和 turtlesim2，这样就可以让 turtlesim2 模仿 turtlesim1 了。

```
</launch>
```

这一行使得 launch 文件的 XML 标签闭合。

4.1.5 使用 roslaunch

现在我们通过 roslaunch 命令来运行 launch 文件：

```
roslaunch beginner_tutorials turtlemimic.launch
```

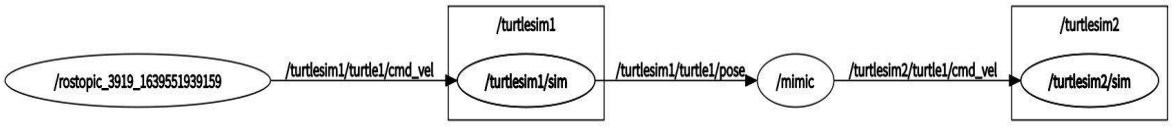
将会有两个新终端被启动，然后我在一个新终端中使用 rostopic 发送以下指令：

```
rostopic pub /turtlesim1/turtle1/cmd_vel geometry_msgs/Twist -r 1 -- '[2.0, 0.0, 0.0]' '[0.0, 0.0, -1.8]'
```

虽然只给 turtlesim1 发布了命令，但是你会发现两个 turtlesims 同时开始移动。

版权所有 © 深圳史河机器人科技有限公司

同时，我们也可以通过 rqt_graph 来更好地理解 launch 文件所做的事情。



五、课堂作业

1、使用 rqt_console 和 rqt_logger_level 修改日志级别。

2、创建一个 launch 文件，同时驱动两个乌龟。

3、通过 rqt_graph 来理解 launch 文件。

版权所有 © 深圳史河机器人科技有限公司

实验 9 自定义消息和服务

一、实验目的

1、学会如何创建和构建 msg 和 srv 文件；

2、学会如何使用 rosmsg、rossrv 和 roscp 命令行工具。

二、实验原理

2.1 msg 和 srv 介绍

• msg（消息）：msg 文件就是文本文件，用于描述 ROS 消息的字段。它们用于为不同编程语言编写的消息生成源代码。

•srv（服务）：一个 srv 文件描述一个服务。它由两部分组成：请求（request）和响应（response）。

msg 文件存放在软件包的 msg 目录下，srv 文件则存放在 srv 目录下。

msg 文件就是简单的文本文件，每行都有一个字段类型和字段名称。可以使用的类型为：

• int8，int16，int32,，int64 (以及 uint*)

• float32，float64

• string

• time，duration

• 其他 msg 文件

• variable-length array[ ] 和 fixed-length array[C]

ROS 中还有一个特殊的数据类型：Header，它含有时间戳和 ROS 中广泛使用的坐标帧信息。在 msg 文件的第一行经常可以看到 Header header。

下面是一个使用了 Header、字符串原语和其他两个消息的示例：这个 msg 文件样例，使用了 Header，string，和其他另外两个消息的类型：

```
Header header
string child_frame_id
geometry_msgs/PoseWithCovariance pose
geometry_msgs/TwistWithCovariance twist
```

srv 文件和 msg 文件一样，只是它们包含两个部分：请求和响应。这两部分用一条---线隔开。下面是一个 srv 文件的示例：

```
int64 A
int64 B
```


版权所有 © 深圳史河机器人科技有限公司

```
---
int64 Sum
```

A 和 B 是请求，Sum 是响应。

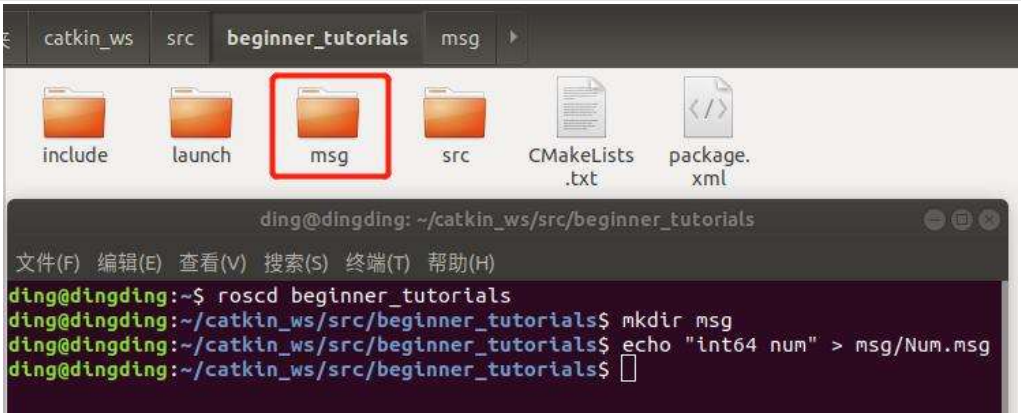
三、实验步骤

3.2 使用 msg

3.2.1 创建 msg

我们在之前创建的软件包（beginner_tutorials）里创建一个新的包（msg）。

```
roscd beginner_tutorials
mkdir msg
echo "int64 num" > msg/Num.msg
```



以上创建的.msg 文件只有一行，你也可以通过添加更多元素来创建一个更复杂的文件。

为了确保 msg 文件能被转换为 C++、Python 和其他语言的源代码，打开 package.xml，确保它包含以下两行且没有被注释。如果没有，添加进去：

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
```

```
<buildtool_depend>catkin</buildtool_depend>
<build_depend>roscpp</build_depend>
<build_depend>rospy</build_depend>
<build_depend>std_msgs</build_depend>
<build_depend>message_generation</build_depend>

<build_export_depend>roscpp</build_export_depend>
<build_export_depend>rospy</build_export_depend>
<build_export_depend>std_msgs</build_export_depend>

<exec_depend>roscpp</exec_depend>
<exec_depend>rospy</exec_depend>
<exec_depend>std_msgs</exec_depend>
<exec_depend>message_runtime</exec_depend>
```


版权所有 © 深圳史河机器人科技有限公司

在 CMakeLists.txt 文件中，为已经存在里面的 find_package 调用添加 message_generation 依赖项，这样就能生成消息了。直接将 message_generation 添加到 COMPONENTS 列表中即可，如下所示：

```
find_package(catkin REQUIRED COMPONENTS
  roscpp
  rospy
  std_msgs
  message_generation
)
```

你需要确保你设置了运行依赖，添加以下代码：

```
CATKIN_DEPENDS message_runtime
```

```
catkin_package(
#  INCLUDE_DIRS include
#  LIBRARIES beginner_tutorials
#  CATKIN_DEPENDS roscpp rospy std_msgs
   CATKIN_DEPENDS message_runtime
#  DEPENDS system_lib
)
```

找到以下码块，将注释去掉，然后进行以下修改：

```
# add_message_files(
#   FILES
#   Message1.msg
#   Message2.msg
# )
```

```
add_message_files(
  FILES
  Num.msg
)
```

现在必须确保 generate_messages()函数被调用，做出以下更改：

```
# generate_messages(
#   DEPENDENCIES
#   std_msgs
# )
```

```
generate_messages(
  DEPENDENCIES
  std_msgs
)
```

如此一来，你就可以从 msg 文件定义中生成源代码文件了。

3.2.2 使用 rosmsg

用法：

```
rosmsg show [消息类型]
rosmsg show beginner_tutorials/Num
```

```
dinq@dinqding:~/catkin_ws/src/beginner_tutorials$ rosmsg show beginner_tutorials/Num
int64 num
```

上面的例子中，消息类型包含两个部分：

- beginner_tutorials -- 定义消息的软件包
- Num -- 消息的名称 Num

版权所有 © 深圳史河机器人科技有限公司

如果你忘记 msg 在哪个包里面，也可以省略包的名称：

```
rosmsg show Num
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$ rosmsg show Num
[beginner_tutorials/Num]:
int64 num
```

3.3 使用 srv

3.3.1 创建 srv

我们继续使用之前创建的包来创建服务：

```
roscd beginner_tutorials
mkdir srv
```

我们将从另一个包复制现有的 srv 定义，而不是手动创建新的 srv。roscp 是一个实用的命令行工具，用于将文件从一个包复制到另一个包。用法：

```
roscp [package_name] [file_to_copy_path] [copy_path]
```

现在我们从 rospy_tutorials 包中复制一个服务：

```
roscp rospy_tutorials AddTwoInts.srv srv/AddTwoInts.srv
```

找到以下码块，进行下图所示的修改：

```
# add_service_files(
#   FILES
#   Service1.srv
#   Service2.srv
# )
```

```
add_service_files(
  FILES
  AddTwoInts.srv
)
```

现在，你可以从 srv 文件中定义生成源代码文件了。

3.3.2 使用 rossrv

用法：

```
rossrv show <服务类型>
rossrv show beginner_tutorials/AddTwoInts
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$ rossrv show beginner_tutorials/AddT
woInts
int64 a
int64 b
---
int64 sum
```

你也可以在不指定包名的情况下使用 rossrv show：

```
rossrv show AddTwoInts
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$ rossrv show AddTwoInts
[beginner_tutorials/AddTwoInts]:
int64 a
int64 b
---
int64 sum

[rospy_tutorials/AddTwoInts]:
int64 a
int64 b
---
int64 sum
```

这里显示了两个服务。第一个是刚刚在 beginner_tutorials 包中创建的，第二个是之前 rospy_tutorials 包中已经存在的。

3.4 获取帮助

在我们学习了许多的 ROS 工具之后，记住每一个命令所需要的参数是十分困难的，这时我们可以使用 ROS 工具提供的帮助：

```
rosmsg -h
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$ rosmsg -h
rosmsg is a command-line tool for displaying information about ROS Message types.

Commands:
        rosmsg show     Show message description
        rosmsg info     Alias for rosmsg show
        rosmsg list     List all messages
        rosmsg md5      Display message md5sum
        rosmsg package  List messages in a package
        rosmsg packages List packages that contain messages

Type rosmsg <command> -h for more detailed usage
```

同样也可以获得子命令的帮助：

```
rosmsg show –h
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$ rosmsg show -h
Usage: rosmsg show [options] <message type>

Options:
  -h, --help            show this help message and exit
  -r, --raw             show raw message text, including comments
  -b BAGFILE, --bag=BAGFILE
                        show message from .bag file
```

四、课堂作业

掌握以下命令行工具的用途和具体用法：

- rospack = ros+pack(age)　　提供与 ROS 包相关的信息
- roscd = ros+cd　　将目录更改为 ROS 包或堆栈
- rosls = ros+ls　　列出 ROS 包中的文件
- roscp = ros+cp　　从/向 ROS 包复制文件

版权所有 © 深圳史河机器人科技有限公司

- rosmsg = ros+msg　　提供与 ROS 消息定义相关的信息
- rossrv = ros+srv　　提供与 ROS 服务定义相关的信息
- catkin_make　　制作（编译）一个 ROS 包
- rosmake = ros+make　　制作(编译)一个 ROS 包(如果你没有使用 catkin 工作区)

版权所有 © 深圳史河机器人科技有限公司

实验 10 编写简单的发布者和订阅者（C++）

一、实验目的

1、掌握编写简单发布者的流程；
2、掌握编写简单订阅者的流程；
3、理解每一句代码的具体含义；
4、掌握如何构建节点。

二、实验设备

计算机；Ubuntu 18.04 操作系统；ROS melodic

三、实验原理

话题（topic）也可译为“主题”，发布者在一个话题上发布消息，订阅者订阅一个话题并取出里面的消息。话题是一个字符串形式的名字，消息是一种数据结构里面填充了特定的内容。

四、实验操作

4.1 编写发布者结点

“节点”是连接到 ROS 网络的可执行文件。在这里，我们将创建 talker（发布者）节点，该节点将不断广播消息。

将当前目录切换到之前创建的 beginner_tutorials 包中：

```
roscd beginner_tutorials
```

4.1.1 代码

在 src 目录下创建 talker.cpp 文件并粘贴以下内容进去：

```
gedit talker.cpp
```

代码：

```
#include "ros/ros.h"
#include "std_msgs/String.h"
#include <sstream>
/**
 * This tutorial demonstrates simple sending of messages over the RO
S system.
 */
int main(int argc, char **argv)
```


版权所有 © 深圳史河机器人科技有限公司

```
{
  ros::init(argc, argv, "talker");
  ros::NodeHandle n;
  ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatte
r", 1000);
  ros::Rate loop_rate(10);
  int count = 0;
  while (ros::ok())
  {
    std_msgs::String msg;
    std::stringstream ss;
    ss << "hello world " << count;
    msg.data = ss.str();
    ROS_INFO("%s", msg.data.c_str());
    chatter_pub.publish(msg);
    ros::spinOnce();
    loop_rate.sleep();
    ++count;
  }
  return 0;
}
```

```
#include "ros/ros.h"
#include "std_msgs/String.h"
```

ros/ros.h 是一个很便利的 include，它包括了使用 ROS 系统中最常见的公共部分所需的全部头文件。

```
ros::init(argc, argv, "talker");
```

初始化 ROS。这使得 ROS 可以通过命令行进行名称重映射——不过目前不重要。这也是我们给节点指定名称的地方。节点名在运行的系统中必须是唯一的。注意：名称必须是基本名称，例如不能包含任何斜杠/

```
ros::NodeHandle n;
```

为这个进程的节点创建句柄。第一个 NodeHandle 实际上将执行节点的初始化，而最后一个被销毁的 NodeHandle 将清除节点所使用的任何资源。

```
ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);
```

告诉主节点我们将要在 chatter 话题上发布一个类型为 std_msgs/String 的消息。这会让主节点告诉任何正在监听 chatter 的节点，我们将在这一话题上发布数据。第二个参数是发布队列的大小。在本例中，如果我们发布得太快，它将最

版权所有 © 深圳史河机器人科技有限公司

多缓存 1000 条消息，不然就会丢弃旧消息。

NodeHandle::advertise()返回一个 ros::Publisher 对象，它有 2 个目的：其一，它包含一个 publish()方法，可以将消息发布到创建它的话题上；其二，当超出范围时，它将自动取消这一宣告操作。

```
ros::Rate loop_rate(10);
```

ros::Rate 对象能让你指定循环的频率。它会记录从上次调用 Rate::sleep()到现在的时间，并休眠正确的时间。在本例中，我/们告诉它希望以 10Hz 运行。

```
int count = 0;
while (ros::ok())
{
```

默认情况下，roscpp 将安装一个 SIGINT 处理程序，它能够处理 Ctrl+C 操作，让 ros::ok()返回 false。

ros::ok()在以下情况会返回 false：

- 收到 SIGINT 信号（Ctrl+C）
- 被另一个同名的节点踢出了网络
- ros::shutdown()被程序的另一部分调用
- 所有的 ros::NodeHandles 都已被销毁

```
std_msgs::String msg;
std::stringstream ss;
ss << "hello world " << count;
msg.data = ss.str();
```

我们使用一种消息自适应的类在 ROS 上广播消息，该类通常由 mag 文件生成。更复杂的数据类型也可以，不过我们现在将使用标准的 String 消息，它有一个成员：data。

```
chatter_pub.publish(msg);
```

将信息广播给了任何已连接的节点。

```
ROS_INFO("%s", msg.data.c_str());
```

ROS_INFO 可用来取代 printf/cout。

版权所有 © 深圳史河机器人科技有限公司

```
ros::spinOnce();
```

调用 ros::spinOnce()对于这个简单程序来说没必要，因为我们没有接收任何回调。然而，如果要在这个程序中添加订阅，但此处没有 ros::spinOnce()的话，回调函数将永远不会被调用。

```
loop_rate.sleep();
```

使用 ros::Rate 在剩下的时间内睡眠，以让我们达到 10Hz 的发布速率。

4.2 编写订阅者节点

4.2.1 代码

在 src 目录下创建 listener.cpp 文件并粘贴以下内容进去：

```
gedit listener.cpp
```

```
#include "ros/ros.h"
#include "std_msgs/String.h"
/**
 * This tutorial demonstrates simple receipt of messages over the RO
S system.
 */
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
  ROS_INFO("I heard: [%s]", msg->data.c_str());
}

int main(int argc, char **argv)
{
  ros::init(argc, argv, "listener");
  ros::NodeHandle n;
  ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback)
;
  ros::spin();

  return 0;
}
```

```
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
   ROS_INFO("I heard: [%s]", msg->data.c_str());
}
```

这是一个回调函数，当有新消息到达 chatter 话题时它就会被调用。

版权所有 © 深圳史河机器人科技有限公司

```
ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback);
```

通过主节点订阅 chatter 话题。每当有新消息到达时，ROS 将调用 chatterCallback()函数。第二个参数是队列大小，以防我们处理消息的速度不够快。在本例中，如果队列达到 1000 条，再有新消息到达时旧消息会被丢弃。

NodeHandle::subscribe()返回一个 ros::Subscriber 对象，你必须保持它，除非想取消订阅。当 Subscriber 对象被析构，它将自动从 chatter 话题取消订阅。

```
ros::spin();
```

ros::spin()启动了一个自循环，它会尽可能快地调用消息回调函数。

一旦 ros::ok()返回 false，ros::spin()就会退出，这意味着 ros::shutdown()被调用了，主节点让我们关闭（或是因为按下 Ctrl+C，它被手动调用）。

总结：

- 初始化 ROS 系统
- 订阅 chatter 话题
- 开始 spin 自循环，等待消息的到达
- 当消息到达后，调用 chatterCallback()函数

4.3 构建节点

修改 beginner_tutorials 里面的 CMakeLists.txt 文件，将以下代码添加到它的底部：

```
add_executable(talker src/talker.cpp)
target_link_libraries(talker {catkin_LIBRARIES})
add_dependencies(talker beginner_tutorials_generate_messages_cpp)

add_executable(listener src/listener.cpp)
target_link_libraries(listener {catkin_LIBRARIES})
add_dependencies(listener beginner_tutorials_generate_messages_cpp)
```

修改后的 CMakeLists.txt 文件具体如下所示：

```
cmake_minimum_required(VERSION 3.0.2)
project(beginner_tutorials)

## Find catkin and any catkin packages
find_package(catkin REQUIRED COMPONENTS roscpp rospy std_msgs genmsg
)

## Declare ROS messages and services
add_message_files(FILES Num.msg)
```

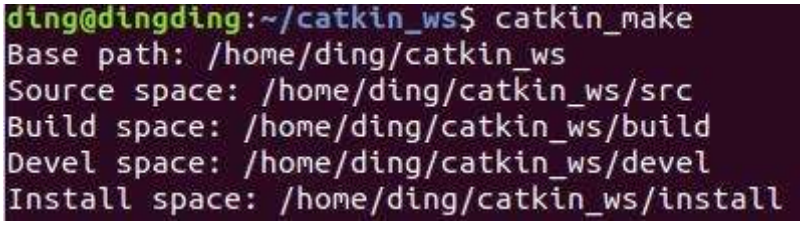

版权所有 © 深圳史河机器人科技有限公司

```
add_service_files(FILES AddTwoInts.srv)
## Generate added messages and services
generate_messages(DEPENDENCIES std_msgs)

## Declare a catkin package
catkin_package()

## Build talker and listener
include_directories(include {catkin_INCLUDE_DIRS})

add_executable(talker src/talker.cpp)
target_link_libraries(talker {catkin_LIBRARIES})
add_dependencies(talker beginner_tutorials_generate_messages_cpp)

add_executable(listener src/listener.cpp)
target_link_libraries(listener {catkin_LIBRARIES})
add_dependencies(listener beginner_tutorials_generate_messages_cpp)
```

这将创建两个可执行文件 talker 和 listener，默认情况下，它们将被放到软件包目录下的 devel 空间中，即~/catkin_ws/devel/lib/beginner_tutorials

然后执行以下指令开始编译：

```
cd ~/catkin_ws
catkin_make
```



版权所有 © 深圳史河机器人科技有限公司

```
[  0%] Built target std_msgs_generate_messages_lisp
[  0%] Built target _beginner_tutorials_generate_messages_check_deps_Num
[  0%] Built target _beginner_tutorials_generate_messages_check_deps_AddTwoInts
[ 11%] Built target beginner_tutorials_generate_messages_lisp
[ 11%] Built target std_msgs_generate_messages_cpp
[ 23%] Built target beginner_tutorials_generate_messages_cpp
[ 23%] Built target std_msgs_generate_messages_eus
[ 41%] Built target beginner_tutorials_generate_messages_eus
[ 41%] Built target std_msgs_generate_messages_py
[ 64%] Built target beginner_tutorials_generate_messages_py
[ 64%] Built target std_msgs_generate_messages_nodejs
[ 76%] Built target beginner_tutorials_generate_messages_nodejs
[ 76%] Built target beginner_tutorials_generate_messages
[ 88%] Built target listener
[100%] Built target talker
```

编译成功之后，我们就编写了一个简单的发布者和订阅者。

4.4 检验发布者和订阅者

4.4.1 运行发布者

开始之前，确保 roscore 已经开启，并且已经 source 过工作空间的 setup.bash 文件

```
roscore
cd ~/catkin_ws
source ./devel/setup.bash
```

然后运行上个实验制作的 talker 的发布者：

```
rosrun beginner_tutorials talker
```

```
ding@dingding:~/catkin_ws$ rosrun beginner_tutorials talker
[ INFO] [1639634004.228022402]: hello world 0
[ INFO] [1639634004.328025573]: hello world 1
[ INFO] [1639634004.428012894]: hello world 2
[ INFO] [1639634004.528077115]: hello world 3
[ INFO] [1639634004.628172060]: hello world 4
[ INFO] [1639634004.728032772]: hello world 5
[ INFO] [1639634004.828234231]: hello world 6
[ INFO] [1639634004.928737260]: hello world 7
[ INFO] [1639634005.028305166]: hello world 8
[ INFO] [1639634005.128150699]: hello world 9
[ INFO] [1639634005.228355602]: hello world 10
```

在发布者节点启动并且运行之后，我们运行订阅者来接受发布者的消息。

4.4.2 运行订阅者

```
rosrun beginner_tutorials listener
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~/catkin_ws$ rosrun beginner_tutorials listener
[ INFO] [1639634163.128822129]: I heard: [hello world 1589]
[ INFO] [1639634163.228912727]: I heard: [hello world 1590]
[ INFO] [1639634163.329467500]: I heard: [hello world 1591]
[ INFO] [1639634163.428560544]: I heard: [hello world 1592]
[ INFO] [1639634163.528543897]: I heard: [hello world 1593]
[ INFO] [1639634163.628635732]: I heard: [hello world 1594]
[ INFO] [1639634163.728769285]: I heard: [hello world 1595]
[ INFO] [1639634163.828424268]: I heard: [hello world 1596]
[ INFO] [1639634163.929047684]: I heard: [hello world 1597]
[ INFO] [1639634164.029473733]: I heard: [hello world 1598]
[ INFO] [1639634164.129504550]: I heard: [hello world 1599]
[ INFO] [1639634164.229446726]: I heard: [hello world 1600]
[ INFO] [1639634164.329056552]: I heard: [hello world 1601]
[ INFO] [1639634164.429181168]: I heard: [hello world 1602]
[ INFO] [1639634164.529310547]: I heard: [hello world 1603]
```

完成之后，停止 listener 和 talker。

五、课堂作业

1、掌握编写发布者节点的流程：

- 初始化 ROS 系统
- 向主节点宣告我们将在 chatter 话题上发布 std_msgs/String 类型的消息
- 以每秒 10 次的速率向 chatter 循环发布消息

2、掌握编写订阅者节点的流程：

- 初始化 ROS 系统
- 订阅 chatter 话题
- 开始 spin 自循环，等待消息的到达
- 当消息到达后，调用 chatterCallback()函数

版权所有 © 深圳史河机器人科技有限公司

实验 11 编写简单的发布者和订阅者（Python）

一、实验目的

1、掌握编写简单发布者的流程；
2、掌握编写简单订阅者的流程；
3、理解每一句代码的具体含义；
4、掌握如何构建节点。

二、实验设备

计算机；Ubuntu 18.04 操作系统；ROS melodic

三、实验原理

话题（topic）也可译为“主题”，发布者在一个话题上发布消息，订阅者订阅一个话题并取出里面的消息。话题是一个字符串形式的名字，消息是一种数据结构里面填充了特定的内容。

四、实验操作

4.1 编写发布者结点

“节点”是连接到 ROS 网络的可执行文件。在这里，我们将创建 talker（发布者）节点，该节点将不断广播消息。

将当前目录切换到之前创建的 beginner_tutorials 包中：

```
roscd beginner_tutorials
```

4.1.2 代码

首先让创建一个 scripts 目录来存放 Python 脚本，在 scripts 目录下创建一个 talker.py 文件

```
mkdir scripts
cd scripts
gedit talker.py
```

将以下代码拷贝到 talker.py 文件里面，然后保存退出。

```
#!/usr/bin/env python
# license removed for brevity
import rospy
from std_msgs.msg import String
```


版权所有 © 深圳史河机器人科技有限公司

```
def talker():
    pub = rospy.Publisher('chatter', String, queue_size=10)
    rospy.init_node('talker', anonymous=True)
    rate = rospy.Rate(10) # 10hz
    while not rospy.is_shutdown():
        hello_str = "hello world %s" % rospy.get_time()
        rospy.loginfo(hello_str)
        pub.publish(hello_str)
        rate.sleep()

if __name__ == '__main__':
    try:
        talker()
    except rospy.ROSInterruptException:
        pass
```

赋予 talker 执行权限

```
chmod +x talker.py
```

然后将以下内容添加到 CMakeLists.txt 文件。这样可以确保正确安装 Python 脚本，并使用合适的 Python 解释器。

```
catkin_install_python(PROGRAMS scripts/talker.py
  DESTINATION {CATKIN_PACKAGE_BIN_DESTINATION})
```

代码分析：

```
#!/usr/bin/env python
```

每个 Python ROS 节点的最开头都有这个声明。第一行确保脚本作为 Python 脚本执行。

```
import rospy
from std_msgs.msg import String
```

如果要编写 ROS 节点，则需要导入 rospy。std_msgs.msg 的导入则是为了使我们能够重用 std_msgs/String 消息类型（即一个简单的字符串容器）来发布。

```
    pub = rospy.Publisher('chatter', String, queue_size=10)
    rospy.init_node('talker', anonymous=True)
```

这部分代码定义了 talker 与其他 ROS 部分的接口。pub = rospy.Publisher("chatter", String, queue_size=10)声明该节点正在使用 String 消息类型发布到 chatter 话题。这里的 String 实际上是 std_msgs.msg.String 类。queue_size 参数是在 ROS Hydro 及更新版本中新增的，用于在订阅者接收消息的

版权所有 © 深圳史河机器人科技有限公司

速度不够快的情况下，限制排队的消息数量。

rospy.init_node(NAME, ...)非常重要，因为它把该节点的名称告诉了 rospy—只有 rospy 掌握了这一信息后，才会开始与 ROS 主节点进行通信。在本例中，你的节点将使用 talker 名称。注意：名称必须是基本名称，例如不能包含任何斜杠/。anonymous = True 会让名称末尾添加随机数，来确保节点具有唯一的名称。

```
    rate = rospy.Rate(10) # 10hz
```

创建一个 Rate 对象 rate。借助其方法 sleep()，它提供了一种方便的方法，来以你想要的速率循环。它的参数是 10，即表示希望它每秒循环 10 次（只要我们的处理时间不超过十分之一秒）！

```
    while not rospy.is_shutdown():
        hello_str = "hello world %s" % rospy.get_time()
        rospy.loginfo(hello_str)
        pub.publish(hello_str)
        rate.sleep()
```

此循环是一个相当标准的 rospy 结构：检查 rospy.is_shutdown()标志，然后执行代码逻辑。你必须查看 is_shutdown()以检查程序是否应该退出。在本例中，代码逻辑即对 public .publish(hello_str)的调用，它将一个字符串发布到 chatter 话题。循环的部分还调用了 rate.sleep()，它在循环中可以用刚刚好的睡眠时间维持期望的速率。

此循环还调用了 rospy.loginfo(str)，它有 3 个任务：打印消息到屏幕上；把消息写入节点的日志文件中；写入 rosout。rosout 是一个方便的调试工具：您可以使用 rqt_console 来拉取消息，而不必在控制台窗口找你节点的输出。

```
try:
    talker()
except rospy.ROSInterruptException:
    pass
```

除了标准的 Python __main__ 检查，它还会捕获一个 rospy.ROSInterruptException 异常，当按下 Ctrl+C 或节点因其他原因关闭时，这一异常就会被 rospy.sleep()和 rospy.Rate.sleep()抛出。引发此异常的原因是你不会意外地在 sleep()之后继续执行代码。

版权所有 © 深圳史河机器人科技有限公司

4.2 编写订阅者节点

在 scripts 目录下创建一个 listener.py 文件

```
roscd beginner_tutorials/scripts/
gedit talker.py
```

将以下代码拷贝到 listener.py 文件中，保存并退出。

```
#!/usr/bin/env python
import rospy
from std_msgs.msg import String

def callback(data):
    rospy.loginfo(rospy.get_caller_id() + "I heard %s", data.data)

def listener():

   # In ROS, nodes are uniquely named. If two nodes with the same
    # name are launched, the previous one is kicked off. The
    # anonymous=True flag means that rospy will choose a unique
    # name for our 'listener' node so that multiple listeners can
    # run simultaneously.
    rospy.init_node('listener', anonymous=True)

    rospy.Subscriber("chatter", String, callback)

    # spin() simply keeps python from exiting until this node is sto
pped
    rospy.spin()

if __name__ == '__main__':
    listener()
```

赋予 listener 执行权限

```
chmod +x listener.py
```

编辑你 CMakeLists.txt 中的 catkin_install_python()调用，如下所示：

```
catkin_install_python(PROGRAMS scripts/talker.py scripts/listener.py
   DESTINATION {CATKIN_PACKAGE_BIN_DESTINATION})
```

代码分析：

```
rospy.init_node('listener', anonymous=True)


   rospy.Subscriber("chatter", String, callback)
```


版权所有 © 深圳史河机器人科技有限公司

```
# spin() simply keeps python from exiting until this node is stopped
rospy.spin()
```

此代码声明你的节点订阅了 chatter 话题，类型是 std_msgs.msgs.String。当接收到新消息时，callback 函数被调用，消息作为第一个参数。

在 rospy.init_node()里面添加了 anonymous=True 关键字参数。ROS 要求每个节点都有一个唯一的名称，如果出现具有相同名称的节点，则会与前一个节点发生冲突，这样一来，出现故障的节点很容易地被踢出网络。anonymous=True 标志会告诉 rospy 为节点生成唯一的名称，这样就很容易可以有多个 listener.py 一起运行。

rospy.spin()只是不让你的节点退出，直到节点被明确关闭。与 roscpp 不同，rospy.spin()不影响订阅者回调函数，因为它们有自己的线程。

4.3 构建节点

回到 catkin 工作空间，然后运行 catkin_make：

```
cd ~/catkin_ws
catkin_make
```

```
ding@dingding:~/catkin_ws/src/beginner_tutorials/scripts$ cd ~/catkin_ws
ding@dingding:~/catkin_ws$ catkin_make
Base path: /home/ding/catkin_ws
Source space: /home/ding/catkin_ws/src
Build space: /home/ding/catkin_ws/build
Devel space: /home/ding/catkin_ws/devel
Install space: /home/ding/catkin_ws/install
####
#### Running command: "make cmake_check_build_system" in "/home/ding/catkin_ws/b
uild"
####
-- Using CATKIN_DEVEL_PREFIX: /home/ding/catkin_ws/devel
-- Using CMAKE_PREFIX_PATH: /home/ding/catkin_ws/devel;/opt/ros/melodic
-- This workspace overlays: /home/ding/catkin_ws/devel;/opt/ros/melodic
-- Found PythonInterp: /usr/bin/python2 (found suitable version "2.7.17", minimu
m required is "2")
-- Using PYTHON_EXECUTABLE: /usr/bin/python2
```

4.4 检验发布者和订阅者

4.4.1 运行发布者

确保 roscore 已经开启：

```
roscore
```

配置环境变量

```
cd ~/catkin_ws
source ./devel/setup.bash
```

运行发布者

```
rosrun beginner_tutorials talker.py
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~/catkin_ws$ rosrun beginner_tutorials talker.py
[INFO] [1642059915.250227]: hello world 1642059915.25
[INFO] [1642059915.350293]: hello world 1642059915.35
[INFO] [1642059915.450718]: hello world 1642059915.45
[INFO] [1642059915.550410]: hello world 1642059915.55
[INFO] [1642059915.650720]: hello world 1642059915.65
[INFO] [1642059915.750330]: hello world 1642059915.75
[INFO] [1642059915.850198]: hello world 1642059915.85
[INFO] [1642059915.950417]: hello world 1642059915.95
```

4.4.2 运行订阅者

rosrun beginner_tutorials listener.py

```
ding@dingding:~/catkin_ws$ rosrun beginner_tutorials listener.py
[INFO] [1642059935.252851]: /listener_4209_16420599349561 heard hello world 1642
059935.25
[INFO] [1642059935.353191]: /listener_4209_16420599349561 heard hello world 1642
059935.35
[INFO] [1642059935.454751]: /listener_4209_16420599349561 heard hello world 1642
059935.45
[INFO] [1642059935.553505]: /listener_4209_16420599349561 heard hello world 1642
059935.55
[INFO] [1642059935.652926]: /listener_4209_16420599349561 heard hello world 1642
059935.65
```

五、课堂作业

1、使用 python 语言编写一个发布者节点。

2、使用 python 语言编写一个订阅者节点。

3、成功构建节点之后检验这两个节点。

版权所有 © 深圳史河机器人科技有限公司

实验 12 编写简单的服务和客户端（C++）

一、实验目的

1、理解服务和客户端之间是如何传递消息的；
2、理解服务节点代码的具体含义；
3、理解客户端节点代码的具体含义。

二、实验设备

Ubuntu18.04 操作系统、ROS melodic、AddTwoInts.srv 服务

三、实验原理

创建简单的服务（Service）节点 add_two_ints_server，该节点将接收两个整数，并在客户端返回它们的和。

四、实验步骤

4.1 编写服务节点

将当前目录切换到之前创建的 beginner_tutorials 包中：

```
roscd beginner_tutorials
```

在 beginner_tutorials 包中创建 src/add_two_ints_server.cpp 文件并粘贴以下内容进去：

```
#include "ros/ros.h"
#include "beginner_tutorials/AddTwoInts.h"

bool add(beginner_tutorials::AddTwoInts::Request  &req,
         beginner_tutorials::AddTwoInts::Response &res)
{
  res.sum = req.a + req.b;
  ROS_INFO("request: x=%ld, y=%ld", (long int)req.a, (long int)req.b)
;
  ROS_INFO("sending back response: [%ld]", (long int)res.sum);
  return true;
}

int main(int argc, char **argv)
{
```


版权所有 © 深圳史河机器人科技有限公司

```
  ros::init(argc, argv, "add_two_ints_server");
  ros::NodeHandle n;

  ros::ServiceServer service = n.advertiseService("add_two_ints", ad
d);
  ROS_INFO("Ready to add two ints.");
  ros::spin();

  return 0;
}
```

```
#include "ros/ros.h"
#include "beginner_tutorials/AddTwoInts.h"
```

beginner_tutorials/AddTwoInts.h 是从之前创建的 srv 文件中生成的头文件。

```
bool add(beginner_tutorials::AddTwoInts::Request  &req,
         beginner_tutorials::AddTwoInts::Response &res)
```

这个函数提供了 AddTwoInts 服务，它接受 srv 文件中定义的请求（request）和响应（response）类型，并返回一个布尔值。

```
{
  res.sum = req.a + req.b;
  ROS_INFO("request: x=%ld, y=%ld", (long int)req.a, (long int)req.b);
  ROS_INFO("sending back response: [%ld]", (long int)res.sum);
  return true;
}
```

两个整数被相加，和已经存储在了响应中。然后记录一些有关请求和响应的信息到日志中。完成后，服务返回 true。

```
  ros::ServiceServer service = n.advertiseService("add_two_ints", add);
```

在这里，服务被创建，并在 ROS 中宣告

4.2 编写客户端节点

在 beginner_tutorials 包中创建 src/add_two_ints_client.cpp 文件并粘贴以下内容进去：

```
#include "ros/ros.h"
#include "beginner_tutorials/AddTwoInts.h"
```


版权所有 © 深圳史河机器人科技有限公司

```
#include <cstdlib>

int main(int argc, char **argv)
{
  ros::init(argc, argv, "add_two_ints_client");
  if (argc != 3)
  {
    ROS_INFO("usage: add_two_ints_client X Y");
    return 1;
  }

  ros::NodeHandle n;
  ros::ServiceClient client = n.serviceClient<beginner_tutorials::Ad
dTwoInts>("add_two_ints");
  beginner_tutorials::AddTwoInts srv;
  srv.request.a = atoll(argv[1]);
  srv.request.b = atoll(argv[2]);
  if (client.call(srv))
  {
    ROS_INFO("Sum: %ld", (long int)srv.response.sum);
  }
  else
  {
    ROS_ERROR("Failed to call service add_two_ints");
    return 1;
  }

  return 0;
}
```

```
ros::ServiceClient client =n.serviceClient<beginner_tutorials: :AddTwoInts>
("add_two_ints");
```

这将为 add_two_ints 服务创建一个客户端。ros::ServiceClient 对象的作用是在稍后调用服务。

```
beginner_tutorials::AddTwoInts srv;
srv.request.a = atoll(argv[1]);
srv.request.b = atoll(argv[2]);
```

这里我们实例化一个自动生成的服务类，并为它的 request 成员赋值。一个服务类包括 2 个成员变量：request 和 response，以及 2 个类定义：Request 和 Response。

版权所有 © 深圳史河机器人科技有限公司

```
if (client.call(srv))
{
  ROS_INFO("Sum: %ld", (long int)srv.response.sum);
}
else
{
  ROS_ERROR("Failed to call service add_two_ints");
  return 1;
}
```

此处实际上调用了服务。由于服务调用被阻塞，它将在调用完成后返回。如果服务调用成功，call()将返回 true，并且 srv.response 中的值将是有效的。如果调用不成功，则 call()将返回 false 且 srv.response 的值将不可用。

4.3 构建节点

下面的代码添加在位于~/catkin_ws/src/beginner_tutorials/CMakeLists.txt 文件的末尾：

```
add_executable(add_two_ints_server src/add_two_ints_server.cpp)
target_link_libraries(add_two_ints_server {catkin_LIBRARIES})
add_dependencies(add_two_ints_server beginner_tutorials_gencpp)
```

```
add_executable(add_two_ints_client src/add_two_ints_client.cpp)
target_link_libraries(add_two_ints_client {catkin_LIBRARIES})
add_dependencies(add_two_ints_client beginner_tutorials_gencpp)
```

执行以下指令开始编译：

```
cd ~/catkin_ws
catkin_make
```

```
ding@dingding:~/catkin_ws$ catkin_make
Base path: /home/ding/catkin_ws
Source space: /home/ding/catkin_ws/src
Build space: /home/ding/catkin_ws/build
Devel space: /home/ding/catkin_ws/devel
Install space: /home/ding/catkin_ws/install
```

```
[ 71%] Built target listener
[ 80%] Built target talker
Scanning dependencies of target beginner_tutorials_gencpp
[ 80%] Built target beginner_tutorials_gencpp
Scanning dependencies of target add_two_ints_server
[ 85%] Building CXX object beginner_tutorials/CMakeFiles/add_two_ints_server.dir/src/add_two_ints_server.cpp.o
[ 90%] Linking CXX executable /home/ding/catkin_ws/devel/lib/beginner_tutorials/add_two_ints_server
[ 90%] Built target add_two_ints_server
Scanning dependencies of target add_two_ints_client
[ 95%] Building CXX object beginner_tutorials/CMakeFiles/add_two_ints_client.dir/src/add_two_ints_client.cpp.o
[100%] Linking CXX executable /home/ding/catkin_ws/devel/lib/beginner_tutorials/add_two_ints_client
[100%] Built target add_two_ints_client
```


版权所有 © 深圳史河机器人科技有限公司

catkin_make 成功之后，将会在~/catkin_ws/devel/lib/beginner_tutorials 中生成两个可执行文件 add_two_ints_server 和 add_two_ints_client。

至此，一个简单的服务端和客户端就已经编写成功了。

4.4 检验服务和客户端

4.4.1 运行服务

```
rosrun beginner_tutorials add_two_ints_server
```

```
ding@dingding:~/catkin_ws$  rosrun beginner_tutorials add_two_ints_server
[ INFO] [1639636800.809336936]: Ready to add two ints.
```

4.4.2 运行客户端

运行客户端时需要加上必要的参数：

```
rosrun beginner_tutorials add_two_ints_client 2 5
```

```
ding@dingding:~/catkin_ws$ rosrun beginner_tutorials add_two_ints_client 2 5
[ INFO] [1639637010.171365003]: Sum: 7
```

到此为止，你已经成功地运行了一个服务和客户端。

五、课堂作业

1、理解客户端是如何利用服务来传输信息的。

2、尝试创建简单的服务（Service）节点 reduce_two_ints_server，该节点将接收两个整数，并返回它们的差。

版权所有 © 深圳史河机器人科技有限公司

实验 13 编写简单的服务和客户端（Python）

一、实验目的

1、理解服务和客户端之间是如何传递消息的；
2、理解服务节点代码的具体含义；
3、理解客户端节点代码的具体含义。

二、实验设备

Ubuntu18.04 操作系统、ROS melodic、AddTwoInts.srv 服务

三、实验原理

创建简单的服务（Service）节点 add_two_ints_server，该节点将接收两个整数，并在客户端返回它们的和。

四、实验步骤

4.1 编写服务节点

将当前目录切换到之前创建的 beginner_tutorials 包中：

```
roscd beginner_tutorials
```

在 beginner_tutorials 包中创建 scripts/add_two_ints_server.py 文件并粘贴以下内容进去：

```
#!/usr/bin/env python

from __future__ import print_function

from beginner_tutorials.srv import AddTwoInts,AddTwoIntsResponse
import rospy

def handle_add_two_ints(req):
    print("Returning [%s + %s = %s]"%(req.a, req.b, (req.a + req.b))
)
    return AddTwoIntsResponse(req.a + req.b)

def add_two_ints_server():
    rospy.init_node('add_two_ints_server')
    s = rospy.Service('add_two_ints', AddTwoInts, handle_add_two_int
s)
```


版权所有 © 深圳史河机器人科技有限公司

```
    print("Ready to add two ints.")
    rospy.spin()

if __name__ == "__main__":
    add_two_ints_server()
```

给节点执行权限

```
chmod +x scripts/add_two_ints_server.py
```

然后将以下内容添加到 CMakeLists.txt 文件。这样可以确保正确安装 Python 脚本，并使用合适的 Python 解释器。

```
catkin_install_python(PROGRAMS scripts/add_two_ints_server.py
  DESTINATION {CATKIN_PACKAGE_BIN_DESTINATION})
```

4.1.1 代码分析

```
s = rospy.Service('add_two_ints', AddTwoInts, handle_add_two_ints)
```

这声明了一个名为 add_two_ints 的新服务，其服务类型为 AddTwoInts。所有的请求（request）都传递给了 handle_add_two_ints 函数。handle_add_two_ints 被 AddTwoIntsRequest 的实例调用，返回 AddTwoIntsResponse 实例。

4.2 编写客户端节点

在 beginner_tutorials 包中创建 scripts/add_two_ints_client.py 文件并粘贴以下内容进去：

```
#!/usr/bin/env python

from __future__ import print_function

import sys
import rospy
from beginner_tutorials.srv import *

def add_two_ints_client(x, y):
    rospy.wait_for_service('add_two_ints')
    try:
        add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts
)
        resp1 = add_two_ints(x, y)
        return resp1.sum
    except rospy.ServiceException as e:
        print("Service call failed: %s"%e)

def usage():
    return "%s [x y]"%sys.argv[0]
```


版权所有 © 深圳史河机器人科技有限公司

```
if __name__ == "__main__":
    if len(sys.argv) == 3:
        x = int(sys.argv[1])
        y = int(sys.argv[2])
    else:
        print(usage())
        sys.exit(1)
    print("Requesting %s+%s"%(x, y))
    print("%s + %s = %s"%(x, y, add_two_ints_client(x, y)))
```

给节点执行权限：

```
chmod +x scripts/add_two_ints_client.py
```

然后，在你的 CMakeLists.txt 中编辑 catkin_install_python()调用：

```
catkin_install_python(PROGRAMS scripts/add_two_ints_server.py
scripts/add_two_ints_client.py
  DESTINATION {CATKIN_PACKAGE_BIN_DESTINATION})
```

4.2.1 代码分析

```
rospy.wait_for_service('add_two_ints')
```

对于客户端来说不需要调用 init_node()。

```
add_two_ints = rospy.ServiceProxy('add_two_ints', AddTwoInts)
```

为服务的调用创建了一个句柄（handle）

```
resp1 = add_two_ints(x, y)
return resp1.sum
```

然后我们可以使用这个句柄，就像普通的函数一样调用它。

4.3 构建节点

切换当前目录到你的 catkin 工作空间，然后运行 catkin_make：

```
cd ~/catkin_ws
catkin_make
```


版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~/catkin_ws/src/beginner_tutorials$ cd ~/catkin_ws
ding@dingding:~/catkin_ws$ catkin_make
Base path: /home/ding/catkin_ws
Source space: /home/ding/catkin_ws/src
Build space: /home/ding/catkin_ws/build
Devel space: /home/ding/catkin_ws/devel
Install space: /home/ding/catkin_ws/install
####
#### Running command: "make cmake_check_build_system" in "/home/ding/catkin_ws/b
uild"
####
-- Using CATKIN_DEVEL_PREFIX: /home/ding/catkin_ws/devel
-- Using CMAKE_PREFIX_PATH: /home/ding/catkin_ws/devel;/opt/ros/melodic
-- This workspace overlays: /home/ding/catkin_ws/devel;/opt/ros/melodic
-- Found PythonInterp: /usr/bin/python2 (found suitable version "2.7.17", minimu
m required is "2")
-- Using PYTHON_EXECUTABLE: /usr/bin/python2
-- Using Debian Python package layout
-- Using empy: /usr/bin/empy
-- Using CATKIN_ENABLE_TESTING: ON
-- Call enable_testing()
-- Using CATKIN_TEST_RESULTS_DIR: /home/ding/catkin_ws/build/test_results
-- Found gtest sources under '/usr/src/googletest': gtests will be built
-- Found gmock sources under '/usr/src/googletest': gmock will be built
-- Found PythonInterp: /usr/bin/python2 (found version "2.7.17")
-- Using Python nosetests: /usr/bin/nosetests-2.7
-- catkin 0.7.29
-- BUILD_SHARED_LIBS is on
-- BUILD_SHARED_LIBS is on
-- ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
-- ~~  traversing 1 packages in topological order:
-- ~~  - beginner_tutorials
```

4.4 检验简单的服务和客户端

4.4.1 运行服务

先运行服务：

```
rosrun beginner_tutorials add_two_ints_server.py
```

```
ding@dingding:~/catkin_ws$ rosrun beginner_tutorials add_two_ints_server.py
Ready to add two ints.
```

4.4.2 运行客户端

运行客户端，并附带必要的参数：

```
rosrun beginner_tutorials add_two_ints_client.py 2 5
```

```
ding@dingding:~/catkin_ws$ rosrun beginner_tutorials add_two_ints_client.py 2 5
Requesting 2+5
2 + 5 = 7
```

五、课堂作业

1、使用 python 语言编写一个服务节点。

2、使用 python 语言编写一个客户端节点。

3、尝试创建简单的服务（Service）节点 reduce_two_ints_server，该节点将接收两个整数，并返回它们的差。

版权所有 © 深圳史河机器人科技有限公司

实验 14 录制和回放数据

一、实验目的

1、学会如何将正在运行的 ROS 系统中的数据记录到一个 bag 文件中；

2、学会如何回放录制的数据。

二、实验设备

Ubuntu18.04 操作系统；ROS melodic；turtlesim_node 节点；turtle_teleop_key 节点

三、实验步骤

3.1 录制数据（创建 bag 文件）

在不同终端中分别执行以下指令：

```
roscore
rosrun turtlesim turtlesim_node
rosrun turtlesim turtle_teleop_key
```

此时，你可以通过键盘上的方向键来控制乌龟的运动。

3.1.1 录制所发布的话题

我们先打开一个新终端，执行以下指令，检查当前系统中发布的话题：

```
rostopic list –v
```

```
ding@dingding:~$ rostopic list -v

Published topics:
 * /turtle1/color_sensor [turtlesim/Color] 1 publisher
 * /turtle1/cmd_vel [geometry_msgs/Twist] 1 publisher
 * /rosout [rosgraph_msgs/Log] 2 publishers
 * /rosout_agg [rosgraph_msgs/Log] 1 publisher
 * /turtle1/pose [turtlesim/Pose] 1 publisher

Subscribed topics:
 * /turtle1/cmd_vel [geometry_msgs/Twist] 1 subscriber
 * /rosout [rosgraph_msgs/Log] 1 subscriber
```

已发布主题的列表是唯一可能被记录在数据日志文件中的消息类型，因为只有发布的消息才能被录制。由 teleop_turtle 发布的/turtle1/cmd_vel 话题是指令消息，作为 turtlesim 进程的输入。消息/turtle1/color_sensor 和/turtle1/pose 是 turtlesim

版权所有 © 深圳史河机器人科技有限公司

发布的输出消息。

现在我们将记录发布的数据。打开一个新终端，输入以下指令：

```
mkdir ~/bagfiles
cd ~/bagfiles
rosbag record –a      //-a 表明所有发布的话题都积累在一个 bag 文件中。
```

```
ding@dingding:~$ mkdir ~/bagfiles
ding@dingding:~$ cd ~/bagfiles
ding@dingding:~/bagfiles$ rosbag record -a
[ INFO] [1639639894.211775337]: Subscribing to /turtle1/color_sensor
[ INFO] [1639639894.213299831]: Recording to '2021-12-16-15-31-34.bag'.
[ INFO] [1639639894.215960444]: Subscribing to /turtle1/cmd_vel
[ INFO] [1639639894.220033405]: Subscribing to /rosout
[ INFO] [1639639894.227901080]: Subscribing to /rosout_agg
[ INFO] [1639639894.230380284]: Subscribing to /turtle1/pose
```

然后我们用键盘控制乌龟运动一段时间，在运行 rosbag record 的窗口中按 Ctrl+C 以退出。现在查看~/bagfiles 目录中的内容，你应该会看到一个以年份、日期和时间开头且扩展名是.bag 的文件。这就是传说中的袋文件，它包含 rosbag record 运行期间由任何节点发布的所有话题。

3.2 检查并回放 bag 文件

现在我们已经使用 rosbag record 命令录制了一个 bag 文件，接下来我们可以使用 rosbag info 查看它的内容，或用 rosbag play 命令回放。首先我们来看看袋子里记录了什么。我们可以执行 info 命令检查 bag 文件的内容而不回放它。在 bag 文件所在的目录下执行以下命令：

```
rosbag info <your bagfile>
```


版权所有 © 深圳史河机器人科技有限公司

```
^Cding@dingding:~/bagfiles$ rosbag info 2021-12-16-15-31-34.bag
path:        2021-12-16-15-31-34.bag
version:     2.0
duration:    6:59s (419s)
start:       Dec 16 2021 15:31:34.23 (1639639894.23)
end:         Dec 16 2021 15:38:33.69 (1639640313.69)
size:        3.5 MB
messages:    52788
compression: none [4/4 chunks]
types:       geometry_msgs/Twist [9f195f881246fdfa2798d1d3eebca84a]
             rosgraph_msgs/Log   [acffd30cd6b6de30f120938c17c593fb]
             turtlesim/Color     [353891e354491c51aabe32df673fb446]
             turtlesim/Pose      [863b248d5016ca62ea2e895ae5265cf9]
topics:      /rosout                  4 msgs    : rosgraph_msgs/Log   (2 connections)
             /turtle1/cmd_vel       383 msgs    : geometry_msgs/Twist
             /turtle1/color_sensor  26201 msgs  : turtlesim/Color
             /turtle1/pose          26200 msgs  : turtlesim/Pose
```

这些信息告诉你 bag 文件中所包含话题的名称、类型和消息数量。我们可以看到，在之前使用 rostopic 命令查看到的五个已公告的话题中，其实只有四个在我们录制期间发布了消息。因为我们带-a 参数选项运行 rosbag record 命令时系统会录制下所有节点发布的所有消息。

然后我们将使用 rosbag play 回放 bag 文件以再现系统运行过程。首先用 Ctrl+C 关闭运行的 turtle_teleop_key，但让 turtlesim 继续运行。在~/bagfiles 下执行以下指令：

```
rosbag play <your bag file>
```

```
ding@dingding:~/bagfiles$ rosbag play 2021-12-16-15-31-34.bag
[ INFO] [1639641395.399435046]: Opening 2021-12-16-15-31-34.bag

Waiting 0.2 seconds after advertising topics... done.

Hit space to toggle paused, or 's' to step.
 [RUNNING]  Bag Time: 1639639894.232971   Duration: 0.000000 / 419.458454
 [RUNNING]  Bag Time: 1639639894.236374   Duration: 0.003402 / 419.458454
 [RUNNING]  Bag Time: 1639639894.336516   Duration: 0.103545 / 419.458454
```

默认模式下，rosbag play 命令在公告每条消息后会等待一段时间（0.2 秒）才真正开始发布 bag 文件中的内容，等待时间可以通过-d 选项来指定。你可以通过-s 参数选项让 rosbag play 不从 bag 文件的开头开始，而是从某个指定的时间开始。最后一个可能比较有趣的参数选项是-r 选项，它允许你通过设定一个参数来改变消息发布速率。

3.3 录制数据子集

当运行一个复杂的系统时，会有几百个话题被发布，有些话题亦会发布大量数据（比如包含摄像头图像流的话题）。如此一来，将包含所有话题的日志写入一个 bag 文件到磁盘通常是不切实际的。rosbag record 命令支持只录制特定的话题到 bag 文件中，这样我们就可以只录制自己想要的话题。

我们重新打开 turtlesim 和键盘控制节点：

```
rosrun turtlesim turtlesim_node
```


版权所有 © 深圳史河机器人科技有限公司

```
rosrun turtlesim turtle_teleop_key
```

在~/bagfiles 目录下执行以下命令：

```
rosbag record /turtle1/cmd_vel
```

上述命令中的 topic 参数告诉 rosbag record 只能订阅这这个指定的话题。然后通过键盘控制乌龟随意移动几秒钟，最后按 Ctrl+C 退出 rosbag record 命令。

现在使用 `rosbag info <your bag file>.bag` 查看 bag 文件中的内容，你应该会看到如下类似信息，里面只包含指定的话题（/turtle1/cmd_vel）：

```
ding@dingding:~/bagfiles$ rosbag info 2021-12-16-21-37-19.bag
path:        2021-12-16-21-37-19.bag
version:     2.0
duration:    17.9s
start:       Dec 16 2021 21:37:23.90 (1639661843.90)
end:         Dec 16 2021 21:37:41.78 (1639661861.78)
size:        793.2 KB
messages:    7606
compression: none [1/1 chunks]
types:       geometry_msgs/Twist [9f195f881246fdfa2798d1d3eebca84a]
topics:      /turtle1/cmd_vel   7606 msgs    : geometry_msgs/Twist
```

四、课堂作业

1、思考为什么在回放 bag 文件时，乌龟的路径没有完全地映射到原先通过键盘控制时产生的路径上。

2、尝试自己录制一个只有/turtle1/color_sensor 话题的 bag 文件。

版权所有 © 深圳史河机器人科技有限公司

实验 15 从 bag 文件中读取消息

一、实验目的

1、学会如何让从 bag 文件中读取消息；

2、熟练掌握 rostopic echo、rosbag play 等命令行工具的用法。

二、实验设备

Ubuntu18.04 操作系统；ROS melodic；一个 bag 文件

三、实验步骤

3.1 立即回放消息并在多个终端中查看输出

要进行这个实验，你需要一个 bag 文件，我们去下载一个示例文件，然而你也可以使用上个实验中自己录制的 bag 文件。

```
wget https://open-source-webviz-ui.s3.amazonaws.com/demo.bag
```

实验前要运行必须的 ROS 节点：

```
roscore
```

1）你需要知道你想要从 bag 文件中读取的正确的话题名，rosbag info 可以检查你的 bag 文件里都有哪些已经发布的话题。

```
rosbag info demo.bag
```

```
ding@dingding:~/bagfiles$ rosbag info demo.bag
path:        demo.bag
version:     2.0
duration:    20.0s
start:       Mar 22 2017 10:37:58.00 (1490150278.00)
end:         Mar 22 2017 10:38:17.00 (1490150298.00)
size:        696.2 MB
messages:    5390
compression: none [600/600 chunks]
types:       bond/Status                          [eacc84bf5d65b6777d4c50f463dfb9c8]
             diagnostic_msgs/DiagnosticArray      [60810da900de1dd6ddd437c3503511da]
             diagnostic_msgs/DiagnosticStatus     [d0ce08bc6e5ba34c7754f563a9cabaf1]
             nav_msgs/Odometry                    [cd5e73d190d741a2f92e81eda573aca7]
             radar_driver/RadarTracks             [6a2de2f790cb8bb0e149d45d297462f8]
             sensor_msgs/Image                    [060021388200f6f0f447d0fcd9c64743]
             sensor_msgs/NavSatFix                [2d3a8cd499b9b4a0249fb98fd05cfa48]
             sensor_msgs/PointCloud2              [1158d486dd51d683ce2f1be655c3c181]
             sensor_msgs/Range                    [c005c34273dc426c67a020a87bc24148]
             sensor_msgs/TimeReference            [fded64a0265108ba86c3d38fb11c0c16]
             tf2_msgs/TFMessage                   [94810edda583a504dfda3829e70d7eec]
             velodyne_msgs/VelodyneScan           [50804fc9533a0e579e6322c04ae70566]
topics:      /diagnostics                          140 msgs    : diagnostic_msgs/DiagnosticArray
             /diagnostics_agg                       40 msgs    : diagnostic_msgs/DiagnosticArray
             /diagnostics_toplevel_state            40 msgs    : diagnostic_msgs/DiagnosticStatus
             /gps/fix                              146 msgs    : sensor_msgs/NavSatFix
             /gps/rtkfix                           200 msgs    : nav_msgs/Odometry
             /gps/time                             192 msgs    : sensor_msgs/TimeReference
             /image_raw                            600 msgs    : sensor_msgs/Image
             /obs1/gps/fix                          30 msgs    : sensor_msgs/NavSatFix
             /obs1/gps/rtkfix                      200 msgs    : nav_msgs/Odometry
             /obs1/gps/time                        136 msgs    : sensor_msgs/TimeReference
             /radar/points                         400 msgs    : sensor_msgs/PointCloud2
             /radar/range                          400 msgs    : sensor_msgs/Range
             /radar/tracks                         400 msgs    : radar_driver/RadarTracks
             /tf                                  1986 msgs    : tf2_msgs/TFMessage
             /velodyne_nodelet_manager/bond         80 msgs    : bond/Status
             /velodyne_packets                     200 msgs    : velodyne_msgs/VelodyneScan
             /velodyne_points                      200 msgs    : sensor_msgs/PointCloud2
```

可以看到，有 30 条消息发布在/obs1/gps/fix 话题上，有 40 条消息发布在/diagnostics_agg 话题上。我们把其中的/diagnostics_agg 提取出来。

版权所有 © 深圳史河机器人科技有限公司

2）在新的终端里订阅/diagnostics_agg 话题并复读该话题上发布的所有内容，同时用 tee 命令转存在一个 yaml 格式的文件中方便查看：

```
rostopic echo /diagnostics_agg | tee topic2.yaml
```

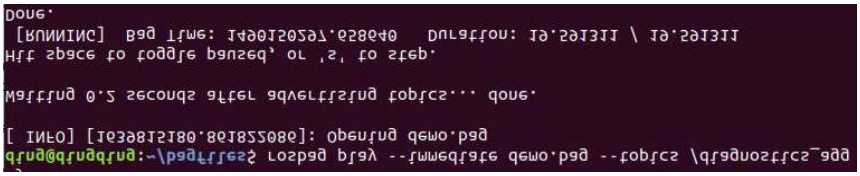
```
ding@dingding:~/bagfiles$ rostopic echo /diagnostics_agg | tee topic2.yaml
WARNING: topic [/diagnostics_agg] does not appear to be published yet
header:
  seq: 1839
  stamp:
    secs: 1490150278
    nsecs:  40781979
  frame_id: ''
status:
  -
    level: 0
    name: "/GPS"
    message: "OK"
    hardware_id: ''
    values:
      -
        key: "piksi_llh_diag: Frequency Status"
        value: "Desired frequency met"
      -
        key: "piksi_rtk_diag: Frequency Status"
        value: "Desired frequency met"
      -
```

3）打开另一个新终端来回放 bag 文件。为了尽可能快地回放 bag 文件（使用--immediate 选项），只会发布我们感兴趣的话题。用法如下：

```
rosbag play --immediate demo.bag --topics /topic1 /topic2 /topic3 /topicN
```

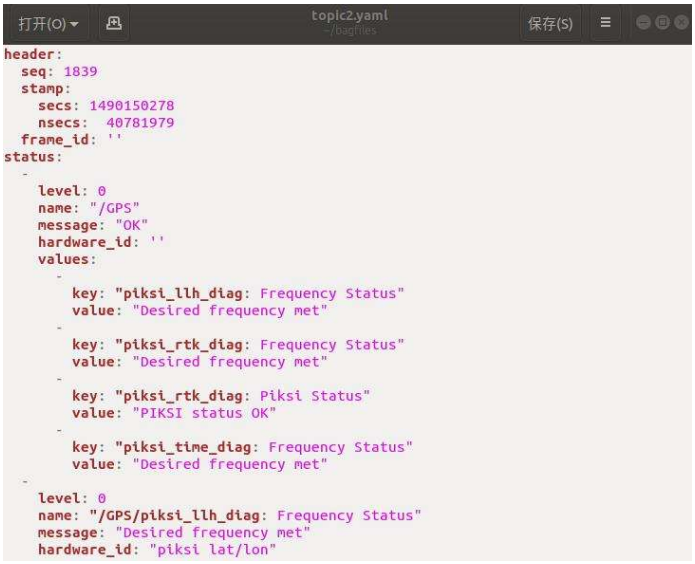
在本例中，具体指令如下：

```
rosbag play --immediate demo.bag --topics /diagnostics_agg
```



4）查看 topic2.yaml 文件

现在查看你的终端，它订阅了一个话题，这个话题类型的所有消息用 yaml 格式输出，每条消息之间用---分割。



版权所有 © 深圳史河机器人科技有限公司

四、课堂作业

1、尝试订阅 demo.bag 文件中的/obs1/gps/fix 话题，并在终端中回放它。

版权所有 © 深圳史河机器人科技有限公司

实验 16　Roswtf 的学习

一、实验目的

学会 Roswtf 的使用方法。

二、实验设备

Ubuntu18.04 操作系统；ROS melodic

三、实验原理

roswtf 是一个用于诊断运行中的 ROS 系统问题的工具。可以把它看作是一个用代码实现的 FAQ。

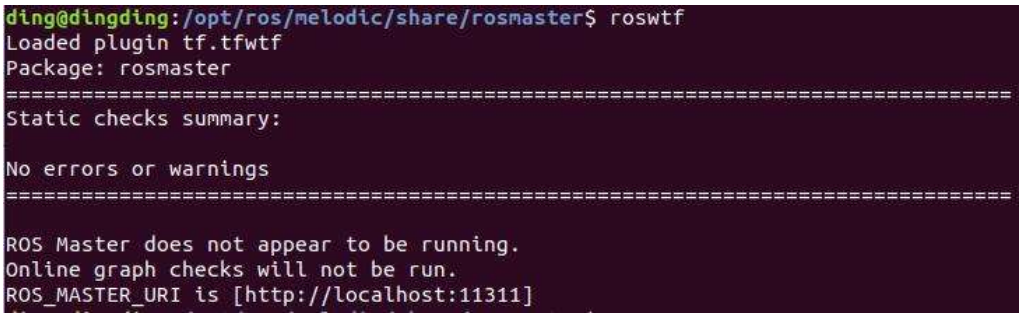
roswtf 将检查你的 ROS 设置，例如你的环境变量，并寻找配置问题。如果你有一个在线的 ROS 系统，它将查看它并检查任何潜在的问题。

四、实验步骤

4.1 安装检查

roswtf 可以检查你的系统并且尝试发现问题，确保 roscore 没有在运行时，执行以下指令：

```
roscd rosmaster
roswtf
```



• Package：rosmaster：roswtf 使用当前目录中的任何内容来确定其执行的检查。这个输出告诉我们是在包 rosmaster 的目录中启动了 roswtf。

• Static checks summary：它会报告任何关于文件系统或未运行时（比如无需 roscore 的场景）的问题。此时显示我们没有错误。

• ROS Master does not appear to be running：roscore 未在运行。roswtf 不会做任何 ROS 在线检查。

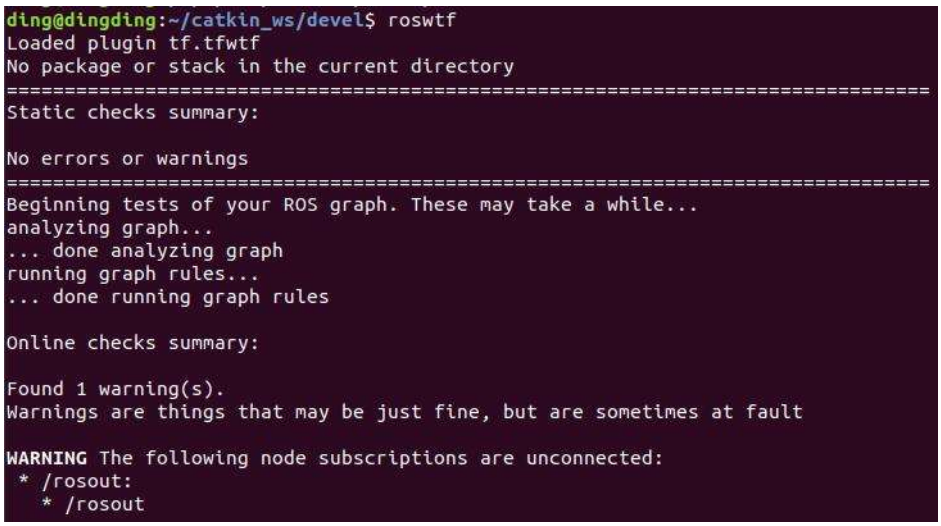
版权所有 © 深圳史河机器人科技有限公司

4.2 在线检查

4.2.1 警告

在做在线检查时，我们需要启动一个 Master，所以要在新终端启动 roscore，然后继续执行以下指令：

```
roscd
roswtf
```



这一次，roscd 在没有参数的情况下运行，这可能会带到一个没有 ROS 包的目录，因此我们看到了一条消息：No package or stack in context。

既然 roscore 已经运行了所以 roswtf 做了一些运行时检查。检查过程的长短取决于正在运行的 ROS 节点数量，可能会花费很长时间才能完成。正如你看到的，这一次出现了警告：

roswtf 发出警告，rosout 节点订阅了一个话题，该话题没有节点向它发送消息。在本例中，这正是所期望看到的，因为除了 roscore 没有任何其它节点在运行，所以我们可以忽略该警告。

4.2.2 错误

roswtf 会对一些系统中看起来异常但可能是正常的运行情况发出警告。也会对确实有问题的情况报告错误。

接下来我们给 ROS_PACKAGE_PATH 环境变量设置一个 *bad* 值，并退出 roscore 以简化检查输出信息。

版权所有 © 深圳史河机器人科技有限公司

```
ding@dingding:~/catkin_ws/devel$ ROS_PACKAGE_PATH=bad:$ROS_PACKAGE_PATH roswtf
Loaded plugin tf.tfwtf
No package or stack in the current directory
================================================================================
Static checks summary:

Found 1 error(s).

ERROR Not all paths in ROS_PACKAGE_PATH [bad:/home/ding/catkin_ws/src:/opt/ros/m
elodic/share] point to an existing directory:
 * bad

================================================================================

ROS Master does not appear to be running.
Online graph checks will not be run.
ROS_MASTER_URI is [http://localhost:11311]
```

正如你看到的，roswtf 发现了一个有关 ROS_PACKAGE_PATH 设置的错误。roswtf 还可以发现很多其它类型的问题，感兴趣的的可以深入研究。

版权所有 © 深圳史河机器人科技有限公司

实验 17　rviz 可视化工具

一、实验目的

1、熟悉 rviz 的操作页面布局；
2、学会使用 rviz 各种显示屏。

二、实验设备

Ubuntu18.04 操作系统；ROS melodic；rviz

三、实验原理

在机器人控制过程中需要显示机器人的各种状态，例如机械臂的运动状态、受力状态等，ROS 称为机器人的操作系统，其强大的功能覆盖各个方面，ROS 提供的 rivz 可以实现各种数据的显示，同时其也提供二次开发的插件，可以打造个性化的机器人的控制界面。

3.1 RViz 安装与启动

安装 ROS 完整版时，rviz 会自动安装，如果没有安装，可以通过下面命令安装。

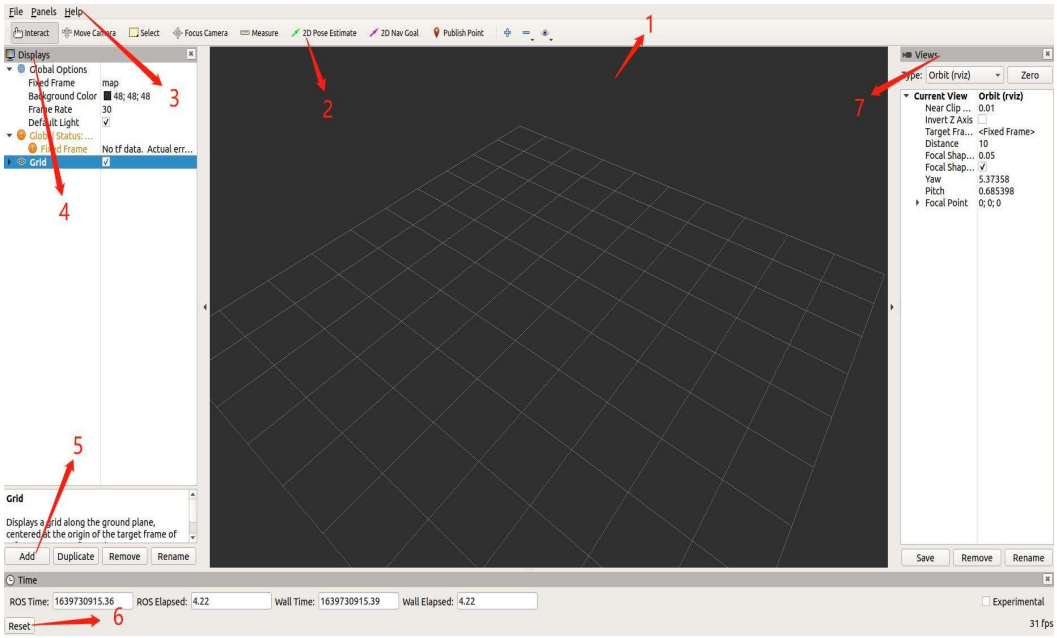
```
sudo apt-get install ros-kinetic-rviz
rosdep install rviz
rosmake rviz
```

启动之前要确保 roscore 在运行：

```
roscore
rosrun rviz rviz
```

RViz 主要页面布局如下：

版权所有 © 深圳史河机器人科技有限公司



1）3D 视图（3D view）：指屏幕的黑色部分。它是可以用三维方式查看各种数据的主屏幕。3D 视图的背景颜 色、固定框架、网格等可以在左侧显示的全局选项（Global Options）和网格（Grid）项目中进行详细设置。

2）工具（Tools）：工具是位于菜单下方的按钮，允许用户用各种功能按键选择多种功能的工具，例如 Interact、Move Camera、Select，Focus Camera、Measure、2D Pose Estimate、2D Navigation Goal 以及 Publish Point 等。

3）菜单（Menu）：菜单位于顶部。用户可以选择保存或读取显示屏状态的命令，还可以选择各种面板。

4）显示屏（Displays）：左侧的显示屏是从各种话题当中选择用户所需的数据的视图的区域。

5）Add：目前有大约 30 种不同的显示屏可供选择，下图有详细介绍。

6）时间（Time）：显示当前时刻（wall time）、ROS Time 以及他们各自经过的时间。这主要用于仿真，如果需要重新启动，请点击底部的[Reset]按钮。

7）视图（Views）：设定三维视图的视点

• Orbit：以指定的视点（在这里称为 Focus）为中心旋转。这是默认情况下最常用的基本视图。

• FPS（第一人称）：显示第一人称视点所看到的画面。

• Third Person Follower：显示以第三人称的视点尾追特定目标的视图。

• Top Down Ortho：这是 Z 轴的视图，与其他视图不同，以直射视图显示，而非透视法。

• XYOrbit：类似于 Orbit 的默认值，但焦点固定在 Z 轴值为 0 的 XY 平面上

版权所有 © 深圳史河机器人科技有限公司

图标	名称	说明
	Axes	显示x、y、z轴
	Camera	从相机的角度创建一个新的渲染窗口，并将图像覆盖在窗口上。
	Fluid Pressure	显示流体的压力，如空气或水。
	Effort	显示施加在机器人的每个旋转关节的力。
	Depth Cloud	显示基于Depth map的点云。将从相机获取的彩色图像覆盖在“点”上面，这里的“点”是通常基于Depth Map和Colorl mage话题的传感器(如Kinect和Xtion)的距离值。
	Grid	显示二维或三维网格。
	Grid Cells	显示网格的每个单元格。主要用于显示导航的costmap中的障碍物。
	Group	这是一个对显示屏进行分组的容器。要使用的显示屏可以作为一个组进行管理。
	Illuminance	指示亮度
	Image	在新的渲染窗口中显示图像。与camera显示屏不同，它不覆盖相机。
	Laser Scan	显示激光扫描值
	Interactive Markers	显示一个或多个Interactive Marker。可以使用鼠标更改位置(x，y，z)和姿态(roll，pitch，yaw)。
	Map	Map将导航中使用的占用地图(occupancy map）显示在地平面(ground plane)上。
	Marker	显示RViz提供的箭头、圆圈、三角形、矩形和圆柱等标志。
	Path	显示导航中使用的机器人的路径

图标	名称	说明
	Marker Array	显示多个标记显示屏
	Odometry	将按照时间推移的测位(odometry)信息以箭头的形式显示。例如，将随着机器人的移动所产生的连续路径（机器人的位置和方向)，沿着时间间隔以箭头的形式显示。
	Point Cloud	显示点云（point cloud）数据显示点云(point cloud)数据。它用于表示Depth camera系列的传感器数据，如RealSense、Kinect和Xtion等。由于Point Cloud和Point Cloud2是有区别的。由于PointCloud2是遵循最新的PCL (Point Cloud Library)中使用的格式，因此一般情况下使用PointCloud2即可。
	Point Cloud2	
	Point Stamped	显示圆形点
	Polygon	显示一个线形的多边形轮廓。主要用于在二维平面上简单显示机器人的轮廓。
	Pose	显示三维姿态(pose，位置+方向)。姿态是一个箭头，箭头的起点是位置(x，y，z),箭头的方向是机器人的方向(roll，pitch，yaw)。例如,机器人模型的位置和方向可以用姿态表示，而导航中可以用目标(goal)点表示。
	Pose Array	显示多个姿态
	Range	用来可视化超声波或红外传感器的测量范围
	Relative Humidity	显示相对湿度
	Robot Model	显示机器人模型
	TF	显示ROS中使用的坐标转换TF。显示方法与上述轴(Axes)一样用xyz轴表示,各轴根据相对坐标用箭头表现分层结构。
	Temperature	显示温度
	Wrench Stamped	用“箭头（力）”和“箭头+圆形（转矩）”表示扭转（wrench）的动作

版权所有 © 深圳史河机器人科技有限公司

3.2 坐标框架（Coordinate Frames）

rviz 使用 tf 变换系统将数据从其到达的坐标系转换为全局参考帧。在可视化工具中有两个重要的坐标框架。

3.2.1 固定框架（The Fixed Frame）

必须取相对世界坐标系静止的物体建立坐标

两帧中更重要的是固定帧。固定帧是用于表示“世界”帧的参考帧。这通常是“地图”，或“世界”，或类似的东西，但也可以是，例如，你的 odometry 框架。

如果将固定框架错误地设置到例如机器人的基座，那么机器人所见过的所有物体将出现在机器人的前方，相对于检测到它们的机器人的位置。为了获得正确的结果，固定框架不应相对于世界移动。

3.2.2 目标框架（The Target Frame）

可取地图（视角为第三视角）；机器人坐标（视角为机器人第一视角）

目标帧是摄像机视图的参考帧。例如，如果您的目标框架是地图，您将看到机器人在地图上行驶。如果您的目标框架是机器人的基础，则机器人将保持在相同的位置，而其他所有内容都相对于它移动。

版权所有 © 深圳史河机器人科技有限公司

第三章 应用类实验

实验 18 BR280 安装与配置

一、实验目的

1、学习 BR280 单目相机和夹持器的安装。

2、了解 BR280 的一些基本情况。

3、学会查看 BR280 的 IP 地址、使用 PC 的远程桌面连接 Beetle 的工控机。

二、实验设备

BR280 机器人、显示器、鼠标、键盘、笔记本电脑、配套单目相机、配套夹持器、机器人遥控器。

三、实验步骤

3.1 摄像头和夹持器的安装

1、按下遥控器的 RB+START 键，让机械臂处于下图所示的位姿。

版权所有 © 深圳史河机器人科技有限公司

2、在机械臂法兰盘下图所示位置上插入两个紧固钉。

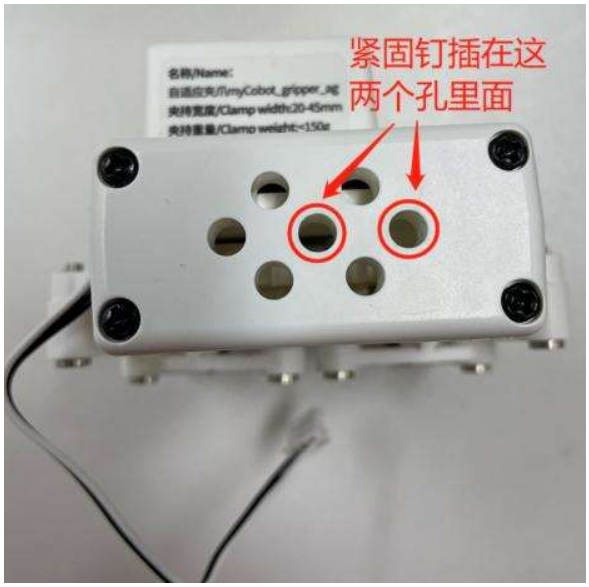
3、按下图所示位置在相机上插入两个固定钉。

4、按下图所示操作，将相机固定在机械臂的法兰盘上。

版权所有 © 深圳史河机器人科技有限公司

5、单目相机安装完毕。

6、将夹持器安装在相机上。



版权所有 © 深圳史河机器人科技有限公司

7、安装完成之后，按下遥控手柄上的 RB+A，让机械臂处于准备姿态，观察相机和夹持器是否安装正确。如果安装正确，则会处于下图所示姿态。

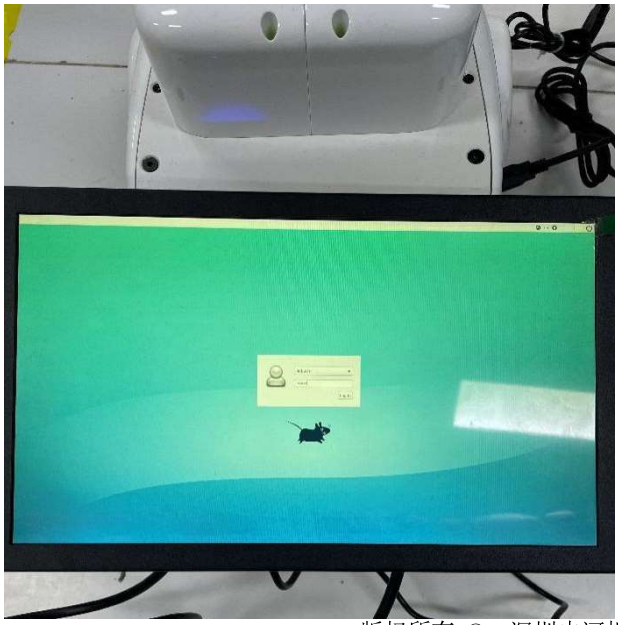
版权所有 © 深圳史河机器人科技有限公司

8、观察相机和夹持器安装无误之后，将夹持器的线连接到机械臂上。到此，相机和夹持器的安装工作就完成了。

3.2 网络配置和 IP 获取

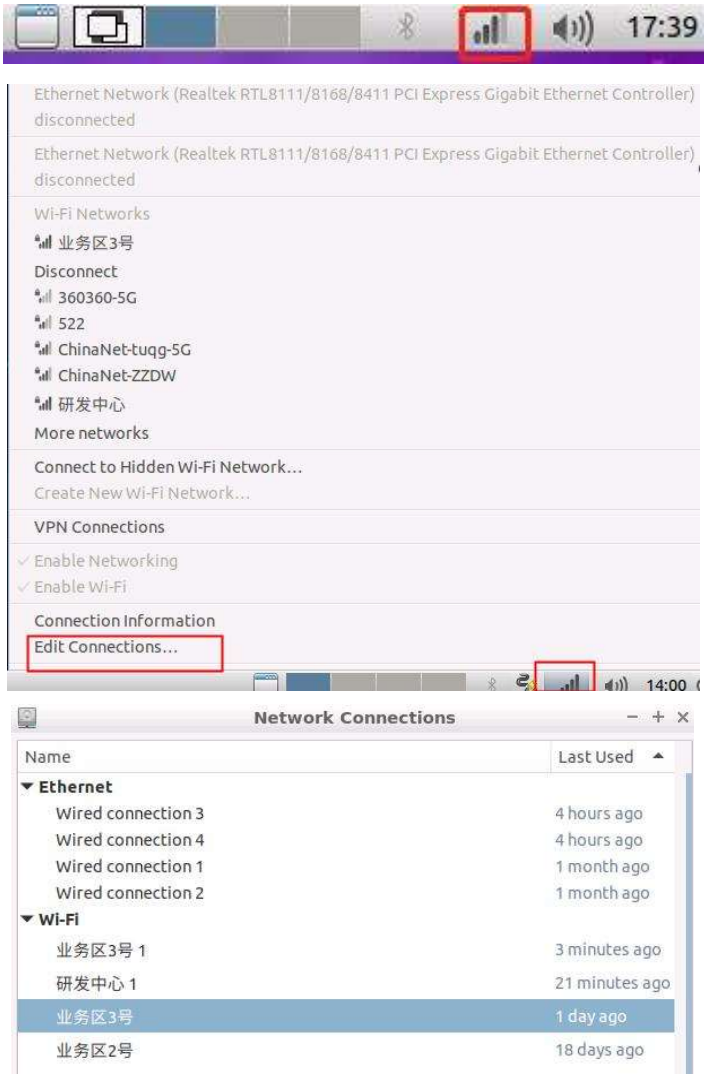
第一次使用 BR280 机器人时，需要配置网络和获取 IP 地址，这样是为了方便使用电脑的远程桌面连接到机器人的工控机。

1）将显示器、鼠标、键盘连接在机器人的主机上，输入密码：123456 登录账户。（配套显示器需先连接电源线开机，再连接 HDMI 线开启机器人使用）

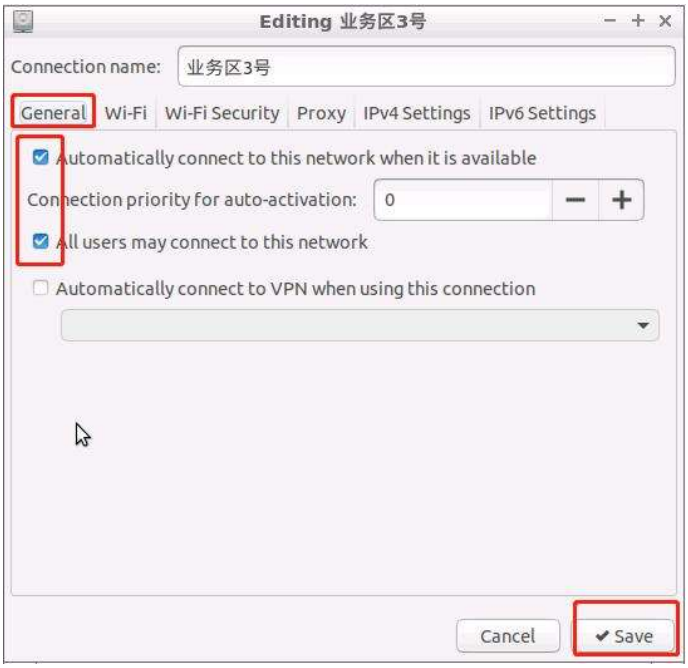


版权所有 © 深圳史河机器人科技有限公司

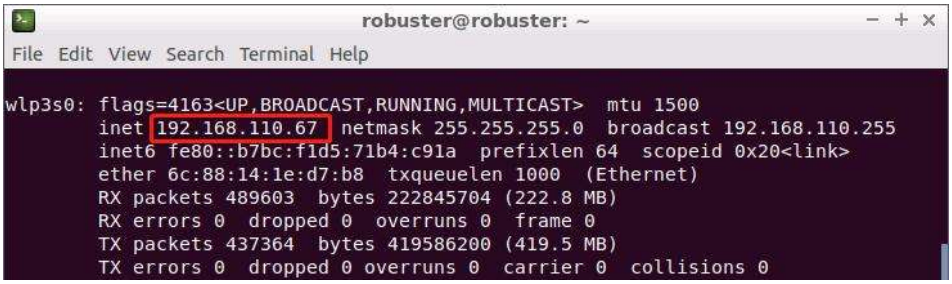
2）配置 WIFI 并设置开机自启动 WIFI。



版权所有 © 深圳史河机器人科技有限公司



3）使用 CTRL+ALT+T 打开终端，输入 ifconfig 查看 IP 地址，参照步骤 2 打开 WIFI 设置功能，并将查看到的 IP 地址设置成静态 IP 地址。



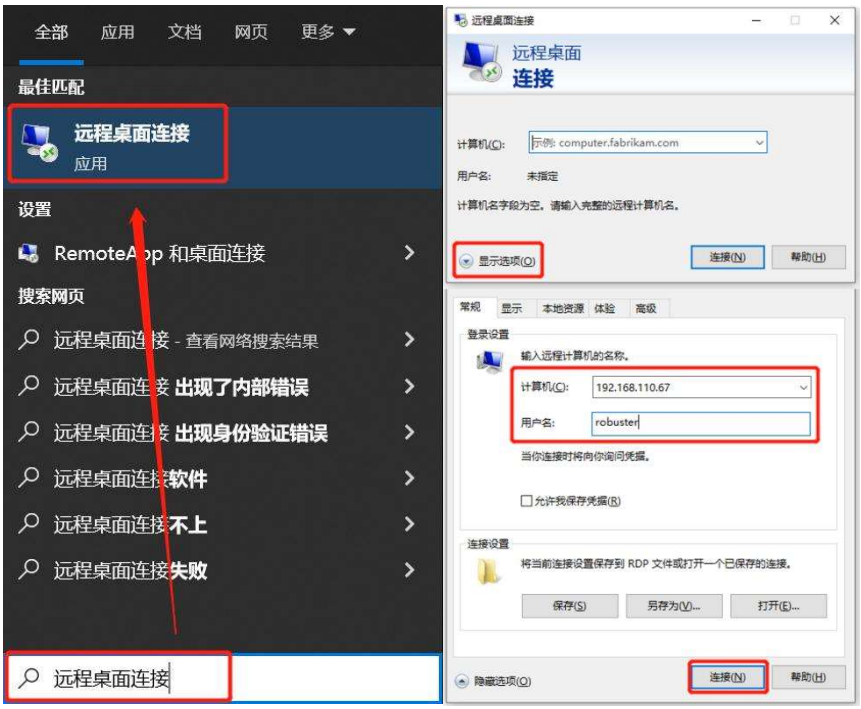
版权所有 © 深圳史河机器人科技有限公司

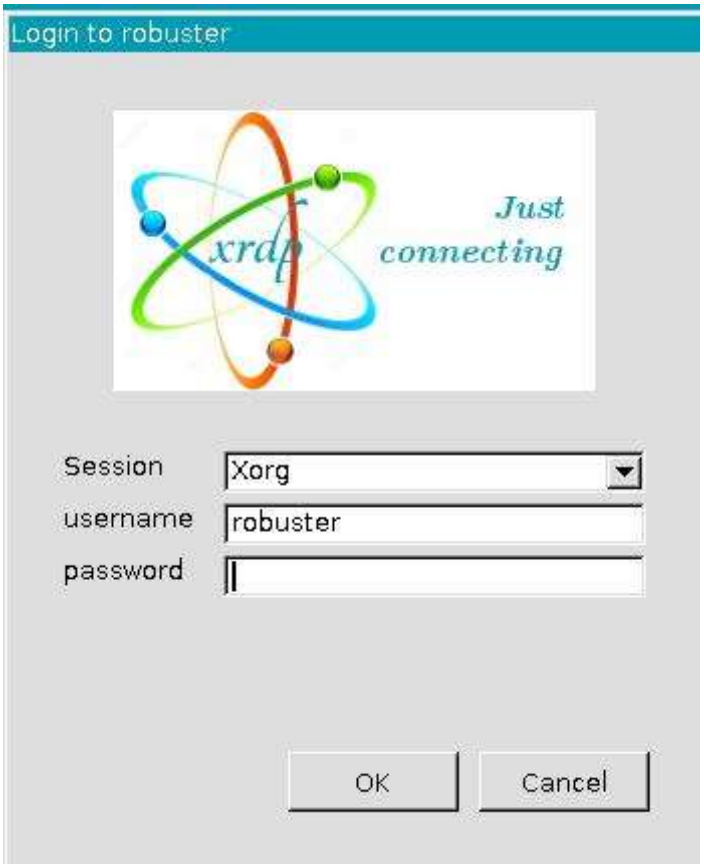
4）退出登录（注意：如果未退出登录，远程桌面则无法连接到机器人）。

3.3 PC 连接远程桌面

打开远程桌面，输入以下 IP 地址：192.168.110.67（每台 Beetle 的 IP 地址都不相同，需要自行获取），用户名：robuster；登录机器人，账号：robuster，密码：123456（连接之前，确保机器人已经开机）

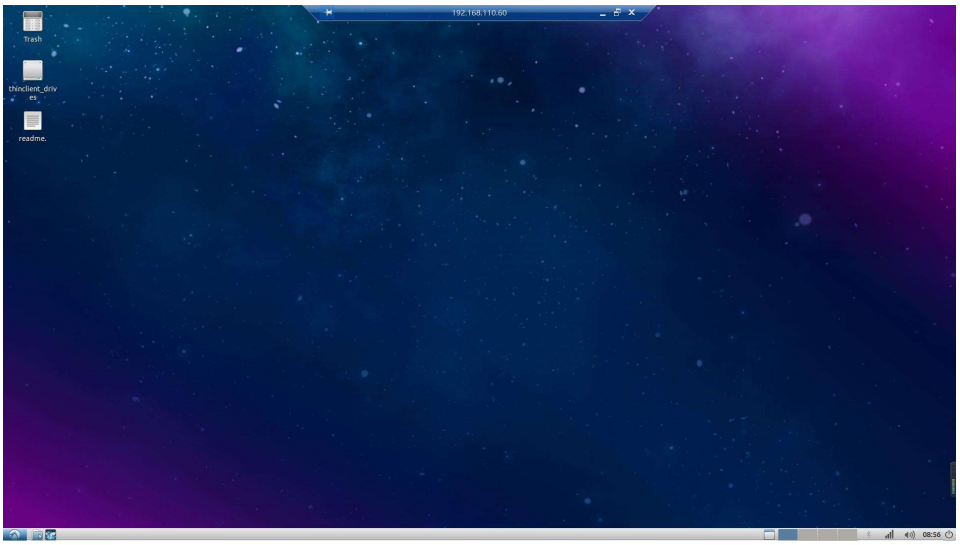
版权所有 © 深圳史河机器人科技有限公司



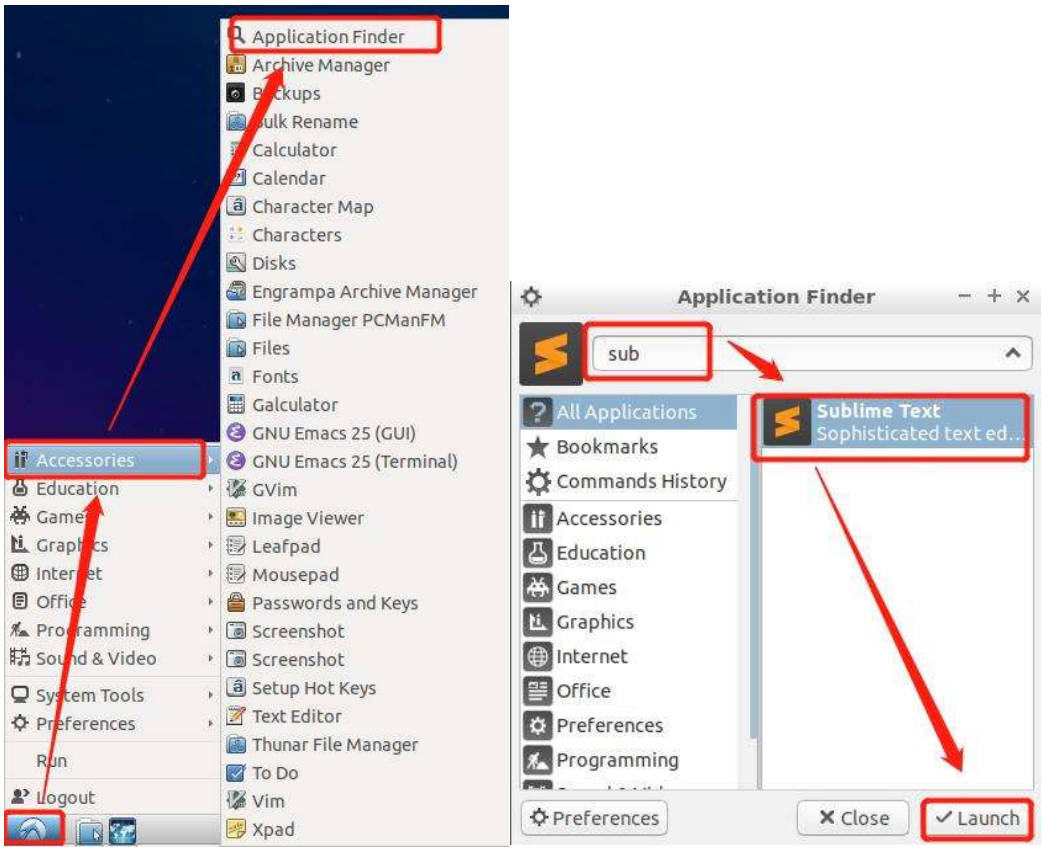


版权所有 © 深圳史河机器人科技有限公司

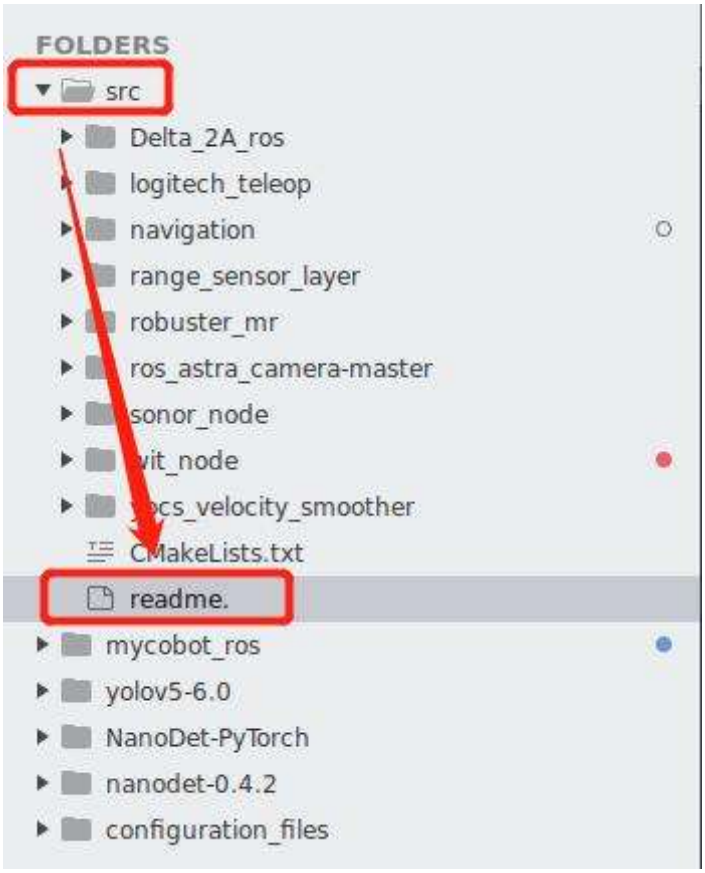
登录成功之后会进入工控机界面：



按下图所示操作找到 sublime text，里面有一个 readme 文件，路径：~/mr300_ws/src/readme，里面包含了一些基本的、今后实验中会用到的指令。



版权所有 © 深圳史河机器人科技有限公司



版权所有 © 深圳史河机器人科技有限公司

实验 19　BR280 基本运动控制

一、实验目的

1、熟悉 BR280 遥控器的按键功能；
2、学会使用遥控器控制 BR280 的运动；
3、学会灵活操作 BR280。

二、实验设备

BR280 移动机器人和配套遥控器

三、实验原理

配套的遥控器可以控制机器人的前进、后退与转弯；方向键可以控制前进后退、转向的最大速度；对应的按键可以控制机械臂完成抓取和放置工作。

四、实验步骤

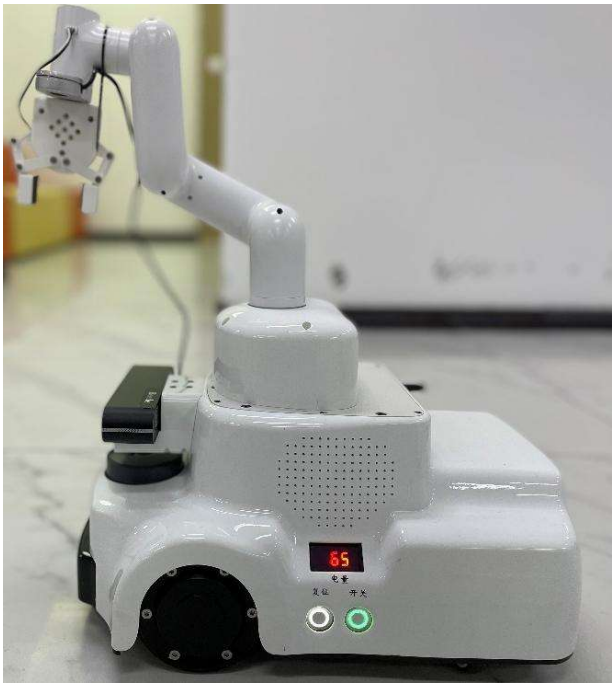
版权所有 © 深圳史河机器人科技有限公司

版权所有 © 深圳史河机器人科技有限公司

名称	操作	功能
D/X	拨动开关	X：遥控失效 D：遥控有效
RB	按键开关	长按遥控生效，不按遥控无效
BACK	按键开关	机械臂处于休息状态
START	按键开关	机械臂处于工作状态
A	按键开关	执行放置工作
B	按键开关	执行图片识别抓取
X	按键开关	进行二维码识别抓取
Y	按键开关	进行颜色识别抓取
遥杆 1	2 向摇杆	控制前进/后退 推杆位置控制速度
遥杆 2	2 向摇杆	控制左右转向 推杆位置控制速度
摇杆 3	4 向拨动开关	上：增加前后移动的限速值 下：减少前后移动的限速值 左：增加旋转的限速值 右：减少旋转的限速值
注：a.使用遥控器时请务必将开关 D/X 拨至 D,否则遥控器按键功能可能与设置功能不符。b.请勿更改遥控器参数设置，遥控器接收机在机器人内部。		

版权所有 © 深圳史河机器人科技有限公司

机械臂处于工作位置：



机械臂处于安全位置（防止掉电机械臂损坏）

五、课堂作业

1、熟练掌握遥控器每个按键的作用。

2、能够熟练地使用遥控器操作 BR280。

版权所有 © 深圳史河机器人科技有限公司

实验 20　BR280 传感器数据获取

一、实验目的

1、了解 BR280 机器人各个传感器的作用。

2、掌握获取每个传感器数据的方法，以及各种参数所代表的含义。

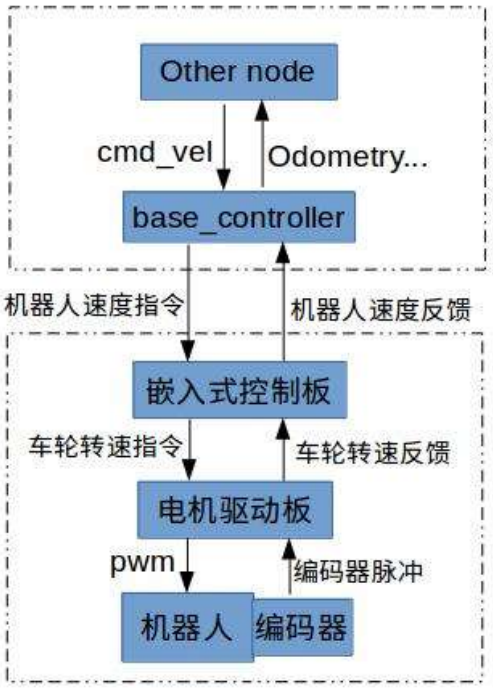
二、实验设备

BR280 机器人和配套遥控器

三、实验原理

1、里程计

里程计是衡量我们从初始位姿到终点位姿的一个标准，通俗的说，我们要实现机器人的定位与导航，就需要知道机器人行进了多少距离，是往哪个方向行进的。　整个移动机器人的控制结构如下图所示，其中 base_controller 节点将订阅的 cmd_vel 信息通过串口或其它通信接口发送给下位机（嵌入式控制板）。下位机中根据机器人运动学公式进行解算，将机器人速度转换为每个轮子的速度，然后通过 CAN 总线（或其它总线接口）将每个轮子的转速发送给电机驱动板控制电机转动。电机驱动板对电机转速进行闭环控制（PID 控制），并统计单位时间内接收到的编码器脉冲数，计算出轮子转速。



版权所有 ©　深圳史河机器人科技有限公司

2、**IMU**（惯性传感器）

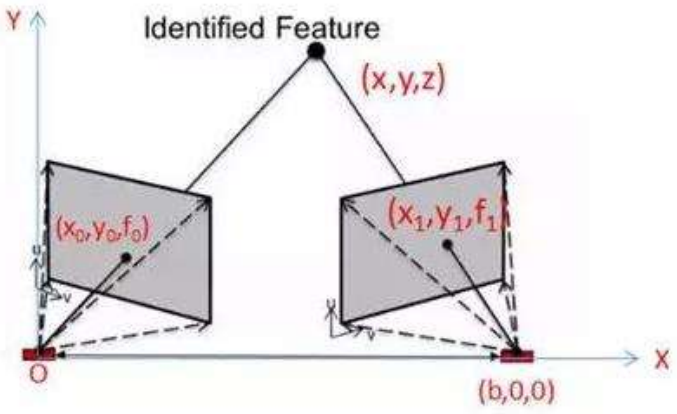
IMU 全称 Inertial Measurement Unit，惯性测量单元，主要用来检测和测量加速度与旋转运动的传感器。其原理是采用惯性定律实现的，最基础的惯性传感器包括加速度计和角速度计（陀螺仪），他们是惯性系统的核心部件，是影响惯性系统性能的主要因素。

3、**激光雷达**

激光雷达犹如人类的眼睛，是一种可以探测物体精确位置的传感器，其主要是通过对目标物发射激光信号，再根据从物体反射回来的信号时间差来计算这段距离，然后再发射激光的角度来确定物体和发射器的角度，从而得出物体与发射器的相对位置，对于移动机器人来说，激光雷达就相当于它的“眼睛”，通过不停扫描来获取二维空间的点阵数据，配合 SLAM 技术，可帮助机器人实现自主定位、地图构建及路径规划等功能。

4、**深度相机**

相比较传统的相机，深度相机在功能上添加了一个深度测量，从而更方便准确的感知周围的环境及变化。双目深度相机：被动式深度相机测量类似人类双眼布置，通过两个位置已标定好的 2D 相机观察同一环境，根据图像内容进行特征点匹配，进而计算深度。在 Beetle 中主要用来建图。



多角立体成像（MulTi-camera）（双目深度相机）

版权所有 © 深圳史河机器人科技有限公司

四、实验步骤

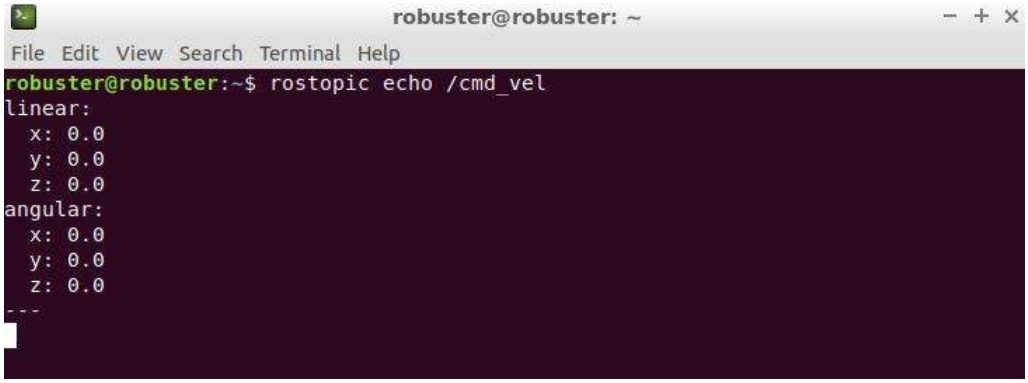
BR280 的几个主要传感器分布情况如下图所示。

1、打开远程桌面，登录机器人账号，进入工控机界面（具体步骤请参考实验 1），按 CTRL+ALT+T 打开终端，准备开始实验。

2、测试基本运动，输入以下指令，通过手柄遥控机器查看数据变化：

```
rostopic echo /cmd_vel
```

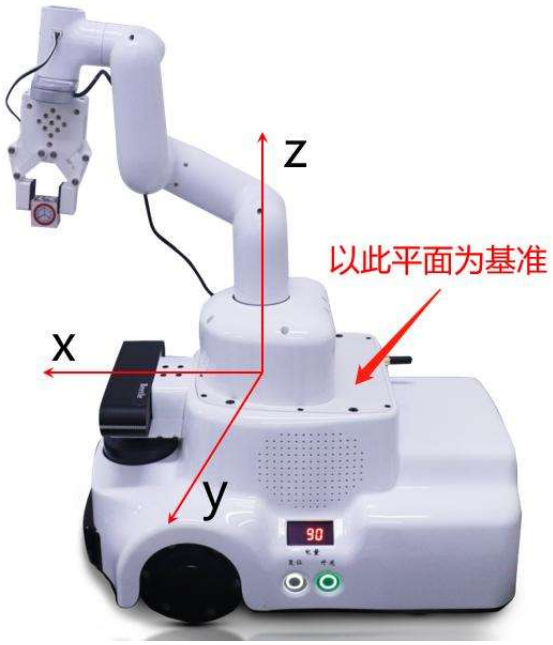
机器人没有移动时，我们可以发现数据全是 0。



机器人开始移动之后，我们可以发现线速度和角度值都发生了变化。Beetle 坐标系的建立遵循以下规则（右手定则）：

以开机时的位置为原点，以相机的朝向为 x 轴的正方向，y 轴垂直于 x 轴且与 x 轴处于同一水平面上，z 轴垂直与 xy 平面。

版权所有 © 深圳史河机器人科技有限公司



因而机器人的前进后退体现为 x 轴坐标的变化，机器人的转弯体现为 xy 平面绕 z 轴旋转。

```
---
linear:
  x: 0.25
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -0.418611943722
---
linear:
  x: 0.25
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -0.429470121861
---
linear:
  x: 0.25
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: -0.445741295815
```

3、测试里程计（通过编码器计算得到），输入以下指令，通过手柄遥控机器查看数据变化：

```
rostopic echo /odom
```


版权所有 © 深圳史河机器人科技有限公司

```
  seq: 14151
  stamp:
    secs: 1640240020
    nsecs: 368989912
  frame_id: "odom"
child_frame_id: "base_link"
pose:
  pose:
    position:
      x: 3.50640895307
      y: 0.989292407606
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: -0.549832447262
      w: 0.835274972652
  covariance: [0.001, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.001, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 10000000000.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 10000000000.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.0, 10000000000.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.001]
twist:
  twist:
    linear:
      x: 0.0
      y: 0.0
      z: 0.0
    angular:
      x: 0.0
      y: 0.0
      z: 0.0
  covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

在机器人运动过程中，我们会发现它的坐标一直在变化（机器人相对于原点的位置）。

4、测试 IMU（需要同时打开两个终端），输入以下指令，遥控机器人运动查看数据变化：

```
roslaunch wit_node wit.launch
rostopic echo /imu_data
```

```
^Crobuster@robuster:~$ roslaunch wit_node wit.launch
... logging to /home/robuster/.ros/log/29be23e4-63b4-11ec-818d-2f50f6433aaf/rosl
aunch-robuster-2984.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://robuster:43337/

SUMMARY
========

PARAMETERS
 * /rosdistro: melodic
 * /rosversion: 1.14.12
 * /use_sim_time: False
 * /wit/port: /dev/ttyS1

NODES
  /
    wit (nodelet/nodelet)
    wit_nodelet_manager (nodelet/nodelet)

ROS_MASTER_URI=http://localhost:11311

process[wit_nodelet_manager-1]: started with pid [3002]
process[wit-2]: started with pid [3003]
```

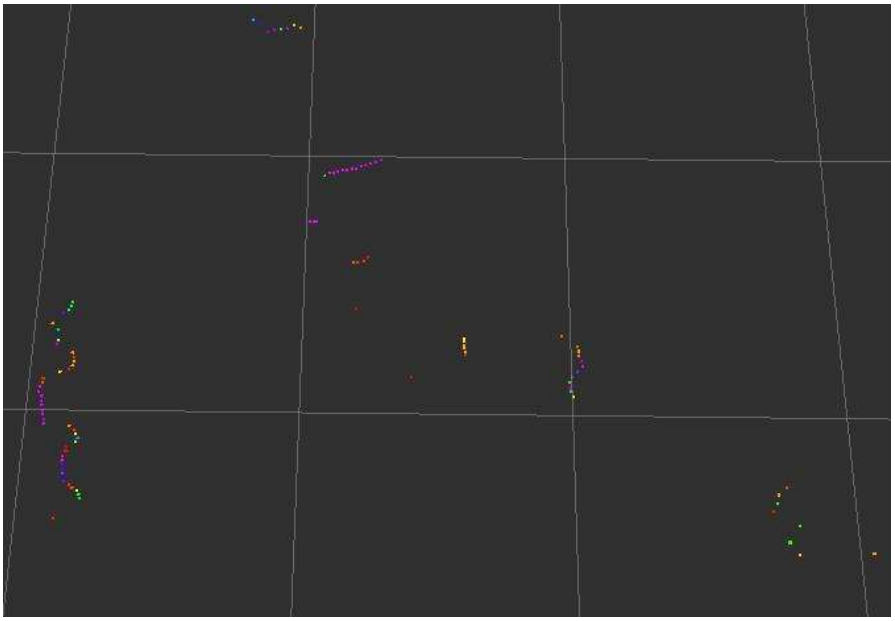

版权所有 © 深圳史河机器人科技有限公司

```
---
header:
  seq: 3169
  stamp:
    secs: 1640338470
    nsecs: 888818090
  frame_id: "imu_link"
orientation:
  x: -0.0104444722233
  y: 0.00621129402761
  z: -0.361634159342
  w: 0.932240884978
orientation_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
angular_velocity:
  x: 0.00106526441786
  y: -0.00213052883572
  z: 0.0234358171929
angular_velocity_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
linear_acceleration:
  x: 0.03828125
  y: -0.2583984375
  z: 9.8861328125
linear_acceleration_covariance: [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
---
```

我们可以清晰的看到，在机器人的移动过程中，它的线加速度和角速度一直在变化，他们就是通过 IMU 测量出来的。

5、测试雷达数据，输入以下指令，通过手柄遥控机器查看数据变化：

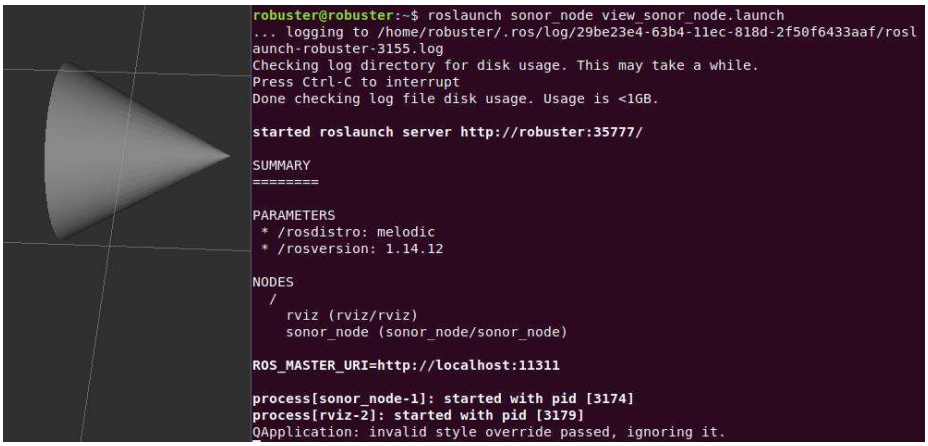
```
roslaunch delta_lidar view_delta_lidar.launch
```



6、测试超声波，输入以下指令，通过手柄遥控机器查看数据变化：

```
roslaunch sonor_node view_sonor_node.launch
```


版权所有 © 深圳史河机器人科技有限公司



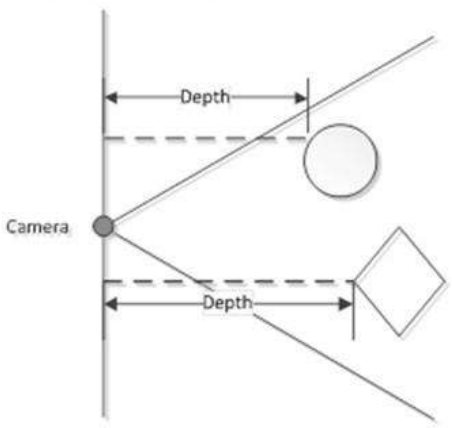
7、测试深度相机，输入以下指令（需要打开两个终端），通过手柄遥控机器查看数据变化：

```
roslaunch astra_camera astrapro.launch
rqt_image_view
```

选择深度图可以看到机器人拍摄的深度图像：

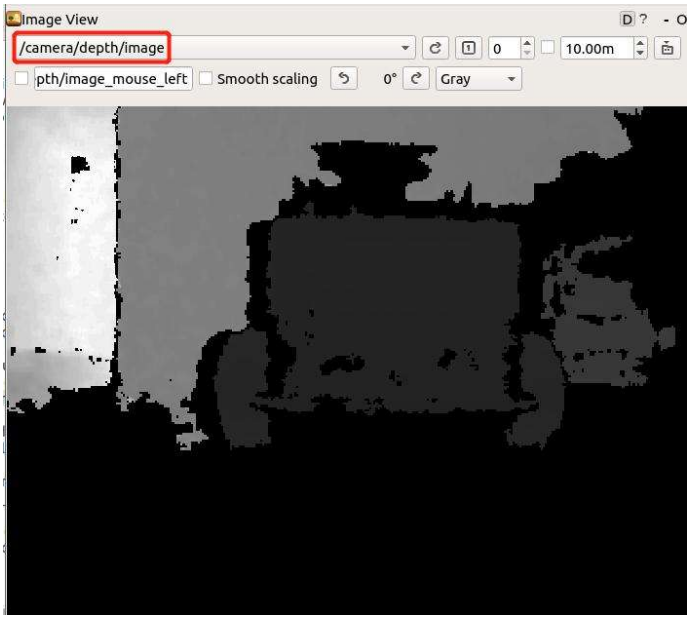
深度图像（depth image)也被称为距离影像（range image），是指将从图像采集器到场景中各点的距离（深度）作为像素值的图像，它直接反映了景物可见表面的几何形状。深度图像经过坐标转换可以计算为点云数据，有规则及必要信息的点云数据也可以反算为深度图像数据。

深度数据流所提供的图像帧中，每一个像素点代表的是在深度感应器的视野中，该特定的（x，y）坐标处物体到离摄像头平面最近的物体到该平面的距离（以毫米为单位）。

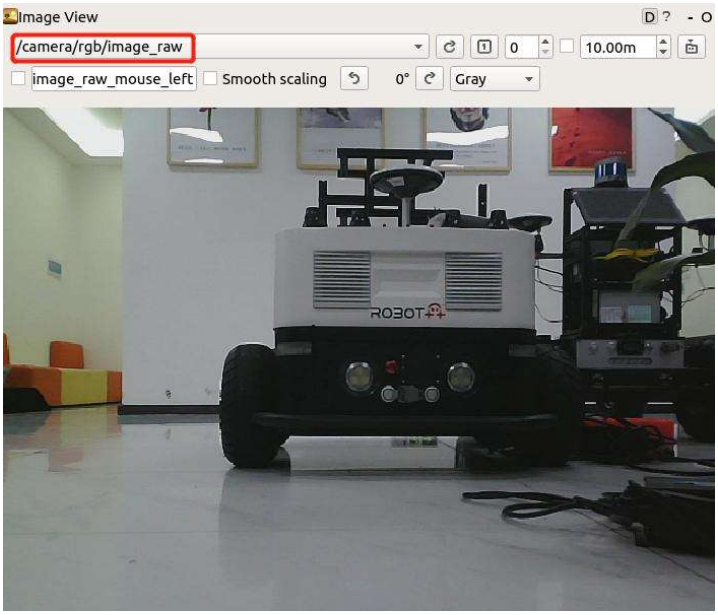


深度数据流值

版权所有 © 深圳史河机器人科技有限公司



选择 rgb 图像可以看到机器人拍摄的真实图像：



五、课堂作业

1、使用指令获取各种传感器的数据。

2、学会理解各类传感器数据代表的含义。

版权所有 © 深圳史河机器人科技有限公司

实验 21　BR280 定位与建图

一、实验目的

1、了解 SLAM 的理论基础。

2、了解 Cartographer 算法的理论基础。

3、学会使用 roslaunch 来进行地图构建。

二、实验设备

PC、BR280 机器人、配套遥控器。

三、实验原理

我们利用 SLAM+Cartographer 技术来实现 Beetle 机器人的地图构建任务：

通过调用激光雷达接口数据，对数据进行处理生成实时地图，采用一种性能损失最小的 SLAM 算法，将算法简化为距离计算与地图更新两个过程。

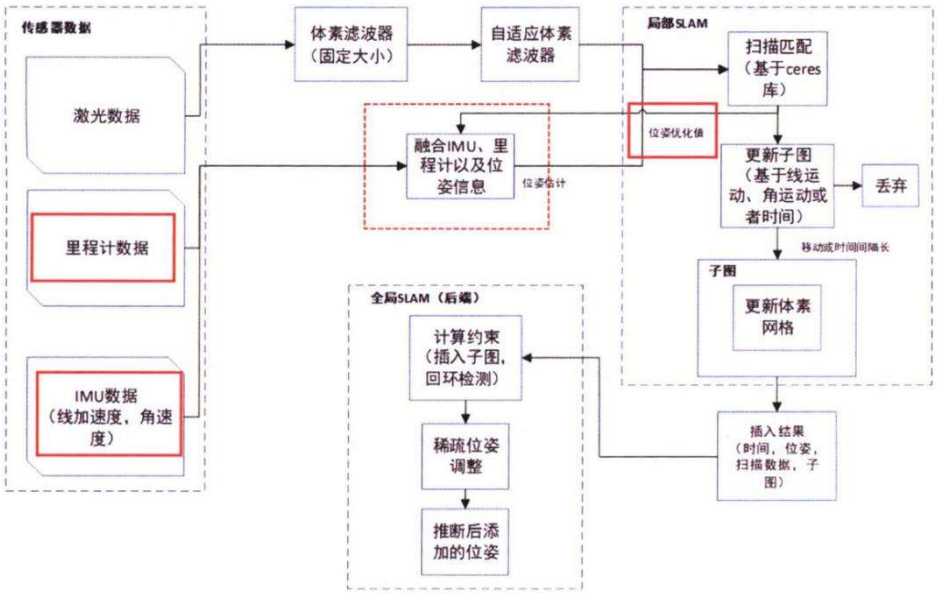
1、**SLAM**（同步定位与地图构建）

移动机器人地图构建就是建立机器人周围环境的空间模型，并指导机器人导航或定位。为了得到地图，机器人必须拥有外部传感器来感知外界环境。它们包含了机器人在每次进行观测时所处的位置信息。机器人仅仅依靠运动是无法确定位姿的，需要融合外部传感器的信息进行定位。地图构建过程依赖于精确的位姿信息，而机器人定位过程反过来又依赖于精确的地图信息，二者既矛盾又相关，必须同时加以考虑。同步定位与地图构建(SLAM)问题可以描述为：移动机器人在未知环境中运动时逐步构建周围环境的地图，同时运用此地图对机器人位置和姿态进行估计。

2、**Cartographer**

Cartographer 是谷歌在 2016 年开源的一个可以在多传感器配置下实现低计算资源消耗的 SLAM 算法框架。Cartographer 是一个实时的室内建图算法，能生成分辨率 r=5cm 的栅格地图。在前端将最新的激光雷达扫描数据在相邻的子图上(整个地图的一小块)完成扫描匹配，得到一个在短时间内准确的最佳插入位置(位姿)后，将扫描插入到子图中。扫描匹配中，位姿估计的误差会在整个地图中随时间逐渐累积，在后端中，通过回环检测加约束进行优化消除误差。下图为 Cartographer 算法框架图。整个框架分为数据源、局部 SLAM 以及全局 SLAM 以及数据融合等部分。

版权所有 © 深圳史河机器人科技有限公司



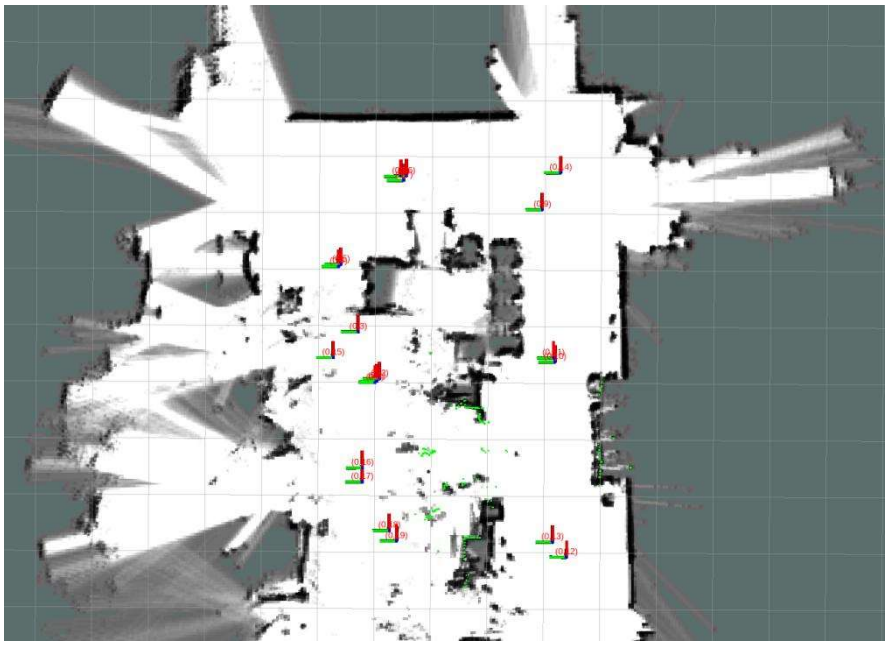
四、实验步骤

1、输入以下指令，开始建图：

```
roslaunch  robuster_mr_navigation mapping.launch
```

通过遥控器控制 Beetle 移动开始建图，注意：建图时要将移动和旋转速度调至最小，否则会导致建图失败。

先原地旋转 2-3 圈，然后每隔 2 米左右旋转 2-3 圈。具体情况根据实际环境而定。



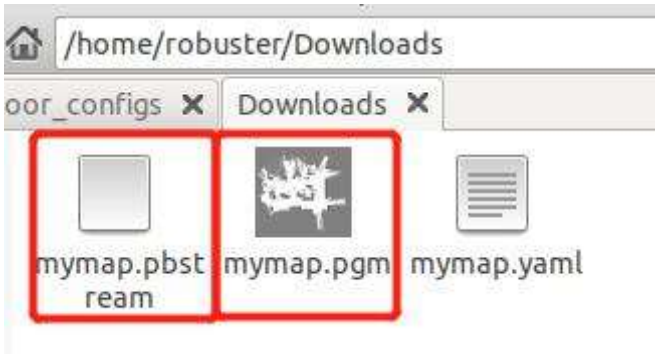
2、保存地图

地图保存在/home/robuster/Downloads 目录下。

版权所有 © 深圳史河机器人科技有限公司

```
    rosservice call /finish_trajectory 0     // 停止接收传感器数据
rosservice call /write_state "{filename: '{HOME}/Downloads/mymap.pbstream'}"
//生成.pbstream 文件
    rosrun cartographer_ros cartographer_pbstream_to_ros_map
    -map_filestem={HOME}/Downloads/mymap
    -pbstream_filename={HOME}/Downloads/mymap.pbstream
    -resolution=0.05
```

地图格式转换：由于用 cartographer_ros 提供的/write_state 方法保存的地图是.pbstream 的格式，而要在后续的自主导航中使用这个地图，我们需要将其转换为 ROS 中通用的 GridMap 格式。cartographer_ros 已经为我们提供了 cartographer_pbstream_to_ros_map 这个节点用于转换的实现。参数 map_filestem，为转换后存放结果的文件路径参数；pbstream_filename 为待转换的.pbstream 文件路径，参数 resolution 为分辨率。




```
^Crobuster@robuster:~$ rosservice call /finish_trajectory 0
status:
  code: 0
  message: "Finished trajectory 0."
robuster@robuster:~$ rosservice call /write_state "{filename: '${HOME}/Downloads
/mymap.pbstream'}"
status:
  code: 0
  message: "State written to '/home/robuster/Downloads/mymap.pbstream'."
robuster@robuster:~$ rosrun cartographer_ros cartographer_pbstream_to_ros_map -m
ap_filestem=${HOME}/Downloads/mymap -pbstream_filename=${HOME}/Downloads/mymap.p
bstream -resolution=0.05
I1223 15:31:23.052848 10112 pbstream_to_ros_map_main.cc:43] Loading submap slice
s from serialized data.
I1223 15:31:23.669100 10112 pbstream_to_ros_map_main.cc:51] Generating combined
map image from submap slices.
```

五、课堂作业

1、使用 BR280 建立一张地图，并且修改地图的格式，将其保存下来。

版权所有 © 深圳史河机器人科技有限公司

实验 22　BR280 自主导航避障

一、实验目的

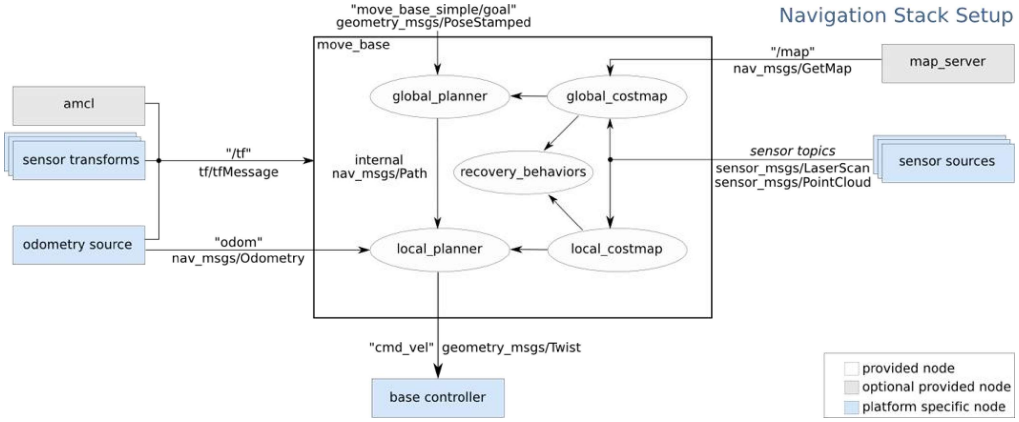
1、根据 ros-navigation 来了解自主导航的基本原理。
2、了解路径规划的基本原理。
3、了解 ros-navigation 功能包里文件的作用。
4、学会使用 rviz 来实现 Beetle 自主导航避障。

二、实验设备

PC、BR280 机器人、配套遥控器。

三、实验原理

我们参考下面这张官方给出的 ros-navigation 来分析自主导航的原理。最中心的是 move_base 节点，是导航过程运动控制的最终执行机构，move_base 订阅用户发布的导航目标 move_base_simple/goal，并将实时运动控制信号 cmd_vel 下发给底盘以实现最终运动控制，move_base 中的各种导航算法模块都是以插件的形式进行调用的，这样可以很方便的替换不同的算法以适应不同的应用，其中 global_planner 用于全局路径规划、local_planner 用于局部路径规划、golobal_costmap 是全局代价地图用于描述全局环境信息、local_costmap 是局部代价地图用于描述局部环境信息、recovery_behaviors 是恢复策略用于机器人碰到障碍后自动进行逃离恢复。然后是 amcl 节点，amcl 节点利用粒子滤波算法实现机器人的全局定位，为机器人导航提供全局位置信息。再然后是 map_server 节点，map_server 节点通过调用前面 SLAM 建图得到的地图为导航提供环境地图信息。最后就是要提供机器人模型相关的 tf 信息、里程计 odom 信息、激光雷达信息 scan。



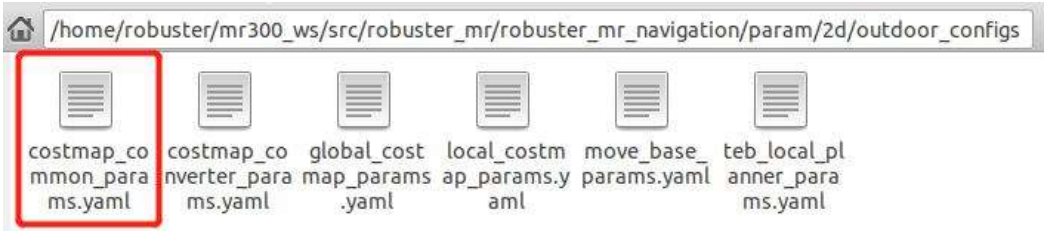
1、ros-navigation 的使用

版权所有 © 深圳史河机器人科技有限公司

ros-navigation 功能包集是一个很强大的导航框架，支持几乎所有的移动机器人。我们只需要按照适当的配置就可以将 ros-navigation 应用到我们自己的机器人。下面依次介绍这些文件的作用。

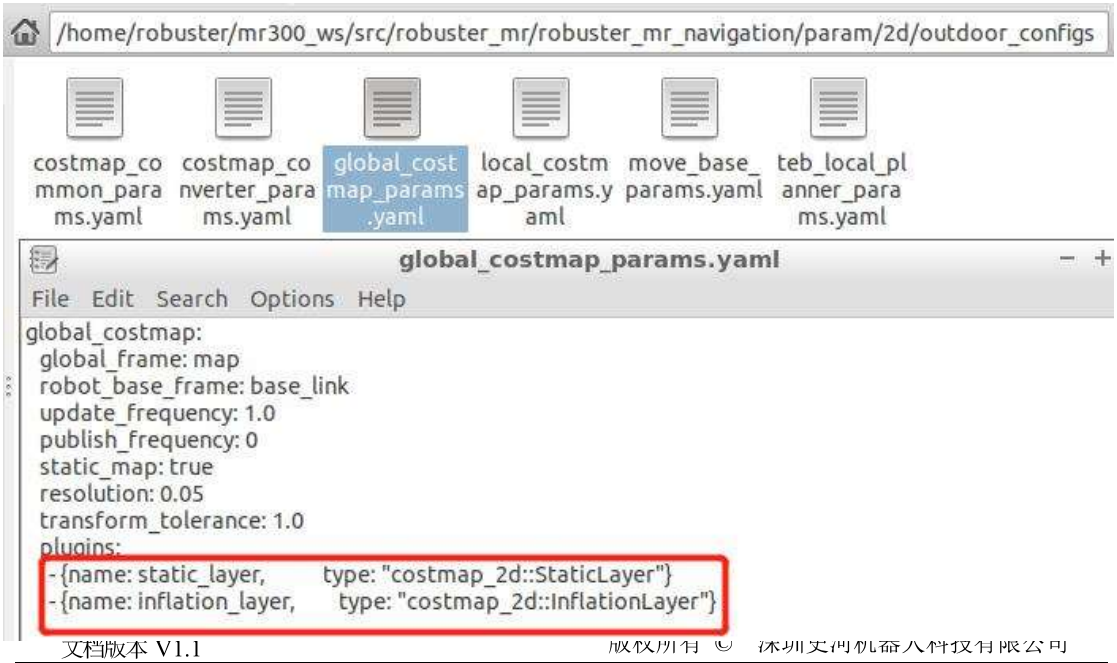
1）代价地图公有参数配置 costmap_common_params.yaml

全局代价地图和局部代价地图公有的参数被放置在 costmap_common_params.yaml 这里面，这样只需要载入这个配置文件就能完成共有参数的配置，避免了在全局代价地图和局部代价地图配置文件中重复配置一些相同参数的冗余，配置参数分为 2 类：机器人形状和代价地图各 Layer 图层。机器人形状可以用多边形或圆形描述。代价地图各 Layer 图层包括：静态层 static_layer（由 SLAM 建立得到的地图提供数据）、障碍层 obstacle_layer（由激光雷达等障碍扫描传感器提供实时数据）、膨胀层 inflation_layer（为代价地图提供膨胀效果）。参数的具体配置方法请参考 wiki 官方教程：http://wiki.ros.org/amcl。



2）全局代价地图参数配置 global_costmap_params.yaml

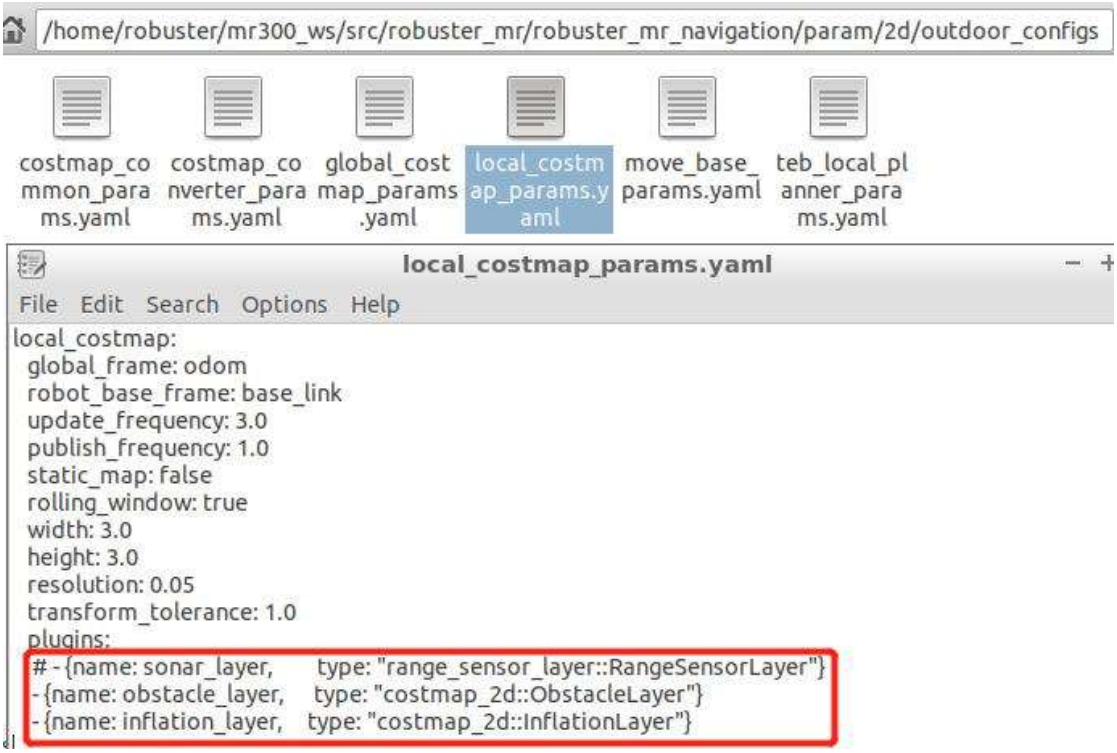
全局代价地图是以插件的形式组建的，在 costmap_common_params.yaml 中定义的各 Layer 图层都可以通过插件的形式放入全局代价地图中，各 Layer 图层的组合可以根据需求自由选择，我们的 Beetle 机器人的全局代价地图只用了 static_layer 和 inflation_layer 两个图层。



版权所有 © 深圳史河机器人科技有限公司

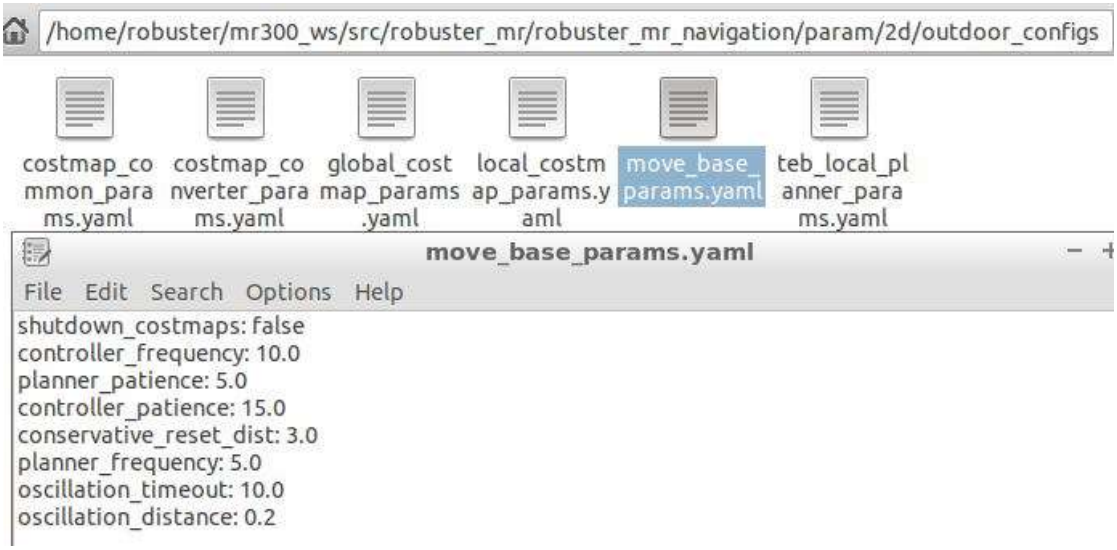
3）局部代价地图参数配置 local_costmap_params.yaml

局部代价地图和全局代价地图类似，我们的 Beetle 机器人的局部代价地图只用了 obstacle_layer、inflation_layer 和 sonar_layer 三个图层。



4）导航核心控制参数配置 move_base_params.yaml

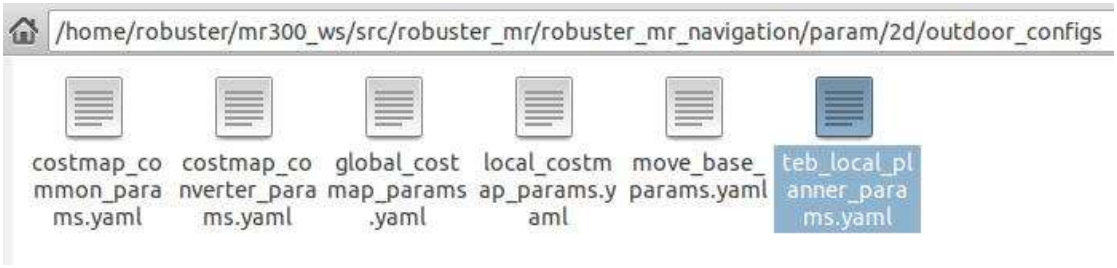
导航核心控制由 move_base 节点最终实现，move_base 在整个导航框架中处于核心地位，全局代价地图、局部代价地图、全局路径规划、局部路径规划、恢复策略等等都将在这里得到具体实现的调用，这里主要配置一些运动控制方面的参数。



5）局部路径规划器参数配置 teb_local_planner_params.yaml

版权所有 © 深圳史河机器人科技有限公司

局部路径规划器也是以插件的形式被使用，这里主要对局部路径规划器 TebLocalPlannerROS 插件及参数进行申明。



2、路径规划

机器人导航中核心的就是路径规划，路径规划是通过利用环境障碍物信息找到一条到达目标并且开销小的路劲。导航中会用到两种路径规划，即全局路径规划 global_planner 和局部路径规划 local_planner。全局路径规划更像是一种战略性策略，需要考虑全局，规划处一条尽量短并且易于执行的路径。在全局路径的指导下，机器人在实际行走时还需要考虑周围实时的障碍物并制定避让策略，这就是局部路径规划要完成的事，可以说机器人的自主导航最终是由局部路径规划一步步完成的。ROS 中推荐的全局路径规划器是 global_planner，基于 A*算法。

四、实验步骤

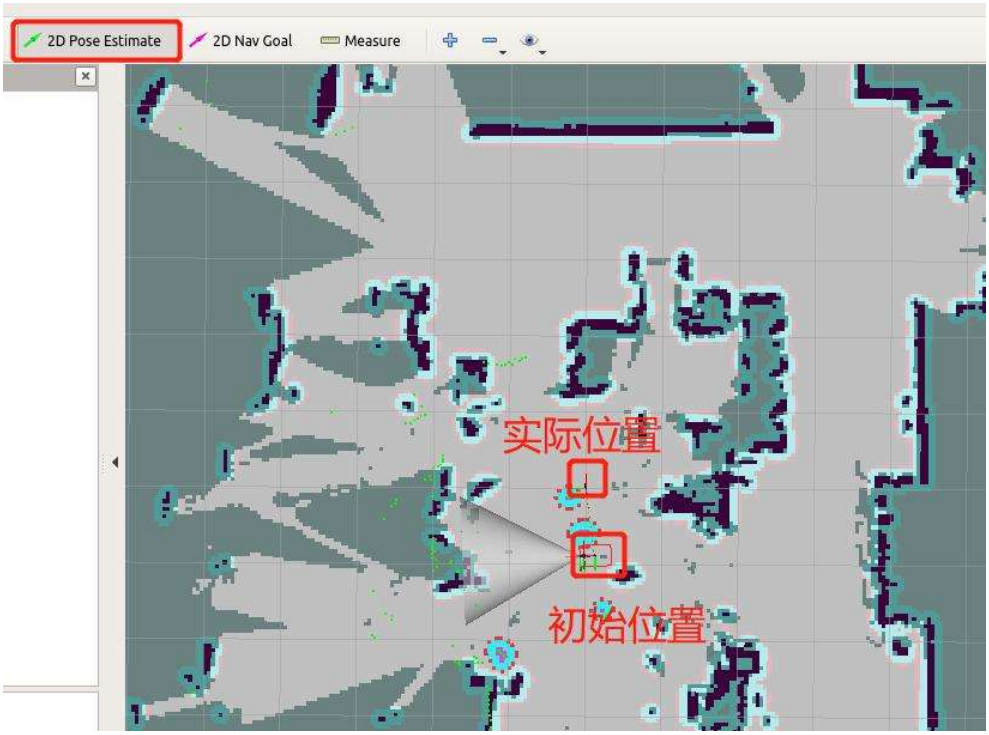
1、输入以下指令加载上个实验保存的地图

```
roslaunch robuster_mr_navigation navigation.launch
```

2、利用 2D Pose Estimate 设置初始位置

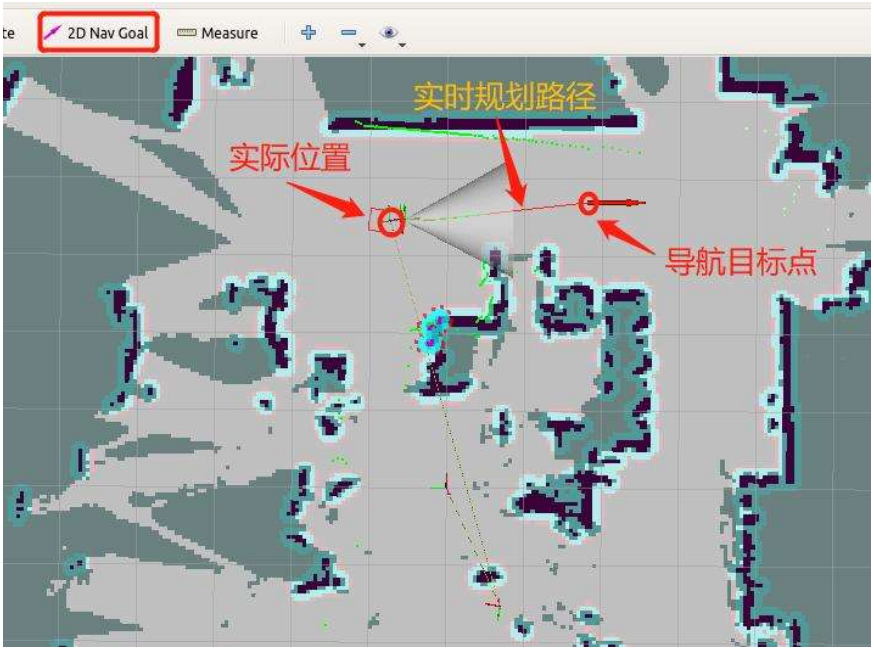
加载完地图之后，观察机器人的初始位置是否正确，如果机器人的初始位置不正确，需要用[2D Pose Estimate]按钮手动给定一个正确的初始位置。点击[2D Pose Estimate]按钮，然后将鼠标放置到机器人在地图中实际应该的位置，按住鼠标并拖动鼠标来完成机器人朝向的设置，最后让 Beetle 原地旋转 3 圈，完成位置的设定。

版权所有 © 深圳史河机器人科技有限公司



3、利用 2D Nav Goal 进行实施导航避障

初始位置设置正确后，就可以用[2D Nav Goal]按钮手动指定导航目标点了。先点击[2D Nav Goal]按钮，然后将鼠标放置到地图中任意的想让机器人到达的空白位置，最后按住鼠标并拖动鼠标来完成机器人朝向的设置。这样机器人就会开始规划路径并自动导航到该指定目标点。



五、课堂作业

1、导入使用 Beetle 建立的地图，并在 rviz 上实现 Beetle 的导航与避障。

版权所有 © 深圳史河机器人科技有限公司

实验 23　BR280 图形化编程控制机械臂

一、实验目的

1、了解 Blockly 的基本知识理论。

2、学会使用 Blockly 编程控制机械臂。

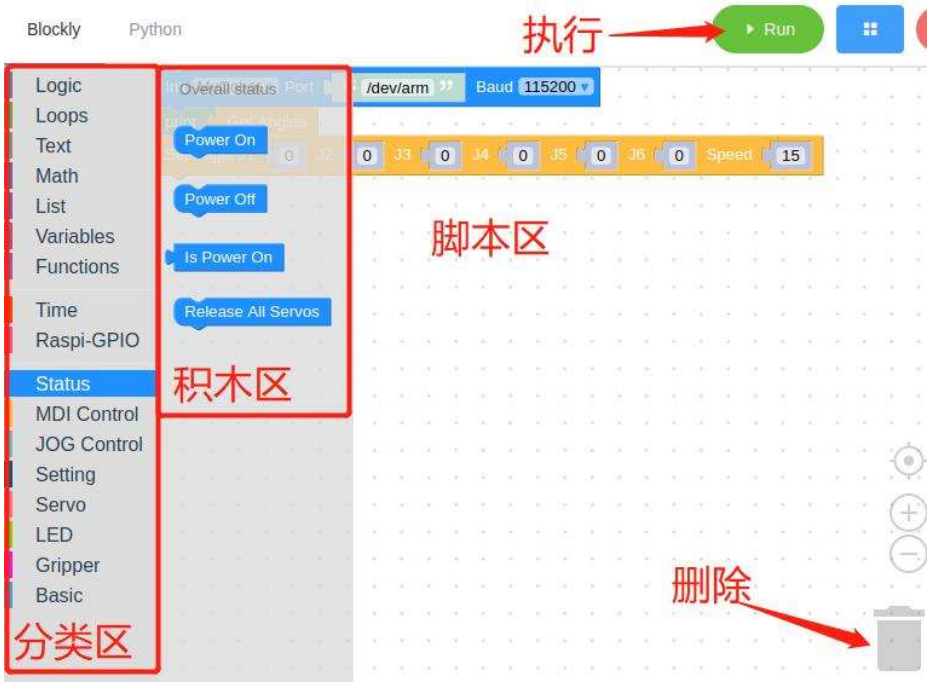
二、实验设备

PC、BR280 机器人、配套遥控器、Blockly

三、实验原理

Blockly 是一个用于构建块编程应用程序的库。Blockly 编辑器使用卡合的图形块来表示代码概念，如变量，逻辑表达式，循环等。它使得用户可以不必关注语法细节就能直接按照编程原则进行编程，让用户自定义组件并向应用程序的某些部分添加功能。

Blockly 的优势不在于定义界面有哪些功能积木，重要的是可灵活定义积木块，自由搭配积木块，使一堆卡合的积木块翻译出一种通用的 XML 语言，运算出相应的结果。界面整体称为工作空间，主要分为三个区域，分类区、积木区、脚本区。



版权所有 © 深圳史河机器人科技有限公司

分类区（Categories）：用于将 Blocks 按模块进行分类，Blockly 定义了多种分类，每种分类使用不同的颜色，有不同功能，譬如 Loops 分类存放循环积木，Math 分类存放数学操作积木等；

积木区（Blocks）：用于存放分类下的积木，不同积木代表不同的功能，在 Blockly 中，积木可以使用开发者工具，或者编写代码灵活定义积木；

脚本区（Scripts）：用于编写代码的区域，将积木区中的积木，拖动到脚本区，积木进行卡合形成积木组，积木组即可成为可执行的代码段，执行代码段获得运算结果；

工作空间（Workspace）：分类区、积木区、脚本区组合为工作空间，Blockly 操作以 Workspace 为核心。

四、实验步骤

1、打开远程桌面，登录机器人账号，进入工控机界面，（具体步骤请参考实验 1）按 CTRL+ALT+T 打开终端，准备开始实验。

2、输入以下指令，打开图形化编程界面：

```
cd ~/catkin_ws/src/mycobot_tools/myblockly/
./myblockly
```

设置有效的 USB 端口（/dev/arm）和波特率（115200），将左侧的积木拖动至脚本区，将每个关节的角度值都设置为 0，速度设置为 15，点击 Run 运行即可。

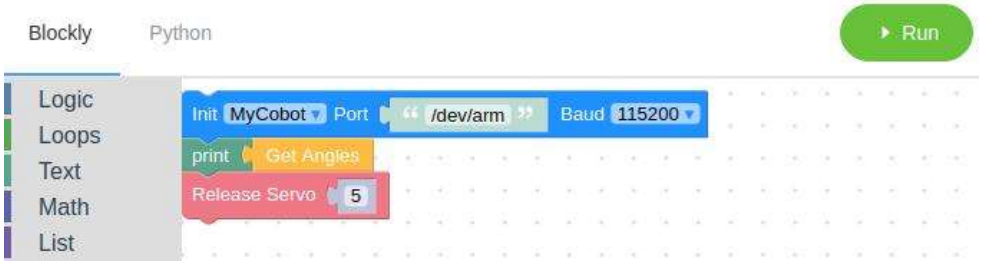
如果机械臂回到下图所示位姿，说明通信设置成功。

版权所有 © 深圳史河机器人科技有限公司

Blockly 打印出了机械臂的每个关节的角度值，我们发现每个角度都不为 0，这是由机械臂自身的误差造成的。

3、释放关节

通过 blockly 可以释放指定关节，让其可以自由转动。



也可以通过延时积木来控制关节释放的时间，比如设定 6 秒的时间，此时关节可以自由转动 6 秒，6 秒之后此关节将会被锁死。

版权所有 © 深圳史河机器人科技有限公司

4、灯光设置

通过颜色积木可以设置机械臂上 LED 灯光的颜色，设置不同的 RGB 值就可以让灯光显示不同的颜色。计算机定义颜色时 R、G、 B 三种成分的取值范围是 0-255。其中，255 亮度最大，0 也是数值之一，表示亮度为 0。三个值全为 0 则为白色，全为 255 则为黑色。

如果输入如下 GRB 值，LED 将会显示紫色。

同样地，我们可以通过延时积木和循环积木来实现 LED 颜色的变化。

版权所有 © 深圳史河机器人科技有限公司

此时，LED 的颜色将一直会循环紫、红、绿、蓝、黑，知道你退出循环。在 python 栏，你将会看到对应的 python 代码。

版权所有 © 深圳史河机器人科技有限公司

五、课堂作业

1、通过 myblockly 自己构造拼图来控制机械臂。

2、自己探索 myblockly 的其他用法。

3、学会将 myblockly 的拼图转换成 python 语言。

版权所有 © 深圳史河机器人科技有限公司

实验 24　BR280 Python 编程控制机械臂

一、实验目的

1、掌握 python 的基本知识理论和基础语法规则。

2、学会使用 python 编程控制机械臂的姿态。

3、学会使用 python 编程控制夹持器。

二、实验设备

PC、BR280 机器人

三、实验原理

1、Python 简介

Python 是一种面向对象的、解释型的、开源的脚本编程语言。它简单易学，功能强大，深受大众喜爱。它有以下优点：

1）语法简单

和传统的 C/C++、Java、C#等语言相比，Python 对代码格式的要求没有那么严格，这种宽松使得用户在编写代码时比较舒服，不用在细枝末节上花费太多精力。

2）开源、免费

Python 是 FLOSS（自由/开放源码软件）之一。使用者可以自由地发布这个软件的拷贝、阅读它的源代码、对它做改动、把它的一部分用于新的自由软件中。FLOSS 是基于一个团体分享知识的概念。

3）高层语言

Python 屏蔽了很多底层细节，比如 Python 会自动管理内存（需要时自动分配，不需要时自动释放）。

4）面向对象

Python 既支持面向过程的编程也支持面向对象的编程。

5）可扩展性、可扩充性

如果需要一段关键代码运行得更快或者希望某些算法不公开，可以部分程序用 C 或 C++编写，然后在 Python 程序中使用它们。

版权所有 © 深圳史河机器人科技有限公司

四、实验步骤

1、在/home/robuster/beetle_ai/scripts 目录下新建文件并命名为 test.py，然后将下面代码拷贝到 test.py 里面。

```
cd ~/Beetle_ai/scripts
gedit test.py
```

```
from pymycobot.mycobot import MyCobot
from pymycobot.genre import Angle
import time
from GrabParams import grabParams
mc = MyCobot(grabParams.usb_dev, grabParams.baudrate)
mc.power_on()
mc.set_color(0,0,255)#blue, arm is busy
coords = grabParams.coords_ready
mc.send_coords(coords,15,1)
time.sleep(6)
mc.set_color(0,255,0)#green, arm is free
```

2、组合键 Ctrl+Alt+T 打开终端，输入 cd ~/Beetle_ai/scripts 然后输入 python test.py，运行程序，此时机械臂会到达工作时的准备姿态。

```
robuster@robuster:~$ cd ~/beetle_ai/scripts/
robuster@robuster:~/beetle_ai/scripts$ python test.py
```

3、如果你对 python 不熟悉，那么你也可以通过 myblockly 来实现相同的功能。

版权所有 © 深圳史河机器人科技有限公司

此时你会发现，使用 myblockly 生成的代码和自己书写的代码几乎是一致的，初学者可以先通过 myblockly 来学习 python 代码的书写，熟悉之后，再将自己书写的代码通过 myblockly 来复现，检查是否正确。

4、关于机械臂的所有 python 代码及其用法，我们都可以通过 myblockly 来获取。只需将积木放在 myblockly 的脚本区，点击 Python，就可以查看积木的 python 代码及其用法。

五、课堂作业

1、使用 python 语言编程来控制机械臂的姿态。

2、通过 myblockly 来理解机械臂 python 指令的使用方法。

3、掌握一些 python 基本的语法。

版权所有 © 深圳史河机器人科技有限公司

实验 25　BR280 ROS+moveit 编程

一、实验目的

1、了解 moveit 的基本理论。

2、学会使用 moveit 对机械臂进行运动规划编程

二、实验设备

PC、BR280 机器人

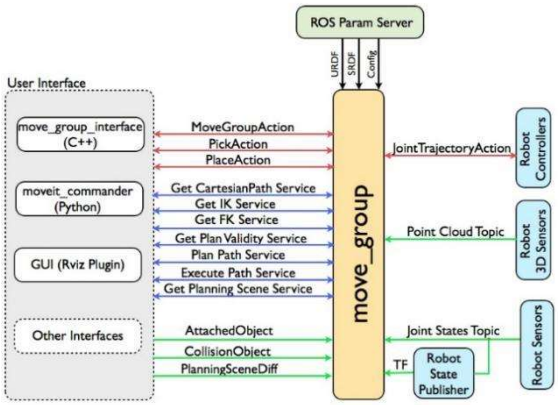
三、实验原理

1、Ros 系统的 moveit 模块介绍

机器人操作系统 ROS 目前最受关注的两个模块是导航(Navigation）和机械臂控制(MoveIt!)。其中，机械臂控制模块（后面简称 Movelt)可以让用户快速建立机械臂模型并实现机械臂的控制（包括建模、运动学求解、运动规划、避障等)。它集成了各种 SOTA 库，包括：

- 运动规划（Motion Planning）
- 轨迹插值（Trajectory Processing）
- 操作（Manipulation）
- 3D 感知（Perception）
- 运动学（Kinematics）
- 碰撞检测（Collision Checking）
- 控制（Control）
- 导航（Navigation）
- 交互（Interaction）

2、moveit 结构介绍



版权所有 © 深圳史河机器人科技有限公司

MoveIt 的核心节点(node)为 move_group，外围的几个部分分别为：ROS ParamServer，Robot Controllers，Robot 3D Sensors，User Interface，Robot Sensors，其中：

ROS Param Server：载入用户定义的模型文件(xacro 或 urdf）和一些配置文件。

Robot Controllers：运动规划的数据由此发给机器人驱动部分。

Robot 3D Sensors：载入 RGB-D 相机或激光雷达等获得的点云数据，用于机械手的抓取或避障等。

User Interface：用户接口，MoveIt 提供一系列的 API（应用程序编程接口）供用户完成自定义的功能。

无论是 C++还是 Python 编程，完整的 MoveIt 程序流程：

1）确定控制对象的规划组

2）设置目标位姿（关节空间或笛卡尔空间）

3）设置运动约束（可选）

4）使用 Moveit 进行运动规划

5）执行规划轨迹

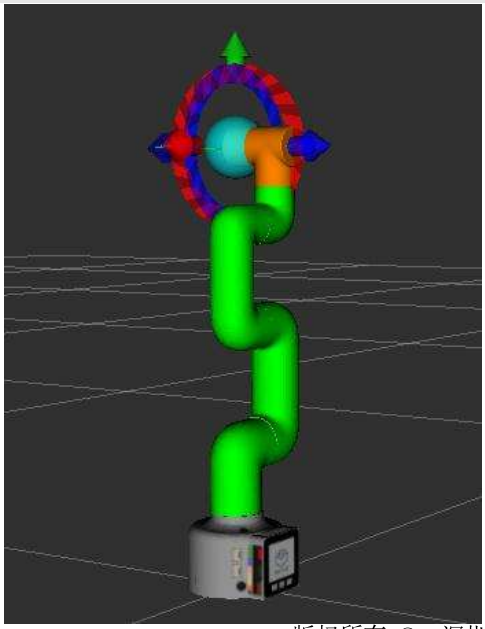
Robot Sensors：这部分是接收机械臂的传感器数据，然后预估出机器人的状态并发布。

四、实验步骤

1、使用 PC 端远程桌面连接到 BR280 工控机。

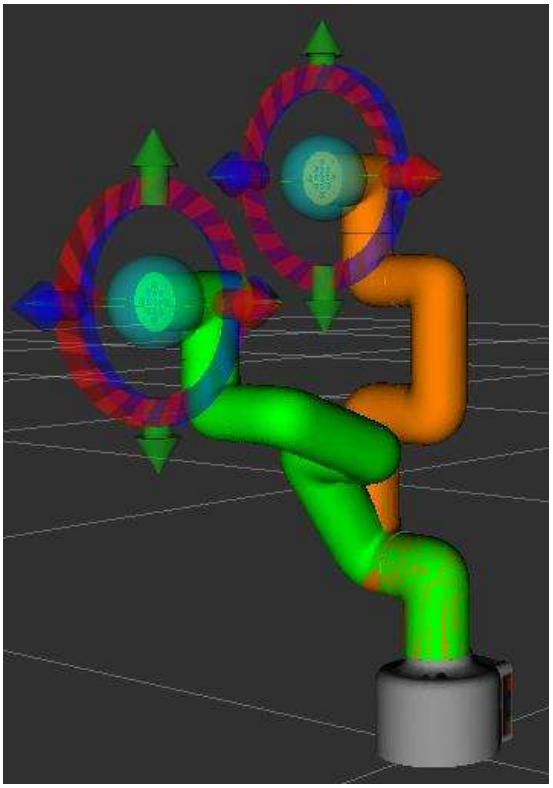
2、按 CTRL+ALT+T 打开终端，输入以下指令，打开 moveit。

```
roslaunch mycobot_280_moveit mycobot_moveit.launch
```

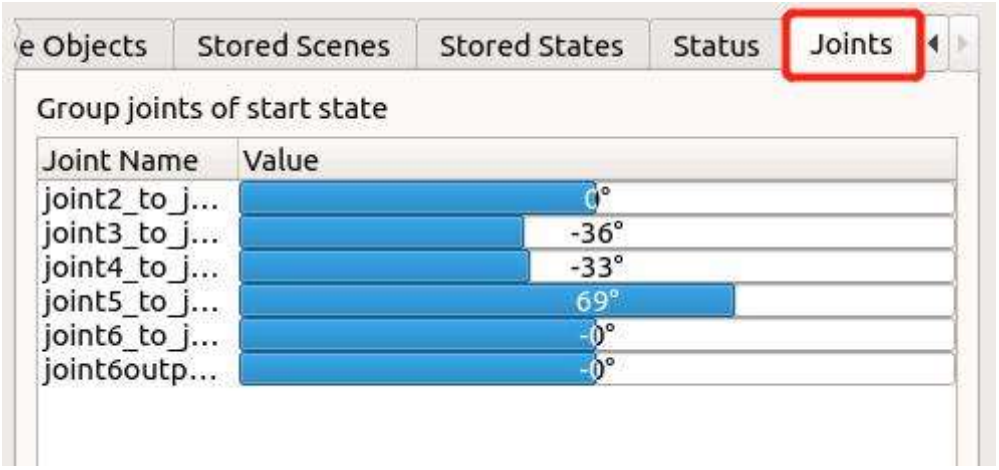


版权所有 © 深圳史河机器人科技有限公司

3、拖动箭头可以让机械臂的姿态变成你想要的姿态。

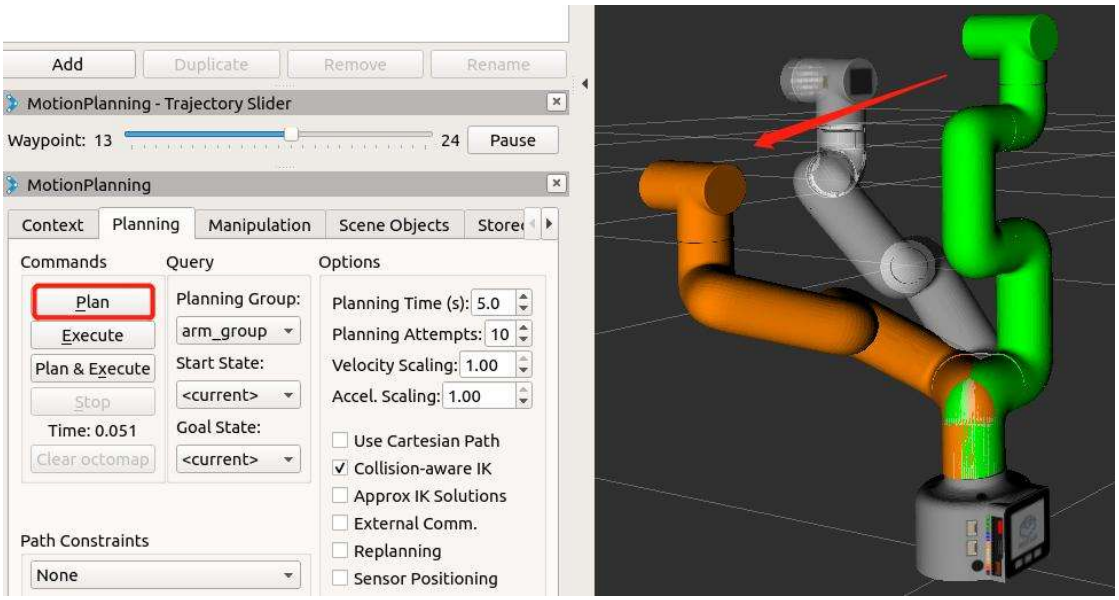


4、你可以在 Joints 里面看到每个关节的角度。你也可以通过调节角度值，让机械臂到达你期望的位置。当角度值全部是 0 的时候，机械臂回到初始状态。



5、点击 Plan，moveit 会自动规划机械臂的轨迹。

版权所有 © 深圳史河机器人科技有限公司



五、课堂作业

1、使用 moveit 完成机械臂的轨迹规划与仿真分析。

版权所有 © 深圳史河机器人科技有限公司

实验 26　BR280 手眼标定

一、实验目的

1、了解手眼标定的相关理论知识。

2、掌握进行手眼标定的流程和操作步骤。

3、理解手眼标定的代码逻辑。

二、实验设备

PC、BR280 机器人、BR280 配套 AI 套件。

三、实验原理

OpenCV 全称是：Open Source Computer Vision Library，它是一个以 BSD 许可证开源的、跨平台的计算机视觉库。它提供了 Python、C++、Java、Matlab 等多种编程语言接口。它集成了很多计算机视觉算法，具有非常强大的功能，是计算机视觉中最为著名的一个库。OpenCV 在诸多领域得到了广泛的应用，例如物体检测、图像识别、运动跟踪、增强现实（AR）、机器人等场景。

机器人视觉系统中要实现像素坐标与实际坐标的转换，首先要进行标定。

1、手眼标定

视觉抓取主要是通过改变机器人的 tool 坐标系或 base 坐标系来实现的。物体位置的变化主要是它位置的 X，Y，Z 方向的变化。而 base 坐标系可根据需要定义用户坐标系。首先，我们要了解一个概念：手眼标定（确定像素坐标系和空间机械手坐标系的坐标转化关系）。为什么会存在手眼标定？举一个简单例子—用手移动物体：

第一步：眼睛观察到三维世界，并将其转换到视网膜平面（三维空间转换到二维平面）传送信息给大脑；

第二步：大脑想要移动某个物体，假设想要将物体从 A 点移动 B 点（二维坐标），但是物体是三维空间中的物体，是三维坐标，需要将二维坐标换算成三维坐标；

第三步：大脑已经获得 A 点和 B 点的三维坐标，大脑给手（执行机构）发出指令去完成这个任务；

其中第二步就是手眼标定，得到二维坐标（像素坐标）到三维坐标的转换矩阵。

在实际控制中，相机检测到目标在图像中的像素位置后，通过标定好的坐标

版权所有 © 深圳史河机器人科技有限公司

转换矩阵将相机的像素坐标变换到机械手的空间坐标系中，从而控制机械手到达指定位置。这个过程中涉及到了图像标定，图像处理，运动学正逆解，手眼标定等。

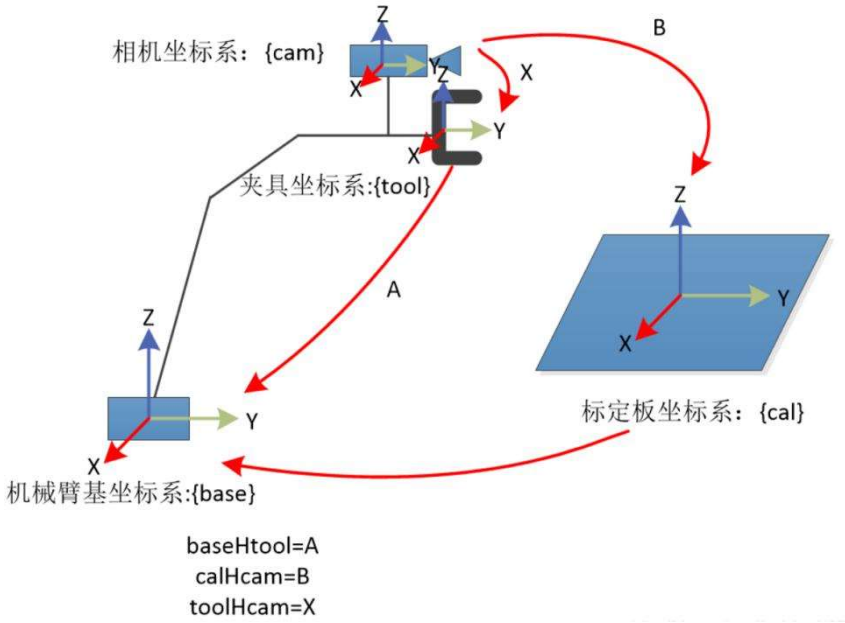
Beetle 的相机搭载方案是眼在手上（eye-in-hand）。在标定求解过程中，会用到四个坐标系，分别是：

1）基础坐标系（用 base 表示）

2）机械手坐标系（用 tool 表示）

3）相机坐标系（用 cam 表示）

4）标定物坐标系（用 cal 表示）



baseHtool：表示机械手坐标系到基础坐标系的转换关系，可以由机器人系统中得出。

toolHcam：表示相机坐标系到机械手坐标系的转换关系；这个转化关系在机械手移动过程中是不变的；

calHcam：表示相机坐标系到标定板坐标系的转换关系（相机外参），可以由相机标定求出；

baseHcal：表示标定板坐标系到基础坐标系的变换，这个是最终想要得到的结果；只要机械手和标定板的相对位置不变，这个变换矩阵不发生变化。

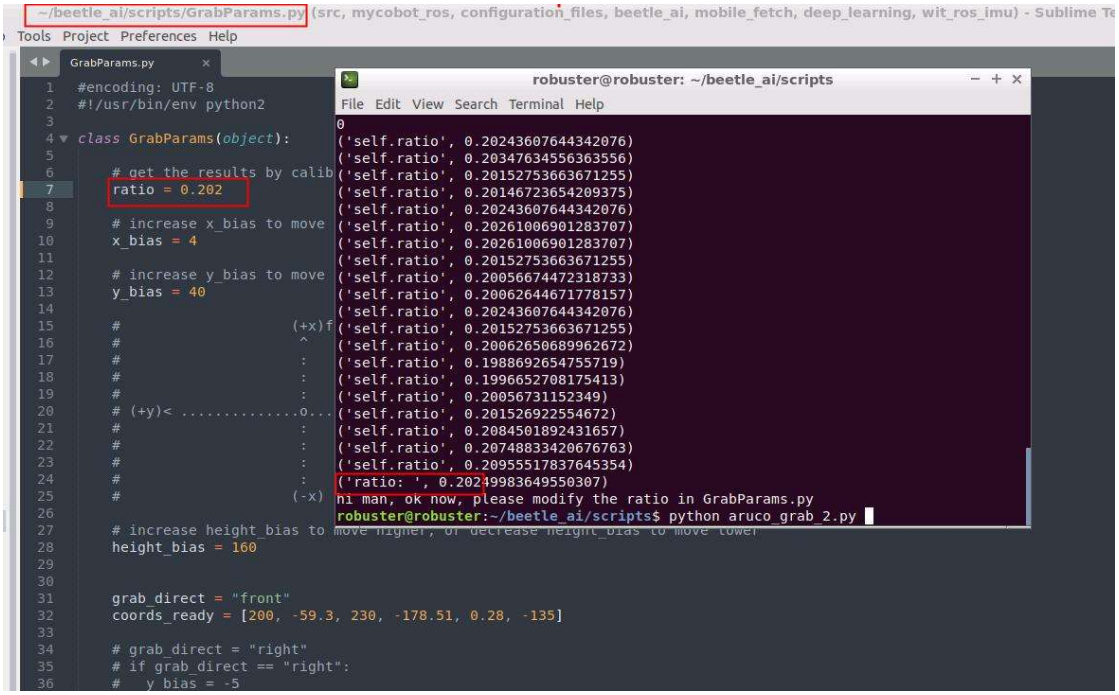
四、实验步骤

1、组合键 Ctrl+Alt+T 打开终端，切换目录 cd ~/beetle_ai/scripts。

2、将二维码积木放置在小桌面上，如下图所示。执行 python grab_calibration.py。

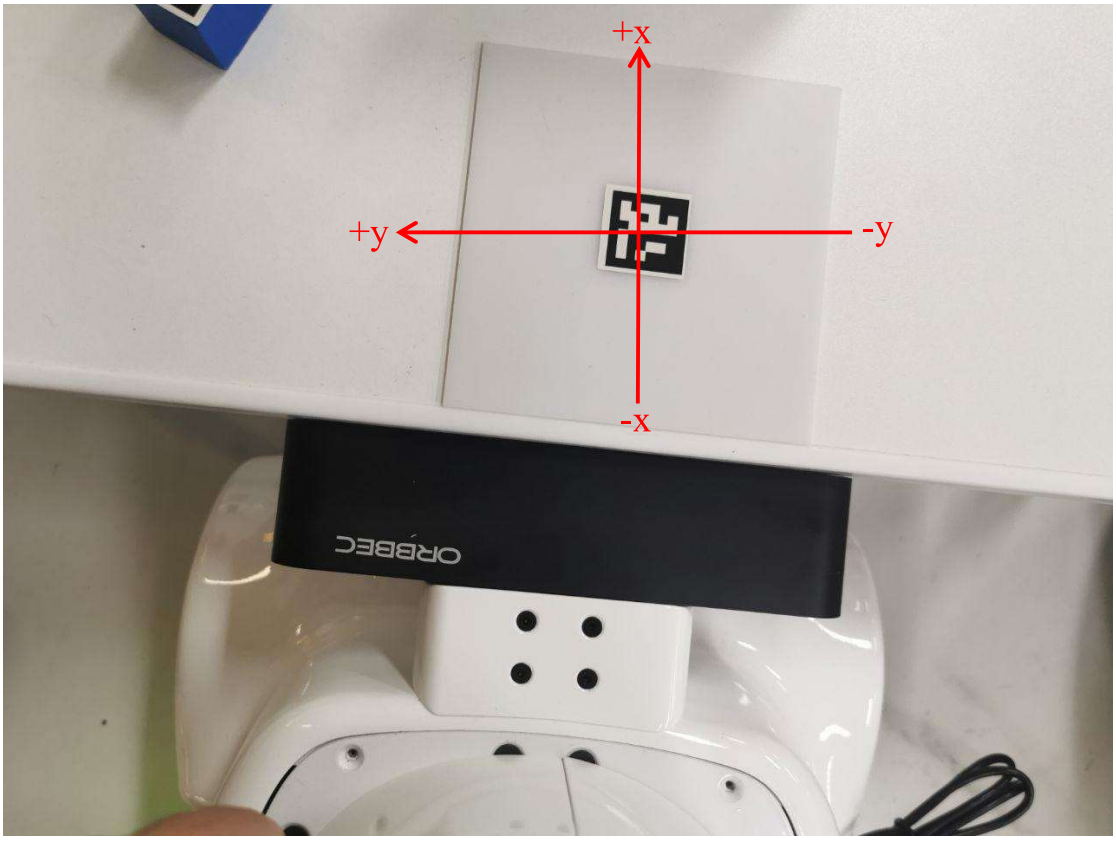
版权所有 © 深圳史河机器人科技有限公司

每次执行标定指令，都会得到一个 ratio 值，这个值在 0.15-0.25 之间都是可行的，超出这个范围，请检查一下摄像头安装是否有问题。重复执行这个指令 2-3 次，取有效的 ratio 值（保留三位小数），替换 GrabParams.py 文件中的 ratio 值并保存（文件位于~/beetle_ai/scripts/GrabParams.py）。



版权所有 © 深圳史河机器人科技有限公司

4、将标定板放置在小桌面上，如下图所示。执行 python aruco_grab_2.py。



观察夹爪夹二维码时（闭合的时候）与二维码中心位置的偏差，如下图所示。调整 x_bias 和 y_bias 的值（注意，它们可以是负的数值），使得下次执行 python aruco_grab_2.py 后夹爪接近二维码的中心位置。

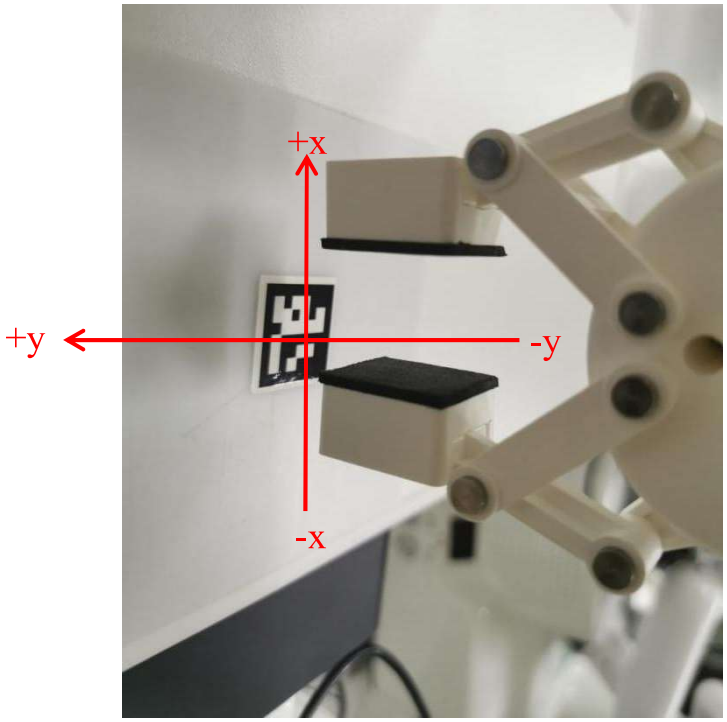
增加 x_bias 的值，夹爪会往前移动；减少 x_bias 的值，夹爪会往后移动；

增加 y_bias 的值，夹爪会往左移动；减少 y_bias 的值，夹爪会往右移动；

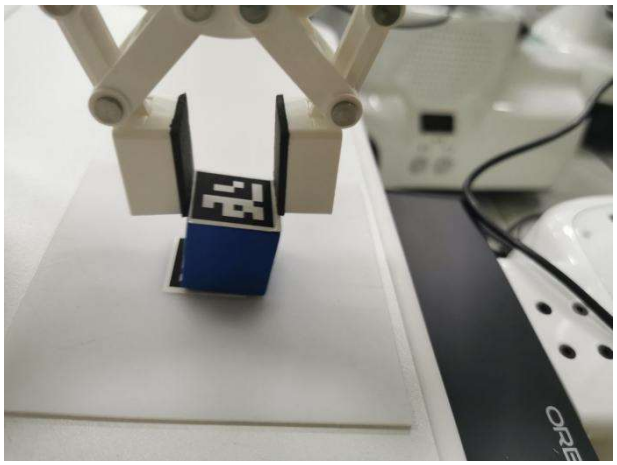
如下图所示，夹爪闭合时，夹爪在二维码中心的前方。为了使得夹爪下次夹的时候向后方移动，我们需要减少 x_bias 的值。打开 ~/beetle_ai/scripts/GrabParams.py 文件，查看里边的 x_bias 值等于 4。这时候我们减少 x_bias 值，改成 0。保存文件。

然后再一次执行 python aruco_grab_2.py。不断调整 x_bias 和 y_bias 的值，使得夹爪处于合理位置，也就是夹爪闭合的位置接近二维码的中心位置。

版权所有 © 深圳史河机器人科技有限公司



5、将二维码积木放在小桌面上，如下图所示。



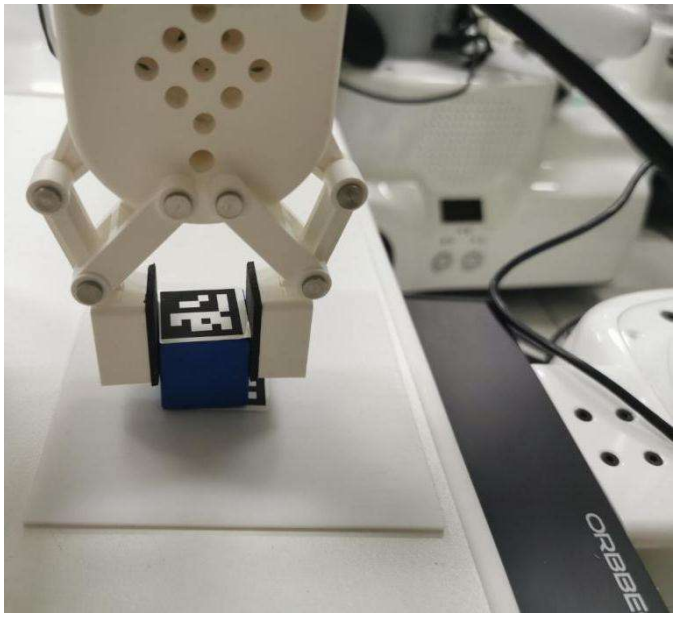
执行 python aruco_grab_2.py。 观察夹爪夹积木时候的高度。调整 height_bias 的值，使得下次夹爪夹积木的 高度合理。建议夹爪夹积木的高度刚好可以夹到一半积木。

增加 height_bias 的值，夹爪会往上移动；减少 height_bias 的值，夹爪会往下移动；

如下图所示，夹爪夹积木时的高度偏高了，很容易夹空。因此，我们需要减少 height_bias 的值，使得夹爪往下移动。打开 ~/beetle_ai/scripts/GrabParams.py 文件，查看里边的 height_bias 值等于 160。这时候我们减少 height_bias 值，改成 150。保存文件。

然后再一次执行 python aruco_grab_2.py。不断调整 height_bias 的值，使得夹爪处于合理位置，也就是夹爪刚好可以夹到一半积木，如下图所示。

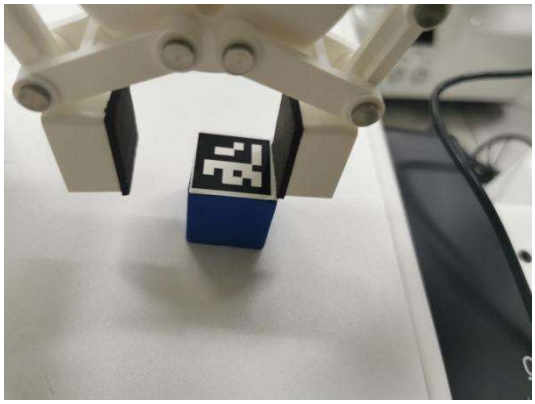
版权所有 © 深圳史河机器人科技有限公司

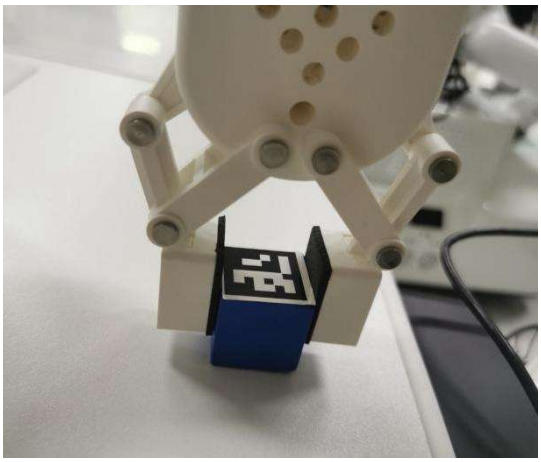


6、拿走标定板，只留下二维码积木，如下图所示。

然后再一次执行 python aruco_grab_2.py。不断调整 x_bias 和 y_bias 的值，使得夹爪处于合理位置，也就是夹爪闭合的位置接近二维码的中心位置。

重复执行 5 次二维码抓取，如果都能成功，说明标定参数合理，标定完成。





版权所有 © 深圳史河机器人科技有限公司

五、课堂作业

1、利用标定板对 Beetle 进行手眼标定。

版权所有 © 深圳史河机器人科技有限公司

实验 27　BR280 二维码识别抓取积木

一、实验目的

1、了解视觉抓取中的关键技术。

2、了解二维码识别的基本原理与步骤。

3、学会使用 Beetle 实现二维码识别抓取积木。

4、理解相关 python 代码。

二、实验设备

PC、BR280 机器人、BR280 配套 AI 套件。

三、实验原理

3.1 视觉抓取中的关键技术

1、手眼标定

视觉抓取主要是通过改变机器人的 tool 坐标系或 base 坐标系来实现的。物体位置的变化主要是它位置的 X，Y，Z 方向的变化。而 base 坐标系可根据需要定义用户坐标系。如此一来，我们就要进行手眼标定（确定像素坐标系和空间机械手坐标系的坐标转化关系）。

2、物体识别与定位

视觉抓取中非常重要的一个部分就是对抓取物体的识别，无论是二维图像还是三维点云，在 ROS 中都可以找到对应的功能包，

3、抓取姿态分析

抓取姿态分析是视觉抓取中的一个难点，这一过程的基础是找到现实世界和图像投影之间的对应点。

4、运动规划

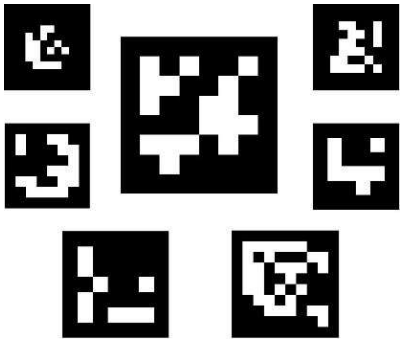
Moveit 为开发者提供了一个易于使用的集成化开发平台。由一系列移动操作的功能包组成，包括运动规划、操作控制、3D 感知、运动学、控制与导航算法等。这些功能算法的实现都是通过插件的方式在 Moveit!中集成，并提供友好的 GUI，可广泛应用于工业、商业、研发和其他领域。

3.2 二维码识别

在 BR280 中采用的是 ArUco Marker，它是一种特殊的二维码，广泛用于机器视觉中。ArUco Marker 有不同的规格，称之为字典。下图是一些 ArUco Marker

版权所有 © 深圳史河机器人科技有限公司

的例子。



一个 ArUco marker 是一个二进制平方标记，它由一个宽的黑边和一个内部的二进制矩阵组成，内部的矩阵决定了它们的 id。黑色的边界有利于快速检测到图像，二进制编码可以验证 id，并且允许错误检测和矫正技术的应用。marker 的大小决定了内部矩阵的大小。例如，一个 4x4 的 marker 由 16bits 组成。

markers 的字典是在一个特殊应用中使用到的 marker 的集合。这仅仅是每个 marker 的二进制编码的链表。字典的主要性质是字典的大小和 marker 的大小：

• 字典的大小是组成字典的 marker 的数量

• marker 的大小是这些 marker 的尺寸（位的个数）

aruco 模块包含了一些预定义的字典，这些字典涵盖了一系列的字典大小和 Marker 尺寸。

1、创建 Marker

在检测之前，我们需要打印 marker，以把它们放到环境中。marker 的图像可以使用 drawMarker（）函数生成。

2、检测 Marker

给定一个可以看见 ArUco marker 的图像，检测程序应当返回检测到的 marker 列表。每个检测到的 marker 包括：

• 图像四个角的位置（按照原始的顺序）

• marker 的 ID

marker 检测过程由以下两个主要步骤构成：

1）检测有哪些 marker。在这一阶段我们分析图像，以找到哪些形状可以被识别为 Markers。首先要做的是利用自适应性阈值来分割 marker，然后从阈值化的图像中提取外形轮廓，并且舍弃那些非凸多边形的，以及那些不是方形的。我们还使用了一些额外的滤波（来剔除那些过小或者过大的轮廓，过于相近的凸多边形等）

2）检测完 marker 之后，我们需要分析它的内部编码来确定它们是否确实是 marker。此步骤首先提取每个标记的标记位。为了达到这个目的，首先，我们需

版权所有 © 深圳史河机器人科技有限公司

要对图像进行透视变换，来得到它规范的形态（正视图）。然后，对规范的图像用 Ossu 阈值化以分离白色和黑色位。这一图像根据 marker 大小和边界大小被分为不同格子，我们统计落在每个格子中的黑白像素数目来决定这是黑色还是白色的位。最终，我们分析这些位数来决定这个 marker 是属于哪个特定字典的，如果有必要的话，需要对错误进行检测。

3、POSE 检测

接下来你想要做的应当是通过 Marker 检测来获取相机 pose。

为了展现相机的 Pose 检测，你需要知道你的相机的校准（Calibration）参数。这是一个相机矩阵和畸变系数。如果你不知道如何校准你的相机，你可以看一看 calibrateCamera() 函数，以及 OpenCV 的校准教程。

4、选择字典

aruco 模块提供了 Dictionary 类来描述 marker 的字典。

除了 marker 大小和字典中的 marker 数目，字典还有一个很重要的参数，就是内部 marker 的距离。内部 marker 的距离是 marker 之间的最小距离，它决定了字典错误检测和纠正能力。一般而言，较小的字典大小和较大的 marker 大小将会产生更大的内部 marker 距离，反之亦然。但是，过大的 Marker 在检测中更加困难，因为我们需要从图像中提取出更多位的信息。

5、检测参数

参数检测的步骤主要分为以下几部分：

1）输入图像的阈值化

2）轮廓滤波

阈值化之后，我们需要检测轮廓。但并不会把所有的轮廓都当作是候选。在不同步骤中，通过滤波剔除一些不太可能是 marker 的轮廓。

3）比特位提取

检测到候选之后，需要分析每个候选的比特位（比特位即 bit，用 0 或 1 来表示比特位的值，越多的比特位可以表现越复杂的图像信息），来确定它们是不是 marker。在分析二进制编码之前，我们需要将对透视变换后的图像使用 Otsu 进行阈值化，来分离黑色和白色像素，提取出比特位。

4）检查提取的编码是否属于 marker 字典

比特位提取之后，接下来的步骤是检查提取的编码是否属于这个 marker 字典，有必要的话，还需要做错误检测步骤。

5）角落细化

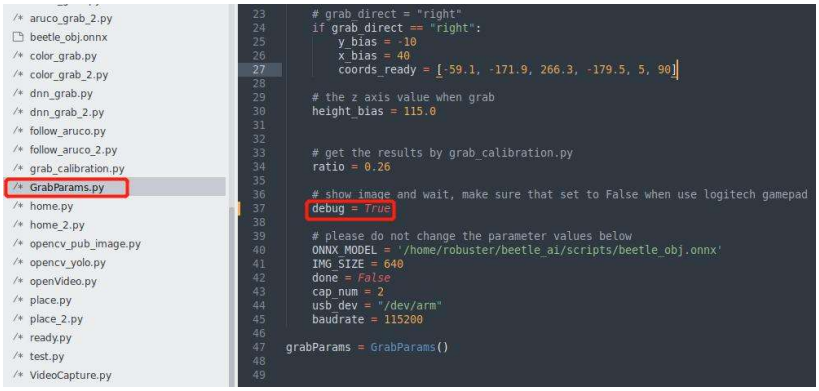
当检测完 marker，并且验证了它们的 ID 之后，最后要做的一步是在角落处的亚像素级的细化。注意，这一步是可选的，仅在我们对 marker 角位置的准确

版权所有 © 深圳史河机器人科技有限公司

性要求很高时才有意义。

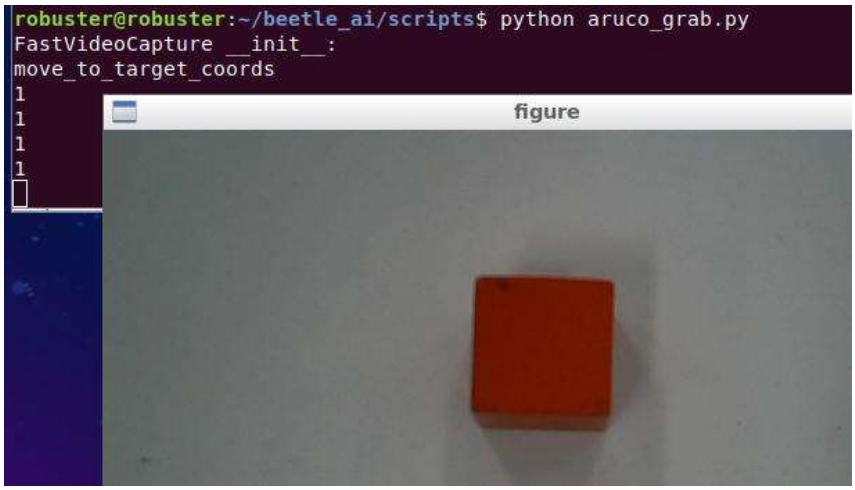
四、实验步骤

1、打开 sublime，将 GrabParams.py 里面的 debug 改为 Ture。

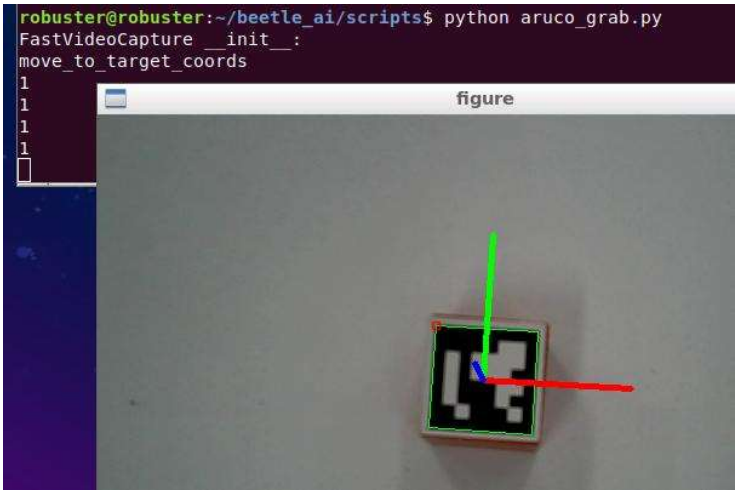


2、组合键 Ctrl+Alt+T 打开终端，切换目录 cd ~/beetle_ai/scripts。

3、执行 python aruco_grab.py。



此时我们发现，相机是无法识别出二维码之外的其他图形的。将小木块换成二维码，然后按 Enter 键。



版权所有 © 深圳史河机器人科技有限公司

二维码识别成功之后，再按 Enter 键，直到机械臂执行抓取任务。

```
robuster@robuster:~/catkin_ws/src/mycobot_tools/myblockly$ cd ~/beetle_ai/scrip
ts/
robuster@robuster:~/beetle_ai/scripts$ python aruco_grab.py
FastVideoCapture __init__:
move_to_target_coords
1
1
1
1
[array([[[ 279.,  174.],
        [ 363.,  179.],
        [ 358.,  264.],
        [ 273.,  258.]]], dtype=float32)]
(5.5250000000000004, 0.45500000000000002)
[array([[[ 280.,  173.],
        [ 363.,  179.],
        [ 357.,  264.],
        [ 273.,  258.]]], dtype=float32)]
(5.5899999999999999, 0.45500000000000002)
[array([[[ 280.,  173.],
        [ 363.,  179.],
        [ 357.,  264.],
        [ 273.,  258.]]], dtype=float32)]
(5.5899999999999999, 0.45500000000000002)
[array([[[ 280.,  173.],
        [ 363.,  179.],
        [ 357.,  264.],
        [ 273.,  258.]]], dtype=float32)]
(5.5899999999999999, 0.45500000000000002)
[array([[[ 280.,  173.],
        [ 363.,  179.],
        [ 357.,  264.],
        [ 273.,  258.]]], dtype=float32)]
(5.5899999999999999, 0.45500000000000002)
[array([[[ 280.,  173.],
        [ 363.,  179.],
        [ 359.,  263.],
        [ 273.,  258.]]], dtype=float32)]
(5.6550000000000002, 0.32500000000000001)
(5.577, 0.45499999999999996)
move_to_target_coords
0
```

4、代码实现

BR280 实现视觉抓取的坐标系按下图所示规则建立：

设置偏移量：x 轴偏移量是单目相机到机械臂末端中心在 x 轴方向的距离。

y 轴偏移量是单目相机到机械臂末端中心在 y 轴方向的距离。

版权所有 © 深圳史河机器人科技有限公司

高度偏移量是机械臂末端到坐标系平面的距离。

```
# y轴偏移量
y_bias = grabParams.y_bias #40
# x轴偏移量
x_bias = grabParams.x_bias #5
# rotation
rotation_angle = 0
#height
height_bias = grabParams.height_bias
done = grabParams.done
cap_num = grabParams.cap_num
```

进行二维码识别之前，先设置机械臂的位姿，确定是否由手柄下达抓取指令。

```
# show image and waitkey
debug = grabParams.debug

coords = grabParams.coords_ready
```

建立识别二维码的类，下列子函数都包括在这个类里面。

```
class Detect_marker(object):
    def __init__(self):
        super(Detect_marker, self).__init__()

        self.mc = MyCobot(grabParams.usb_dev, grabParams.baudrate)
        self.mc.power_on()

        # Creating a Camera Object
        self.cap = fastVideoCapture
        # Get ArUco marker dict that can be detected.
        self.aruco_dict = cv.aruco.Dictionary_get(cv.aruco.DICT_6X6_250)
        # Get ArUco marker params.
        self.aruco_params = cv.aruco.DetectorParameters_create()
        self.calibrationParams = cv.FileStorage("calibrationFileName.xml", cv.FILE_STORAGE_READ)
        # Get distance coefficient.
        self.dist_coeffs = self.calibrationParams.getNode("distCoeffs").mat()

        height = self.cap.getHeight()
        focal_length = width = self.cap.getWidth()
        center = [width / 2, height / 2]
        # Calculate the camera matrix.
        self.camera_matrix = np.array(
            [
                [focal_length, 0, center[0]],
                [0, focal_length, center[1]],
                [0, 0, 1],
            ],
            dtype=np.float32,
        )
        self.c_x, self.c_y = width/2, height/2

        self.ratio = grabParams.ratio
```

移动抓取设置，包括机械臂坐标、机械臂移动的速度。

版权所有 © 深圳史河机器人科技有限公司

```
# Grasping motion
def move(self, x, y):
    global height_bias, done


    self.move_to_target_coords([coords[0]+int(x), coords[1]+int(y), height_bias, coords[3], coords[4], coords[5]], 15)

    # self.rotate_ange()

    self.grap(True)
    time.sleep(1)

    self.move_to_target_coords(coords, 15)
    # time.sleep(4)

    done = True
    print("Done")
    self.mc.set_color(0,255,0)#green, arm is free


def move_to_target_coords(self,coords,speed):
    print("move_to_target_coords")
    self.mc.send_coords(coords,speed,1)
    is_finised = self.mc.is_in_position(coords,1);
    print(is_finised)
    count = 0
    is_finised_count = 0
    while is_finised_count < 3 and count < 10:
        print(is_finised)
        if self.mc.is_in_position(coords,1):
            is_finised_count += 1
        time.sleep(0.5)
        count = count + 1
```

初始化机械臂。

```
# init mycobot
def init_mycobot(self):

    self.move_to_target_coords(coords,30)

    self.grap(False)
    time.sleep(0.5)
```

计算积木和机械臂之间的坐标

```
# calculate the coords between cube and mycobot
def get_position(self, corners):
    x = corners[0][0][0][0] + corners[0][0][1][0] + corners[0][0][2][0] + corners[0][0][3][0]
    y = corners[0][0][0][1] + corners[0][0][1][1] + corners[0][0][2][1] + corners[0][0][3][1]
    x = x/4.0
    y = y/4.0
    wx = wy = 0
    if grabParams.grab_direct == "front":
        wx = (self.c_y - y) * self.ratio
        wy = (self.c_x - x) * self.ratio
    elif grabParams.grab_direct == "right":
        wx = (self.c_x - x) * self.ratio
        wy = (y - self.c_y) * self.ratio
    return wx, wy
```

开始执行抓取动作

```
def run(self):
    self.mc.set_color(0,0,255)#blue, arm is busy
    global y_bias, x_bias, rotation_angle
    self.init_mycobot()
    num = sum_x = sum_y = 0
    count = 5
    while not done:
        img = self.cap.read()
        # if not success:
        #     print("It seems that the image cannot be acquired correctly.")
        #     break

        # transfrom the img to model of gray
        gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
        # Detect ArUco marker.
        corners, ids, rejectImaPoint = cv.aruco.detectMarkers(
            gray, self.aruco_dict, parameters=self.aruco_params
        )
```


版权所有 © 深圳史河机器人科技有限公司

五、课堂作业

1、在 PC 端工控机上输入指令，让机械臂通过二维码识别来抓取积木。

2、理解执行二维码识别的各部分 python 代码。

版权所有 © 深圳史河机器人科技有限公司

实验 28　BR280 颜色识别抓取积木

一、实验目的

1、了解颜色识别的原理。

2、学会如何通过颜色识别来抓取积木。

3、理解相关的 python 代码。

二、实验设备

PC、BR280 机器人、BR280 配套 AI 套件。

三、实验原理

OpenCV 全称是：Open Source Computer Vision Library，它是一个以 BSD 许可证开源的、跨平台的计算机视觉库。它提供了 Python、C++、Java、Matlab 等多种编程语言接口。它集成了很多计算机视觉算法，具有非常强大的功能，是计算机视觉中最为著名的一个库。OpenCV 在诸多领域得到了广泛的应用，例如物体检测、图像识别、运动跟踪、增强现实（AR）、机器人等场景。

RGB 彩色图像的每个像素拥有 3 个通道，各 8 位，分别表示红色、绿色、蓝色 3 个分量，各自的取值范围都为 0～255。将这 3 种颜色组合，可以得到更多的颜色表示方式。例如，红色和绿色结合可以产生黄色等。这 3 种颜色分量也可以用来表示不同等级的灰色。例如，当 3 个颜色都为 0 时，表示的颜色是黑色；当 3 个颜色都为最大值 255 时，将得到白色。

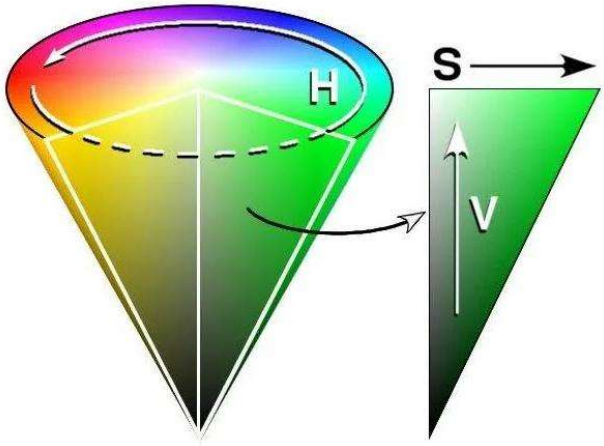
HSV 模型中颜色的参数分别是：色调（H：hue），饱和度（S：saturation），亮度（V：value）。由 A. R. Smith 在 1978 年创建的一种颜色空间, 也称六角锥体模型(Hexcone Model)。

(1)色调（H：hue）：用角度度量，取值范围为 0°～360°，从红色开始按逆时针方向计算，红色为 0°，绿色为 120°,蓝色为 240°。它们的补色是：黄色为 60°，青色为 180°,品红为 300°；

(2)饱和度（S：saturation）：取值范围为 0.0～1.0，值越大，颜色越饱和。

(3)亮度（V：value）：取值范围为 0(黑色)～255(白色）

版权所有 © 深圳史河机器人科技有限公司

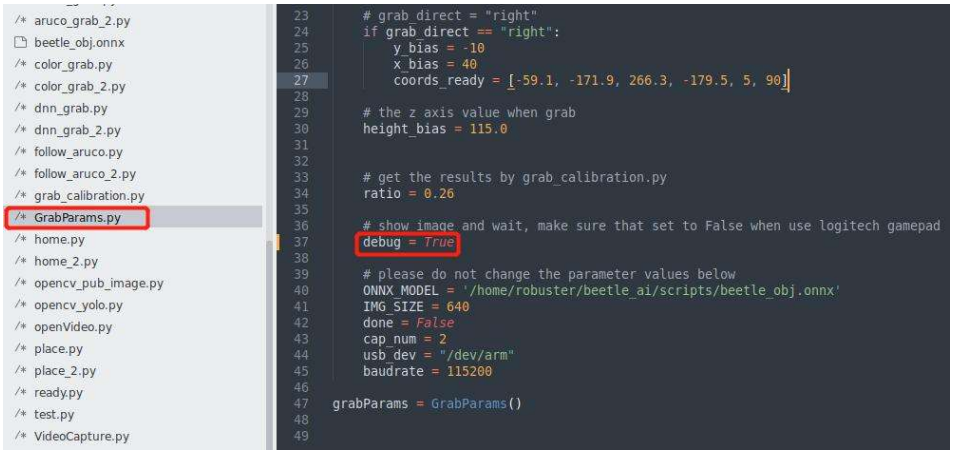


在图像处理的过程中，有时仅参考 RGB 颜色空间无法得到理想的结果，就需要对颜色空间做一些转换，使用 HSV 颜色空间，可以通过色调、饱和度、亮度信息来对图像进行处理。

OpenCV 下有个函数可以直接将 RGB 模型转换为 HSV 模型，在 OpenCV 中，0≤H<180，0≤H≤255，0≤V≤255。我们知道 H 分量基本能表示一个物体的颜色，但是 S 和 V 的取值也要在一定范围内，因为 S 代表的是 H 所表示的那个颜色和白色的混合程度，也就说 S 越小，颜色越发白，也就是越浅；V 代表的是 H 所表示的那个颜色和黑色的混合程度，也就说 V 越小，颜色越发黑。

四、实验步骤

1、打开 sublime，将 GrabParams.py 里面的 debug 改为 Ture。

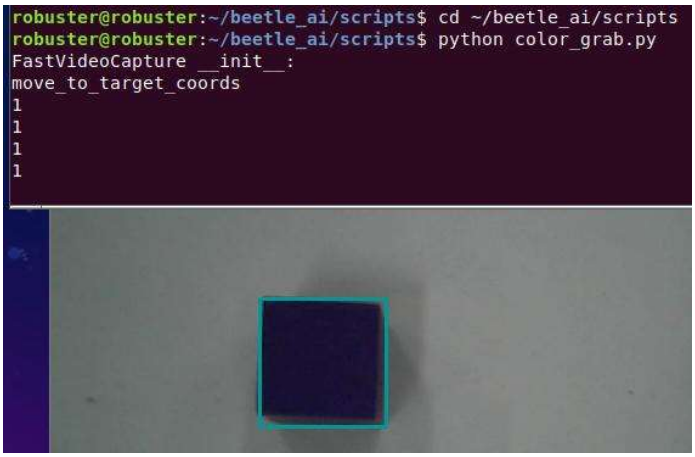


2、组合键 Ctrl+Alt+T 打开终端，切换目录 cd ~/beetle_ai/scripts。

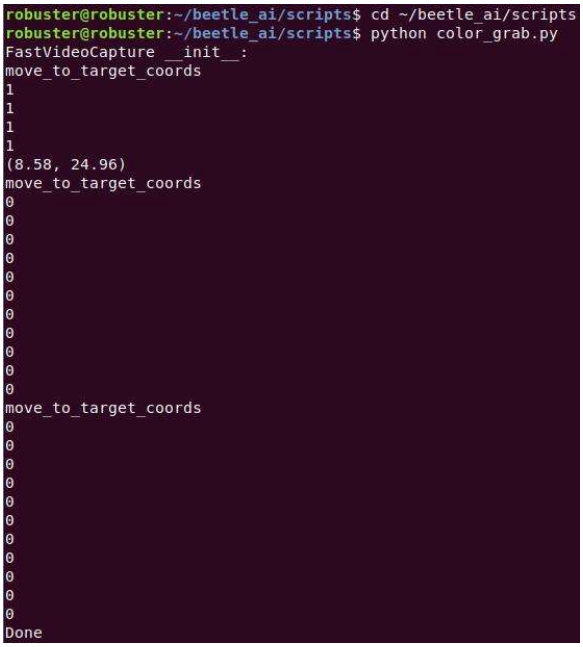
3、执行 python color_grab.py。

版权所有 © 深圳史河机器人科技有限公司

相机无法选中积木，此时需要手动移动积木位置，然后按 Enter 键，重新进行识别。



识别成功之后，再按 Enter 键，即可执行抓取任务。



版权所有 © 深圳史河机器人科技有限公司

4、代码实现

创建检测目标的类

```
class Detect_marker(object):
    def __init__(self):
        super(Detect_marker, self).__init__()

        self.mc = MyCobot(grabParams.usb_dev, grabParams.baudrate)
        self.mc.power_on()

        self.yolo = yolo()
```

设置待检测的颜色的 HSV 值

```
# set color HSV
self.HSV = {
    "yellow": [np.array([11, 115, 70]), np.array([40, 255, 245])],
    "red": [np.array([0, 43, 46]), np.array([8, 255, 255])],
    "green": [np.array([35, 43, 46]), np.array([77, 255, 255])],
    "blue": [np.array([110, 43, 46]), np.array([124, 255, 255])],
    "purple": [np.array([120, 43, 46]), np.array([200, 255, 255])],
}
```

移动到目标的位置

```
def move_to_target_coords(self,coords,speed):
    print("move_to_target_coords")
    self.mc.send_coords(coords,speed,1)
    is_finised = self.mc.is_in_position(coords,1);
    print(is_finised)
    count = 0
    is_finised_count = 0
    while is_finised_count < 3 and count < 10:
        print(is_finised)
        if self.mc.is_in_position(coords,1):
            is_finised_count += 1
        time.sleep(0.5)
        count = count + 1
```

控制夹持器闭合与松开。

```
#grap
def grap(self,flag):
    if flag:
        # close
        # self.mc.set_gripper_state(1, 0)
        self.mc.set_gripper_value(50,20)
    else:
        # open
        # self.mc.set_gripper_state(0, 0)
        self.mc.set_gripper_value(255,20)
```

抓取行动，LED 灯为绿色时，机械臂处于空闲状态。

版权所有 © 深圳史河机器人科技有限公司

```
# Grasping motion
def move(self, x, y):
    self.move_to_target_coords([self.coords[0]+int(x), self.coords[1]+int(y), grabParams.height_bias, self.coords[3], self.coords[4], self.coords[5]], 15)


    self.grap(True)
    time.sleep(1)

    self.move_to_target_coords(self.coords, 15)


    self.done = True
    print("Done")
    self.mc.set_color(0,255,0)#green, arm is free
```

获取定位板上两个二维码的位置

```
# get points of two aruco
def get_calculate_params(self, img):
    # Convert the image to a gray image
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    # Detect ArUco marker.
    corners, ids, rejectImaPoint = cv2.aruco.detectMarkers(
        gray, self.aruco_dict, parameters=self.aruco_params
    )

    # print corners, len(corners)

    """
    Two Arucos must be present in the picture and in the same order.
    There are two Arucos in the Corners, and each aruco contains the pixels of its four corners.
    Determine the center of the aruco by the four corners of the aruco.
    """
    if len(corners) > 0:
        if ids is not None:
            if len(corners) <= 1 or ids[0] == 1:
                return None
            x1 = x2 = y1 = y2 = 0
            point_11, point_21, point_31, point_41 = corners[0][0]
            x1, y1 = int(point_11[0]), int(point_11[1])
            point_1, point_2, point_3, point_4 = corners[1][0]
            x2, y2 = int(point_1[0]), int(point_1[1])
            return x1, x2, y1, y2
    return None
```

设置摄像头的抓取参数

```
# set camera clipping parameters
def set_cut_params(self, x1, y1, x2, y2):
    self.x1 = int(x1)
    self.y1 = int(y1)
    self.x2 = int(x2)
    self.y2 = int(y2)
    print(self.x1, self.y1, self.x2, self.y2)
```

计算积木与机械臂之间的坐标关系

```
# set parameters to calculate the coords between cube and mycobot
def set_params(self,  ratio):

    self.ratio = 200.0/ratio

def get_position(self, x, y):
    wx = wy = 0
    if grabParams.grab_direct == "front":
        wx = (self.c_y - y) * self.ratio
        wy = (self.c_x - x) * self.ratio
    elif grabParams.grab_direct == "right":
        wx = (self.c_x - x) * self.ratio
        wy = (y - self.c_y) * self.ratio
    return wx, wy
```


版权所有 © 深圳史河机器人科技有限公司

检测积木的颜色

```
# detect cube color
def color_detect(self, img):
    # set the arrangement of color'HSV
    x = y = 0
    for mycolor, item in self.HSV.items():
        redLower = np.array(item[0])
        redUpper = np.array(item[1])
        # transfrom the img to model of gray
        hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
        # wipe off all color expect color in range
        mask = cv2.inRange(hsv, item[0], item[1])
        # a etching operation on a picture to remove edge roughness
        erosion = cv2.erode(mask, np.ones((1, 1), np.uint8), iterations=2)
        # the image for expansion operation, its role is to deepen the color depth in the picture
        dilation = cv2.dilate(erosion, np.ones(
            (1, 1), np.uint8), iterations=2)
        # adds pixels to the image
        target = cv2.bitwise_and(img, img, mask=dilation)
        # the filtered image is transformed into a binary image and placed in binary
        ret, binary = cv2.threshold(dilation, 127, 255, cv2.THRESH_BINARY)
        # get the contour coordinates of the image, where contours is the coordinate value, here only the contour is detected
        contours, hierarchy = cv2.findContours(
            dilation, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

机械臂工作时，LED 灯为蓝色

```
def run(self):
    self.mc.set_color(0,0,255)#blue, arm is busy
    self.init_mycobot()
```

主函数调用子函数，实现颜色识别抓取的操作。

```
if __name__ == "__main__":
    detect = Detect_marker()
    detect.run()

    cap = fastVideoCapture
    time.sleep(0.5)


    init_num = 0
    nparams = 0
    num = 0
    miss = 0
    while cv2.waitKey(1) < 0 and  not detect.done:
        frame = cap.read()

        # deal img
        frame = detect.transform_frame(frame)


        # get detect result
        detect_result = detect.color_detect(frame)
        detect.show_image(frame)
        if detect_result is None:
            continue
        else:
            x, y = detect_result
            # calculate real coord between cube and mycobot, unit mm
            real_x, real_y = detect.get_position(x, y)
            print(real_x, real_y)
            detect.move(real_x + x_bias, real_y + y_bias)
```

五、课堂作业

1、在 PC 端工控机上输入指令，让机械臂通过颜色识别来抓取积木。

2、理解执行颜色识别的各部分 python 代码。

版权所有 © 深圳史河机器人科技有限公司

实验 29　BR280 物体识别抓取积木

一、实验目的

1、了解物体识别的原理。

2、学会如何通过物体识别来抓取积木。

3、理解相关的 python 代码。

二、实验设备

PC、BR280 机器人、BR280 配套 AI 套件。

三、实验原理

1、物体识别简介

物体识别将图像分类（预测图像中一个对象的类别）和对象定位（识别图像一个或多个对象的位置，并在其周围绘制边框）结合起来，对图像中的一个或多个对象进行定位和分类。

2、YOLOV5 简介

YOLOv5 是一种单阶段目标检测算法，该算法在 YOLOv4 的基础上添加了一些新的改进思路，使其速度与精度都得到了极大的性能提升。主要的改进思路如下所示：

• 输入端：在模型训练阶段，改进部分主要包括 Mosaic 数据增强、自适应锚框计算、自适应图片缩放。输入端表示输入的图片。该网络的输入图像大小为 608*608，该阶段通常包含一个图像预处理阶段，即将输入图像缩放到网络的输入大小，并进行归一化等操作。在网络训练阶段，YOLOv5 使用 Mosaic 数据增强操作提升模型的训练速度和网络的精度；并提出了一种自适应锚框计算与自适应图片缩放方法。

•基准网络：融合其它检测算法中的一些特点，主要包括：Focus 结构与 CSP 结构。基准网络通常是一些性能优异的分类器种的网络，该模块用来提取一些通用的特征表示。YOLOv5 中不仅使用了 CSPDarknet53 结构，而且使用了 Focus 结构作为基准网络。

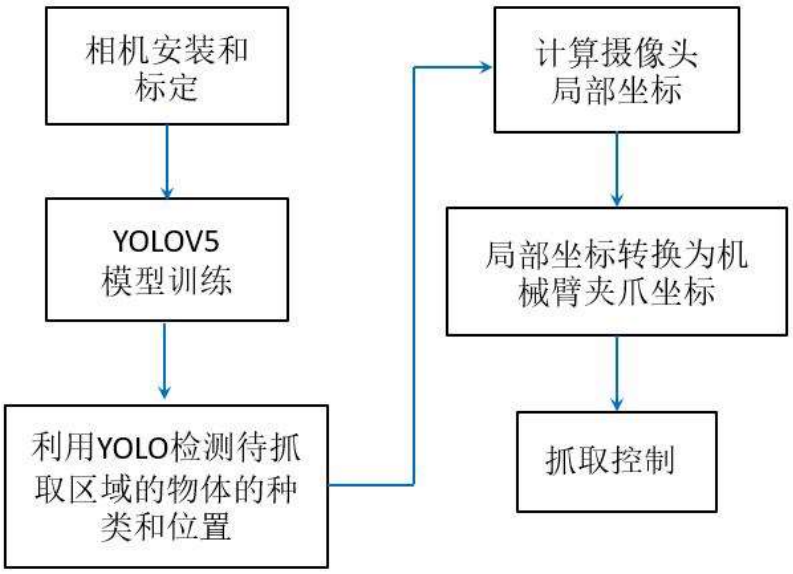
• Neck 网络：目标检测网络在 BackBone 与最后的 Head 输出层之间往往会插入一些层，Yolov5 中添加了 FPN+PAN 结构。Neck 网络通常位于基准网络和头网络的中间位置，利用它可以进一步提升特征的多样性及鲁棒性。虽然 YOLOv5 同样用到了 SPP 模块、FPN+PAN 模块，但是实现的细节会有所不同。

版权所有 © 深圳史河机器人科技有限公司

• Head 输出层：输出层的锚框机制与 YOLOv4 相同，主要改进的是训练时的损失函数 GIOU_Loss，以及预测框筛选的 DIOU_nms。Head 用来完成目标检测结果的输出。针对不同的检测算法，输出端的分支个数不尽相同，通常包含一个分类分支和一个回归分支。YOLOv4 利用 GIOU_Loss 来代替 Smooth L1 Loss 函数，从而进一步提升算法的检测精度。

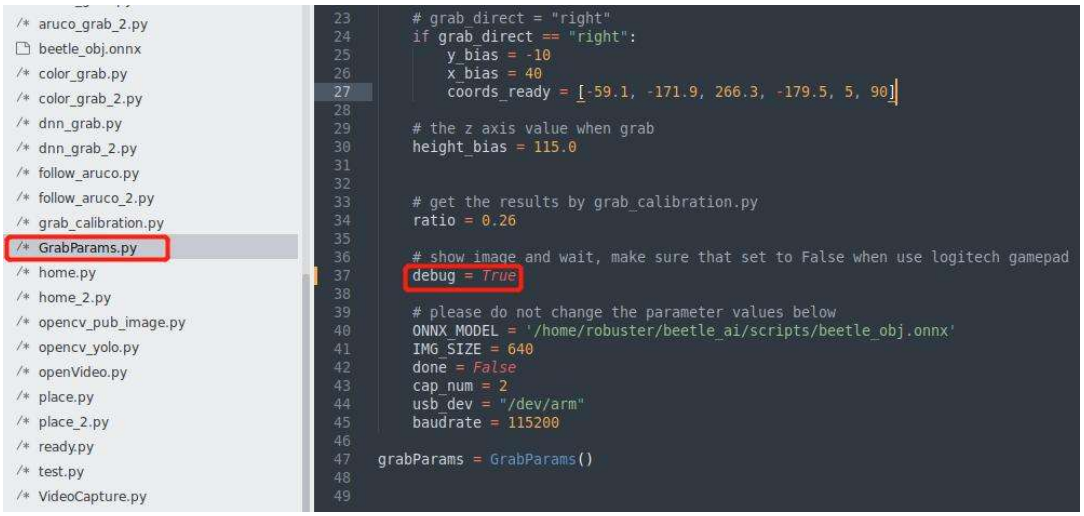
3、物体识别抓取步骤

实现物体识别抓取的主要流程如下：



四、实验步骤

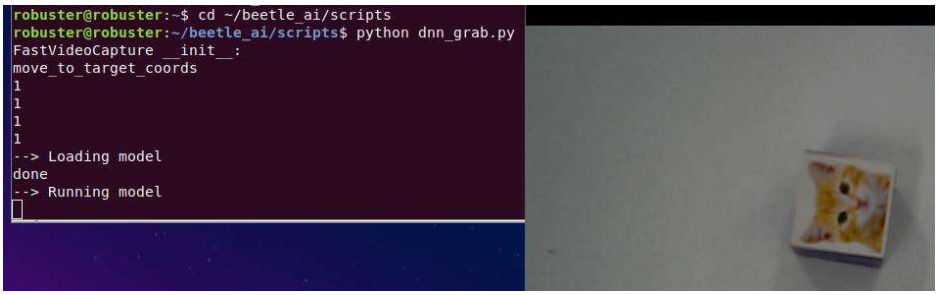
1、打开 sublime，将 GrabParams.py 里面的 debug 改为 Ture。



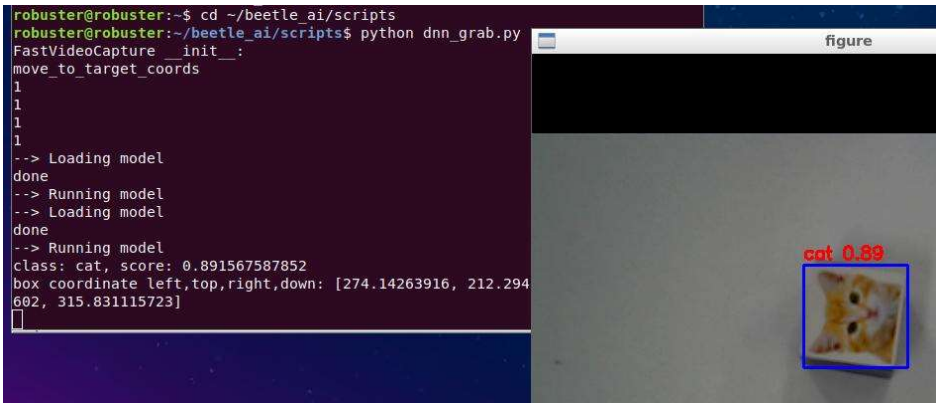
2、组合键 Ctrl+Alt+T 打开终端，切换目录 cd ~/beetle_ai/scripts。

3、执行 python dnn_grab.py。

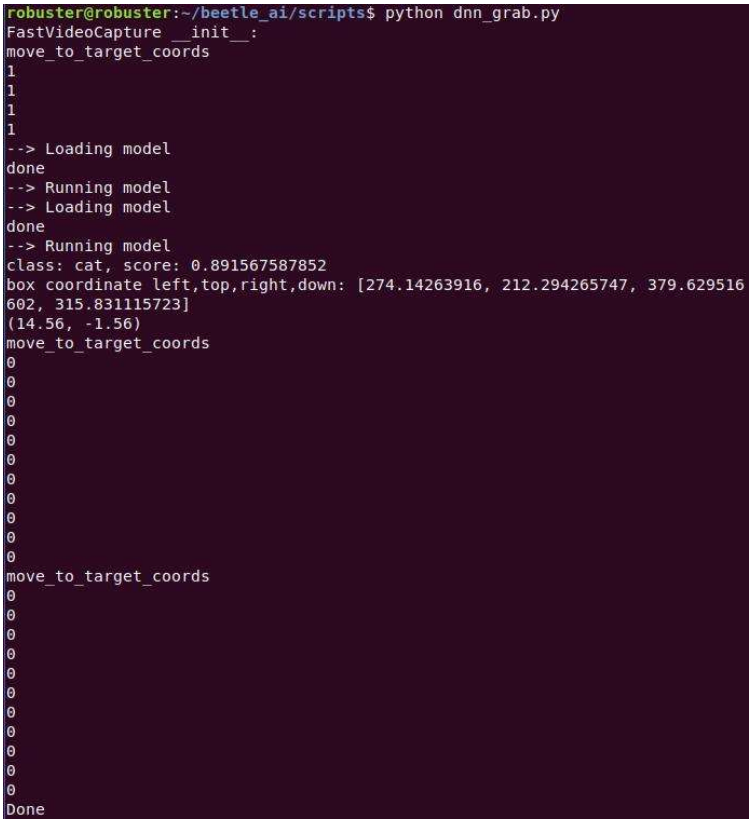
版权所有 © 深圳史河机器人科技有限公司



按下 Enter 键进行图片识别。



识别成功之后，按下 Enter 键执行抓取任务。



版权所有 © 深圳史河机器人科技有限公司

4、代码实现

创建检测目标的类

```
class Detect_marker(object):

    def __init__(self):
        super(Detect_marker, self).__init__()

        self.mc = MyCobot(grabParams.usb_dev, grabParams.baudrate)
        self.mc.power_on()

        self.yolo = yolo()

        self.aruco_dict = cv2.aruco.Dictionary_get(cv2.aruco.DICT_6X6_250)
        # Get ArUco marker params.
        self.aruco_params = cv2.aruco.DetectorParameters_create()
```

以下的内容属于创建的检测目标的类，并且前面实验有的，这里就不再赘述。

检测目标

```
# detect object
def obj_detect(self, img):
    x=y=0
    # Load ONNX model
    print('--> Loading model')
    net = cv2.dnn.readNetFromONNX(grabParams.ONNX_MODEL)
    print('done')

    t1 = time.time()
    blob = cv2.dnn.blobFromImage(img, 1 / 255.0, (grabParams.IMG_SIZE, grabParams.IMG_SIZE), [0, 0, 0], swapRB=True, crop=False)
    net.setInput(blob)
    t1 = time.time()
    # Inference
    print('--> Running model')
    outputs = net.forward(net.getUnconnectedOutLayersNames())[0]


    # simple post process
    boxes, classes, scores = self.yolo.yolov5_post_process_simple(outputs)
    t2 = time.time()

    # img_0 = cv2.cvtColor(img, cv2.COLOR_RGB2BGR)
    if boxes is not None:
        self.yolo.draw(img, boxes, scores, classes)
        left, top, right, bottom = boxes[0]
        x = int((left+right)/2)
        y = int((top+bottom)/2)
        # print x, y
    # cv2.imshow("direct result", img)

    # Print time (inference-only)
    # print "Done."+ str(t2 - t1)+"s"


    if x+y > 0:
        return x, y
    else:
        return None
```

主函数调用子函数实现物体识别抓取。

版权所有 © 深圳史河机器人科技有限公司

```
if __name__ == "__main__":


    detect = Detect_marker()
    detect.run()

    cap = fastVideoCapture
    time.sleep(0.5)

    init_num = 0
    nparams = 0
    num = 0
    miss = 0
    while cv2.waitKey(1) < 0 and  not detect.done:
        # read camera
        frame = cap.read()
        # deal img
        frame = detect.transform_frame(frame)
        detect.show_image(frame)


        # get detect result
        detect_result = detect.obj_detect(frame)
        detect.show_image(frame)
        if detect_result is None:
            continue
        else:
            x, y = detect_result
            # calculate real coord between cube and mycobot, unit mm
            real_x, real_y = detect.get_position(x, y)
            print(real_x, real_y)
            detect.move(real_x + grabParams.x_bias, real_y + grabParams.y_bias)
```

五、课堂作业

1、在 PC 端工控机上输入指令，让机械臂通过图片识别来抓取积木。

2、理解执行图片识别的各部分 python 代码。

版权所有 © 深圳史河机器人科技有限公司

实验 30　BR280 深度学习实验 1—数据集采集与标注

一、实验目的

1、了解深度学习的基本理论

2、掌握如何采集数据集

3、掌握如何标注数据集

二、实验设备

PC、BR280 机器人、BR280 配套 AI 套件。

三、实验原理

1、深度学习的基本概念

深度学习(DL，Deep Learning)是机器学习(ML，Machine Learning)领域中一个新的研究方向，它被引入机器学习使其更接近于最初的目标——人工智能(AI，Artificial Intelligence)。

深度学习是学习样本数据的内在规律和表示层次，这些学习过程中获得的信息对诸如文字，图像和声音等数据的解释有很大的帮助。它的最终目标是让机器能够像人一样具有分析学习能力，能够识别文字、图像和声音等数据。

深度学习的概念源于人工神经网络的研究，但是并不完全等于传统神经网络。而很多深度学习算法中都会包含“神经网络”这个词，所以可以说深度学习是在传统神经网络基础上的升级，约等于神经网络。

2、深度学习与机器学习的相似点

他们都会对数据进行以下操作：

- 数据清洗
- 数据标签
- 归一化
- 去噪
- 降维

3、深度学习与机器学习的区别

传统机器学习的特征提取主要依赖人工，针对特定简单任务的时候人工提取特征会简单有效，但是并不能通用。

深度学习的特征提取并不依靠人工，而是机器自动提取的。深度学习的可解释性很差，因为有时候深度学习虽然能有好的表现，但是我们并不知道他的原理

版权所有 © 深圳史河机器人科技有限公司

是什么。

4、深度学习优点

- 学习能力强
- 覆盖范围广、适应好
- 数据驱动、上限高
- 可移植性好

5、深度学习缺点

- 计算量大、便携性差
- 硬件要求高
- 模型设计复杂
- 没有“人性”，容易存在偏见

6、数据标注

数据标注是大部分人工智能算法得以有效运行的关键环节。数据标注越准确、标注的数据量越大，算法的性能就越好。

数据标注相当于人类从书本中获取已有知识的认知行为。具体操作时，数据标注把需要计算机识别和分辨的图片事先打上标签，让计算机不断地识别这些图片的特征，最终实现计算机能够自主识别。数据标注为人工智能企业提供了大量带标签的数据，供机器训练和学习，保证了算法模型的有效性。

目前，学术界尚未对数据标注的概念形成一个统一的认识，比较认可的是由王翀和李飞飞等人提出的定义。他们认为，标注是对未处理的初级数据，包括语音、图片、文本、视频等进行加工处理，并转换为机器可识别信息的过程。原始数据一般通过数据采集获得，随后的数据标注相当于对数据进行加工，然后输送到人工智能算法和模型里完成调用。

目前数据标注有 3 种常用的划分方式：

(1)按照标注对象进行分类，包括图像标注、视频标注、语音标注和文本标注；

(2)根据标注的构成形式，将其分为结构化标注、非结构化标注和半结构化标注；

(3)根据标注者类型，分为人工标注和机器标注。

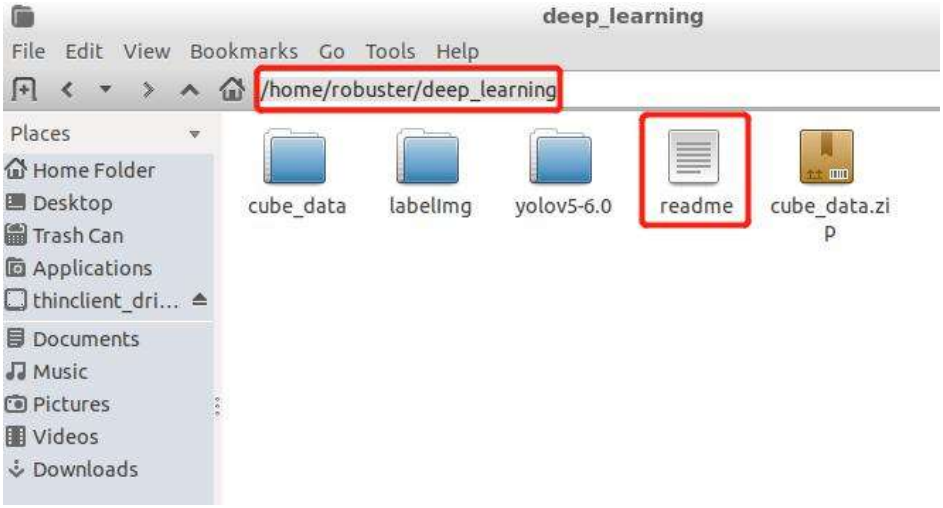
图像标注包括图像标注和视频标注，因为视频也是由连续播放的图像组成。图像标注一般要求标注人员使用不同的颜色来对不同的目标标记物进行轮廓识别，然后给相应的轮廓打上标签，用标签来概述轮廓内的内容。以便让算法模型能够识别图像中的不同标记物。

版权所有 © 深圳史河机器人科技有限公司

四、实验步骤

4.1 数据集采集

1、在/home/robuster/deep_learning 目录下存在一个 readme 文件，里面书写了关于深度学习的所有操作指令，可自行打开查看。

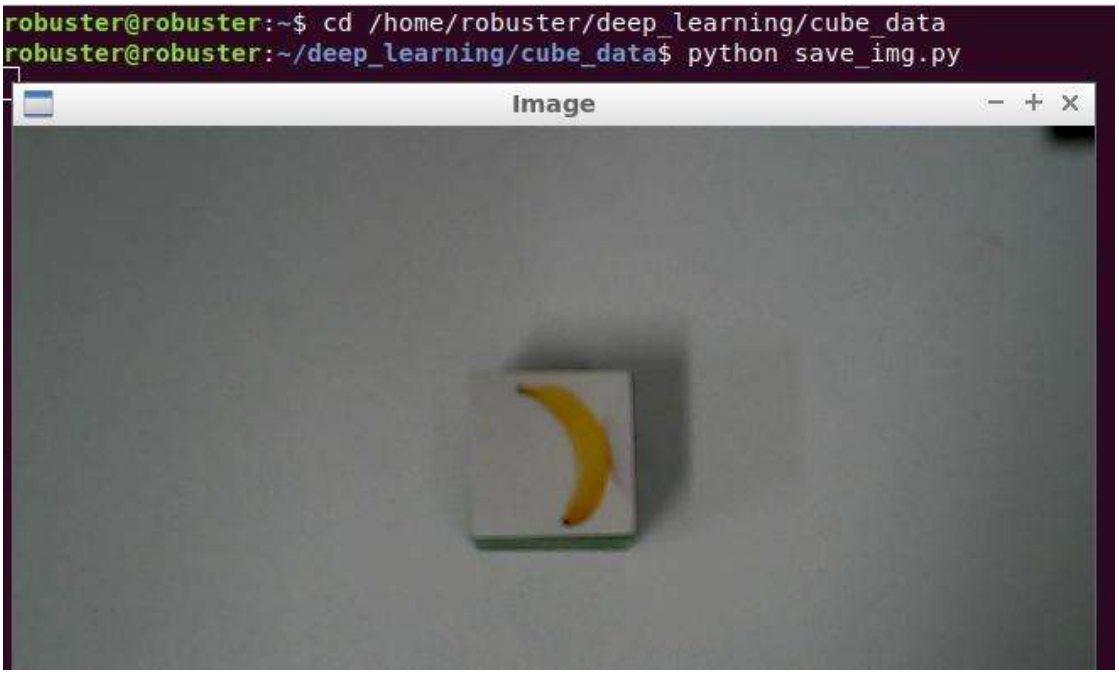


2、进行图片采集之前，首先将机械臂调整到下图所示位置。手柄的 A 键可以达到这个目的。

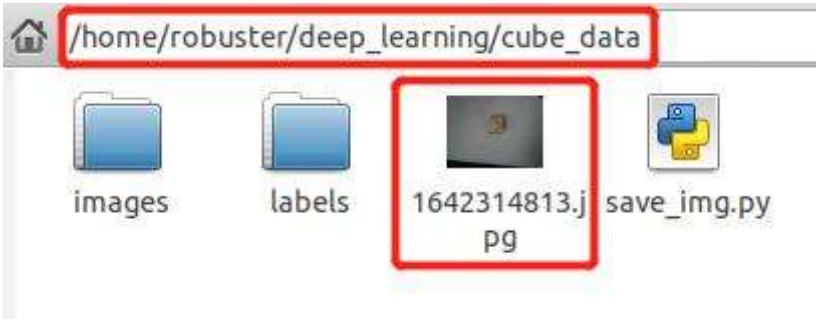
3、通过使用单目相机拍摄图片来获取想要的数据集，为后面的训练模型做准备。在 PC 端工控机打开终端，执行以下指令。

```
cd /home/robuster/deep_learning/cube_data
python save_img.py
```


版权所有 © 深圳史河机器人科技有限公司



4、按 S 键保存图片（保存图片之前，需要用鼠标点一下 Image 界面），此时图片被保存在/home/robuster/deep_learning/cube_data 目录下。



5、将积木旋转 90°，一直按 Enter 键直到界面刷新，然后按 S 键保存。

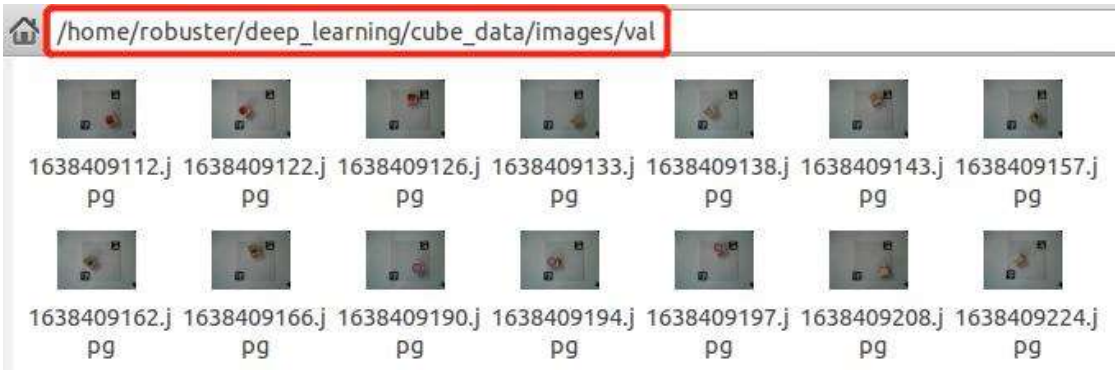
6、重复上述操作，每个位置拍 4 张不一样的图片，每次旋转 90°。

7、将积木分别放置在左上角和右下角，重复上述操作，每个位置拍摄 4 张图片。也就是说，每个物体都要拍 12 张图片来作为训练的数据。

版权所有 © 深圳史河机器人科技有限公司

8、将拍摄的图片拷贝到/home/robuster/deep_learning/cube_data/images/train 目录下。因为我们要识别的图片有 5 类，所以我们总共要拍摄 60 张图片。

9、每个物体还需要拍摄 3 张不同的图片，作为对照使用。注：这 3 张图片不能与上面已经拍摄的图片相同，你可以将积木摆在右上角和左下角，亦或是你喜欢的位置，但是不能与前面的重复。具体操作方法与上面相同。将拍好的 15 张图片拷贝到/home/robuster/deep_learning/cube_data/images/val 目录下。



10、拍摄完所有的图片之后，按 Q 键退出图片采集。（先点击一下 Image 界面）

4.2 数据标注

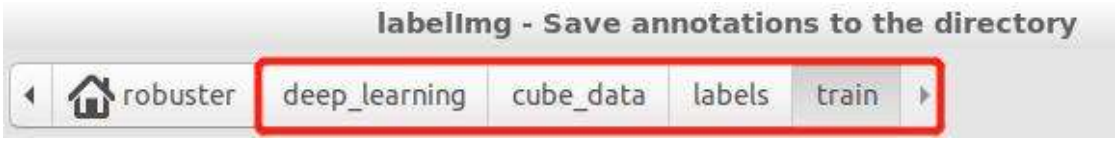
1、打开终端，执行以下指令，开始进行数据标注。

```
cd ~/deep_learning/labelImg
conda activate yolov5
python labelImg.py
```

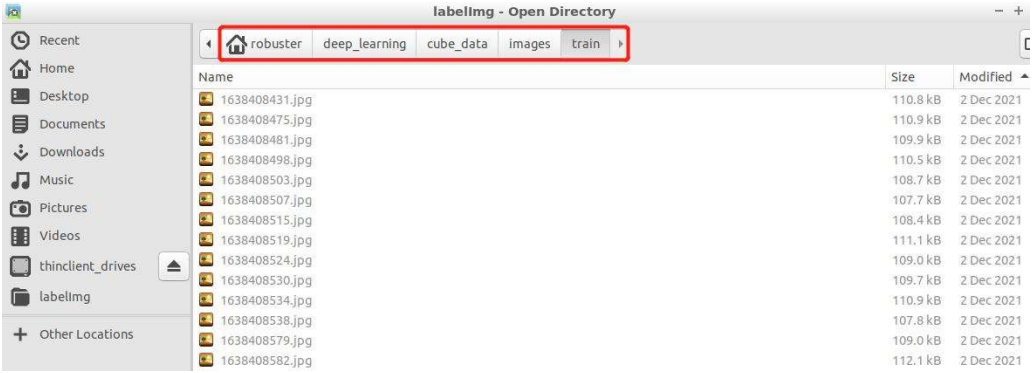

版权所有 © 深圳史河机器人科技有限公司

```
robuster@robuster:~/deep_learning/cube_data$ cd ~/deep_learning/labelImg
robuster@robuster:~/deep_learning/labelImg$ conda activate yolov5
(yolov5) robuster@robuster:~/deep_learning/labelImg$
```

2、先标注 train 文件里面的 60 张图片，点击 Change Save Dir，修改标签的保存路径，将路径修改为/home/robuster/deep_learning/cube_data/labels/train



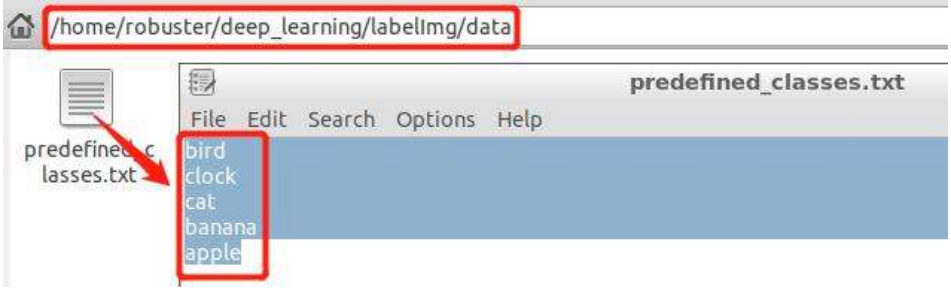
3、点击 Open，找到你将要标注的图片，这些图片被保存在了/home/robuster/deep_learning/cube_data/images/train 路径下。也可以选择 Open Dir，一次打开某个文件夹里面的所有图片，然后逐一进行标注。



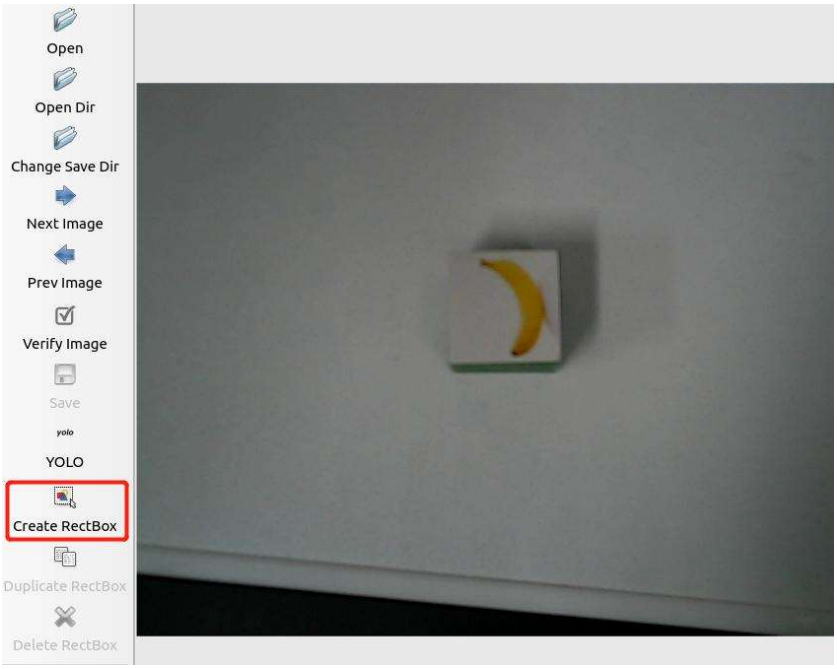
4、进行标注之前，需要给图片分类，因为只拍摄了 5 种物体的图片，所以

版权所有 © 深圳史河机器人科技有限公司

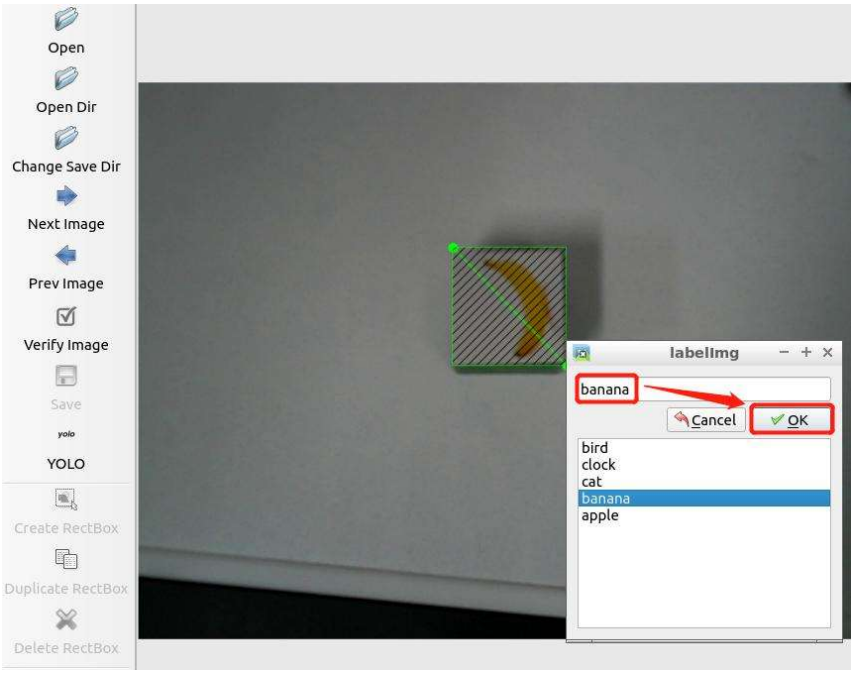
设置 5 类标签，标签的存放路径为/home/robuster/deep_learning/labelImg/data。



5、点击 Create RectBox 开始标注。



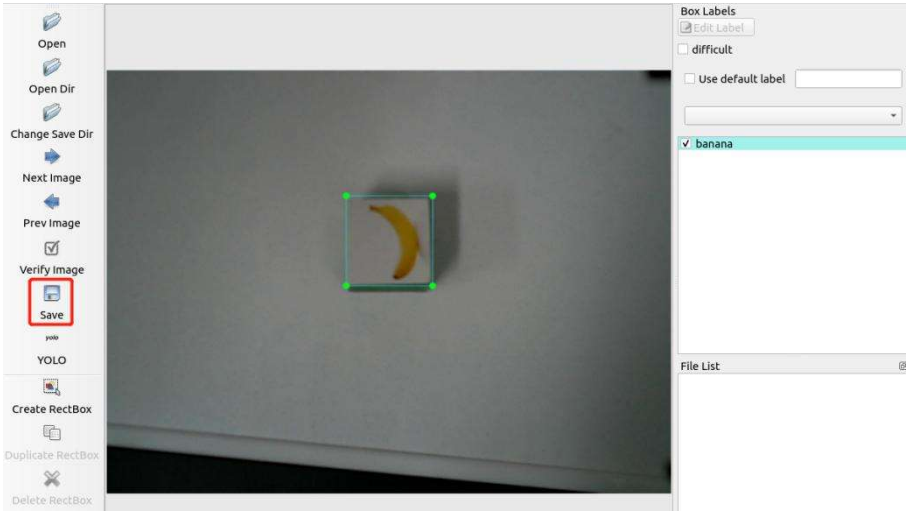
6、创建边框，然后选择对应的标签分类。



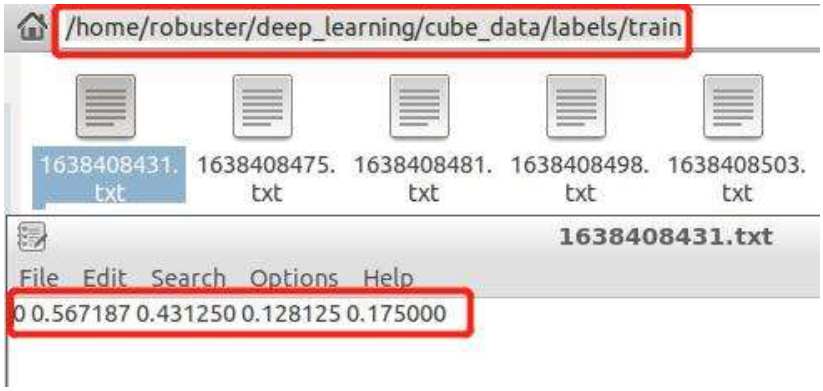
7、点击 Save 保存标签。在/home/robuster/deep_learning/cube_data/labels/train

版权所有 © 深圳史河机器人科技有限公司

里可以找到你保存的标签，它们以.txt 文件的格式存在。



里面的数字分别代表了



8、标注完 train 文件里面的 60 张图片，再标注 val 文件里面的图片。将标签保存路径修改为/home/robuster/deep_learning/cube_data/labels/val。重复上述操作，完成所有图片的标注工作。

五、课堂作业

1、完成 60 张训练样本数据的采集工作。
2、完成 15 张对照样本数据的采集工作。
3、完成 60 张图片的标注工作。
4、完成 15 张图片的标注工作。

版权所有 © 深圳史河机器人科技有限公司

实验 31　BR280 深度学习实验 2—模型训练与目标检测

一、实验目的

1、掌握深度学习模型的训练；
2、掌握深度学习目标的检测。

二、实验设备

PC、BR280 机器人、BR280 配套 AI 套件。

三、实验原理

3.1 什么是目标检测

目标检测的最终目的就是对图像中的目标物体位置和大小进行判定，但是不同物体有着不同的特征，再加上一些外界因素的影响，所以目标检测具有很大的难度，这也是机器视觉领域比较头疼的一个问题。

我们拿到一张图片后需要将其分为三个步骤进行理解：

第一步：分类。用事前确定好的类别或实例 ID 对化为信息的图像结构进行描述。

第二步：检测。上一步是对整张图片内容的描述，这一步则需要选定一个物体目标进行检测，获取物体所处位置以及类别信息。

第三步：分割。这一步需要对语义和实例进行分割，并得出像素属于哪个目标物体或哪个场景的结论。

3.2 基于深度学习的目标检测算法

基于深度学习的主流目标检测算法根据有无候选框生成阶段分为双阶段目标检测算法和单阶段目标检测算法两类。双阶段目标检测算法先对图像提取候选框，然后基于候选区域做二次修正得到检测结果，检测精度较高，但检测速度较慢；单阶段目标检测算法直接对图像进行计算生成检测结果，检测速度快，但检测精度低。

3.2.1 双阶段目标检测算法

该算法需要先借助 SelectiveSearch 选出图像中的候选区域，之后还需要对候选区域进行再次检测，从而得出最后检测结果，比较常用的算法主要有 OverFeat、R-CNN、MaskRCNN 等。

版权所有 © 深圳史河机器人科技有限公司

3.2.2 单阶段目标检测算法

该算法依据的是回归分析思想，所以也被称作回归分析目标检测算法。该算法之所以被称作单阶段目标检测算法是因为该算法不需要生成候选区域，而是直接对整个图像进行检测，从而获得目标位置类别和位置信息，比较常用的检测算法主要有 YOLO 和 SSD。

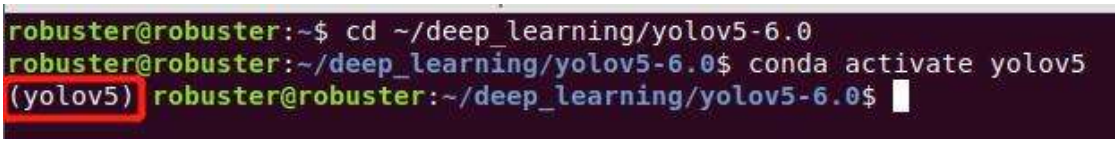
Beetle 使用的就是 YOLO 算法。JosephRedmon 等人在 2016 年的时候提出了由卷积层和 FC 层构成的 YOLO 目标检测算法，先要在最顶层特征图中标出边界框，之后就可以对每个类别概率进行预测，最后再激活函数就可以得到最终信息。该算法并不是针对整幅图像进行检测，而是将整幅图像分为了多个网格单元，对每个网格中心目标进行检测即可，该算法不用生成候选区域，在一个卷积网络中就可以完成特征提取、分类回归等任务，检测过程得到了简化，检测速度也变得更快，但该算法对于小尺度目标的检测不够准确，如果图像中存在重叠遮挡等现象就可能出现遗漏。

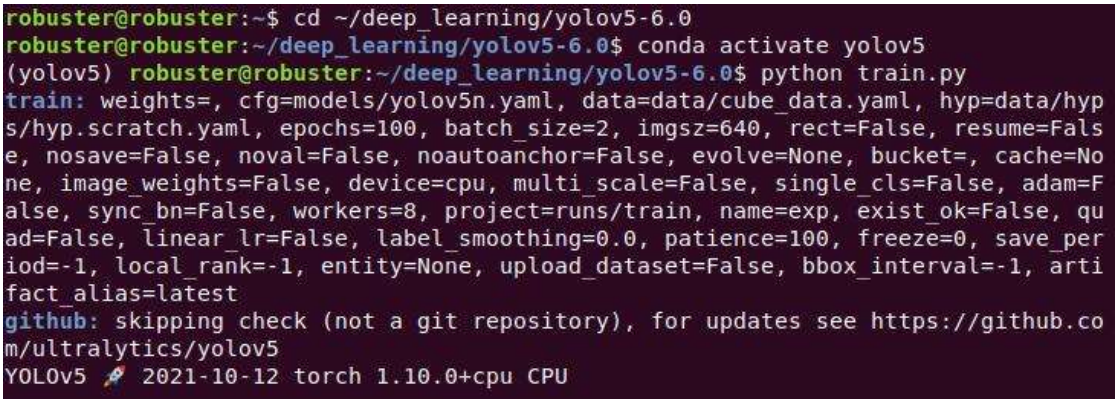
四、实验步骤

4.1 模型训练

1、打开终端，执行以下指令开始模型训练工作。

```
cd ~/deep_learning/yolov5-6.0
conda activate yolov5
python train.py
```





在完成第一次训练之后，我们可以看到 P（准确率）为 0，R（召回率：检测图片里是否有物体）为 0，这表示还不能检测到物体。

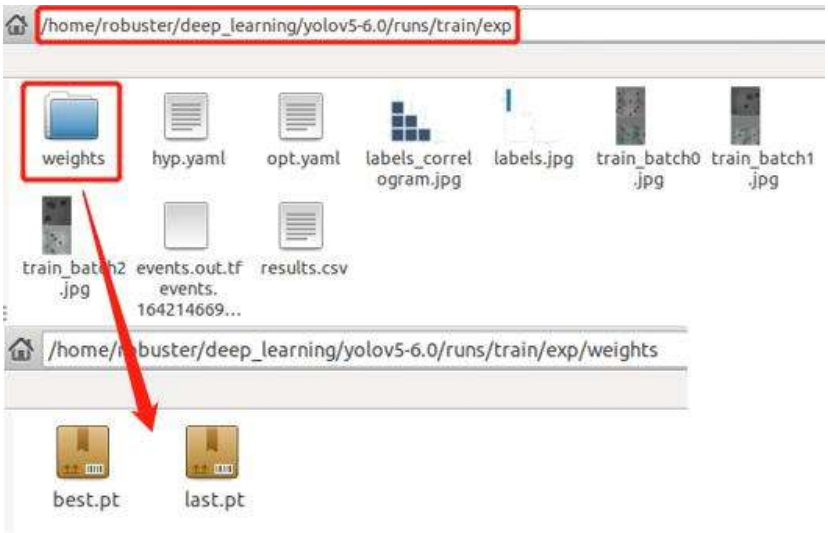
版权所有 © 深圳史河机器人科技有限公司

```
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available
     0/99        0G    0.1096   0.02925   0.04999         6       640:  97%|
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available
     0/99        0G    0.1099   0.02915   0.04997         3       640: 100%|
               Class      Images      Labels           P           R     mAP@.5 mAP@.
                 all          15           0           0           0           0

     Epoch    gpu_mem       box       obj       cls    labels  img_size
  0%|
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available
     1/99        0G     0.117   0.02558   0.05312         3       640:   3%|
```

通过 13 次训练之后，P 和 R 的值都有了变化，训练次数越多，他们的值越大，他们的最大值都是 1。

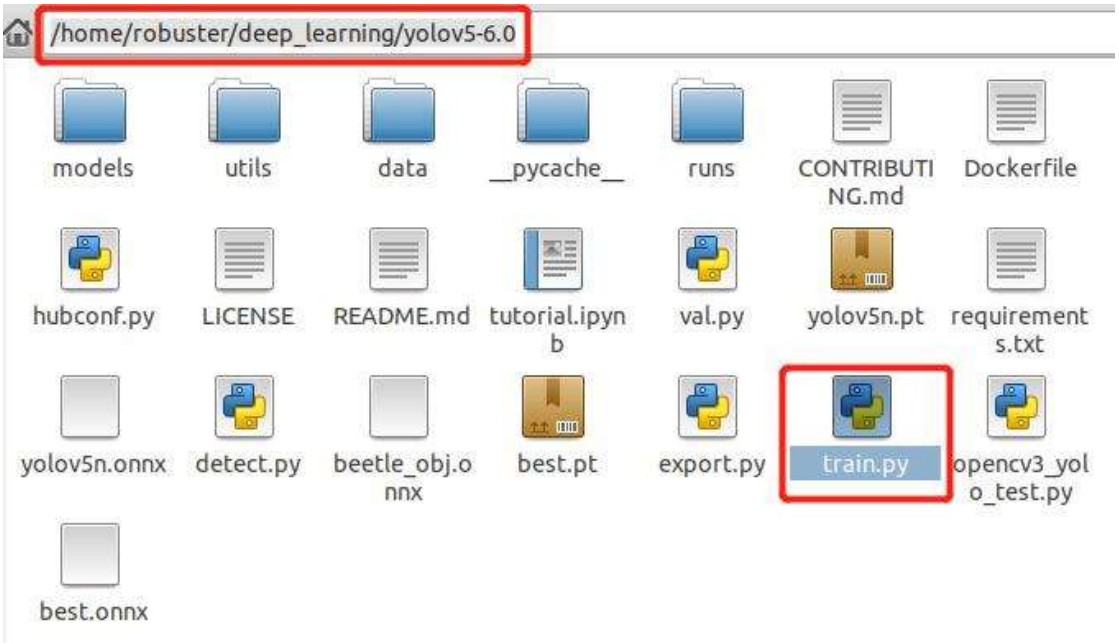
```
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_mode.py:141: Us
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available. Disabling')
    12/99        0G    0.0811   0.03321   0.04771         5       640:  90%|
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_mode.py:141: Us
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available. Disabling')
    12/99        0G   0.08084   0.03328   0.04772         4       640:  93%|
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_mode.py:141: Us
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available. Disabling')
    12/99        0G   0.08097     0.033   0.04774         3       640:  97%|
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_mode.py:141: Us
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available. Disabling')
    12/99        0G    0.0811   0.03285   0.04775         3       640: 100%|
               Class      Images      Labels           P           R     mAP@.5 mAP@.5:.95: 100%|
                 all          15          15    0.000174      0.0667    0.000119    3.57e-05

     Epoch    gpu_mem       box       obj       cls    labels  img_size
  0%|12表示这是第13次训练，99表示总共要训练100次
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/autocast_mode.py:141: Us
  warnings.warn('User provided device_type of \'cuda\', but CUDA is not available. Disabling')
    13/99        0G   0.06087   0.02503   0.04647         2       640:   3%|
```

2、训练完成之后，会生成一个 exp 文件夹，这里存放了一些训练生成的文件，你可以在/home/robuster/deep_learning/yolov5-6.0/runs/train 路径下找到它。在 weights 文件夹里面有两个.pt 文件，其中 best.pt 表示训练效果最好，last.pt 存放的是最后一次的训练结果。



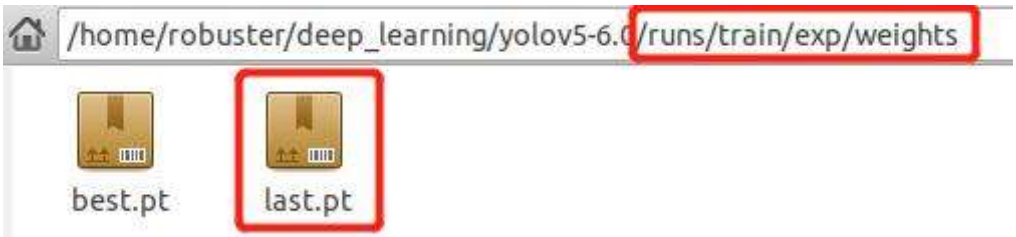
版权所有 © 深圳史河机器人科技有限公司

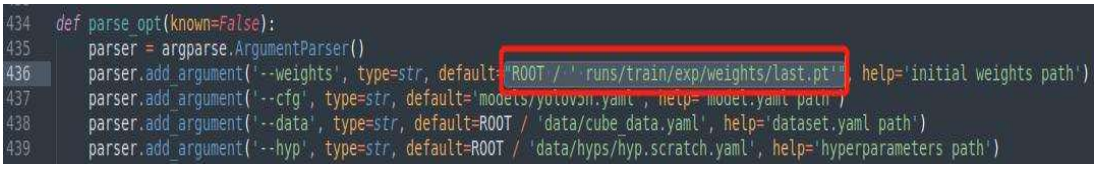
3、如果在训练还未完成时，你中途退出了训练，我们也可以在上一次训练的基础上继续开始训练。找到/home/robuster/deep_learning/yolov5-6.0 里的 train.py，修改下图所示位置里的代码。



第 436 行代码的 default=“ ”表示每次训练都重新开始。440 行的 default 表示样本训练的次数，441 行的 default 表示每次取 2 张图片来训练。

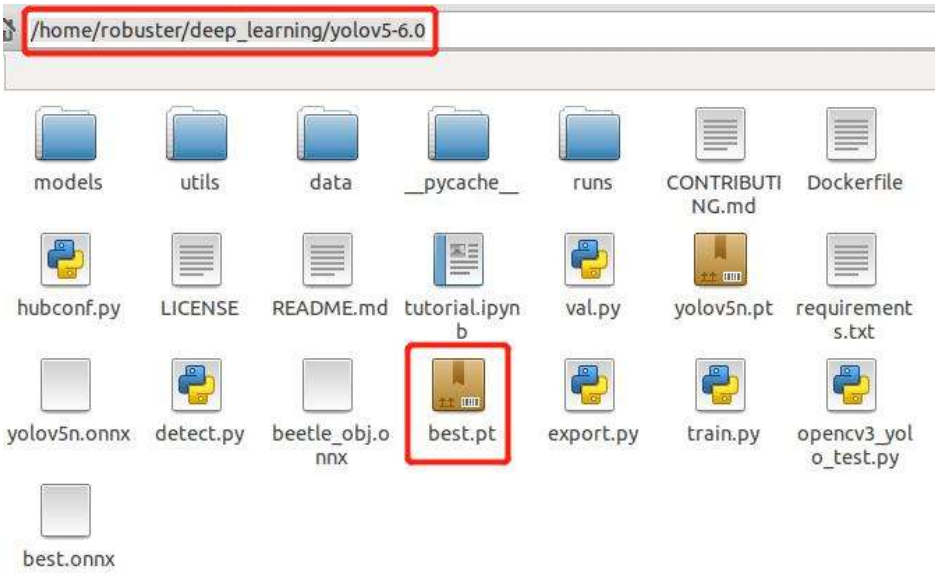
如果要训练未完成的模型，将上次训练的模型的路径添加进去即可。例如我们继续训练下面这个模型，则添加此路径：ROOT / ' runs/train/exp/weights/last.pt'





版权所有 © 深圳史河机器人科技有限公司

4、将 weights 里面的 best.pt 文件拷贝到 /home/robuster/deep_learning/yolov5-6.0 路径下。

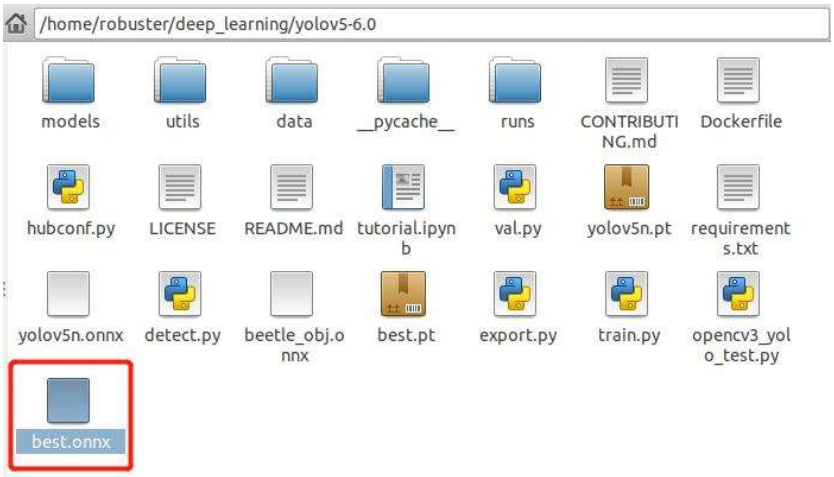


5、将 best.pt 文件导出为 best.onnx 文件。ONNX 全称 Open Neural Network Exchange（开放神经网络交换），ONNX 格式是一个用于表示深度学习模型的标准，可使模型在不同框架之间进行转移（一般用于中间部署阶段）。ONNX 文件不仅仅存储了神经网络模型的权重，同时也存储了模型的结构信息以及网络中每一层的输入输出和一些其它的辅助信息。执行以下指令：

```
cd ~/deep_learning/yolov5-6.0
conda activate yolov5
python export.py
```

```
(yolov5) robuster@robuster:~/deep_learning/yolov5-6.0$ python export.py
export: data=data/cube_data.yaml, weights=best.pt, imgsz=[640, 640], batch_size=1, d
evice=cpu, half=False, inplace=False, train=False, optimize=False, int8=False, dynam
ic=False, simplify=False, opset=12, topk_per_class=100, topk_all=100, iou_thres=0.45
, conf_thres=0.25, include=['onnx']
YOLOv5 🚀 2021-10-12 torch 1.10.0+cpu CPU

Fusing layers...
[W NNPACK.cpp:79] Could not initialize NNPACK! Reason: Unsupported hardware.
/home/robuster/miniconda3/envs/yolov5/lib/python3.6/site-packages/torch/functional.p
y:445: UserWarning: torch.meshgrid: in an upcoming release, it will be required to p
ass the indexing argument. (Triggered internally at  ../aten/src/ATen/native/TensorS
hape.cpp:2157.)
  return _VF.meshgrid(tensors, **kwargs)  # type: ignore[attr-defined]
Model Summary: 213 layers, 1765930 parameters, 0 gradients, 4.2 GFLOPs

PyTorch: starting from best.pt (14.7 MB)

ONNX: starting export with onnx 1.10.2...
/home/robuster/deep_learning/yolov5-6.0/models/yolo.py:60: TracerWarning: Converting
 a tensor to a Python boolean might cause the trace to be incorrect. We can't record
 the data flow of Python values, so this value will be treated as a constant in the
future. This means that the trace might not generalize to other inputs!
  if self.grid[i].shape[2:4] != x[i].shape[2:4] or self.onnx_dynamic:
ONNX: export success, saved as best.onnx (7.5 MB)
ONNX: run --dynamic ONNX model inference with: 'python detect.py --weights best.onnx
'

Export complete (10.05s)
Results saved to /home/robuster/deep_learning/yolov5-6.0
Visualize with https://netron.app
```


版权所有 © 深圳史河机器人科技有限公司

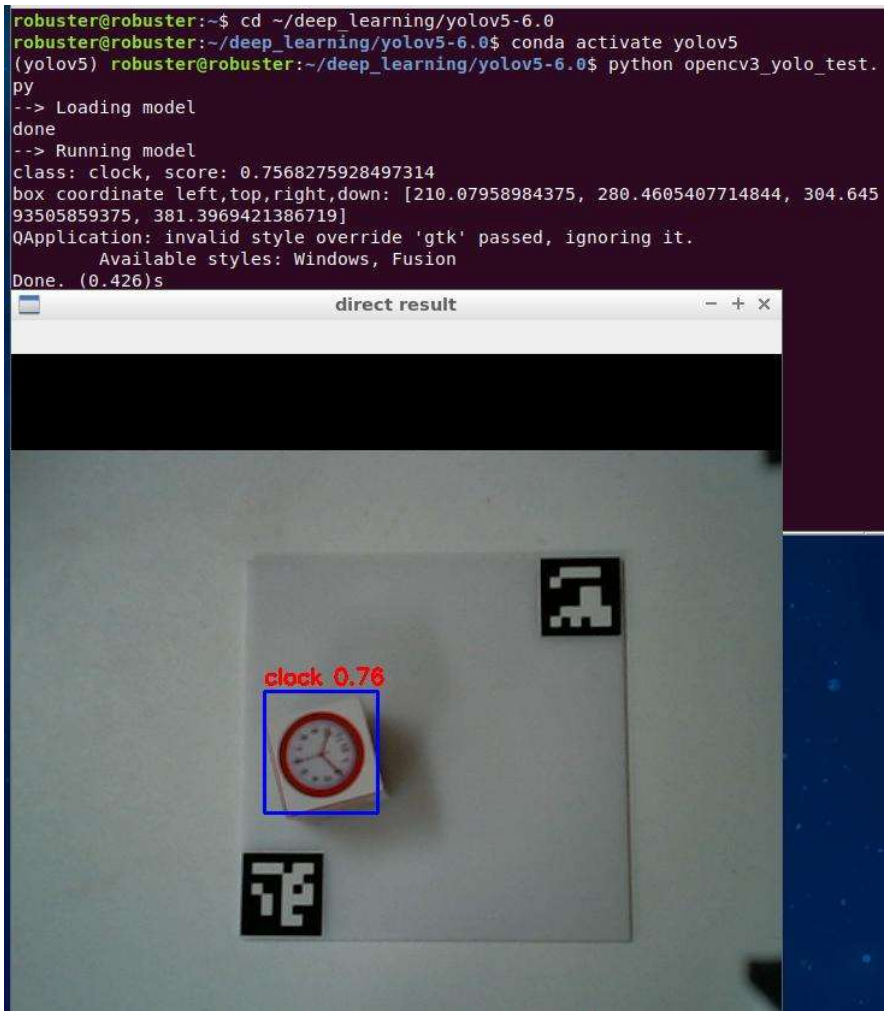


4.2 目标检测

执行以下指令开始进行目标检测测试：

```
cd ~/deep_learning/yolov5-6.0
conda activate yolov5
cp Beetle_obj.onnx best.onnx
python opencv3_yolo_test.py
```


版权所有 © 深圳史河机器人科技有限公司



根据训练好的模型，单目相机可以检测出物体。

五、课堂作业

1、自己根据需求训练一个模型。

2、修改代码，在训练未完成的模型上继续训练。

版权所有 © 深圳史河机器人科技有限公司

实验 32　BR280 自主移动抓取实验

一、实验目的

掌握移动抓取综合应用

二、实验设备

PC、BR280 机器人、BR280 配套 AI 套件、小桌子两张。

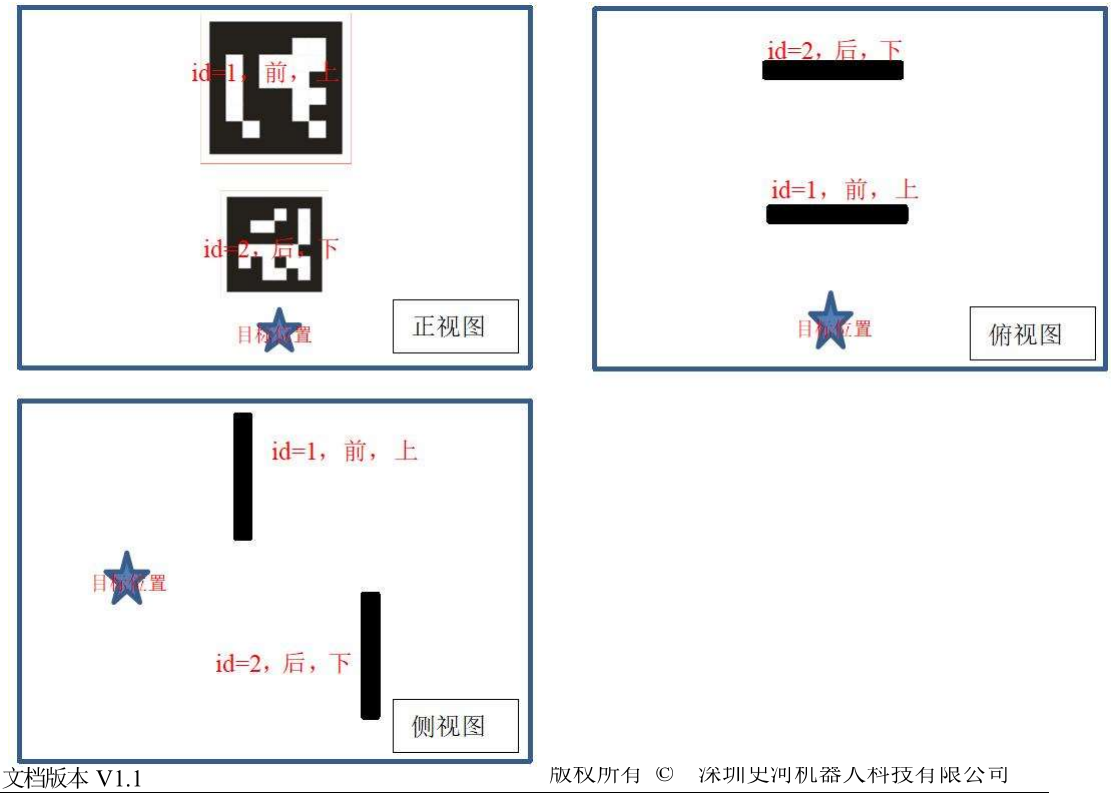
三、实验原理

3.1 什么是移动抓取

移动抓取是一项集成了建图定位，导航避障，视觉定位，机械臂控制等功能的综合应用，它完成了自主移动到抓取区域进行抓取，并导航到放置区域进行放置的功能。

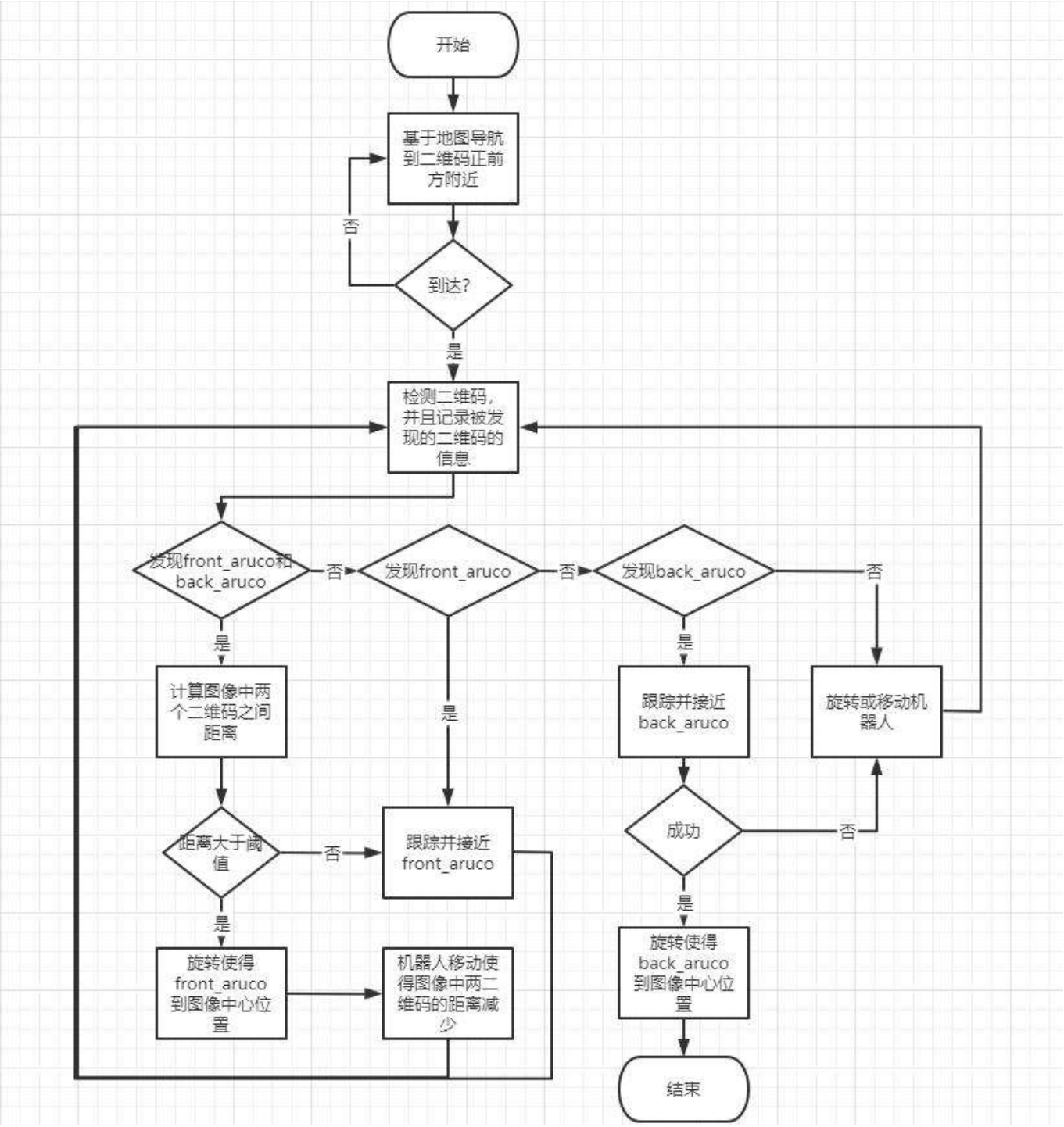
3.2 自主移动和视觉精准对位功能

利用两个二维码来进行机器人目标位置精准控制，详细的布置如下图所示。两个二维码尺寸大小都一样，id=1 的二维码在前面看起来就大一些，id=2 的二维码在后面看起来就小一些。目标位置在二维码的正前方



版权所有 © 深圳史河机器人科技有限公司

控制机器人到达目标位置的流程图如下图所示：



3.3 二维码积木抓取

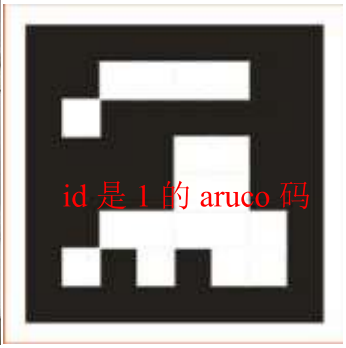
详见实验 27

版权所有 © 深圳史河机器人科技有限公司

四、实验步骤

4.1 改造小桌子

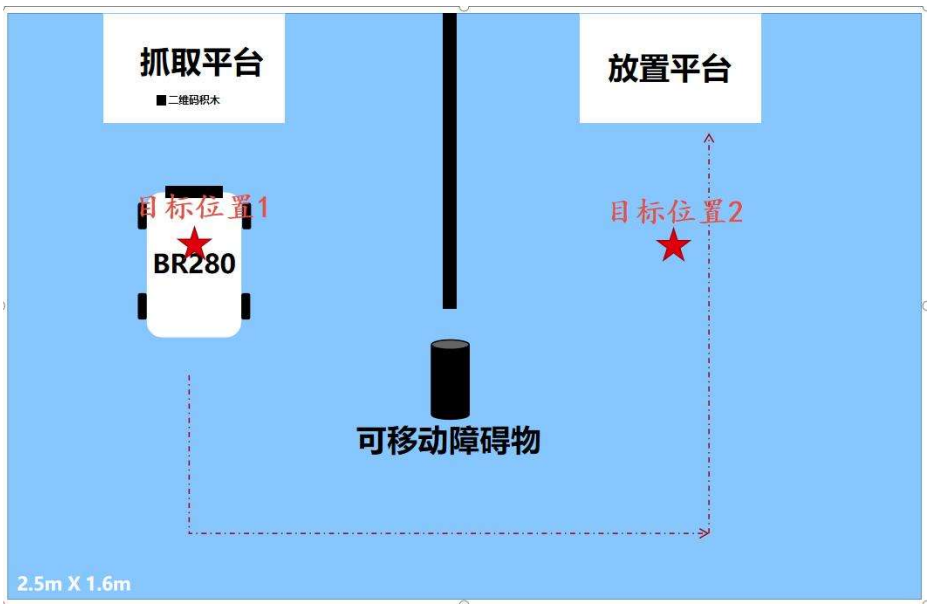
将带有 id 是 1 的 aruco 码的标定板固定在小桌的中心位置，将带有 id 是 2 的 aruco 码的积木固定在小桌背后，如下图所示。两张桌子都是如此改造。



4.2 场地布置

如下图所示，在抓取平台和放置平台处各放置一张改造后的小桌子。并在抓取平台的小桌子桌面正前方处放一个带有 aruco 码的积木。

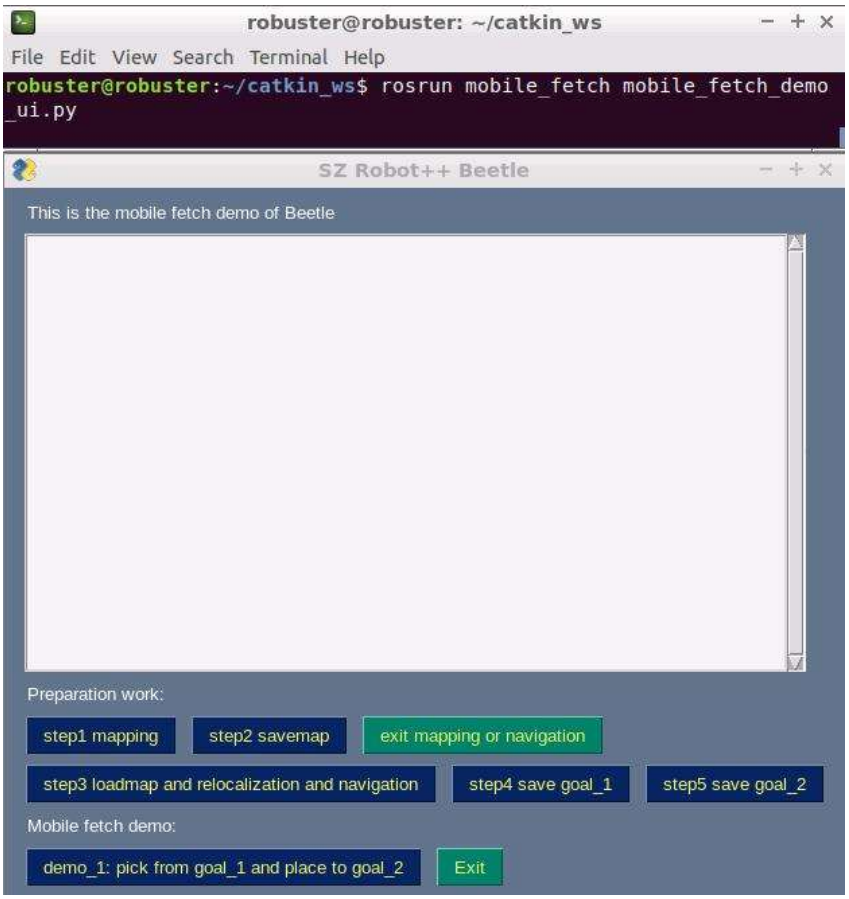
版权所有 © 深圳史河机器人科技有限公司



4.3 软件操作

打开终端，执行以下指令打开移动抓取 UI 界面。

```
rosrun mobile_fetch mobile_fetch_demo_ui.py
```



版权所有 © 深圳史河机器人科技有限公司

第 1 步：在 UI 界面上点击按键“step1 mapping”开始建图，利用遥控手柄控制机器人运动，直至建图完毕；

第 2 步：在 UI 界面上点击按键“step2 savemap”结束建图，系统会自动保存地图到机器人本地存储空间；

第 3 步：在 UI 界面上点击按键“step3 loadmap and relocalization and navigation”加载地图。然后在 rviz 界面上进行重定位。重定位方法参照实验 22 的实验步骤第 2 点。

第 4 步：遥控控制机器人到达抓取小桌正前方 40cm 处，然后在 UI 界面上点击按键“step4 save goal_1”，等待 10 秒钟，系统会自动保存目标位置 1；

第 5 步：遥控控制机器人到达放置小桌正前方 40cm 处，然后在 UI 界面上点击按键“step5 save goal_2”，等待 10 秒钟，系统会自动保存目标位置 2；

第 6 步：在 UI 界面上点击按键“demo_1:pick from goal_1and place to goal_2”开始移动抓取。机器人会自主从抓取平台的小桌上抓取积木，放置到放置平台的小桌子上。接着机器人会自主从放置平台的小桌上抓取积木，放回到抓取平台的小桌上。

五、课堂作业

1、理解整个移动抓取所覆盖的知识点。

2、按实验步骤操作一遍移动抓取功能。

3、尝试修改代码，将二维码抓取换成物体识别抓取。

版权所有 © 深圳史河机器人科技有限公司